

Proyecto Global Integrador - Año 2025

Control de Accionamientos y Automatización de Grúa Portacontenedores de Muelle tipo Pórtico STS (ship-to-shore)

Autómatas y Control Discreto

Profesor

Ing. Gabriel L. Julián

Alumnos

Martín Duci, Ignacio – 13560

Sorrentino, Nicolás – 13423

Resumen

Se presenta el diseño, modelado, simulación e implementación de un sistema híbrido de control y protección para una grúa portacontenedores de muelle tipo pórtico (*ship-to-shore*, STS). El sistema se organiza en tres niveles jerárquicos: el Nivel 2 realiza el control continuo en tiempo discreto de los subsistemas de traslación (carro) e izaje, incorporando realimentación de estados con observador de Luenberger, compensación de no linealidades y control de oscilación de carga con ganancia adaptativa según la longitud de cable; el Nivel 1 implementa la lógica supervisora mediante un autómata de estados jerárquico programado en SFC bajo el estándar IEC-61131-3, gestionando los modos de operación manual y automático, la planificación y coordinación de trayectorias, la estimación de masa del contenedor y la habilitación coordinada de accionamientos y frenos; el Nivel 0 opera de forma independiente como sistema de seguridad y protección, atendiendo condiciones de emergencia con selectividad jerárquica.

La implementación se desarrolló siguiendo un flujo *Model-in-the-Loop* $\rightarrow$ *Software-in-the-Loop*: la lógica del autómata fue diseñada y validada en Stateflow/Simulink y luego traducida a CODESYS en dos vías complementarias: el control supervisor en lenguaje SFC desarrollado manualmente, y el código en lenguaje ST generado automáticamente mediante Simulink PLC Coder, con adaptaciones manuales para satisfacer las restricciones semánticas del entorno de destino. La co-simulación se realizó integrando Simulink y CODESYS a través de OPC UA y UDP. Se implementó además una interfaz de operación con mando de joystick físico y visualización 2D del entorno portuario en tiempo real.

Los resultados obtenidos verificaron el cumplimiento de las especificaciones: el ángulo de oscilación de la carga se mantuvo por debajo de los límites establecidos en todas las condiciones operativas, la productividad en ciclo simple superó el umbral de 30 contenedores por hora, y la estimación de masa alcanzó un error relativo inferior al 1 %. El Nivel 0 respondió correctamente ante las condiciones de emergencia evaluadas, incluyendo activación de finales de carrera, sobrevelocidad y parada de emergencia.

Índice

1. Introducción	5
2. Modelado matemático del sistema físico	7
2.1. Subsistema <i>Trolley</i>	7
2.1.1. Accionamiento de traslación del carro	8
2.1.2. Modelo de cable elástico con amortiguamiento del carro	10
2.1.3. Ecuación de movimiento del carro	11
2.1.4. Subsistema <i>Trolley</i> completo	12
2.2. Subsistema <i>Hoist</i>	13
2.2.1. Accionamiento de izaje de carga	13
2.2.2. Modelo de cable elástico-amortiguado de izaje	17
2.2.3. Movimiento de la carga	19
2.2.4. Ángulo de balanceo y longitud real del cable	22
2.2.5. Subsistema <i>Hoist</i> completo	23
2.3. Generación del perfil de obstáculos $y_{co}(x, t)$	24
2.4. Fines de carrera y detectores de sobrevelocidad	30
2.5. Moduladores de torque de los accionamientos	32
2.6. Sistema físico completo	34
3. Sistema híbrido de control y protección	37
3.1. Nivel 2: Control Regulatorio	37
3.1.1. Controlador de traslación del carro	37
3.1.2. Controlador de izaje de la carga	46
3.1.3. Controlador de oscilación de la carga	52
3.1.4. Nivel 2 completo	57
3.2. Conversión de los diagramas de bloques del Nivel 2 a <i>MATLAB Function</i>	58
3.2.1. Controlador de traslación del carro	58
3.2.2. Observador de estados del carro	59
3.2.3. Controlador de izaje de carga	60
3.2.4. Observador de estados del izaje	60
3.2.5. Controlador paralelo de balanceo	61
3.3. Nivel 1: Control Supervisor	61
3.3.1. Estructura general — Heartbeat y estado OFF	62
3.3.2. Lógica de frenos	63
3.3.3. Comandos de frenos — <code>BRK_Commander</code>	64
3.3.4. Fines de carrera	65
3.3.5. Modos de operación	66
3.3.6. Lógica de <i>twistlocks</i>	68
3.3.7. Control de balanceo de carga	68
3.3.8. Mapa LIDAR	69
3.3.9. Detector de sobrecarga — <code>Overload</code>	70
3.3.10. Banderas de movimiento	71
3.3.11. Agrupador de alarmas — <code>Any_Alarms</code>	72
3.3.12. Planificador de trayectorias	73
3.3.13. Ciclo operativo — <code>Pickup_And_Place_Loop</code>	74

3.3.14. Generación de trayectorias	77
3.4. Nivel 0: Seguridad y Protección	79
3.4.1. Emergencias	80
3.4.2. Subestado Trolley_Emgy	82
3.4.3. Subestado Hoist_Emgy	84
3.4.4. Subestado Global_Emgy	86
3.4.5. Watchdog	87
4. Implementación de los Niveles 0 y 1 en CODESYS	89
4.1. Estructura general del proyecto	89
4.2. Nivel 0: Seguridad y Protección	93
4.3. Nivel 1: Control Supervisor	96
4.3.1. Heartbeat Generator	96
4.3.2. Supervisory Control	97
5. Generación automática de código ST mediante PLC Coder	112
6. Interfaz HMI	114
6.1. Control de mando — Joystick PS4	114
6.2. Visualización en tiempo real — RT_VIEW	118
6.3. Panel de monitoreo en Simulink — Dashboard	122
7. Resultados	123
7.1. Rendimiento: ciclo simple	123
7.2. Pruebas adicionales	124
7.2.1. Evolución del ángulo de balanceo en múltiples ciclos dobles	124
7.2.2. Fines de carrera	125
7.2.3. Parada de emergencia	126
7.2.4. Estimación de masa	128
7.3. Enlaces a videos	128
8. Conclusiones	129
9. Nota sobre el uso de agentes de Inteligencia Artificial	131
Referencias	132

1. Introducción

Las grúas portacontenedoras de muelle tipo pórtico, conocidas internacionalmente como *Ship-to-Shore Container Gantry Cranes* (STS), constituyen el eslabón crítico entre la navegación comercial y la logística terrestre en los puertos de contenedores modernos. Su función principal consiste en la transferencia de contenedores estandarizados entre las bodegas de los buques portacontenedores y los vehículos de transporte interno de la terminal, mediante una **operación semi-automática coordinada** de dos movimientos continuos y simultáneos en el plano vertical 2D: la *traslación horizontal del carro* (*Trolley*, paralelo al eje x) sobre las vigas principal y móvil de la estructura de pórtico, y el *izaje/descenso de la carga* (*Hoist*, paralelo al eje y) a través del cabezal extensible o *headblock/spreader*. Ambos movimientos son impulsados mediante cables de acero (*wireropes*) desde mecanismos rotativos con tambores, reductores, frenos mecánicos y motores eléctricos ubicados en la *Casa de Máquinas* (Machinery House), con accionamientos electrónicos de velocidad variable.

Desde el punto de vista dinámico, la carga suspendida se comporta como un péndulo de longitud variable con 2 grados de libertad en el plano vertical, cuya oscilación lateral (*sway*) constituye la perturbación dominante a controlar. La longitud del cable de izaje varía simultáneamente con el movimiento del carro, conformando un sistema no lineal de parámetros variables que exige estrategias de control sofisticadas para garantizar precisión de posicionamiento, amortiguación de balanceo y ciclos de operación seguros.

Relevancia económica y optimización de tiempos

El comercio marítimo moviliza aproximadamente el 80 % del volumen total de mercancías intercambiadas a nivel global [2], siendo los contenedores estandarizados el formato dominante en los flujos de carga general. La productividad de una grúa STS se mide en contenedores movidos por hora: un incremento de apenas 2–3 movimientos por hora puede representar reducciones de estancia del buque de varias horas, con un impacto económico que para buques de gran porte (*Ultra Large Container Vessels*, ULCV) supera los US\$100 000 por día de demora [3]. La especificación de productividad del presente proyecto establece como objetivo mínimo **30 contenedores movidos por hora** (ciclo completo de ida cargado y retorno vacío en no más de 2 minutos), lo que impone restricciones directas sobre las trayectorias de movimiento, la coordinación entre izaje y traslación, y la eficacia del control de balanceo.

Presencia global y tecnologías alternativas

Las grúas STS dominan de forma casi excluyente las operaciones de transferencia buque–tierra en terminales de aguas profundas. Según datos del sector, el fabricante chino ZPMC (*Zhenhua Port Machinery Company*) concentra alrededor del 70 % de la flota mundial instalada [4], con más de 3 000 unidades desplegadas en más de 100 países. Frente a tecnologías alternativas como los puentes grúa de patio (*Rubber-Tyred Gantry*, RTG) o los vehículos de guiado autónomo (AGV), las grúas STS son insustituibles en la interfaz directa con el buque por su alcance de pluma (*outrreach*) de hasta 72 m, su capacidad de carga superior a 65 t y su compatibilidad con los mayores buques en servicio (clase *Megamax*, 24 contenedores de manga).

Alcance del presente trabajo

El presente trabajo aborda el desarrollo e implementación de un **Sistema Híbrido de Control y Protección con Operación Semi-Automática coordinada** para una grúa STS, que integra control continuo para la regulación de movimientos con lógica discreta para la supervisión y la seguridad operativa. El sistema se estructura en una arquitectura jerárquica de tres niveles:

- **Nivel 2 – Control Regulatorio** ($T_{s2} = 1$ ms): control de lazo cerrado de estados continuos muestreados en tiempo discretizado (CTDS) para el seguimiento de trayectorias de traslación e izaje, incorporando control automático retroalimentado de amortiguación de balanceo (*sway control*) mediante compensador PD/PID con *gain scheduling* en función de la longitud de péndulo equivalente.
- **Nivel 1 – Control Supervisor** ($T_{s1} = 20$ ms): autómata de estados discretos activados por eventos (DEDS) para la coordinación de secuencias operativas, detección de condiciones de falla (sobrecarga, cable flojo, fines de carrera), operación de twistlocks y frenos mecánicos, y optimización de trayectorias en modo automático.
- **Nivel 0 – Seguridad y Protección** ($T_{s0} = 20$ ms): autómata *separado* del Nivel 1, en *standby* permanente, que toma el control ante fallas críticas (pulsador de emergencia, sobrevelocidad, fines de carrera últimos, *watchdog*) llevando el sistema a un estado seguro con selectividad jerárquica según origen y criticidad de la falla.

La metodología de desarrollo sigue el enfoque de *Model-based Design* en dos etapas. Primero, la simulación *Model-in-the-Loop* (MIL) mediante **Stateflow**/Simulink: Stateflow modela los autómatas de Niveles 0 y 1 (DEDS), mientras Simulink modela el Nivel 2 (CTDS) y la Planta (estados continuos). Segundo, la implementación en PLC con **CODESYS** bajo el estándar IEC 61131-3 [5], comparando dos métodos alternativos: (1) codificación manual en *Sequential Function Chart* (SFC) con funciones complementarias en *Structured Text* (ST), y (2) generación automática de código ST desde los Statecharts de Stateflow mediante el toolbox **PLC Coder** de Simulink. Ambas implementaciones se verifican mediante co-simulación *Software-in-the-Loop* (SIL) con comunicación **OPC UA** [6].

2. Modelado matemático del sistema físico

El modelado matemático de la grúa portacontenedores considera los dos movimientos principales que intervienen de manera coordinada durante la operación: la traslación horizontal del carro (*Trolley*) sobre las vigas principal y móvil, y el izaje/descenso vertical de la carga (*Hoist*) bajo la posición del carro, junto con los mecanismos que producen dichos movimientos. Asimismo, incorpora la dinámica del balanceo de la carga suspendida, la cual se modela como un sistema pendular acoplado a los movimientos del carro y del mecanismo de izaje.

El sistema completo se representa en el plano vertical mediante un sistema de coordenadas globales (x, y) , donde el eje x corresponde a la traslación horizontal del carro sobre la viga principal, y el eje y representa la altura de izaje de la carga suspendida.

Para el desarrollo de las ecuaciones que describen el comportamiento dinámico del sistema, se adoptan las siguientes **hipótesis simplificativas**, que reducen la complejidad del modelo sin comprometer su representatividad para el análisis y diseño del sistema de control:

- **Estructura rígida:** se asume estructura completamente rígida, sin deformaciones ni vibraciones, para la viga principal, la viga móvil y el pórtico.
- **Carro como cuerpo rígido:** el carro se considera apoyado sobre la viga principal con desplazamiento exclusivamente horizontal, accionado mediante cables.
- **Carga como masa concentrada:** la carga (headblock/spreader con o sin container) se modela como masa concentrada en el centro de la base del *spreader*, con 2 grados de libertad en el plano vertical, suspendida mediante cables.
- **Mecanismos equivalentes simplificados:** se consideran transmisiones rígidas para la traslación del carro y el izaje de la carga, con un cable de acero equivalente por mecanismo.
- **Modelos de cable:** el cable de carro se modela con comportamiento elástico amortiguado en dirección longitudinal (solo tracción, tensado); el cable de izaje se asume de masa despreciable y comportamiento elástico únicamente a tracción.
- **Efectos externos:** se desprecia la resistencia aerodinámica frontal al viento. Los únicos efectos externos considerados son la gravedad sobre la carga suspendida y la interacción de contacto por apilado del *spreader* con el perfil de obstáculos al tomar o liberar una carga.

2.1. Subsistema *Trolley*

Este subsistema comprende el conjunto de elementos relacionados con la traslación horizontal del carro sobre la viga principal, entre ellos el mecanismo de accionamiento (motor + freno operativo + caja reductora + tambor), el cable de acero que transmite la fuerza de tracción, y la dinámica del carro como cuerpo rígido.

2.1.1. Accionamiento de traslación del carro

El mecanismo de accionamiento del carro integra un motor eléctrico, un freno de operación, una caja reductora y un tambor. La Figura 1 ilustra el esquema simplificado del conjunto.

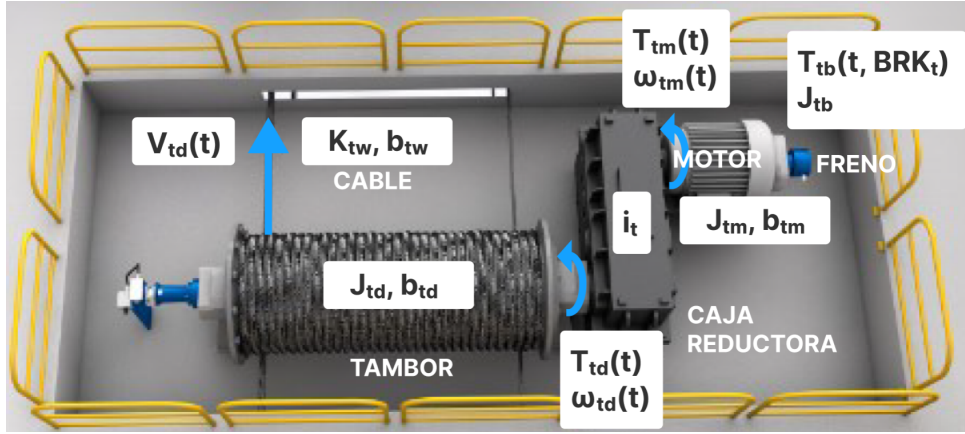


Figura 1: Esquema simplificado del mecanismo de accionamiento de traslación del carro.

Las ecuaciones de movimiento del tambor (eje lento) y del conjunto motor–freno (eje rápido) son, respectivamente:

$$J_{td} \frac{d\omega_{td}(t)}{dt} = T_{td}(t) - b_{td} \omega_{td}(t) - T_{tdl}(t) \quad (1)$$

$$J_{tm+tb} \frac{d\omega_{tm}(t)}{dt} = T_{tm}(t) + T_{tb}(t, BRK_t) - b_{tm} \omega_{tm}(t) - T_{tml}(t) \quad (2)$$

Ambas ecuaciones se vinculan mediante las relaciones de transmisión de la caja reductora:

$$\omega_{td}(t) i_t = \omega_{tm}(t), \quad T_{td}(t) = i_t T_{tml}(t) \quad (3)$$

Despejando $T_{tdl}(t)$ de (1), sustituyendo en (2) mediante (3), y aplicando la relación $T_{tdl}(t) = F_{tw}(t) r_{td}$, se obtiene:

$$J_{tm+tb} \frac{d\omega_{tm}(t)}{dt} = T_{tm}(t) + T_{tb}(t, BRK_t) - b_{tm} \omega_{tm}(t) - \frac{J_{td}}{i_t^2} \frac{d\omega_{tm}(t)}{dt} - \frac{b_{td}}{i_t^2} \omega_{tm}(t) - \frac{r_{td}}{i_t} F_{tw}(t) \quad (4)$$

Agrupando los términos de inercia y amortiguamiento en cada miembro, la ecuación en forma de espacio de estados resulta:

$$\frac{d\omega_{tm}(t)}{dt} = \frac{1}{J_{tm+tb} + \frac{J_{td}}{i_t^2}} \left[T_{tm}(t) + T_{tb}(t, BRK_t) - \left(b_{tm} + \frac{b_{td}}{i_t^2} \right) \omega_{tm}(t) - \frac{r_{td}}{i_t} F_{tw}(t) \right] \quad (5)$$

El torque $T_{tb}(t, BRK_t)$ que aparece en (5) corresponde al freno de operación del carro, cuyo modelo se detalla a continuación.

Freno de operación del carro (eje rápido motor). El freno de operación es un freno mecánico de disco con mordazas (*disc and calipers*), de tipo *Normalmente Cerrado* (NC): cuando está desenergizado (*off*) el resorte cierra las mordazas generando un torque de frenado por fricción; cuando está energizado (*on*) el actuador electro-hidráulico abre las mordazas y el torque es nulo.

OFF: Freno **desenergizado** (cerrado) $\Rightarrow$ torque de frenado por fricción, saturado:

$$T_{tb}(t, BRK_t = \text{Off}) = \min(+T_{tb\text{Max}}, \max(-T_{tb\text{Max}}, -b_{tb} \omega_{tm}(t))) \quad (6)$$

Como función alternativa continua, más efectiva y abrupta, puede usarse la tangente hiperbólica escalada:

$$T_{tb}(t, BRK_t = \text{Off}) = -T_{tb\text{Max}} \cdot \tanh(b_{tb} \omega_{tm}(t)) \quad (7)$$

ON: Freno **energizado** (abierto) $\Rightarrow$ torque nulo:

$$T_{tb}(t, BRK_t = \text{On}) = 0 \text{ N m} \quad (8)$$

El modelo de saturación de (6) refleja que el torque de fricción es proporcional a la velocidad angular del eje rápido pero no puede superar el torque máximo de frenado $T_{tb\text{Max}}$. La formulación con $\tanh$ de (7) aproxima la misma saturación con una función suave y diferenciable en todo el dominio, ventajosa para simulación continua. En operación normal el Nivel 1 (Control Supervisor) energiza el freno antes de iniciar cualquier movimiento y lo desenergiza al detener el carro, garantizando la inmovilidad del sistema cuando el motor no está activo.

La Figura 2 muestra el diagrama de bloques del modelo del freno de operación del carro implementado en Simulink, correspondiente a las ecuaciones (6) y (8).

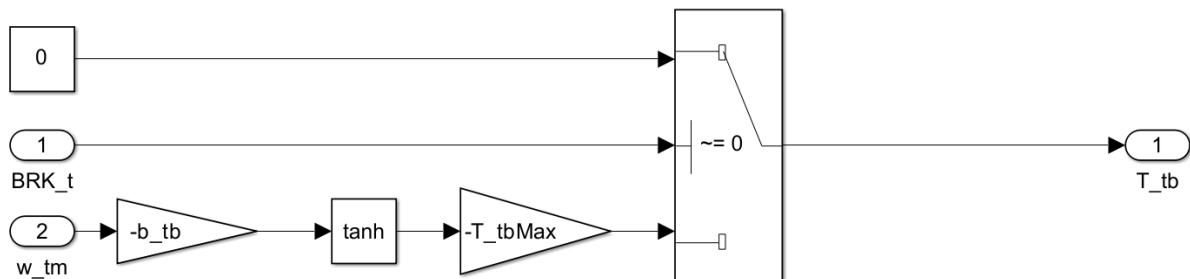


Figura 2: Diagrama de bloques del modelo del freno de operación del carro (Simulink).

La Figura 3 muestra el diagrama de bloques del accionamiento de traslación del carro implementado en Simulink, que integra los modelos del tambor, la caja reductora, el motor y el freno de operación descritos en esta sección.

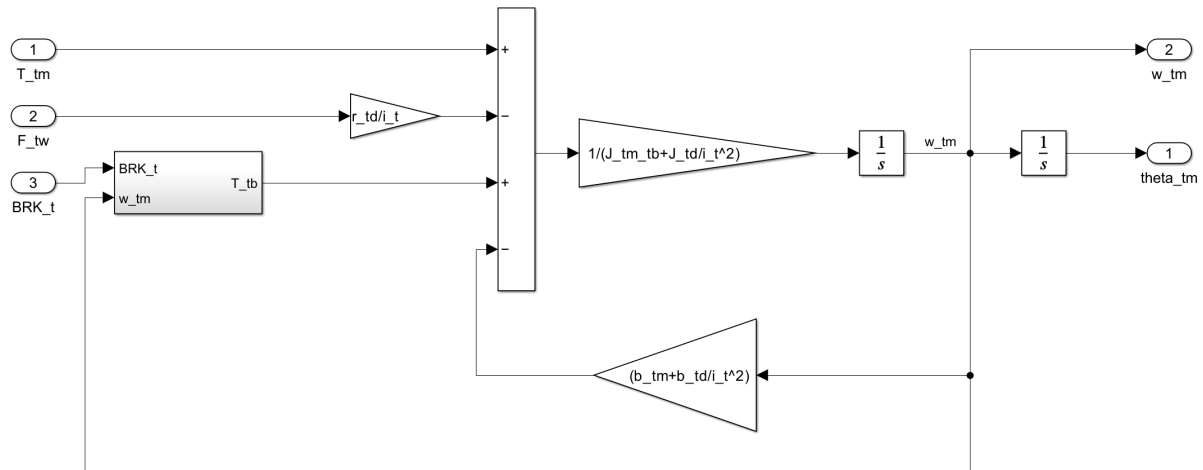


Figura 3: Diagrama de bloques del accionamiento de traslación del carro (Simulink).

2.1.2. Modelo de cable elástico con amortiguamiento del carro

El cable de acero que transmite la fuerza de tracción desde el tambor hasta el carro se modela como un elemento elástico con amortiguamiento en dirección longitudinal, trabajando exclusivamente a tracción (tensado). La fuerza que ejerce sobre el carro depende de la diferencia entre la posición y velocidad de referencia impuesta por el tambor y las del carro mismo:

$$F_{tw}(t) = K_{tw} (x_{td}(t) - x_t(t)) + b_{tw} (v_{td}(t) - v_t(t)) \quad (9)$$

donde K_{tw} es la rigidez equivalente total a tracción del cable tensado del carro, b_{tw} el amortiguamiento interno equivalente, $x_{td}(t)$ y $v_{td}(t)$ la posición y velocidad de referencia del extremo del cable en el tambor, y $x_t(t)$, $v_t(t)$ la posición y velocidad del carro.

La Figura 4 muestra el diagrama de bloques de este modelo implementado en Simulink.

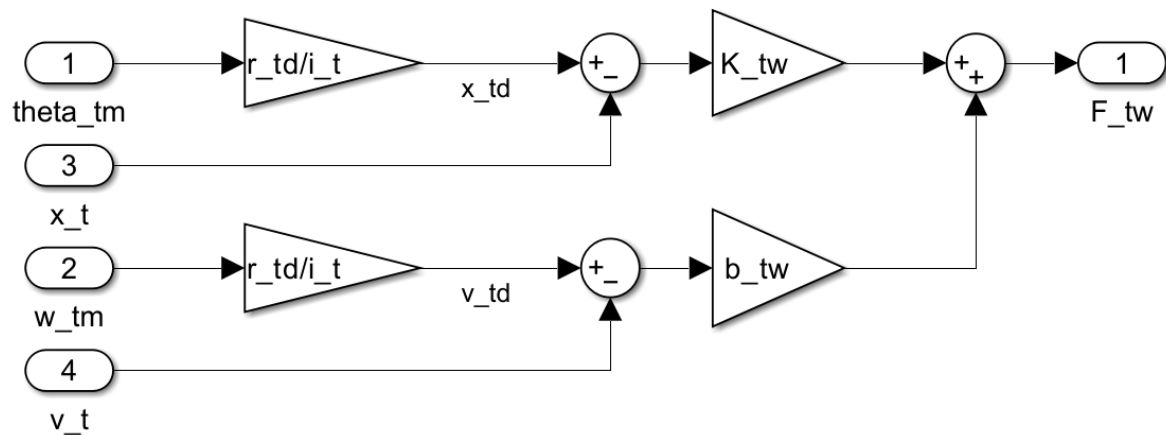


Figura 4: Diagrama de bloques del modelo de cable elástico-amortiguado del carro (Simulink).

2.1.3. Ecuación de movimiento del carro

El carro se modela como un cuerpo rígido de masa equivalente M_t con desplazamiento exclusivamente horizontal. La Figura 5 ilustra las variables y fuerzas involucradas en su dinámica.

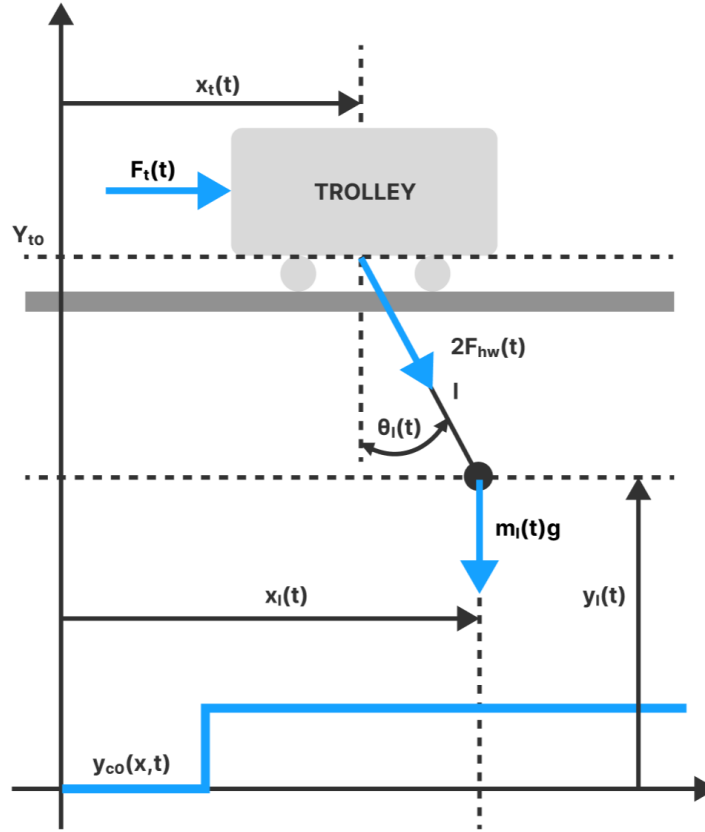


Figura 5: Esquema de variables del subsistema Carro–Cable–Carga.

Aplicando la segunda ley de Newton en la dirección horizontal, las fuerzas que actúan sobre el carro son: la fuerza de tracción del cable $F_{tw}(t)$ proveniente del accionamiento (9), el amortiguamiento viscoso equivalente $b_t v_t(t)$, y la componente horizontal de la fuerza del cable de izaje $2F_{hw}(t, l(t) > l_h(t))$ sobre el carro, que acopla la dinámica del balanceo de la carga con el movimiento de traslación:

$$M_t \frac{dv_t(t)}{dt} = F_{tw}(t) - b_t v_t(t) + 2 F_{hw}(t, l(t) > l_h(t)) \sin(\theta_l(t)) \quad (10)$$

El término $2 F_{hw} \sin \theta_l$ representa la reacción que ejerce el cable de izaje sobre el carro en dirección horizontal cuando el cable está tensado ($l(t) \geq l_h(t)$); su signo actúa oponiéndose al movimiento del carro cuando la carga queda rezagada, constituyendo el principal mecanismo de acoplamiento dinámico entre los dos subsistemas. Cuando el cable está flojo ($l(t) < l_h(t)$), $F_{hw} = 0$ y el carro se desacopla de la carga.

Llevando (10) a la forma de espacio de estados:

$$\frac{dv_t(t)}{dt} = \frac{F_{tw}(t) - b_t v_t(t) + 2 F_{hw}(t, l(t) > l_h(t)) \sin(\theta_l(t))}{M_t} \quad (11)$$

La Figura 6 muestra el diagrama de bloques correspondiente implementado en Simulink.

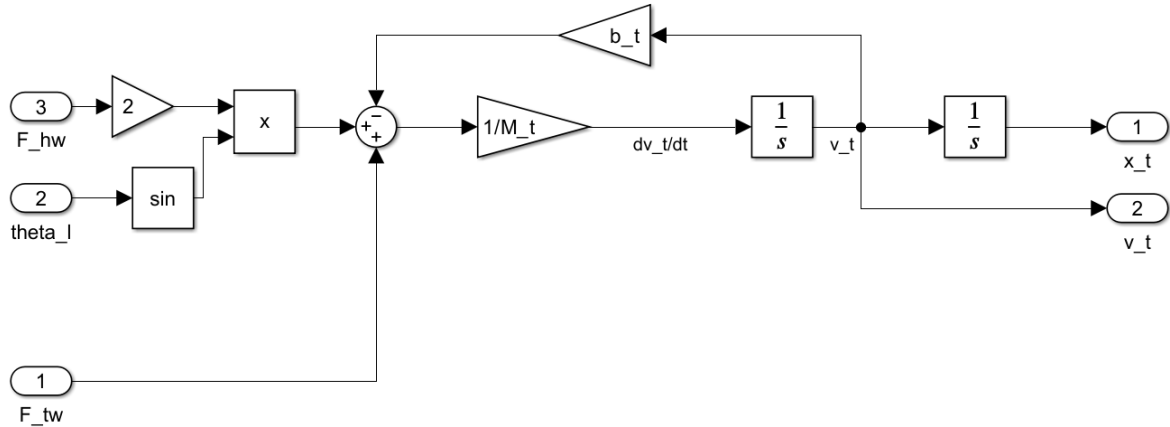


Figura 6: Diagrama de bloques de la ecuación de movimiento del carro (Simulink).

2.1.4. Subsistema *Trolley* completo

La Figura 7 muestra el diagrama de bloques del subsistema *Trolley* completo implementado en Simulink, integrando los tres modelos desarrollados en las secciones anteriores: el accionamiento del carro, la ecuación de movimiento del carro y el modelo de cable elástico-amortiguado.

Para dar claridad al modelado propio de este subsistema, las señales provenientes del subsistema *Hoist* —el ángulo de balanceo $\theta_l(t)$ y la fuerza del cable de izaje $F_{hw}(t)$ — se representan como entradas externas independientes. El acoplamiento con el subsistema *Hoist* se incorporará en la sección 2.6, al integrar ambos subsistemas en el modelo físico completo.

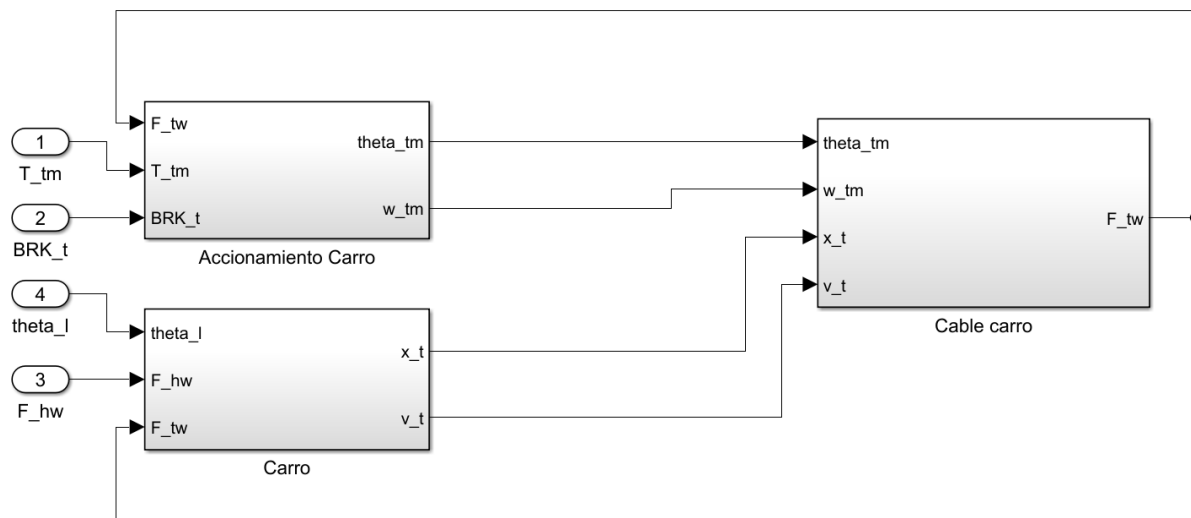


Figura 7: Diagrama de bloques del subsistema *Trolley* completo (Simulink). Las entradas θ_l y F_{hw} se muestran desacopladas del subsistema *Hoist* para mayor claridad.

2.2. Subsistema *Hoist*

El subsistema *Hoist* comprende el conjunto de elementos responsables del izaje y descenso de la carga en dirección vertical (eje y): el mecanismo de accionamiento (motor + freno de emergencia + freno de operación + caja reductora + tambor), el cable de acero equivalente de izaje, y la dinámica de la carga suspendida como masa concentrada.

A diferencia del subsistema *Trolley*, el *Hoist* presenta tres particularidades que incrementan la complejidad de su modelado. En primer lugar, la gravedad actúa directamente como perturbación externa constante sobre la carga, lo que exige que el freno de emergencia permanezca activo ante cualquier falla del sistema de control. En segundo lugar, la masa total suspendida $m_l(t)$ es variable según el estado de los twistlocks (spreader vacío o con container), con posibilidad de condición de sobrecarga. En tercer lugar, la dinámica de izaje está acoplada con la traslación del carro mediante el ángulo de balanceo $\theta_l(t)$, siendo esta interacción el origen del comportamiento pendular que el sistema de control debe amortiguar.

2.2.1. Accionamiento de izaje de carga

El mecanismo de accionamiento de izaje integra un motor eléctrico, un freno de operación (eje rápido), una caja reductora, un tambor y un freno de emergencia (eje lento). A diferencia del carro, la presencia de dos frenos obedece a la necesidad de garantizar la retención de la carga ante cualquier falla: el freno de emergencia actúa directamente sobre el tambor e inmoviliza la carga independientemente del estado del motor. La Figura 8 ilustra el esquema simplificado del conjunto.

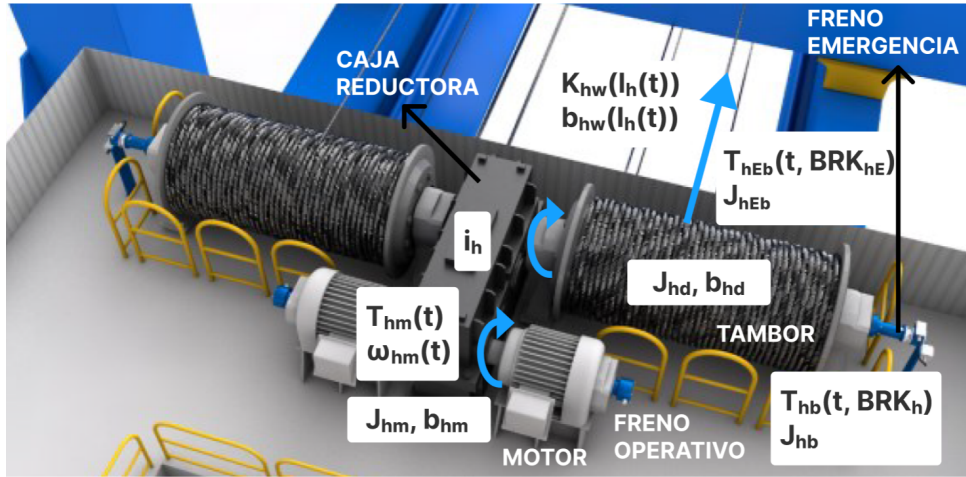


Figura 8: Esquema simplificado del mecanismo de accionamiento de izaje.

Se modela el sistema incluyendo tambor, disco de freno de emergencia, caja reductora, motor y freno de operación. La posición de la carga en el eje y queda definida por:

$$Y_{t0} - l_h(t) \equiv y_h(t) \Rightarrow -\frac{dl_h(t)}{dt} = \frac{dy_h(t)}{dt} \equiv v_h(t) \quad (12)$$

La relación entre el movimiento lineal de la carga y el movimiento rotatorio del tambor, incorporando el factor de transmisión 2:1 del despliegue abierto de cables (para que la carga suba 1 m el cable del tambor debe desplazarse 2 m), es:

$$2 v_h(t) = r_{hd} \omega_{hd}(t), \quad F_{hw}(t, l(t) > l_h(t)) r_{hd} = T_{hdl}(t) \quad (13)$$

El balance de torques en el eje del tambor (eje lento) y la ecuación de posición angular asociada son:

$$J_{hd+hEb} \frac{d\omega_{hd}(t)}{dt} = T_{hd}(t) + T_{hEb}(t, BRK_{hE}) - b_{hd} \omega_{hd}(t) - T_{hdl}(t) \quad (14)$$

$$\frac{d\theta_{hd}(t)}{dt} \equiv \omega_{hd}(t) \quad (15)$$

La relación motor–tambor dada por la caja reductora es:

$$\omega_{hd}(t) i_h = \omega_{hm}(t), \quad T_{hd}(t) = i_h T_{hml}(t) \quad (16)$$

de donde se despeja:

$$\omega_{hd}(t) = \frac{\omega_{hm}(t)}{i_h}, \quad T_{hml}(t) = \frac{T_{hd}(t)}{i_h} \quad (17)$$

El balance de torques en el eje del motor (eje rápido) y su ecuación de posición angular son:

$$J_{hm+hEb} \frac{d\omega_{hm}(t)}{dt} = T_{hm}(t) + T_{hb}(t, BRK_h) - b_{hm} \omega_{hm}(t) - T_{hml}(t) \quad (18)$$

$$\frac{d\theta_{hm}(t)}{dt} \equiv \omega_{hm}(t) \quad (19)$$

Para referir todo al eje del motor, se despeja $T_{hd}(t)$ de (14):

$$T_{hd}(t) = J_{hd+hEb} \frac{d\omega_{hd}(t)}{dt} - T_{hEb}(t, BRK_{hE}) + b_{hd} \omega_{hd}(t) + T_{hdl}(t) \quad (20)$$

Aplicando la relación de transmisión de velocidad (17):

$$T_{hd}(t) = \frac{J_{hd+hEb}}{i_h} \frac{d\omega_{hm}(t)}{dt} - T_{hEb}(t, BRK_{hE}) + b_{hd} \frac{\omega_{hm}(t)}{i_h} + T_{hdl}(t) \quad (21)$$

Sustituyendo en (18) mediante (17) y $T_{hdl}(t) = r_{hd} F_{hw}(t, l(t) > l_h(t))$:

$$\begin{aligned} J_{hm+hb} \frac{d\omega_{hm}(t)}{dt} &= T_{hm}(t) + T_{hb}(t, BRK_h) - b_{hm} \omega_{hm}(t) \\ &\quad - \frac{J_{hd+hEb}}{i_h^2} \frac{d\omega_{hm}(t)}{dt} + \frac{T_{hEb}(t, BRK_{hE})}{i_h} \\ &\quad - \frac{b_{hd}}{i_h^2} \omega_{hm}(t) - \frac{r_{hd}}{i_h} F_{hw}(t, l(t) > l_h(t)) \end{aligned} \quad (22)$$

Agrupando los términos de inercia y amortiguamiento:

$$\begin{aligned} \left(J_{hm+hb} + \frac{J_{hd+hEb}}{i_h^2} \right) \frac{d\omega_{hm}(t)}{dt} &= T_{hm}(t) + T_{hb}(t, BRK_h) + \frac{T_{hEb}(t, BRK_{hE})}{i_h} \\ &\quad - \left(b_{hm} + \frac{b_{hd}}{i_h^2} \right) \omega_{hm}(t) - \frac{r_{hd}}{i_h} F_{hw}(t, l(t) > l_h(t)) \end{aligned} \quad (23)$$

Llevando a la forma de espacio de estados:

$$\begin{aligned} \frac{d\omega_{hm}(t)}{dt} &= \frac{1}{J_{hm+hb} + \frac{J_{hd+hEb}}{i_h^2}} \left[T_{hm}(t) + T_{hb}(t, BRK_h) + \frac{T_{hEb}(t, BRK_{hE})}{i_h} \right. \\ &\quad \left. - \left(b_{hm} + \frac{b_{hd}}{i_h^2} \right) \omega_{hm}(t) \right. \\ &\quad \left. - \frac{r_{hd}}{i_h} F_{hw}(t, l(t) > l_h(t)) \right] \end{aligned} \quad (24)$$

Frenos de operación y de emergencia del izaje. El freno de operación actúa sobre el eje rápido (motor) con el mismo modelo que el freno del carro (6)–(8):

$$T_{hb}(t, BRK_h = \text{Off}) = \min(+T_{hbMax}, \max(-T_{hbMax}, -b_{hb} \omega_{hm}(t))) \quad (25)$$

$$T_{hb}(t, BRK_h = \text{On}) = 0 \text{ N m} \quad (26)$$

El freno de emergencia actúa sobre el eje lento (tambor) y su torque se expresa en términos de la velocidad angular del tambor $\omega_{hd}(t)$:

$$T_{hEb}(t, BRK_{hE} = \text{Off}) = \min(+T_{hEbMax}, \max(-T_{hEbMax}, -b_{hEb} \omega_{hd}(t))) \quad (27)$$

$$T_{hEb}(t, BRK_{hE} = \text{On}) = 0 \text{ N m} \quad (28)$$

El freno de emergencia permanece cerrado por defecto (NC) y solo se abre cuando el sistema de control está activo y habilitado (Nivel 0). Ante cualquier falla crítica se cierra inmediatamente, inmovilizando el tambor e impidiendo el descenso libre de la carga, con independencia del estado del motor y del freno de operación.

Las Figuras 9 y 10 muestran los diagramas de bloques del freno de operación y del freno de emergencia del izaje, respectivamente, implementados en Simulink.

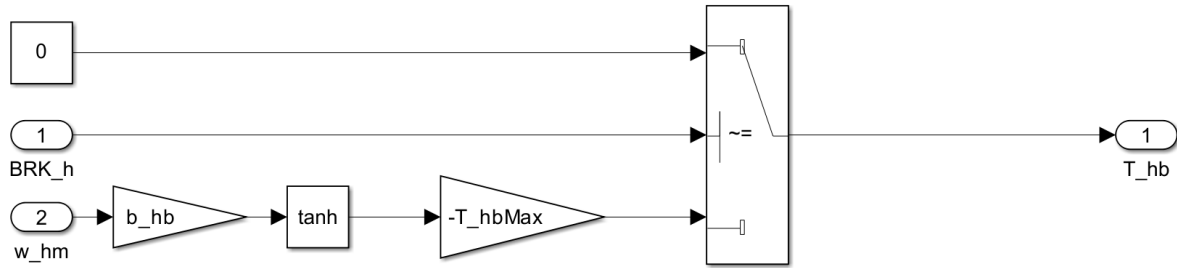


Figura 9: Diagrama de bloques del modelo del freno de operación del izaje (Simulink).

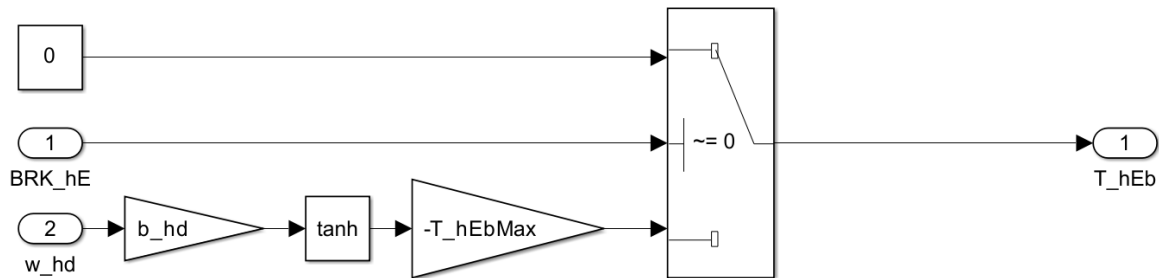


Figura 10: Diagrama de bloques del modelo del freno de emergencia del izaje (Simulink).

La Figura 11 muestra el diagrama de bloques del accionamiento de izaje completo implementado en Simulink.

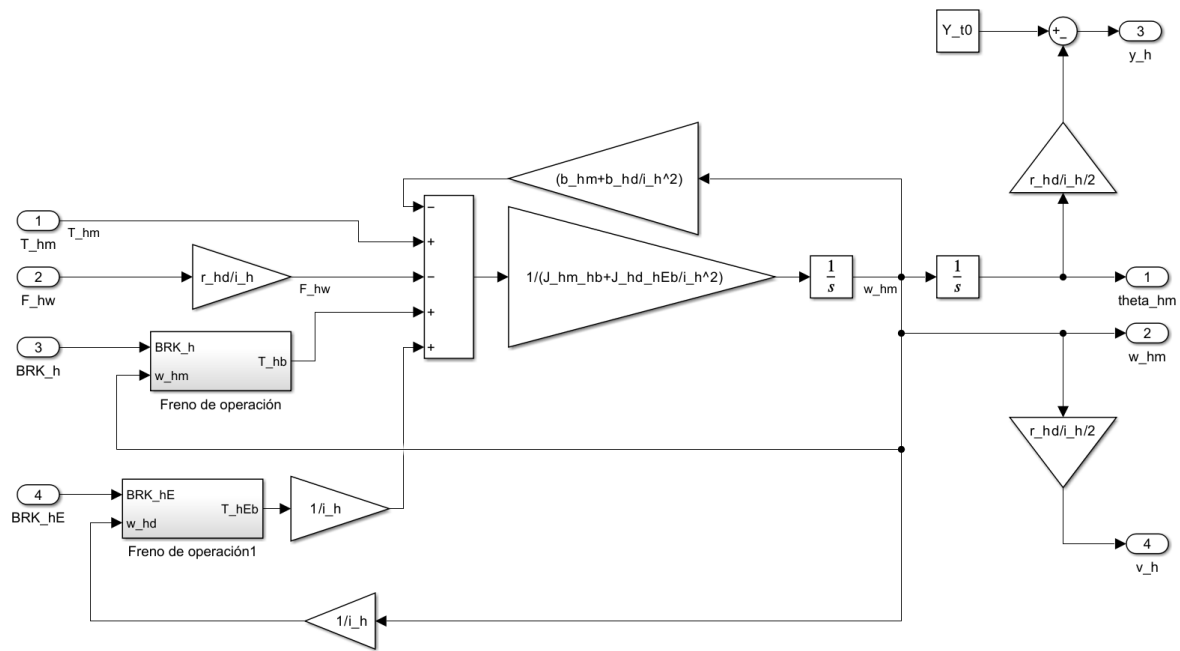


Figura 11: Diagrama de bloques del accionamiento de izaje de carga (Simulink).

2.2.2. Modelo de cable elástico-amortiguado de izaje

El cable de izaje se modela como un único cable de acero equivalente con despliegue abierto que resiste **solo tracción**. Su comportamiento depende del estado de tensión:

- **Cable tensado** (*On*): $l(t) \geq l_h(t)$ — el cable soporta carga con rigidez y amortiguamiento elástico longitudinal.
- **Cable flojo** (*Off*): $l(t) < l_h(t)$ — el cable se panea o flexiona sin resistencia, la fuerza es nula.

ON — Cable tensado. Cuando $l(t) \geq l_h(t)$, la fuerza del cable de izaje es:

$$F_{hw}(t, l(t) \geq l_h(t)) = K_{hw}(l_h(t)) \cdot 2 \cdot (l(t) - l_h(t)) + b_{hw}(l_h(t)) \cdot 2 \cdot \left(\frac{dl(t)}{dt} - \frac{dl_h(t)}{dt} \right) \quad (29)$$

donde los coeficientes de rigidez K_{hw} y amortiguamiento b_{hw} dependen de la longitud de cable desplegada $l_h(t)$ y se calculan como:

$$K_{hw}(l_h(t)) \equiv \frac{k_{hwu}}{2l_h(t) + L_{h0}} \quad (30)$$

$$b_{hw}(l_h(t)) \equiv b_{hwu} \cdot (2l_h(t) + L_{h0}) \quad (31)$$

La Figura 12 muestra el diagrama de bloques de (29) implementado en Simulink, y las Figuras 13 y 14 los cálculos de K_{hw} y b_{hw} respectivamente.

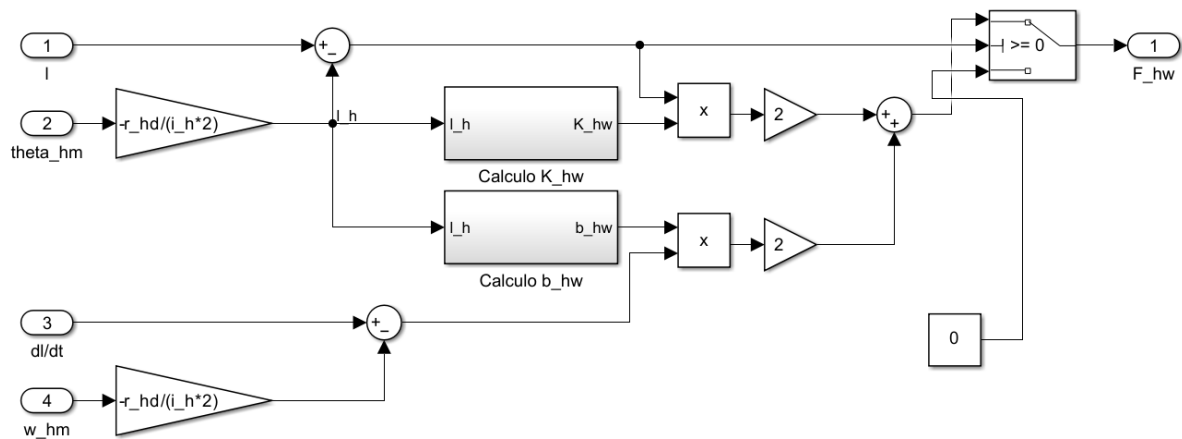


Figura 12: Diagrama de bloques del modelo del cable de izaje — cálculo de F_{hw} (Simulink).

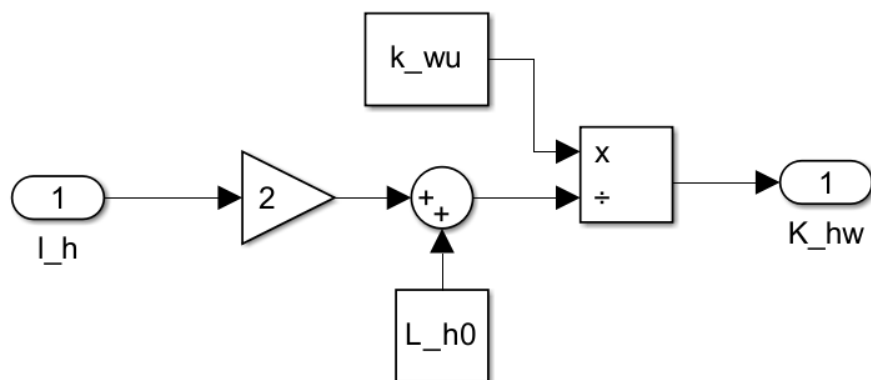


Figura 13: Diagrama de bloques del cálculo de la rigidez equivalente $K_{hw}(l_h)$ (Simulink).

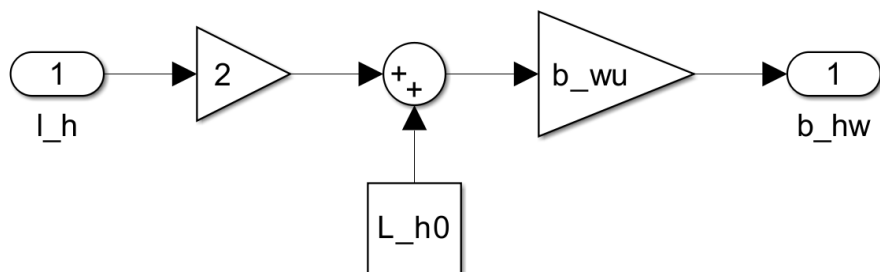


Figura 14: Diagrama de bloques del cálculo del amortiguamiento equivalente $b_{hw}(l_h)$ (Simulink).

OFF — Cable flojo. Cuando $l(t) < l_h(t)$, la carga está apoyada y el cable no transmite fuerza:

$$F_{hw}(t, l(t) < l_h(t)) = 0 \text{ N} \quad (32)$$

2.2.3. Movimiento de la carga

La carga se modela como una masa concentrada con 2 grados de libertad en el plano vertical. Sus propiedades físicas dependen del estado de los **twistlocks** (TLK), variable booleana discreta que indica si el spreader está vacío (abierto, $TLK = \text{Off}$) o cargado con un container (cerrado, $TLK = \text{On}$):

$$m_l(TLK) = \begin{cases} M_s & TLK = \text{Off} \quad (\text{spreader vacío}) \\ M_s + M_c(t) & TLK = \text{On} \quad (\text{con container}) \end{cases} \quad (33)$$

$$h_c(TLK) = \begin{cases} 0 \text{ m} & TLK = \text{Off} \\ H_c & TLK = \text{On} \end{cases} \quad (34)$$

donde h_c es la altura del container que debe descontarse a la posición vertical de la base del spreader para determinar el contacto con el perfil de obstáculos.

Ecuación de movimiento en el eje x (horizontal). Aplicando Newton en dirección horizontal, la componente horizontal de la fuerza del cable de izaje y la fuerza de contacto horizontal actúan sobre la carga:

$$m_l(TLK) \frac{dv_{lx}(t)}{dt} = -2 F_{hw}(t, l(t) > l_h(t)) \sin(\theta_l(t)) + F_{cx}(t, (y_l(t) - h_c(TLK)) \leq y_{c0}(x_l, t)) \quad (35)$$

$$\frac{dx_l(t)}{dt} \equiv v_{lx}(t) \quad (36)$$

donde la fuerza de contacto horizontal F_{cx} es fricción de arrastre cuando la carga está apoyada, y nula cuando está suspendida:

$$F_{cx}(t, (y_l - h_c) \leq y_{c0}) = \begin{cases} 0 & \text{carga suspendida} \\ -b_{cx} v_{lx}(t) & \text{carga apoyada} \end{cases} \quad (37)$$

Ecuación de movimiento en el eje y (vertical). En dirección vertical actúan la componente vertical del cable de izaje, la gravedad y la reacción de contacto con el perfil de obstáculos:

$$m_l(TLK) \frac{dv_{ly}(t)}{dt} = 2 F_{hw}(t, l(t) > l_h(t)) \cos(\theta_l(t)) + F_{cy}(t, (y_l(t) - h_c(TLK)) \leq y_{c0}(x_l, t)) - m_l(TLK) g \quad (38)$$

$$\frac{dy_l(t)}{dt} \equiv v_{ly}(t) \quad (39)$$

La fuerza de contacto vertical F_{cy} modela la reacción elástica amortiguada del apoyo (solo compresión) cuando la base del container toca el perfil de obstáculos:

$$F_{cy}(t, (y_l - h_c) \leq y_{c0}) = \begin{cases} 0 & \text{suspendida} \\ K_{cy}(y_{c0} - y_l + h_c) - b_{cy} v_{ly}(t) & \text{apoyada} \end{cases} \quad (40)$$

Las Figuras 15, 16 y 17 muestran el diagrama de bloques completo del movimiento de la carga y los cálculos de F_{cy} y F_{cx} respectivamente, implementados en Simulink.

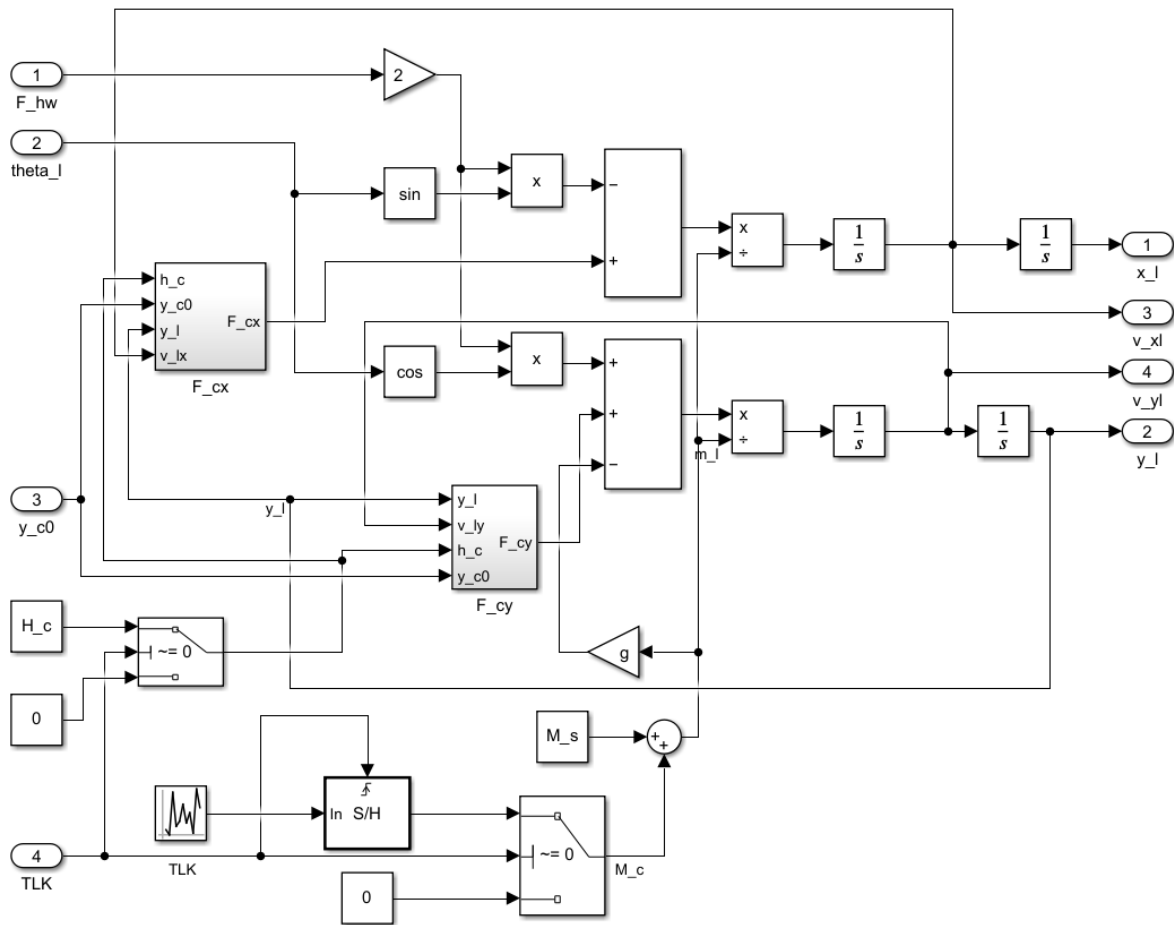


Figura 15: Diagrama de bloques del movimiento de la carga completo (Simulink).

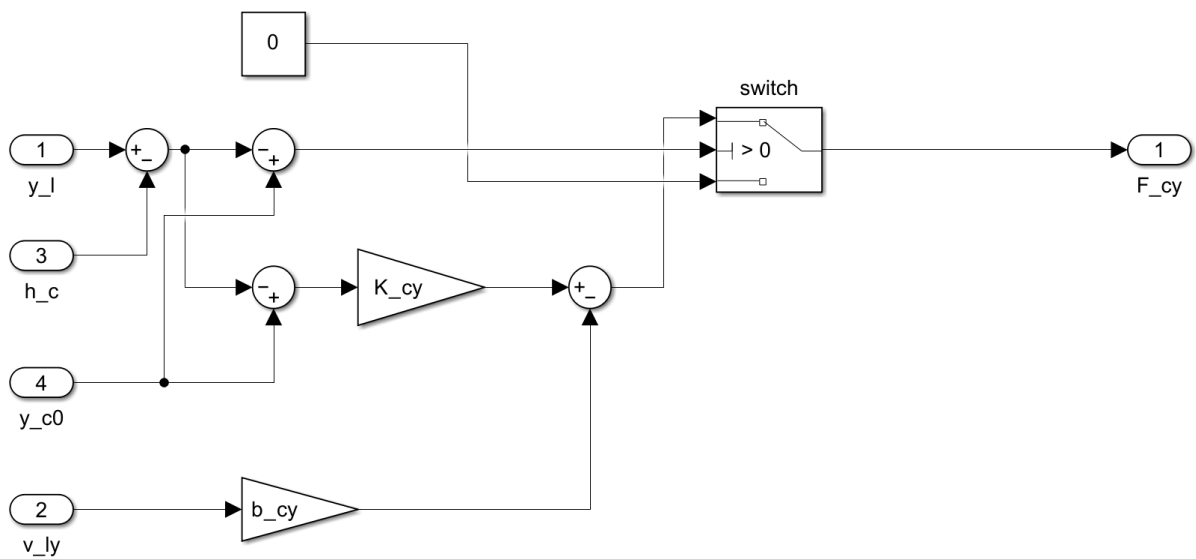


Figura 16: Diagrama de bloques del cálculo de la fuerza de contacto vertical F_{cy} (Simulink).

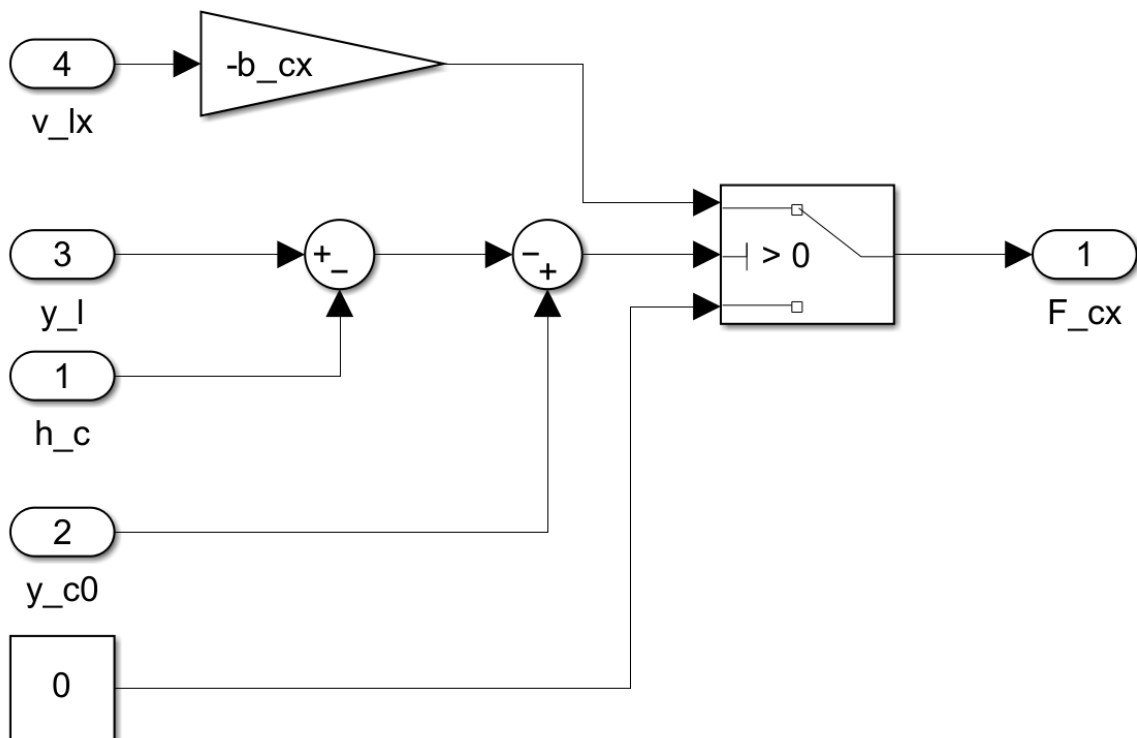


Figura 17: Diagrama de bloques del cálculo de la fuerza de contacto horizontal F_{cx} (Simulink).

Generación de masa aleatoria del container. En una operación real, la masa del container $M_c(t)$ varía entre ciclo y ciclo dentro del rango $[M_{c,min}, M_{c,max}]$ y es desconocida

a priori por el sistema de control. Para emular este comportamiento en la simulación, $M_c(t)$ se genera como un valor aleatorio uniforme en dicho rango cada vez que los twistlocks pasan de abiertos a cerrados, es decir, en el **flanco ascendente** de la señal TLK (Off $\rightarrow$ On). El valor se mantiene constante mediante un bloque *Sample & Hold* hasta el próximo flanco, momento en que se toma un nuevo container con masa diferente. Esto garantiza que la masa total suspendida $m_l(TLK = \text{On}) = M_s + M_c(t)$ sea realista y variable en cada ciclo de carga/descarga, exigiendo al sistema de control estimar $M_c(t)$ a partir de la fuerza del cable $F_{hw}(t)$ durante la maniobra inicial de despegue.

2.2.4. Ángulo de balanceo y longitud real del cable

El ángulo de balanceo de la carga respecto de la vertical, $\theta_l(t)$, y la longitud real del cable de izaje, $l(t)$, se determinan a partir de relaciones geométricas entre las posiciones del carro y de la carga en el plano vertical. Utilizando la función atan2 de cuatro cuadrantes:

$$\theta_l(t) = \text{atan2}\left(\frac{x_l(t) - x_t(t)}{Y_{t0} - y_l(t)}\right) \quad (41)$$

La longitud real del cable, considerando la elongación producida por la carga suspendida, se obtiene como:

$$l(t) = \sqrt{(x_l(t) - x_t(t))^2 + (Y_{t0} - y_l(t))^2} \quad (42)$$

y su derivada temporal, necesaria para el modelo del cable de izaje (29):

$$\frac{dl(t)}{dt} = \frac{(x_l(t) - x_t(t))(v_{lx}(t) - v_t(t)) - (Y_{t0} - y_l(t))v_{ly}(t)}{l(t)} \quad (43)$$

Adicionalmente, en lugar de obtener $\dot{\theta}_l(t)$ por diferenciación numérica de $\theta_l(t)$, se incorpora su cálculo directo a partir de las ecuaciones de movimiento de la carga:

$$\dot{\theta}_l(t) = \frac{(\dot{x}_l(t) - \dot{x}_t(t))l(t) - (x_l(t) - x_t(t))\dot{l}(t)}{l(t)(Y_{t0} - y_l(t))} \quad (44)$$

La Figura 18 muestra el diagrama de bloques unificado implementado en Simulink, que calcula simultáneamente $l(t)$, $\dot{l}(t)$, $\theta_l(t)$ y $\dot{\theta}_l(t)$.

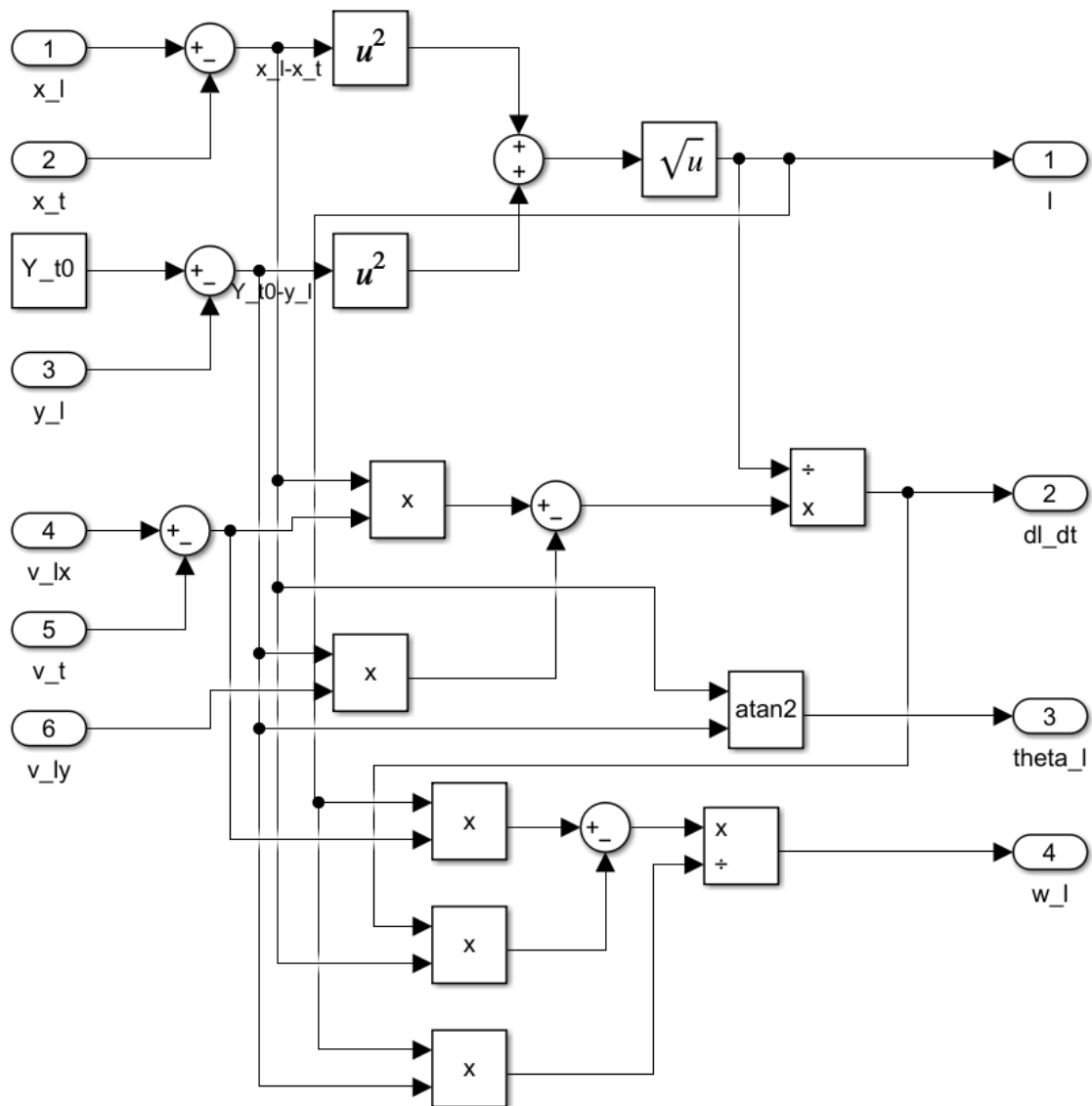


Figura 18: Diagrama de bloques para el cálculo de $l(t)$, $\dot{l}(t)$, $\theta_l(t)$ y $\dot{\theta}_l(t)$ (Simulink).

2.2.5. Subsistema *Hoist* completo

La Figura 19 muestra el diagrama de bloques del subsistema *Hoist* completo implementado en Simulink, integrando el accionamiento de izaje, el modelo del cable, el movimiento de la carga, y el cálculo del ángulo de balanceo y longitud real del cable.

Para dar claridad al modelado propio de este subsistema, las entradas externas se representan de forma independiente y se clasifican en tres grupos:

- **Del subsistema *Trolley*:** posición $x_t(t)$ y velocidad $v_t(t)$ del carro.
- **Del entorno:** perfil de obstáculos $y_{co}(x, t)$ (superficie de apoyo variable).
- **Variables de comando:** torque del motor $T_{hm}(t)$ proveniente del modulador de

torque (Nivel 2); señales booleanas BRK_h , BRK_{hE} (frenos de operación y emergencia) y TLK (twistlocks), generadas por el Control Supervisor (Nivel 1).

El acoplamiento completo con el subsistema *Trolley* se incorpora en la sección 2.6, al ensamblar el modelo físico completo.

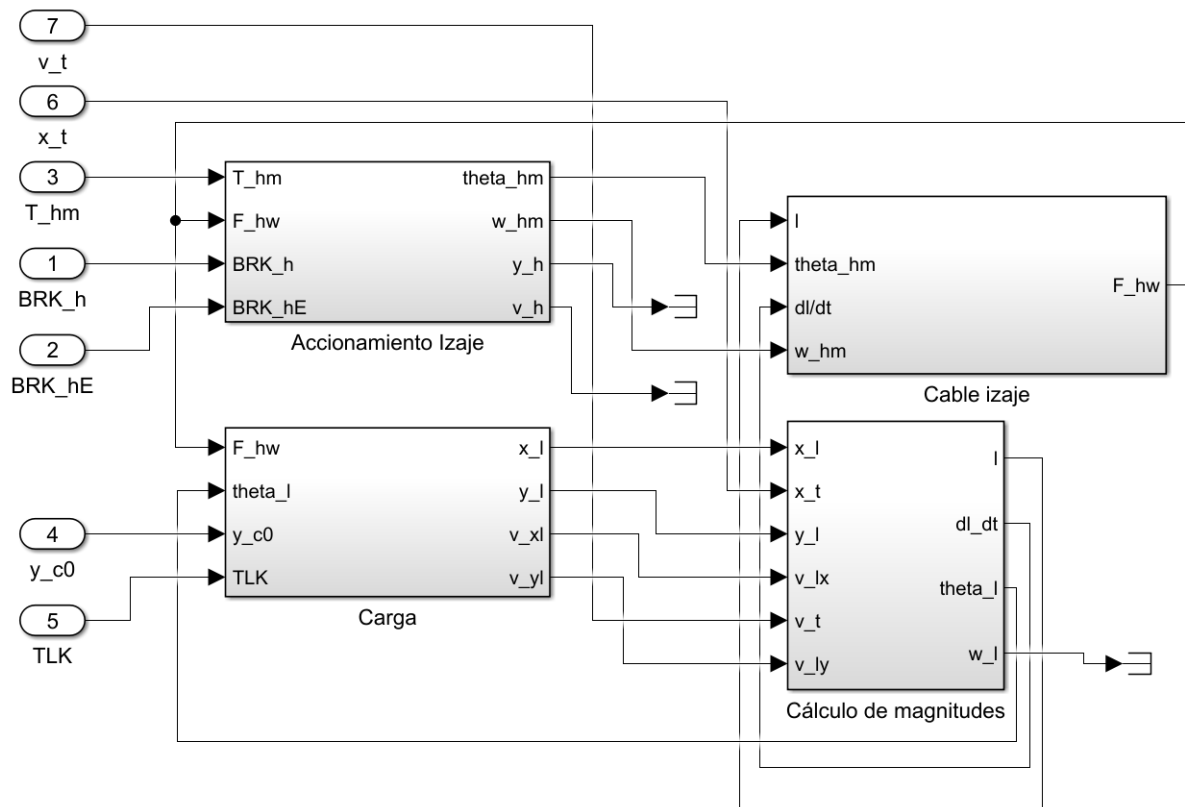


Figura 19: Diagrama de bloques del subsistema *Hoist* completo (Simulink). Las entradas externas se muestran desacopladas del resto del sistema para mayor claridad.

2.3. Generación del perfil de obstáculos $y_{c0}(x, t)$

El perfil de obstáculos $y_{c0}(x, t)$ representa la altura de la superficie de apoyo en función de la posición horizontal x y el tiempo t . Se implementa como una *MATLAB Function* en Simulink con variables persistentes que mantienen el estado completo del entorno entre pasos de simulación. La versión extendida del modelo incorpora, además del apilado de contenedores en patio y barco, la presencia dinámica de camiones de transporte terrestre, la gestión de escenarios de carga/descarga y una lectura anticipada del sensor LIDAR. La función expone seis salidas:

- $y_{c0_contacto}$: altura real bajo la posición de la carga $x_l(t)$, utilizada en el modelo de fuerzas de contacto F_{cx} y F_{cy} (37)(40).
- y_{c0_lidar} : altura anticipada leída por el sensor LIDAR, ubicado a x_{lidar_offset} por delante del spreader.

- `containers_pillars_state`: vector con la altura (en número de contenedores) de cada una de las 19 posiciones de apilado del patio/barco.
- `trucks_state`: vector de presencia de los 5 camiones (1 = presente, 0 = ausente).
- `trucks_has_load`: indica qué camiones llevan contenedor.
- `carga_tomada`: bandera que refleja si el spreader está actualmente enganchado a una carga (señal *TLK* activa).

La implementación se describe a continuación dividida en bloques funcionales.

Firma e inicialización de estado persistente.

```
1 function [y_c0_contacto, y_c0_lidar, containers_pillars_state, ...
2         trucks_state, trucks_has_load, carga_tomada] = ...
3         perfil_suelo(x_c0_left, x_c0_right, Y_sb, W_c, y_h_min, H_c, ...
4                     x_lidar_offset, x_l, container_pillars, TLK, x_t, ...
5                     trucks0, H_truck, Ts, rand_in, escenario, ...
6                     x_containers_left_lim, x_containers_right_lim, ...
7                     t_camion_transicion)
8
9 persistent n_containers x_grid2 tlk_prev dx_grid idx_posiciones
10 persistent trucks_state_p trucks_has_load_p idx_trucks lambda_truck
   grupo_camion
11 persistent idx_zona_min idx_zona_max
12 persistent truck_en_transicion truck_timer
13 persistent carga_tomada_p
14
15 coder.varsize('x_grid2', [1, 10000], [false, true]);
16 coder.varsize('n_containers', [1, 10000], [false, true]);
17
18 if isempty(n_containers)
19     dx_grid = 0.01;
20     x_grid2 = x_c0_left : dx_grid : x_c0_right;
21     n_containers = zeros(1, length(x_grid2));
22     tlk_prev = false;
23
24     % Limites de la zona valida de contenedores
25     idx_zona_min = round((x_containers_left_lim - x_c0_left) / dx_grid)
26     + 1;
27     idx_zona_max = round((x_containers_right_lim - x_c0_left) / dx_grid)
28     + 1;
29     idx_zona_min = max(1, min(length(x_grid2), idx_zona_min));
30     idx_zona_max = max(1, min(length(x_grid2), idx_zona_max));
31
32     % Carga inicial: 19 posiciones de pilas estaticas
33     n_pilares = 19;
34     x_posiciones = linspace(x_containers_left_lim,
35                             x_containers_right_lim, n_pilares);
36     idx_posiciones = zeros(1, 19);
37     medio_ancho_idx = round((W_c/2) / dx_grid);
38     for k = 1:n_pilares
39         idx_pos = round((x_posiciones(k) - x_c0_left) / dx_grid) + 1;
40         idx_pos = max(1, min(length(x_grid2), idx_pos));
```

```
38     idx_posiciones(k) = idx_pos;
39     idx_ini = max([1, idx_pos - medio_ancho_idx, idx_zona_min]);
40     idx_fin = min([length(x_grid2), idx_pos + medio_ancho_idx,
41                  idx_zona_max]);
42     n_containers(idx_ini:idx_fin) = container_pillars(k);
43 end
44 % Inicializacion de camiones (5 posiciones fijas en zona muelle)
45 x_trucks_left = [-25 -20 -15 -10 -5];
46 idx_trucks = zeros(1, 5);
47 for k = 1:5
48     idx_t = round((x_trucks_left(k) - x_c0_left) / dx_grid) + 1;
49     idx_trucks(k) = max(1, min(length(x_grid2), idx_t));
50 end
51
52 trucks_state_p      = double(trucks0(:)');
53 trucks_has_load_p   = zeros(1, 5);
54 lambda_truck        = -log(0.5) / 60;
55 grupo_camion        = ones(1, 5);
56 truck_en_transicion = zeros(1, 5);
57 truck_timer         = zeros(1, 5);
58 carga_tomada_p      = false;
59 end
```

Listing 1: Firma e inicialización de la *MATLAB Function* perfil_suelo.

La grilla de perfil se construye con resolución de 1 cm ($dx = 0,01$ m) sobre el rango $[x_{c0,left}, x_{c0,right}]$. Las 19 posiciones fijas de apilado se distribuyen uniformemente dentro de la zona válida de contenedores y se inicializan con los valores provistos por el vector `container_pillars`. Cada pila ocupa una ventana de ancho W_c sobre la grilla, evitando solapamientos con otras zonas del entorno. Los 5 camiones se colocan inicialmente en posiciones negativas del eje x (zona de muelle), con sus índices de grilla precalculados. La tasa de reaparición λ_{truck} se calibra para una vida media de 60 s.

Detección de flancos de TLK y estado de enganche.

```
1 %% Indice de la carga y ventana de escritura
2 idx = round((x_l - x_c0_left) / dx_grid) + 1;
3 idx = max(1, min(length(x_grid2), idx));
4 medio_ancho_idx = round((W_c/2) / dx_grid);
5 idx_ini_l = max([1, idx - medio_ancho_idx, idx_zona_min]);
6 idx_fin_l = min([length(x_grid2), idx + medio_ancho_idx, idx_zona_max]);
7
8 %% Flancos de TLK
9 tlk_bool      = logical(TLK);
10 flanco_bajada = tlk_prev && ~tlk_bool;    % Soltar carga (1->0)
11 flanco_subida = ~tlk_prev && tlk_bool;     % Tomar carga (0->1)
12 tlk_prev      = tlk_bool;
13
14 if flanco_subida
15     carga_tomada_p = true;
16 elseif flanco_bajada
17     carga_tomada_p = false;
18 end
19 carga_tomada = carga_tomada_p;
```

Listing 2: Detección de flancos de la señal TLK y actualización de `carga_tomada`.

La posición de la carga $x_l(t)$ se mapea a un índice de grilla; la ventana de escritura queda siempre confinada dentro de la zona válida de contenedores para evitar interferencias con otras regiones del perfil. La señal TLK se monitorea flanco a flanco: el **flanco ascendente** ($0 \rightarrow 1$) indica que el spreader acaba de enganchar una carga, y el **flanco descendente** ($1 \rightarrow 0$) indica que la acaba de soltar.

Lógica de interacción con camiones y actualización del perfil.

```
1 %% Identificar si la carga esta sobre un camion
2 W_trucks_idx = round((W_c/2) / dx_grid);
3 sobre_camion_k = 0;
4 for k = 1:5
5     if abs(idx - idx_trucks(k)) <= W_trucks_idx && trucks_state_p(k) ==
6         1
7         sobre_camion_k = k;
8         break;
9     end
10 end
11 %% Asignacion de grupo segun escenario
12 if escenario ~= 0
13     switch escenario
14         case 1, grupo_camion = ones(1,5);           % Solo carga
15         case 2, grupo_camion = 2*ones(1,5);         % Solo descarga
16         case 3, grupo_camion = [2 2 1 1 1];         % Mixto
17     end
18 end
19
20 %% Transferencia de carga entre camion y spreader
21 if sobre_camion_k > 0 && escenario ~= 0
22     k = sobre_camion_k;
23     if flanco_subida && grupo_camion(k) == 1 && trucks_has_load_p(k) ==
24         1
25         trucks_has_load_p(k) = 0; % Spreader toma la carga del
26         camion
27         truck_en_transicion(k) = 1;
28         truck_timer(k) = 0;
29     elseif flanco_bajada && grupo_camion(k) == 2 && trucks_has_load_p(k)
30         == 0
31         trucks_has_load_p(k) = 1; % Spreader deposita la carga en el
32         camion
33         truck_en_transicion(k) = 1;
34         truck_timer(k) = 0;
35     end
36 else
37     % Actualizar perfil de patio/barco si no hay camion bajo la carga
38     if idx_fin_l >= idx_ini_l
39         [dist_min, pilar_cercano] = min(abs(idx - idx_posiciones));
40         if dist_min <= medio_ancho_idx
41             % Snap al pilar mas cercano
42             idx_centro = idx_posiciones(pilar_cercano);
```

```
39         idx_ini_write = max([1, idx_centro - medio_ancho_idx,
40                             idx_zona_min]);
41         idx_fin_write = min([length(x_grid2), idx_centro +
42                             medio_ancho_idx, idx_zona_max]);
43     else
44         idx_ini_write = idx_ini_l;
45         idx_fin_write = idx_fin_l;
46     end
47     if flanco_bajada
48         altura_actual = max(n_containers(idx_ini_write:idx_fin_write
49                                     ));
50         n_containers(idx_ini_write:idx_fin_write) = ...
51             max(n_containers(idx_ini_write:idx_fin_write),
52                 altura_actual + 1);
53     elseif flanco_subida
54         altura_actual = max(n_containers(idx_ini_write:idx_fin_write
55                                     ));
56         n_containers(idx_ini_write:idx_fin_write) = max(0,
57                 altura_actual - 1);
58     end
59 end
60 end
```

Listing 3: Identificación del camión bajo la carga y actualización del perfil de contenedores.

Cuando la carga queda sobre un camión activo, la transferencia es directa entre el spreader y ese camión (sin tocar el perfil de patio). Cuando la carga se deposita sobre el patio o el barco, se aplica un mecanismo de *snap*: si la posición está dentro de la tolerancia de $W_c/2$ de un pilar predefinido, el depósito se alinea exactamente con ese pilar para preservar la coherencia geométrica del apilado. Los tres escenarios configurables determinan el rol de cada camión: **caso 1** (solo carga desde muelle), **caso 2** (solo descarga hacia muelle) y **caso 3** (operación mixta).

Reaparición probabilística de camiones y cálculo de altura total.

```
1 %% Reaparicion probabilistica de camiones
2 if escenario ~= 0
3     p_ciclo = 1 - exp(-lambda_truck * Ts);
4     for k = 1:5
5         if trucks_state_p(k) == 0 && rand_in(k) < p_ciclo
6             trucks_state_p(k) = 1;
7             trucks_has_load_p(k) = (grupo_camion(k) == 1);
8             truck_en_transicion(k) = 0;
9             truck_timer(k) = 0;
10        end
11    end
12 end
13
14 %% Altura fija segun zona bajo la carga
15 if x_l < -1,
16     y_fijo = 0;
17 elseif x_l <= 0,
18     y_fijo = Y_sb;
19 else,
20     y_fijo = y_h_min;
21 end
22
23 %% Aporte de altura de camiones
```



```
21 h_truck_l = 0;
22 for k = 1:5
23     if abs(idx - idx_trucks(k)) <= W_trucks_idx && trucks_state_p(k) ==
        1
24         h_truck_l = H_truck + trucks_has_load_p(k) * H_c;
25     end
26 end
27
28 %% Altura total de contacto bajo la carga
29 if idx_fin_l >= idx_ini_l
30     altura_containers_l = max(n_containers(idx_ini_l:idx_fin_l));
31 else
32     altura_containers_l = 0;
33 end
34 y_c0_contacto = y_fijo + altura_containers_l * H_c + h_truck_l;
35
36 %% Resolucion de camiones en transicion (basada en tiempo)
37 for k = 1:5
38     if truck_en_transicion(k) == 1
39         truck_timer(k) = truck_timer(k) + Ts;
40         if truck_timer(k) >= t_camion_transicion
41             trucks_state_p(k) = 0;
42             trucks_has_load_p(k) = 0;
43             truck_en_transicion(k) = 0;
44             truck_timer(k) = 0;
45         end
46     end
47 end
```

Listing 4: Reparición de camiones, altura de contacto y resolución de transiciones.

La reaparición de cada camión sigue un proceso de Poisson discretizado: en cada paso de simulación, un camión ausente reaparece con probabilidad $p_{ciclo} = 1 - e^{-\lambda_{truck} T_s}$, donde $\lambda_{truck} = \ln(2)/60\text{s}^{-1}$ corresponde a una vida media de 60 s. La altura de contacto bajo la carga agrega: la altura de zona fija (muelle, *sill beam* o barco), la contribución de los contenedores apilados en esa celda, y la altura del camión presente (plataforma sola H_{truck} o plataforma con contenedor $H_{truck} + H_c$). Una vez que el spreader transfiere la carga a un camión (o viceversa), se activa un temporizador que, al superar $t_{camion_transicion}$, retira el camión del escenario y libera su ranura para una futura reaparición.

Lectura anticipada del LIDAR y salidas.

```
1 %% Lectura de LIDAR (posicion adelantada)
2 x_laser = x_t + x_lidar_offset;
3 idx_lidar = round((x_laser - x_c0_left) / dx_grid) + 1;
4 idx_lidar = max(1, min(length(x_grid2), idx_lidar));
5
6 if x_laser < -1,           y_fijo_lidar = 0;
7 elseif x_laser <= 0,      y_fijo_lidar = Y_sb;
8 else,                    y_fijo_lidar = y_h_min;
9 end
10
11 h_truck_lidar = 0;
12 for k = 1:5
```

```

13     if abs(idx_lidar - idx_trucks(k)) <= W_trucks_idx && trucks_state_p(
14         k) == 1
15         h_truck_lidar = H_truck + trucks_has_load_p(k) * H_c;
16     end
17 end
18 y_c0_lidar = y_fijo_lidar + n_containers(idx_lidar) * H_c +
19     h_truck_lidar;
20 %% Salidas garantizadas
21 containers_pillars_state = n_containers(idx_posiciones);
22 trucks_state             = trucks_state_p;
23 trucks_has_load          = trucks_has_load_p;

```

Listing 5: Lectura del sensor LIDAR y generación de salidas.

La lectura del LIDAR evalúa el perfil en la posición $x_t + x_{lidar_offset}$, es decir, por delante del carro (no de la carga), lo que permite al Nivel 1 anticipar obstáculos antes de que el spreader llegue a esa posición. Aplica la misma lógica de zona fija y aporte de camiones que la lectura de contacto. El vector `containers_pillars_state` extrae del perfil continuo los valores en las 19 posiciones de pilar, de modo que el supervisor tenga información compacta del estado actual del patio.

La Figura 20 muestra un ejemplo del perfil $y_{c0}(x, t)$ generado durante una secuencia de operación, junto con la lectura anticipada del LIDAR.

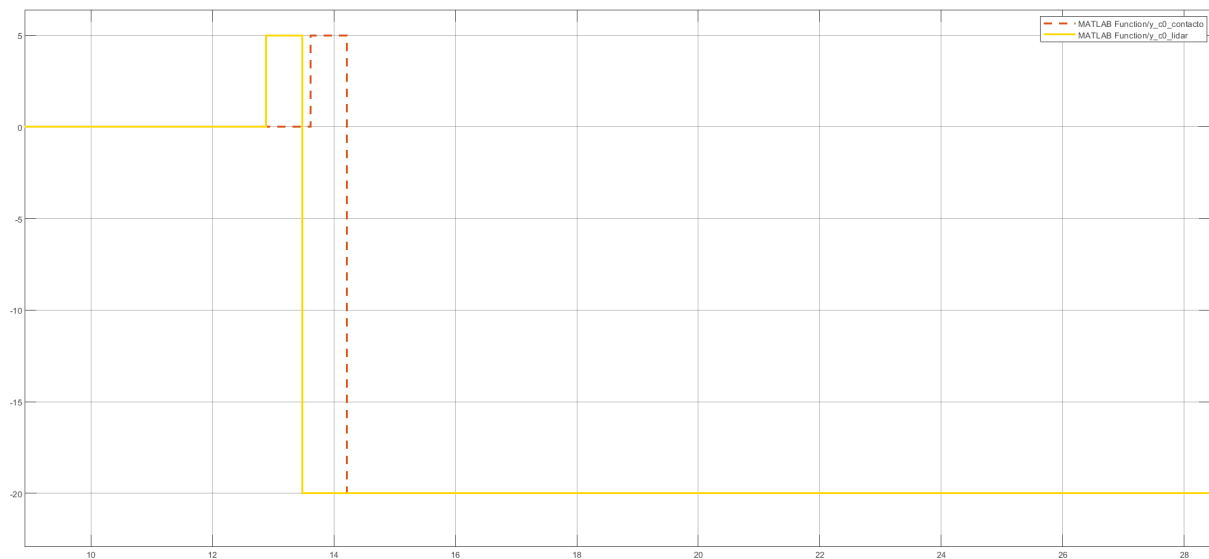


Figura 20: Perfil de obstáculos $y_{c0}(x, t)$ (línea continua) y lectura anticipada del sensor LIDAR $y_{c0,lidar}(x, t)$ (línea discontinua) durante una secuencia de carga/descarga.

2.4. Fines de carrera y detectores de sobrevelocidad

Los fines de carrera y detectores de sobrevelocidad son sensores discretos que generan señales booleanas al superarse determinados umbrales de posición o velocidad. En el modelo, estas señales actúan como entradas digitales hacia el sistema de control híbrido: los fines de carrera de operación son atendidos por el Nivel 1 (Control Supervisor) para detener el movimiento de forma coordinada, mientras que los fines de carrera de emergencia

y el detector de sobrevelocidad escalan al Nivel 0 (Seguridad) para una parada inmediata independientemente del estado del control superior.

Fines de carrera de traslación del carro. Se definen cuatro umbrales de posición $x_t(t)$ sobre el eje horizontal, simétricos para cada extremo del recorrido:

- **Límites normales de operación:** activados al alcanzar los extremos del recorrido nominal. El Nivel 1 inicia la desaceleración y detención del carro.
- **Límites de emergencia:** ubicados 0,50 m más allá de los límites normales. Su activación escala al Nivel 0, que cierra frenos y desactiva el motor inmediatamente.

Fines de carrera de izaje. Para el movimiento de izaje se utilizan fines de carrera rotativos en el tambor, equivalentes a umbrales sobre la longitud de cable $l_h(t)$, con la misma jerarquía de operación y emergencia que en el carro.

Detector de sobrevelocidad. Se modela un detector de sobrevelocidad (*overspeed switch*, OS) en el tambor de cada mecanismo (carro e izaje), que se activa cuando la velocidad angular supera el 115 % de la velocidad máxima nominal:

$$OS_i(t) = \begin{cases} 1 & |\omega_i(t)| > 1,15 \omega_{i,\text{máx}} \\ 0 & \text{en caso contrario} \end{cases} \quad i \in \{\text{carro, izaje}\} \quad (45)$$

La activación de OS_i es atendida directamente por el Nivel 0, que desactiva el motor y aplica frenos sin intervención del Nivel 1.

La Figura 21 muestra el diagrama de bloques de la implementación de estos sensores en Simulink.

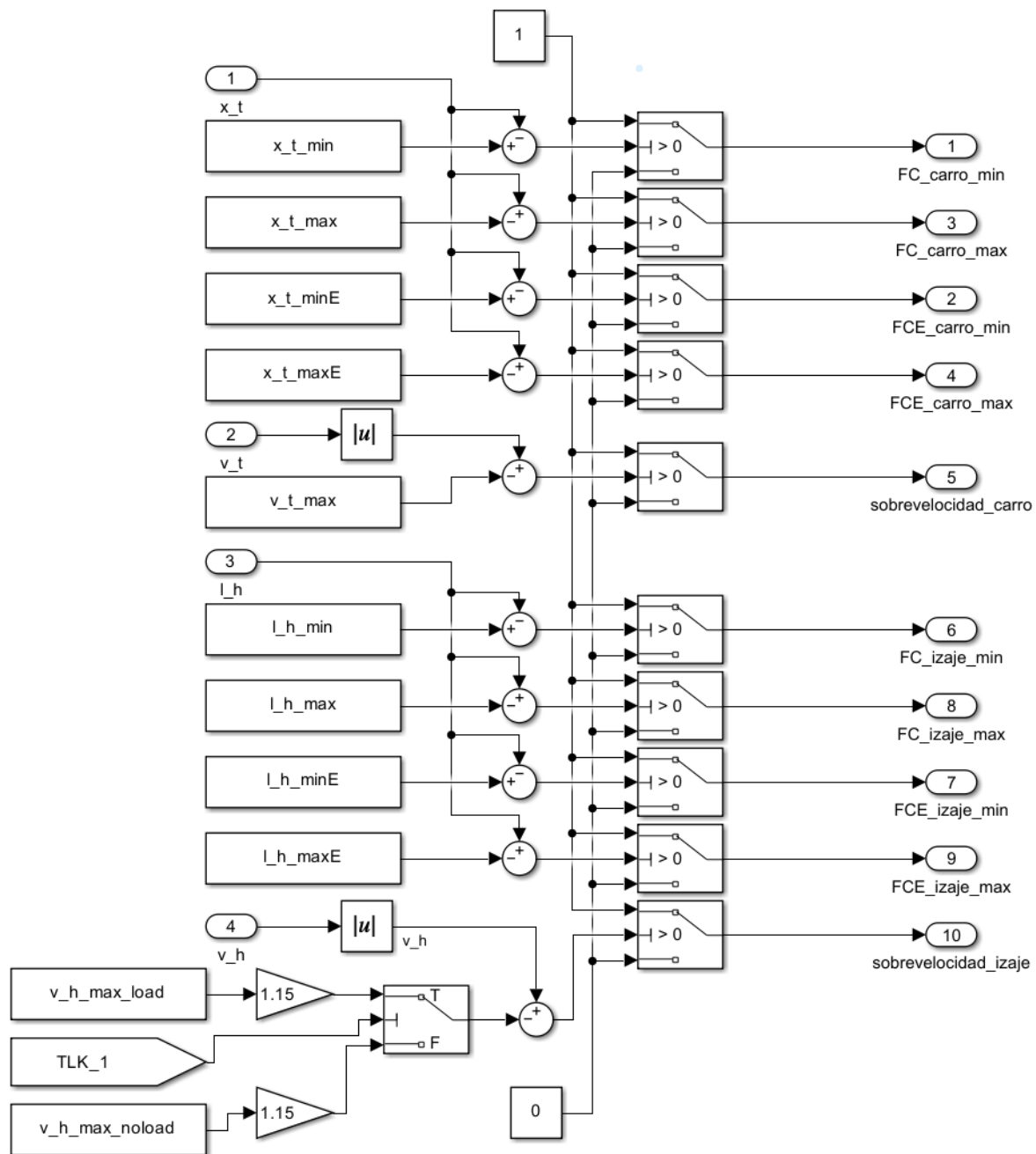


Figura 21: Diagrama de bloques de fines de carrera y detectores de sobrevelocidad (Simulink).

2.5. Moduladores de torque de los accionamientos

Los moduladores de torque son modelos simplificados de primer orden LTI en tiempo continuo que representan el lazo interno de control de torque del variador de frecuencia (*motor-drive*). Reciben una consigna de torque $T^*(k \cdot T_{s2})$ en tiempo discretizado (generada por el Nivel 2 cada T_{s2}) e incluyen un limitador de torque máximo para protección del motor.

Modulador del carro. La dinámica del modulador de torque del carro es:

$$\tau_{tm} \frac{dT_{tm}(t)}{dt} = T_{tm}^* Lim(t) - T_{tm}(t) \quad (46)$$

donde la consigna limitada, mantenida constante durante cada período de muestreo, es:

$$T_{tm}^* Lim(k T_{s2} \leq t < (k+1)T_{s2}) = \min(+T_{tm Max}, \max(-T_{tm Max}, T_{tm}^*(k T_{s2}))) \quad (47)$$

El torque máximo $T_{tm Max}$ es constante para el carro, aplicando una saturación fija $\pm T_{tm Max}$.

Modulador del izaje — con *Field Weakening*. La dinámica del modulador de torque del izaje es análoga:

$$\tau_{hm} \frac{dT_{hm}(t)}{dt} = T_{hm}^* Lim(t) - T_{hm}(t) \quad (48)$$

$$T_{hm}^* Lim(k T_{s2} \leq t < (k+1)T_{s2}) = \min(+T_{hm Max}, \max(-T_{hm Max}, T_{hm}^*(k T_{s2}))) \quad (49)$$

A diferencia del carro, el torque máximo del izaje no es constante sino que se reduce cuadráticamente con la velocidad angular cuando ésta supera la velocidad nominal del motor (*Field Weakening*):

$$T_{hm Max}(t) = T_{hm MAX} \cdot \min\left(1, \left(\frac{\omega_{hm RATED}}{\omega_{hm}(t)}\right)^2\right) \quad (50)$$

donde $T_{hm MAX}$ es el torque máximo absoluto del motor, $\omega_{hm RATED}$ la velocidad nominal y $\omega_{hm}(t)$ la velocidad instantánea. Para velocidades $|\omega_{hm}| \leq \omega_{hm RATED}$ el límite es constante e igual a $T_{hm MAX}$; por encima de la nominal, la potencia se mantiene constante reduciendo el torque máximo disponible. La Figura 22 muestra el diagrama de bloques del cálculo de $T_{hm Max}(t)$ según (50).

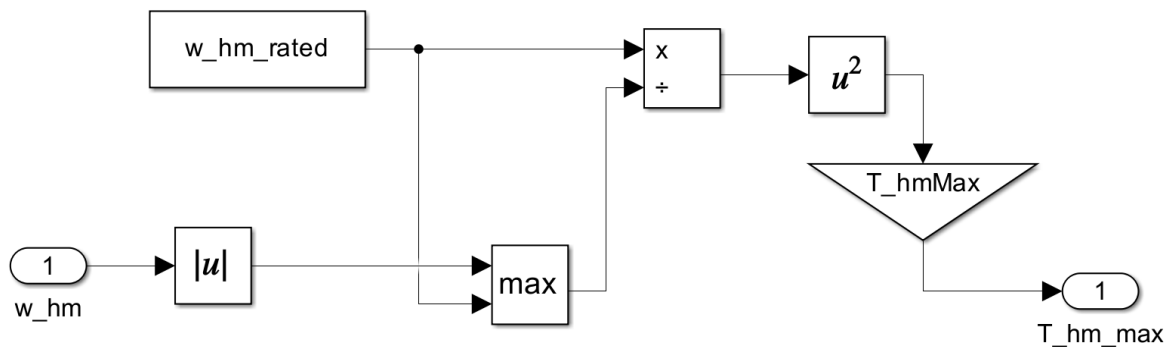


Figura 22: Diagrama de bloques del cálculo de $T_{hm Max}(t)$ con *Field Weakening* (Simulink).

La Figura 23 muestra el diagrama de bloques del subsistema completo de moduladores de torque de ambos accionamientos implementado en Simulink.

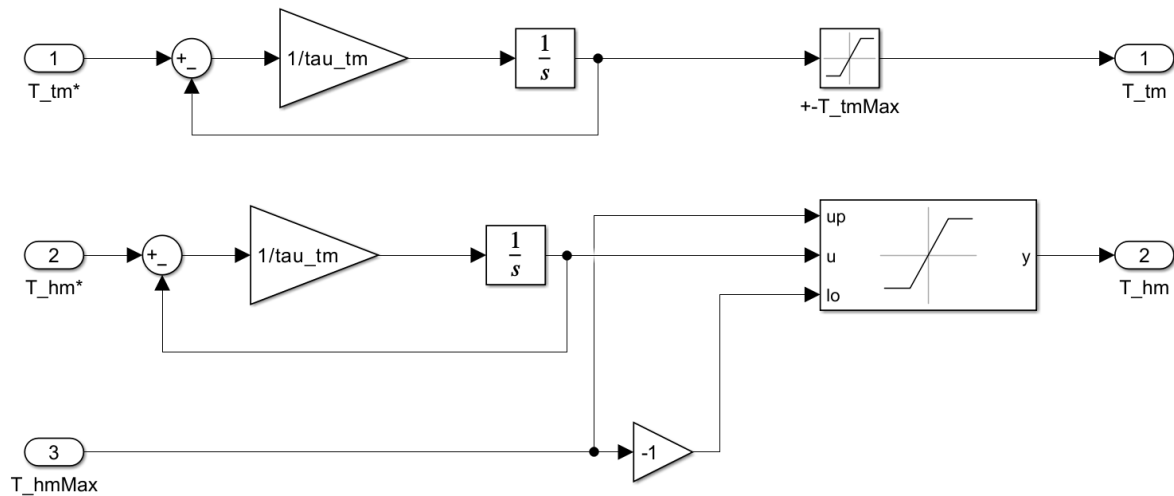


Figura 23: Diagrama de bloques del subsistema de moduladores de torque del carro y del izaje (Simulink).

2.6. Sistema físico completo

El modelo del sistema físico completo resulta del ensamble e interconexión de todos los subsistemas desarrollados en las secciones anteriores:

- **Subsistema *Trolley***: accionamiento de traslación del carro (motor, freno de operación, caja reductora y tambor), modelo de cable elástico-amortiguado y ecuación de movimiento del carro.
- **Subsistema *Hoist***: accionamiento de izaje (motor, freno de operación, freno de emergencia, caja reductora y tambor), modelo de cable de izaje elástico-amortiguado, movimiento de la carga en 2D, y cálculo de ángulo de balanceo $\theta_l(t)$ y longitud real del cable $l(t)$.
- **Perfil de obstáculos $y_{c0}(x,t)$** : generación dinámica de la superficie de apoyo a partir de la posición de la carga y el estado de los twistlocks, con lectura anticipada por LIDAR.
- **Fines de carrera y detectores de sobrevelocidad**: señales booleanas de límite de recorrido (operación y emergencia) y sobrevelocidad para carro e izaje.
- **Moduladores de torque**: modelos de primer orden de los lazos internos de control de torque de los variadores de frecuencia, con saturación fija (carro) y variable con *Field Weakening* (izaje).

Al integrar todos los subsistemas, los acoplamientos que se mostraban desconectados en los diagramas individuales quedan completamente interconectados: la fuerza del cable

de izaje $F_{hw}(t)$ actúa sobre la ecuación de movimiento del carro a través de $\theta_l(t)$, y la posición y velocidad del carro $x_t(t)$, $v_t(t)$ realimentan el cálculo del ángulo de balanceo y la longitud del cable.

Las entradas externas al sistema completo provienen exclusivamente del sistema de control: las consignas de torque $T_{tm}^*(t)$ y $T_{hm}^*(t)$ del Nivel 2, y las señales booleanas BRK_t , BRK_h , BRK_{hE} y TLK generadas por el Nivel 1.

Salidas sensadas. El sistema expone las siguientes salidas hacia el sistema de control:

- $\theta_{tm}(t)$: posición angular del motor de carro (encoder).
- $\theta_{hm}(t)$: posición angular del motor de izaje (encoder).
- $\theta_l(t)$, $\dot{\theta}_l(t)$: ángulo de balanceo y velocidad angular de la carga (sensor de ángulo).
- $F_{hw}(t)$: fuerza del cable de izaje (celda de carga).
- $y_{c0,lidar}(x, t)$: perfil de obstáculos leído por el sensor LIDAR.

Señales de fines de carrera y sobrevelocidad. El bloque *Finales de carrera* genera diez señales booleanas que se distribuyen mediante bloques **Goto** de Simulink, de modo que los niveles de control superiores las consumen mediante bloques **From** sin necesidad de conexiones explícitas en el diagrama:

- **FC_carro_min**, **FC_carro_max**: fines de carrera normales del carro (atendidos por Nivel 1).
- **FCE_carro_min**, **FCE_carro_max**: fines de carrera de emergencia del carro (atendidos por Nivel 0).
- **sobrevelocidad_carro**: detector de sobrevelocidad del carro (atendido por Nivel 0).
- **FC_izaje_min**, **FC_izaje_max**: fines de carrera normales de izaje (atendidos por Nivel 1).
- **FCE_izaje_min**, **FCE_izaje_max**: fines de carrera de emergencia de izaje (atendidos por Nivel 0).
- **sobrevelocidad_izaje_noload**: detector de sobrevelocidad de izaje sin carga (atendido por Nivel 0).

La Figura 24 muestra el diagrama de bloques del sistema físico completo ensamblado en Simulink.

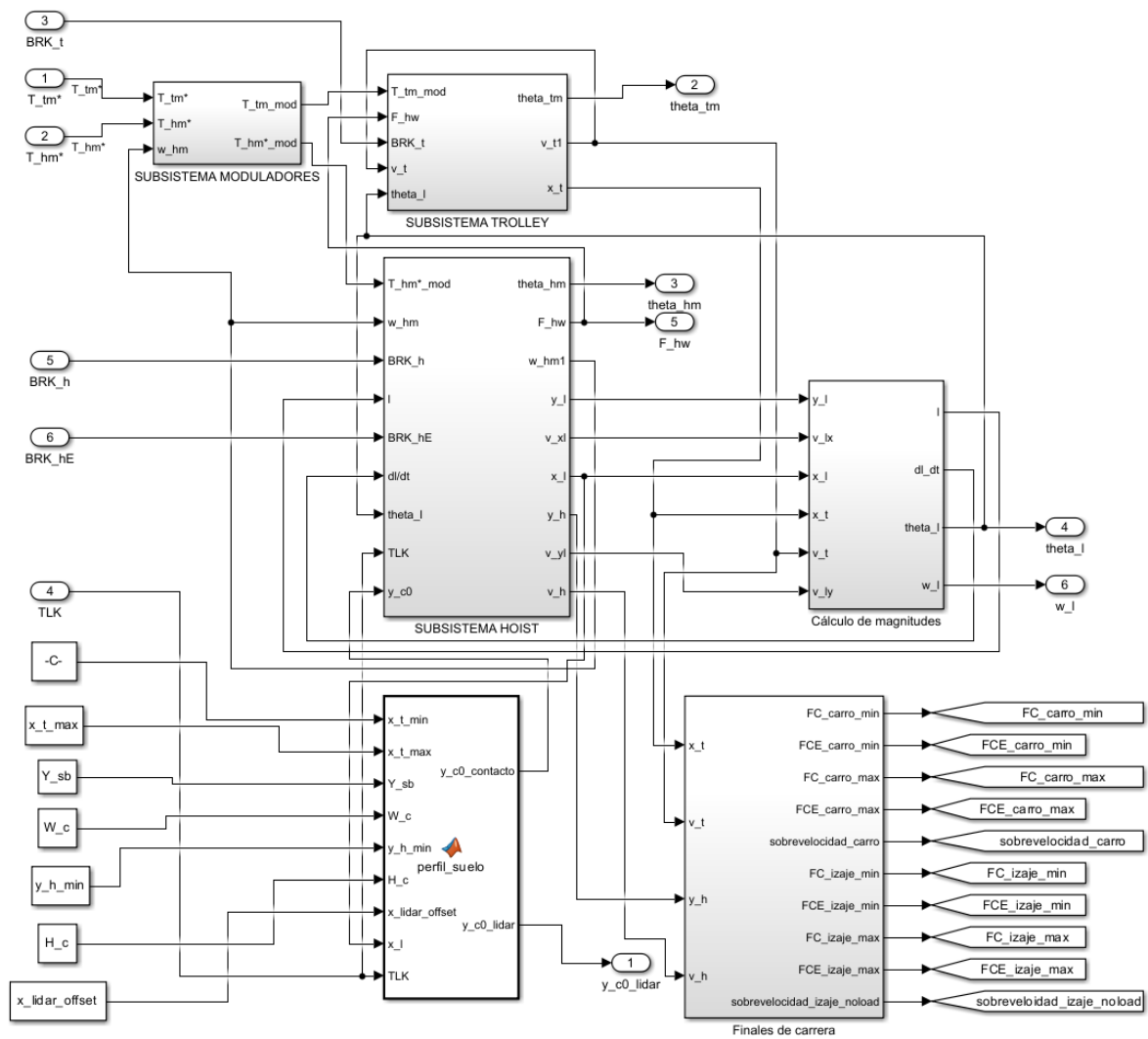


Figura 24: Diagrama de bloques del sistema físico completo (Simulink).

3. Sistema híbrido de control y protección

El sistema de control y protección diseñado para la grúa STS es un **Sistema Híbrido** que integra, en una arquitectura jerárquica de tres niveles, el control continuo de los accionamientos con la lógica discreta de supervisión y seguridad. Cada nivel opera con su propio período de muestreo y tiene responsabilidades bien delimitadas:

- **Nivel 2 – Control Regulatorio** ($T_{s2} = 1$ ms): controladores de movimiento de lazo cerrado en tiempo discretizado (CTDS) para traslación, izaje y amortiguación de balanceo; generan la consigna de torque $T_m^*(k T_{s2})$ para los moduladores de torque de cada accionamiento.
- **Nivel 1 – Control Supervisor** ($T_{s1} = 20$ ms): autómata de estados discretos activados por eventos (DEDS) que coordina las secuencias operativas, gestiona frenos, twistlocks, fines de carrera normales y genera las consignas de movimiento para el Nivel 2.
- **Nivel 0 – Seguridad y Protección** ($T_{s0} = 20$ ms): autómata *separado* del Nivel 1, en *standby* permanente, que toma el control ante fallas críticas (pulsador de emergencia, sobrevelocidad, fines de carrera últimos, *watchdog*), llevando el sistema a un estado seguro con selectividad jerárquica según el origen y criticidad de la falla.

3.1. Nivel 2: Control Regulatorio

El Nivel 2 implementa los **controladores de movimiento de lazo cerrado** para los dos accionamientos principales de la grúa: la traslación del carro y el izaje de la carga. Opera en tiempo discretizado con período de muestreo $T_{s2} = 1$ ms, recibiendo consignas de posición y velocidad del Nivel 1 y entregando consignas de torque $T_{tm}^*(k T_{s2})$ y $T_{hm}^*(k T_{s2})$ a los moduladores de torque de cada accionamiento.

Cada controlador de movimiento se diseña a partir de la representación en espacio de estados del subsistema correspondiente, identificando las variables de estado accesibles mediante encoder y las que deben estimarse mediante un **observador de estados**. Adicionalmente, el controlador de traslación incorpora en paralelo un lazo de **control automático de amortiguación de balanceo** (*sway control*) con ganancias programadas (*gain scheduling*) en función de la longitud de péndulo equivalente $l(t)$, dado que en la operación dinámica de la grúa STS no es válida la aproximación lineal simple $\sin \theta_l \approx \theta_l$.

3.1.1. Controlador de traslación del carro

Análisis de controlabilidad y observabilidad. A partir de las ecuaciones de movimiento del accionamiento (5), del cable elástico (9) y del carro (11) desarrolladas en la Sección 2.6, el subsistema de traslación puede representarse en forma de espacio de estados lineal con vector de estado $\mathbf{x}_t(t) = [\theta_{tm}(t), \omega_{tm}(t), x_t(t), v_t(t)]^\top$, donde $\theta_{tm}(t)$ y $\omega_{tm}(t)$ son la posición y velocidad angulares del eje rápido (motor), y $x_t(t)$, $v_t(t)$ la posición y velocidad del carro. Definiendo los parámetros equivalentes referidos al eje del motor:

$$J_{eq,t} = J_{tm+tb} + \frac{J_{td}}{i_t^2}, \quad b_{eq,t} = b_{tm} + \frac{b_{td}}{i_t^2}, \quad \rho_t = \frac{r_{td}}{i_t} \quad (51)$$

las matrices del sistema resultan:

$$A_t = \begin{bmatrix} 0 & 1 & 0 & 0 \\ -\frac{\rho_t^2 K_{tw}}{J_{eq,t}} & -\frac{b_{eq,t} + \rho_t^2 b_{tw}}{J_{eq,t}} & \frac{\rho_t K_{tw}}{J_{eq,t}} & \frac{\rho_t b_{tw}}{J_{eq,t}} \\ 0 & 0 & 0 & 1 \\ \frac{\rho_t K_{tw}}{M_t} & \frac{\rho_t b_{tw}}{M_t} & -\frac{K_{tw}}{M_t} & -\frac{b_{tw} + b_t}{M_t} \end{bmatrix}, \quad B_t = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}, \quad C_t = [1 \ 0 \ 0 \ 0] \quad (52)$$

La entrada de control es el torque del motor $T_{tm}(t)$ y la salida medida es la posición angular del encoder $\theta_{tm}(t)$. Para que sea posible diseñar un controlador de movimiento y un observador que estime los estados no medidos ($\omega_{tm}(t)$, $x_t(t)$, $v_t(t)$), es necesario verificar las propiedades de **controlabilidad** y **observabilidad** del par (A_t, B_t, C_t) .

La **matriz de controlabilidad** $\mathcal{W}_c$ y la **matriz de observabilidad** $\mathcal{W}_o$ de un sistema de orden $n = 4$ son:

$$\mathcal{W}_c = [B_t \ A_t B_t \ A_t^2 B_t \ A_t^3 B_t] \in \mathbb{R}^{4 \times 4} \quad (53)$$

$$\mathcal{W}_o = \begin{bmatrix} C_t \\ C_t A_t \\ C_t A_t^2 \\ C_t A_t^3 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (54)$$

Dado que el sistema es de orden $n = 4$, el cálculo numérico de ambas matrices y la verificación de sus rangos se realizan en el script `Cy0_carro.m`, obteniendo:

$$\text{rango}(\mathcal{W}_c) = 4 = n \quad \Rightarrow \quad \text{sistema } \mathbf{completamente controlable} \text{ desde } T_{tm}(t) \quad (55)$$

$$\text{rango}(\mathcal{W}_o) = 4 = n \quad \Rightarrow \quad \text{sistema } \mathbf{completamente observable} \text{ desde } \theta_{tm}(t) \quad (56)$$

Ambas condiciones se satisfacen, lo que garantiza que es posible diseñar un controlador de movimiento con polos de lazo cerrado en posiciones arbitrarias y un observador de estados (estimador de Luenberger) que reconstruye los estados no medidos a partir de la única señal sensada $\theta_{tm}(t)$.

Diseño del observador de estados del carro.

El controlador de traslación requiere los cuatro estados $\mathbf{x}_t = [\theta_{tm}, \omega_{tm}, x_t, v_t]^\top$, de los cuales solo θ_{tm} es sensado directamente. Para estimar los estados no medidos se diseña un **observador de Luenberger** de orden pleno que replica la dinámica del sistema (52) corrigiendo la estimación mediante la diferencia entre la salida medida y la estimada, ponderada por la matriz de ganancias $\mathbf{L}_t \in \mathbb{R}^{4 \times 1}$:

$$\dot{\hat{\mathbf{x}}}_t(t) = A_t \hat{\mathbf{x}}_t(t) + B_{ct} \mathbf{u}_t(t) + \mathbf{L}_t [\theta_{tm}(t) - C_t \hat{\mathbf{x}}_t(t)] \quad (57)$$

Definiendo el error $\mathbf{e}_t = \mathbf{x}_t - \hat{\mathbf{x}}_t$, su dinámica es:

$$\dot{\mathbf{e}}_t(t) = (\mathbf{A}_t - \mathbf{L}_t \mathbf{C}_t) \mathbf{e}_t(t) \quad (58)$$

Los polos del observador se ubican con un **factor de separación de 10** respecto a la frecuencia dominante del controlador PID ω_t , con ligera dispersión entre los cuatro polos para garantizar la operación numérica de place:

$$p_{obs,i} = -10 \omega_t \alpha_i, \quad \alpha_i \in \{1,00, 1,02, 1,05, 1,07\} \quad (59)$$

La matriz $\mathbf{L}_t$ se obtiene por dualidad:

$$\mathbf{L}_t = \text{place}(\mathbf{A}_t^\top, \mathbf{C}_t^\top, \{p_{obs,i}\})^\top \quad (60)$$

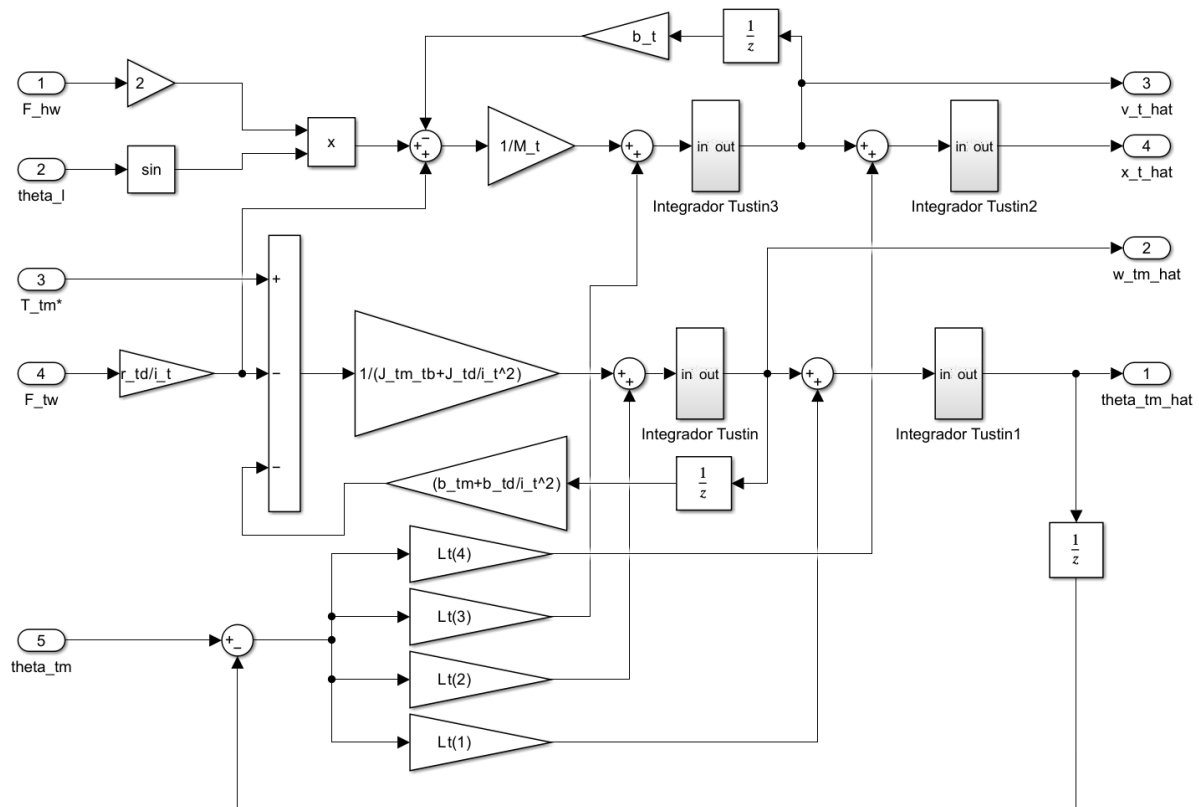


Figura 25: Observador de estados del carro implementado por bloques en Simulink ($T_{s2} = 1 \text{ ms}$), con las ganancias de retroalimentación $\mathbf{L}_t$.

En el diagrama se aprecian las ganancias del observador $\mathbf{L}_t$ y los retardos de un paso de muestreo en las señales retroalimentadas para evitar lazos algebraicos.

Polos a lazo abierto del carro. Para el diseño del controlador se parte de las ecuaciones de movimiento del carro (10) y del accionamiento (5), junto con la restricción cinemática que vincula la velocidad lineal del carro con la velocidad angular del eje rápido:

$$v_t(t) = \frac{r_{td}}{i_t} \omega_{tm}(t) \Rightarrow \omega_{tm}(t) = \frac{i_t}{r_{td}} v_t(t), \quad \frac{d\omega_{tm}(t)}{dt} = \frac{i_t}{r_{td}} \frac{dv_t(t)}{dt} \quad (61)$$

Despejando $F_{tw}(t)$ de (10):

$$F_{tw}(t) = M_t \frac{dv_t(t)}{dt} + b_t v_t(t) - 2 F_{hw}(t, l(t) > l_h(t)) \sin(\theta_l(t)) \quad (62)$$

y sustituyendo (61) y (62) en (5):

$$\begin{aligned} \left(J_{tm+tb} + \frac{J_{td}}{i_t^2} \right) \frac{i_t}{r_{td}} \frac{dv_t(t)}{dt} = T_{tm}(t) + T_{tb}(t, BRK_t) - \left(b_{tm} + \frac{b_{td}}{i_t^2} \right) \frac{i_t}{r_{td}} v_t(t) \\ - \frac{r_{td}}{i_t} \left[M_t \frac{dv_t(t)}{dt} + b_t v_t(t) - 2 F_{hw}(t) \sin(\theta_l(t)) \right] \end{aligned} \quad (63)$$

Agrupando los términos de inercia y amortiguamiento y definiendo los parámetros equivalentes que incluyen la masa de la carga suspendida:

$$M_{t,eq} = \frac{i_t}{r_{td}} \left(J_{tm+tb} + \frac{J_{td}}{i_t^2} \right) + \frac{r_{td}}{i_t} (M_t + m_l(TLK)) \quad [\text{kg m}], \quad b_{t,eq} = \frac{i_t}{r_{td}} \left(b_{tm} + \frac{b_{td}}{i_t^2} \right) + \frac{r_{td}}{i_t} b_t \quad (64)$$

la ecuación de movimiento equivalente resulta:

$$M_{t,eq} \frac{dv_t(t)}{dt} = T_{tm}(t) + T_{tb}(t, BRK_t) - b_{t,eq} v_t(t) + \frac{2r_{td}}{i_t} F_{hw}(t) \sin(\theta_l(t)) \quad (65)$$

Considerando nulas las perturbaciones ($F_{hw} \sin \theta_l = 0$) y el freno de operación ($T_{tb} = 0$, con $BRK_t = \text{On}$), y aplicando la Transformada de Laplace, se obtienen las funciones de transferencia torque $\rightarrow$ velocidad y torque $\rightarrow$ posición:

$$\frac{V_t(s)}{T_{tm}(s)} = \frac{1}{M_{t,eq} s + b_{t,eq}} \quad (66)$$

$$\frac{X_t(s)}{T_{tm}(s)} = \frac{1}{s(M_{t,eq} s + b_{t,eq})} \quad (67)$$

Los polos a lazo abierto de (67) son:

$$p_1 = 0 \frac{\text{rad}}{\text{s}}, \quad p_2 = -\frac{b_{t,eq}}{M_{t,eq}} \approx -0,1736 \frac{\text{rad}}{\text{s}} \quad (68)$$

Diseño del controlador de posición del carro. Considerando freno y perturbación de izaje nulos, de (65) se obtiene:

$$M_{t,eq} s^2 X_t(s) + b_{t,eq} s X_t(s) = T_{tm}(s) \quad (69)$$

Se propone un controlador PID de posición que genera la consigna de torque $T_{tm}^*(s)$ a partir del error $E(s) = X_t^*(s) - X_t(s)$:

$$T_{tm}^*(s) = \left(s b_{ta} + k_{tsa} + \frac{k_{tsia}}{s} \right) E(s) \quad (70)$$

Igualando $T_{tm}^*(s) = T_{tm}(s)$ y agrupando:

$$X_t(s) \left[s^3 M_{t,eq} + s^2 (b_{t,eq} + b_{ta}) + s k_{tsa} + k_{tsia} \right] = X_t^*(s) \left(s^2 b_{ta} + s k_{tsa} + k_{tsia} \right) \quad (71)$$

de donde la función de transferencia de lazo cerrado es:

$$G_t(s) = \frac{X_t(s)}{X_t^*(s)} = \frac{s^2 b_{ta} + s k_{tsa} + k_{tsia}}{s^3 M_{t,eq} + s^2 (b_{t,eq} + b_{ta}) + s k_{tsa} + k_{tsia}} \quad (72)$$

Las ganancias se sintonizan por el **método de ubicación en serie**, igualando el denominador de (72) al polinomio característico deseado de tercer orden $(s + \omega_d)(s + \eta \omega_d)(s + \eta^2 \omega_d)$, con frecuencia deseada ω_d y factor de espaciado η . Expandiendo y normalizando:

$$s^3 + s^2 \frac{b_{t,eq} + b_{ta}}{M_{t,eq}} + s \frac{k_{tsa}}{M_{t,eq}} + \frac{k_{tsia}}{M_{t,eq}} = s^3 + \omega_d(1 + \eta + \eta^2) s^2 + \omega_d^2 \eta(1 + \eta + \eta^2) s + \omega_d^3 \eta^3 \quad (73)$$

Identificando coeficiente a coeficiente se obtienen las expresiones de las ganancias del controlador:

$$b_{ta} = M_{t,eq} \omega_d (1 + \eta + \eta^2) - b_{t,eq} \quad (74)$$

$$k_{tsa} = M_{t,eq} \omega_d^2 \eta (1 + \eta + \eta^2) \quad (75)$$

$$k_{tsia} = M_{t,eq} \omega_d^3 \eta^3 \quad (76)$$

Lazo externo de posición — generación de consigna de velocidad. La consigna de posición $x_{t,ref}(t)$ proviene del planificador de trayectorias del Nivel 1 (Control Supervisor). Sobre ella opera un lazo proporcional externo cuya salida es la consigna de velocidad $v_{t,ref}(t) = \text{sat}_{v_{max}}(K_p e_x(t))$, con $e_x(t) = x_{t,ref}(t) - \hat{x}_t(t)$ y $v_{max} = 4,0 \text{ m/s}$, que alimenta al PID interno de torque.

Asumiendo que el lazo interno de velocidad es suficientemente rápido como para aproximarse a ganancia unitaria ($v_t \approx v_{t,ref}$), la planta vista por este controlador externo es un integrador puro ($\dot{x}_t = v_t$, es decir $P(s) = 1/s$). La función de transferencia de lazo cerrado resulta entonces de primer orden:

$$\frac{X_t(s)}{X_{t,ref}(s)} = \frac{K_p/s}{1 + K_p/s} = \frac{K_p}{s + K_p} \quad (77)$$

con constante de tiempo $\tau = 1/K_p$. De aquí se desprende que K_p tiene unidades de s^{-1} y fija la rapidez de convergencia del lazo de posición.

La ganancia $K_p = 0,11 \text{ s}^{-1}$ ($\tau \approx 9,1 \text{ s}$) se determinó de forma empírica: se partió de un valor inicial $K_p = 1$ y se aplicó una reducción por *bisección sucesiva* —dividiendo a la mitad en cada iteración— hasta que la respuesta dejó de presentar oscilaciones sostenidas alrededor de la referencia; a partir de ese punto se realizó un ajuste fino manual que llevó al valor definitivo, el cual ofrece un seguimiento sin sobrepaso apreciable dentro del rango de operación normal del carro.

```

1 function v_t_ref = controlador_posicion(x_t_ref, x_hat, v_t_ff, v_td_hat
  , v_max)
2
3     Kp = 0.11;
4     Kd = 0;
5
6     e_x = x_t_ref - x_hat;
7
8     v_calculada = Kp * e_x + v_t_ff * Kd;
9
10    if v_calculada > v_max
11        v_t_ref = v_max;
12    elseif v_calculada < -v_max
13        v_t_ref = -v_max;
14    else
15        v_t_ref = v_calculada;
16    end
17 end

```

Listing 6: Lazo externo de posición del carro: generación de consigna de velocidad.

Implementación discreta en Simulink. El controlador y el observador de estados se implementan en Simulink mediante bloques independientes, operando en tiempo discretizado con período de muestreo $T_{s2} = 1$ ms. La discretización de los integradores se realiza mediante el **método de Tustin** (transformación bilineal), que preserva la estabilidad y la respuesta en frecuencia del equivalente continuo. El integrador de Tustin se encapsula como un **subsistema reutilizable** que se emplea de forma consistente a lo largo de todo el proyecto, tanto en los controladores como en los observadores de carro e izaje.

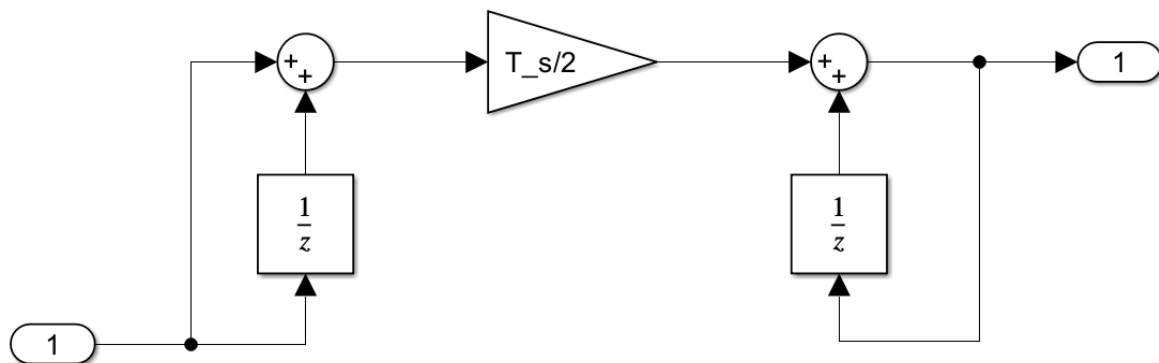


Figura 26: Subsistema integrador discreto por método de Tustin, reutilizado en todos los controladores y observadores del proyecto.

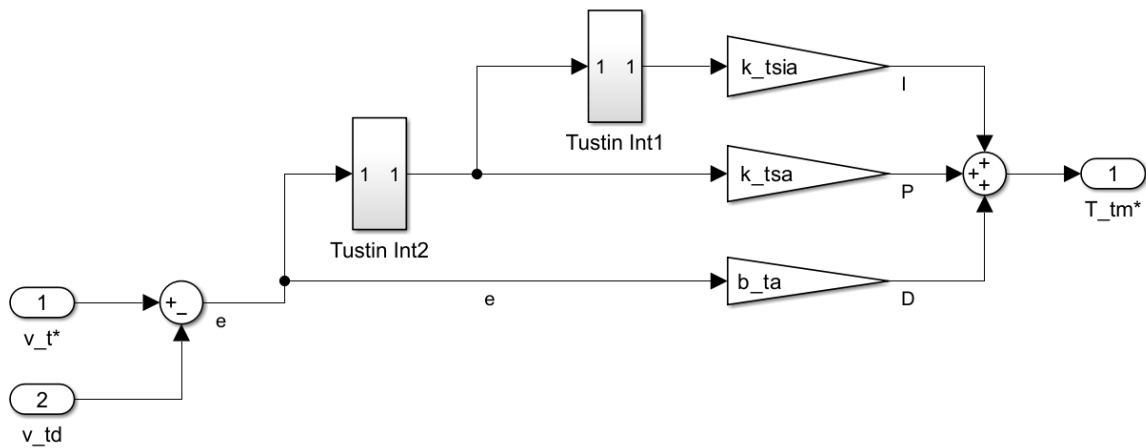


Figura 27: Diagrama de bloques del controlador de traslación del carro discretizado (Simulink, $T_{s2} = 1$ ms).

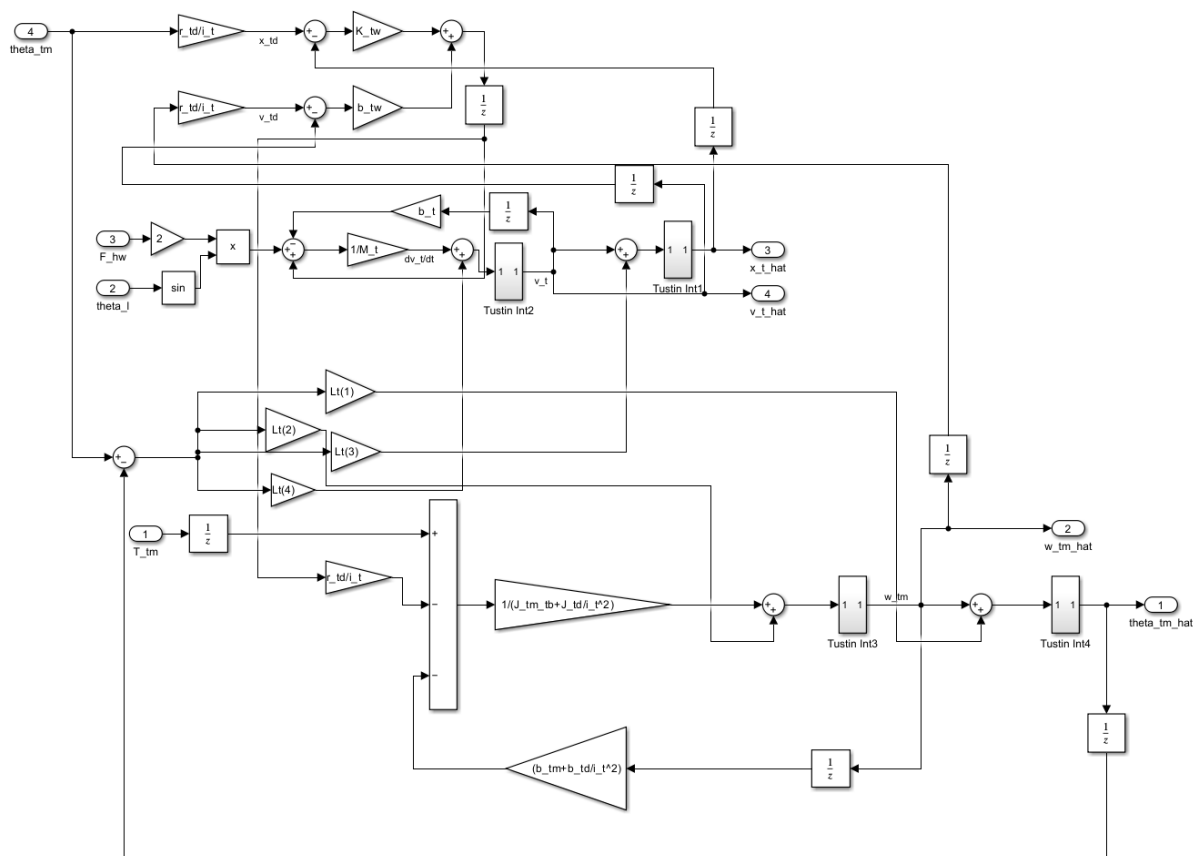


Figura 28: Diagrama de bloques del observador de estados del carro discretizado (Simulink, $T_{s2} = 1$ ms).

Desempeño del controlador de traslación. Para evaluar el desempeño se simula el escenario con **spreader vacío** ($m_l(TLK = \text{Off}) = M_s$) y longitud de péndulo intermedia $l_h = 30$ m, con un **perfil trapezoidal de velocidad suavizado**: el carro parte desde $x_t = -25$ m (lado muelle) y se desplaza hasta $x_t = +45$ m (lado barco), recorriendo 70 m dentro de los límites normales de traslación. La velocidad de consigna asciende en rampa con $a_{t,max} = \pm 0,80$ m/s² hasta $v_{t,max} = \pm 4,0$ m/s, se mantiene constante y desacelera simétricamente. El controlador paralelo de balanceo de carga se mantiene desconectado para aislar el comportamiento del sistema de traslación. Se presentan tres gráficas: comparativa de velocidad (consigna, estimada por el observador y simulada), comparativa de posición (consigna, estimada y simulada), y comparativa de torque entre consigna $T_{tm}^*(t)$ y salida del modulador $T_{tm}(t)$.

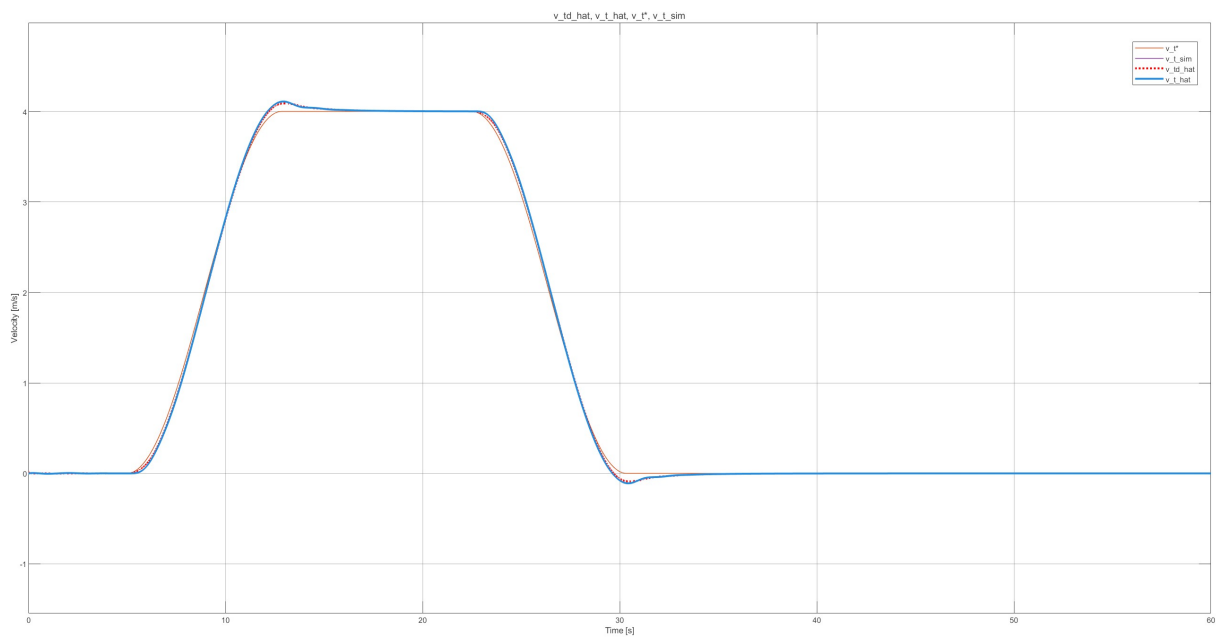


Figura 29: Traslación del carro — velocidad: consigna $v_t^*(t)$, estimada $\hat{v}_t(t)$, velocidad en el tambor $\hat{v}_{td}(t)$ y simulada $v_t(t)$ (spreader vacío, perfil trapezoidal).

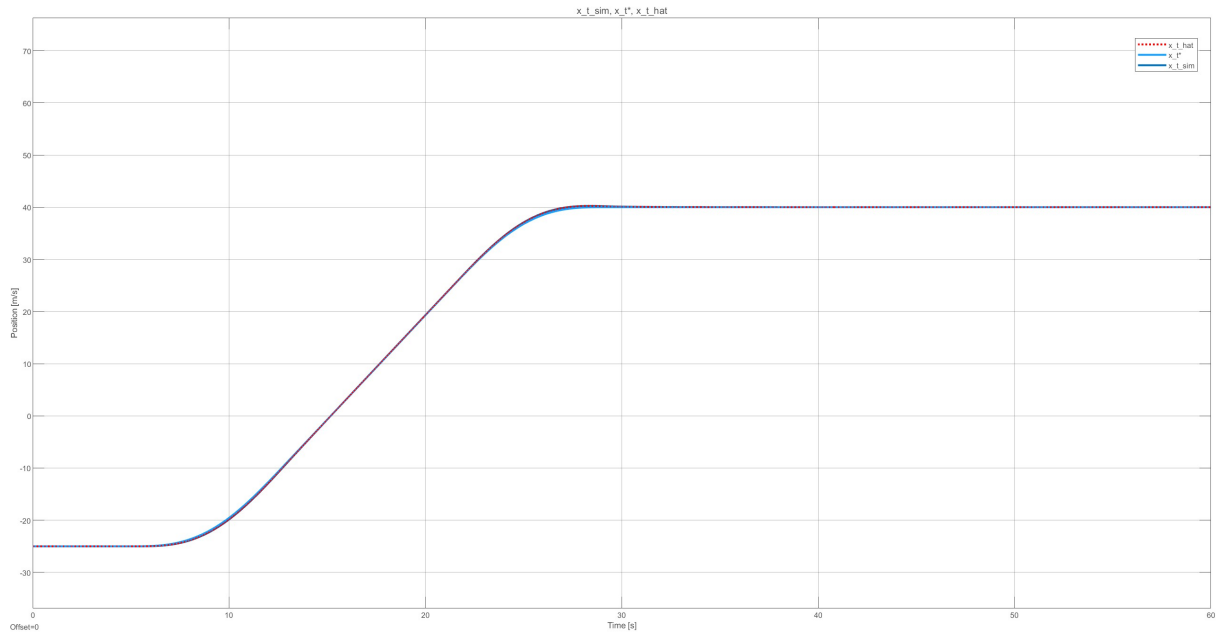


Figura 30: Traslación del carro — posición: consigna $x_t^*(t)$, estimada $\hat{x}_t(t)$ y simulada $x_t(t)$ (carga máxima, perfil trapezoidal).

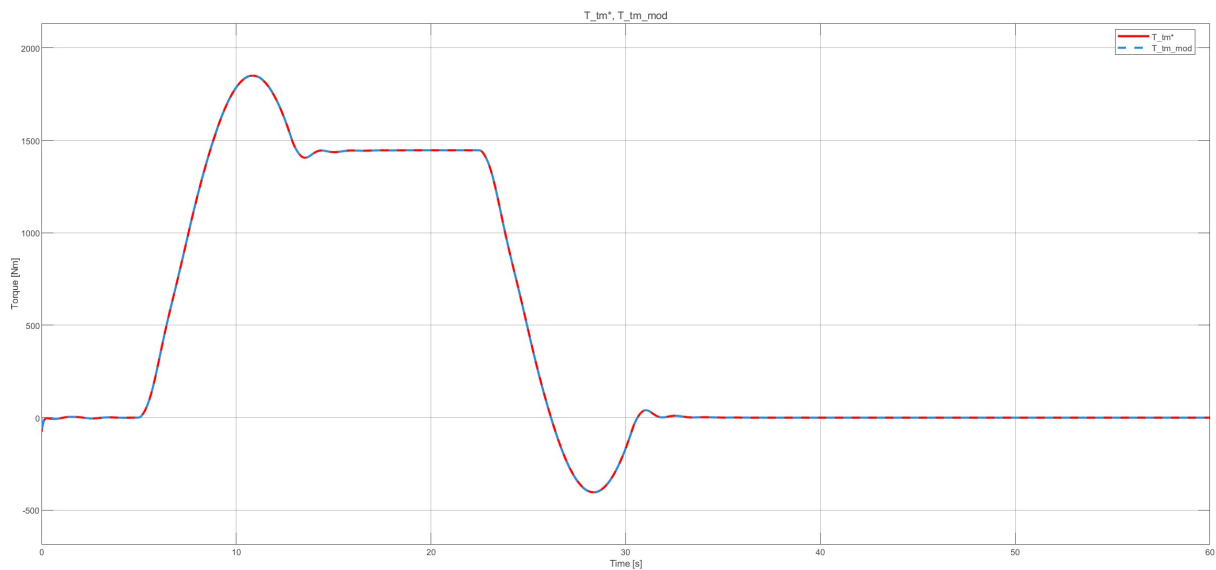


Figura 31: Traslación del carro — comparativa de torque: consigna $T_{tm}^*(t)$ y salida del modulador $T_{tm}(t)$ (spreader vacío, perfil trapezoidal).

El elevado valor de torque durante la fase de velocidad constante se debe principalmente a la necesidad de vencer la fricción viscosa del motor referida al cable. A velocidad máxima $v_{t,max} = 4 \text{ m/s}$, dicha contribución vale:

$$T_{fric} = v_t \frac{i_t}{r_{td}} b_{tm} = 4 \cdot \frac{30}{0,50} \cdot 6 = 1440 \text{ Nm} \quad (78)$$

3.1.2. Controlador de izaje de la carga

Análisis de controlabilidad y observabilidad. El subsistema de izaje se representa en espacio de estados con vector de estado $\mathbf{x}_h(t) = [\theta_{hm}(t), \omega_{hm}(t)]^\top$, donde $\theta_{hm}(t)$ y $\omega_{hm}(t)$ son la posición y velocidad angulares del eje rápido (motor de izaje). Definiendo los parámetros equivalentes referidos al eje del motor:

$$J_{eq,h} = J_{hm+hb} + \frac{J_{hd+hb} b_{hd}}{i_h^2}, \quad b_{eq,h} = b_{hm} + \frac{b_{hd}}{i_h^2} \quad (79)$$

las matrices del sistema resultan:

$$A_h = \begin{bmatrix} 0 & 1 \\ 0 & -\frac{b_{eq,h}}{J_{eq,h}} \end{bmatrix}, \quad B_{ch} = \begin{bmatrix} 0 & 0 & 0 \\ \frac{1}{J_{eq,h}} & \frac{1}{J_{eq,h}} & \frac{1}{i_h J_{eq,h}} \end{bmatrix}, \quad B_{dh} = \begin{bmatrix} 0 \\ -\frac{r_{hd}}{i_h J_{eq,h}} \end{bmatrix}, \quad C_h = \begin{bmatrix} 1 & 0 \end{bmatrix} \quad (80)$$

Las entradas de control son $T_{hm}(t)$, $T_{hb}(t, BRK_h)$ y $T_{hEb}(t, BRK_{hE})$; la perturbación es $F_{hw}(t)$ y la salida medida es $\theta_{hm}(t)$ (encoder). Para un sistema de orden $n = 2$, las matrices de controlabilidad y observabilidad son:

$$\mathcal{W}_{c,h} = [B_{ch}(:, 1) \quad A_h B_{ch}(:, 1)] \in \mathbb{R}^{2 \times 2} \quad (81)$$

$$\mathcal{W}_{o,h} = \begin{bmatrix} C_h \\ C_h A_h \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{2 \times 2} \quad (82)$$

La observabilidad puede verificarse analíticamente: $\mathcal{W}_{o,h}$ es la identidad, con $\text{rango}(\mathcal{W}_{o,h}) = 2 = n$. El cálculo numérico de ambas matrices y la verificación de sus rangos se realiza en `Cy0_izaje.m`, obteniendo:

$$\text{rango}(\mathcal{W}_{c,h}) = 2 = n \Rightarrow \text{sistema } \mathbf{completamente controlable} \text{ desde } T_{hm}(t) \quad (83)$$

$$\text{rango}(\mathcal{W}_{o,h}) = 2 = n \Rightarrow \text{sistema } \mathbf{completamente observable} \text{ desde } \theta_{hm}(t) \quad (84)$$

Ambas condiciones garantizan el diseño de un controlador de movimiento y de un observador de Luenberger que reconstruye $\omega_{hm}(t)$ a partir de la señal sensada $\theta_{hm}(t)$.

Diseño del observador de estados del izaje.

La estructura es análoga a la del carro (sección 3.1.1): un observador de Luenberger de orden pleno con matriz de ganancias $\mathbf{L}_h \in \mathbb{R}^{2 \times 1}$:

$$\dot{\hat{\mathbf{x}}}_h(t) = A_h \hat{\mathbf{x}}_h(t) + B_{ch} T_{hm}(t) + \mathbf{L}_h [\theta_{hm}(t) - C_h \hat{\mathbf{x}}_h(t)] \quad (85)$$

A diferencia del carro, los polos se ubican con factor 10 respecto al **polo a lazo abierto no nulo** $p_{LA} = -b_{eq,h}/J_{eq,h}$, separados un 10 % entre sí:

$$p_{base} = -10 |p_{LA}| \quad (86)$$

$$p_{obs,1} = p_{base}, \quad p_{obs,2} = 1,10 p_{base} \quad (87)$$

$$\mathbf{L}_h = \text{place}(A_h^\top, C_h^\top, \{p_{obs,1}, p_{obs,2}\})^\top \quad (88)$$

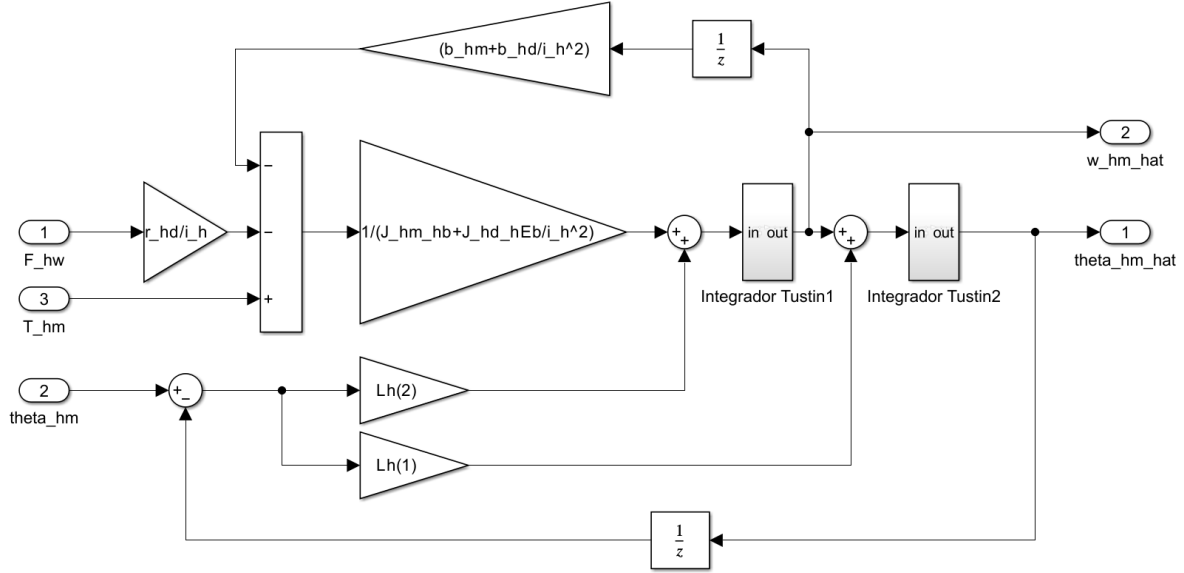


Figura 32: Observador de estados del izaje implementado por bloques en Simulink ($T_{s2} = 1 \text{ ms}$), con las ganancias de retroalimentación $\mathbf{L}_h$.

En el diagrama se aprecian las ganancias $\mathbf{L}_h$ y los retardos de un paso de muestreo en las señales retroalimentadas para evitar lazos algebraicos.

Polos a lazo abierto del izaje. El punto de partida es la ecuación de movimiento del accionamiento (24). La variable de control elegida es la longitud de cable en relajación $l_h(t)$. A partir de la relación de transmisión y del factor de multiplicación del despliegue abierto $2v_h(t) = r_{hd}\omega_{hd}(t)$ de (13):

$$\omega_{hm}(t) = -\frac{2i_h}{r_{hd}}\dot{l}_h(t), \quad \frac{d\omega_{hm}(t)}{dt} = -\frac{2i_h}{r_{hd}}\ddot{l}_h(t) \quad (89)$$

Sustituyendo (89) en (24) y agrupando:

$$-\frac{2i_h}{r_{hd}}J_{eq,h}\ddot{l}_h(t) = T_{hm}(t) + T_{hb}(t, BRK_h) + \frac{T_{hEb}(t, BRK_{hE})}{i_h} + \frac{2i_h}{r_{hd}}b_{eq,h}\dot{l}_h(t) - \frac{r_{hd}}{i_h}F_{hw}(t, l(t) > l_h(t)) \quad (90)$$

Despejando F_{hw} de la ecuación de movimiento de la carga (38), asumiendo $\theta_l(t) \approx 0$ y sin contacto ($F_{cy} = 0$), y despreciando la elasticidad del cable ($v_{ly}(t) \approx -\dot{l}_h(t)$):

$$F_{hw}(t, l(t) > l_h(t)) = \frac{m_l(TLK)}{2}(-\ddot{l}_h(t) + g) \quad (91)$$

El término de gravedad se compensa mediante *feedforward*, por lo que se omite en el diseño del controlador. Sustituyendo (91) en (90) y definiendo los parámetros equivalentes referidos al cable:

$$J_{h,eq} = \frac{2i_h}{r_{hd}} J_{eq,h}, \quad b_{h,eq} = \frac{2i_h}{r_{hd}} b_{eq,h} \quad (92)$$

la ecuación de movimiento equivalente resulta:

$$-\left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK)\right) \ddot{l}_h(t) = T_{hm}(t) + T_{hb}(t, BRK_h) + \frac{T_{hEb}(t, BRK_{hE})}{i_h} + b_{h,eq} \dot{l}_h(t) \quad (93)$$

Considerando nulos los frenos y aplicando la Transformada de Laplace, la función de transferencia torque $\rightarrow$ longitud de cable es:

$$\frac{L_h(s)}{T_{hm}(s)} = \frac{-1}{s \left[\left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK) \right) s + b_{h,eq} \right]} \quad (94)$$

El signo negativo refleja la convención de dirección: torque positivo en el motor produce descenso de carga (aumento de l_h). Los polos a lazo abierto de (94) son:

$$p_1 = 0 \frac{\text{rad}}{\text{s}}, \quad p_2 = \frac{-b_{h,eq}}{J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK)} \quad (95)$$

Diseño del controlador de longitud de cable de izaje. Se propone un controlador PID cuya variable de error es $E_h(s) = L_h^*(s) - L_h(s)$, con consigna de torque:

$$T_{hm}^*(s) = \left(s b_{ha} + k_{hsa} + \frac{k_{hsia}}{s} \right) E_h(s) \quad (96)$$

Igualando $T_{hm}^*(s) = T_{hm}(s)$, multiplicando por s y agrupando:

$$L_h(s) \left[s^3 \left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK) \right) + s^2(b_{h,eq} + b_{ha}) + s k_{hsa} + k_{hsia} \right] = -L_h^*(s) (s^2 b_{ha} + s k_{hsa} + k_{hsia}) \quad (97)$$

Las ganancias se sintonizan por el **método de ubicación en serie**, igualando el denominador al polinomio deseado $(s + \omega_{d,h})(s + \eta_h \omega_{d,h})(s + \eta_h^2 \omega_{d,h})$ y normalizando:

$$b_{ha} = - \left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK) \right) \omega_{d,h} (1 + \eta_h + \eta_h^2) + b_{h,eq} \quad (98)$$

$$k_{hsa} = - \left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK) \right) \omega_{d,h}^2 (\eta_h + \eta_h^2 + \eta_h^3) \quad (99)$$

$$k_{hsia} = - \left(J_{h,eq} + \frac{r_{hd}}{2i_h} m_l(TLK) \right) \eta_h^3 \omega_{d,h}^3 \quad (100)$$

Implementación discreta en Simulink. Al igual que para el carro, el controlador y el observador de estados de izaje se implementan en Simulink con bloques independientes y período de muestreo $T_{s2} = 1$ ms, discretizando los integradores mediante el **método de Tustin** y reutilizando el mismo subsistema de la Figura 26.

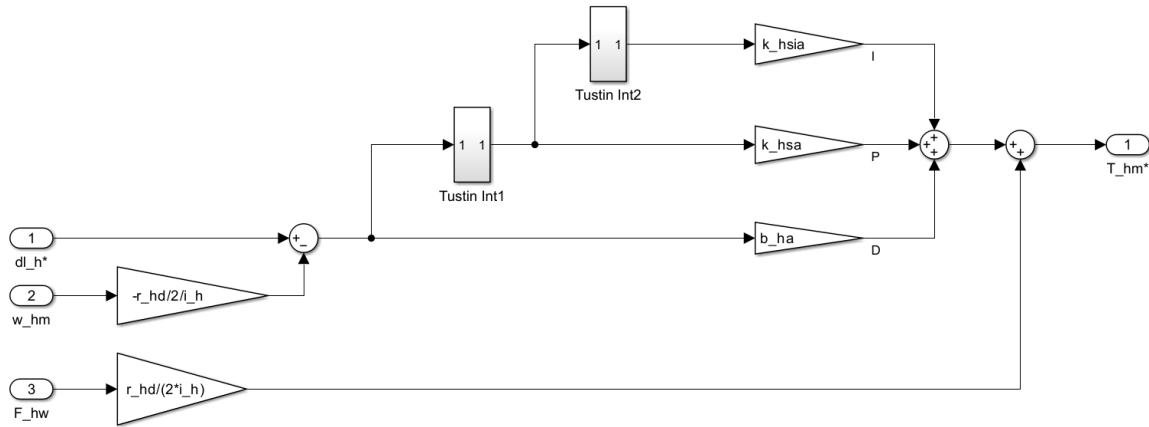


Figura 33: Diagrama de bloques del controlador de izaje de carga discretizado (Simulink, $T_{s2} = 1$ ms).

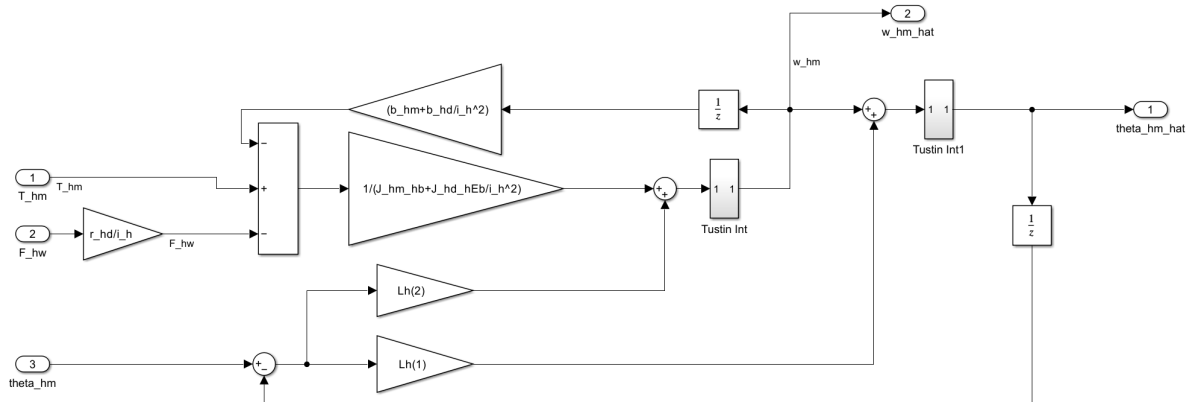


Figura 34: Diagrama de bloques del observador de estados del izaje discretizado (Simulink, $T_{s2} = 1$ ms).

Lazo externo de posición de izaje — generación de consigna de velocidad de cable.

De manera análoga al carro (77), el lazo externo de izaje cierra un controlador proporcional $K_{p,h} = 0,5 \text{ s}^{-1}$ ($\tau_h = 2 \text{ s}$) sobre el integrador $1/s$ de longitud de cable, resultando en la misma estructura de primer orden $K_{p,h}/(s + K_{p,h})$. La diferencia respecto al carro reside en la saturación de velocidad de cable $\dot{l}_{h,ref}$, que es variable en función de la masa estimada $\hat{m}$: cuando $\hat{m} > 16\,000 \text{ kg}$ (contenedor pesado) la saturación se reduce a $\pm 1,5 \text{ m/s}$ para limitar las solicitaciones mecánicas; en caso contrario se permite hasta $\pm 3,0 \text{ m/s}$.

```

1 function dl_h_ref = controlador_posicion(l_h_ref, l_h_sensor, dl_h_ff,
2     m_est)
3
4     Kp = 0.5;
5
6     e_l_h = l_h_ref - l_h_sensor;
7
8     dl_h_ref = Kp * e_l_h;
9
10    if m_est > 16000
11        dl_h_ref = min([1.5, max([dl_h_ref, -1.5])]);
12    else
13        dl_h_ref = min([3, max([dl_h_ref, -3])]);
14    end
15 end

```

Listing 7: Lazo externo de posición de izaje: generación de consigna de velocidad de cable.

Desempeño del controlador de izaje. Para evaluar el desempeño se simula un ascenso a **potencia constante máxima** con **carga máxima** ($m_l(TLK = On) = M_s + M_{c,max}$), con el carro quieto. El recorrido parte desde $l_h = 60$ m ($y_h = -15$ m, cable casi completamente desenrollado, dentro del barco) y asciende hasta $l_h = 5$ m ($y_h = 40$ m, posición de spreader elevado). Se presentan tres gráficas: comparativa de velocidad de cable $\dot{l}_h$ (consigna, estimada por el observador y simulada), comparativa de longitud de cable l_h (consigna, estimada y simulada), y comparativa de torque entre consigna $T_{hm}^*(t)$ y salida del modulador $T_{hm}(t)$.

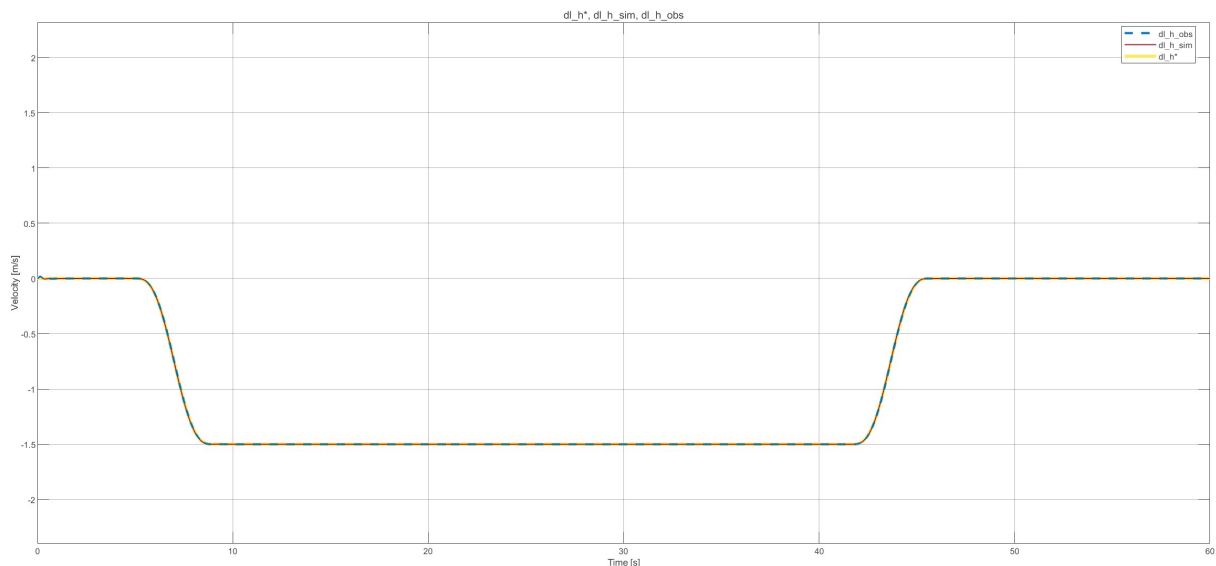


Figura 35: Izaje de carga — velocidad de cable: consigna $\dot{l}_h^*(t)$, estimada $\hat{\dot{l}}_h(t)$ y simulada $\dot{l}_h(t)$ (carga máxima, perfil trapezoidal).

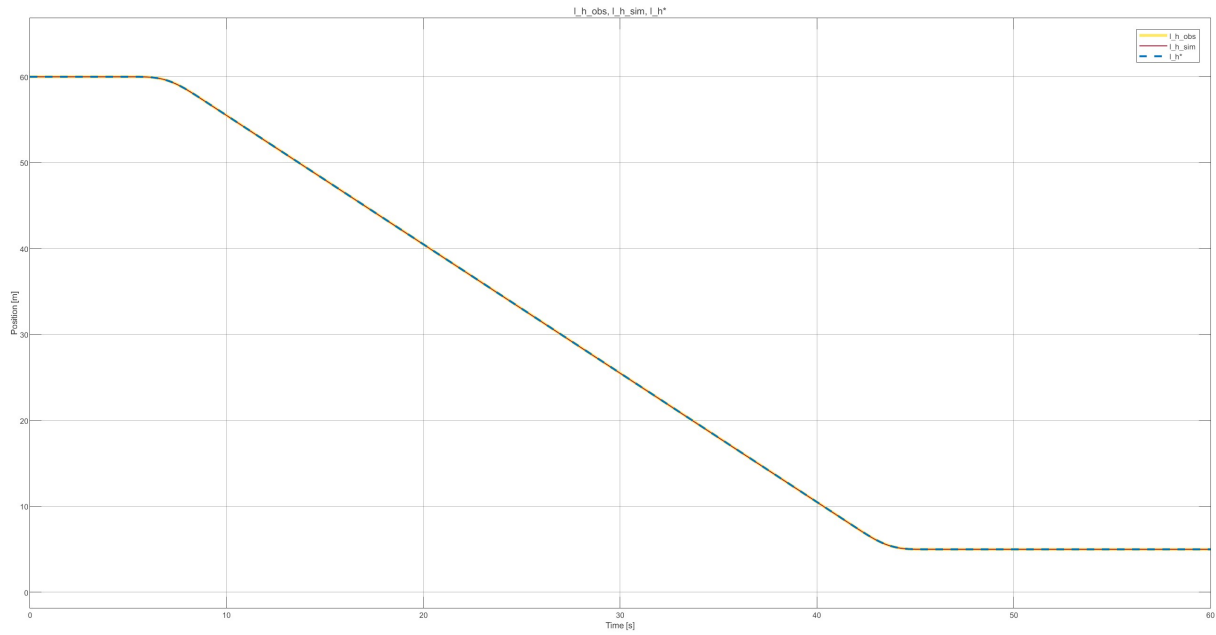


Figura 36: Izaje de carga — posición: consigna $l_h^*(t)$, estimada $\hat{l}_h(t)$ y simulada $l_h(t)$ (carga máxima, perfil trapezoidal).

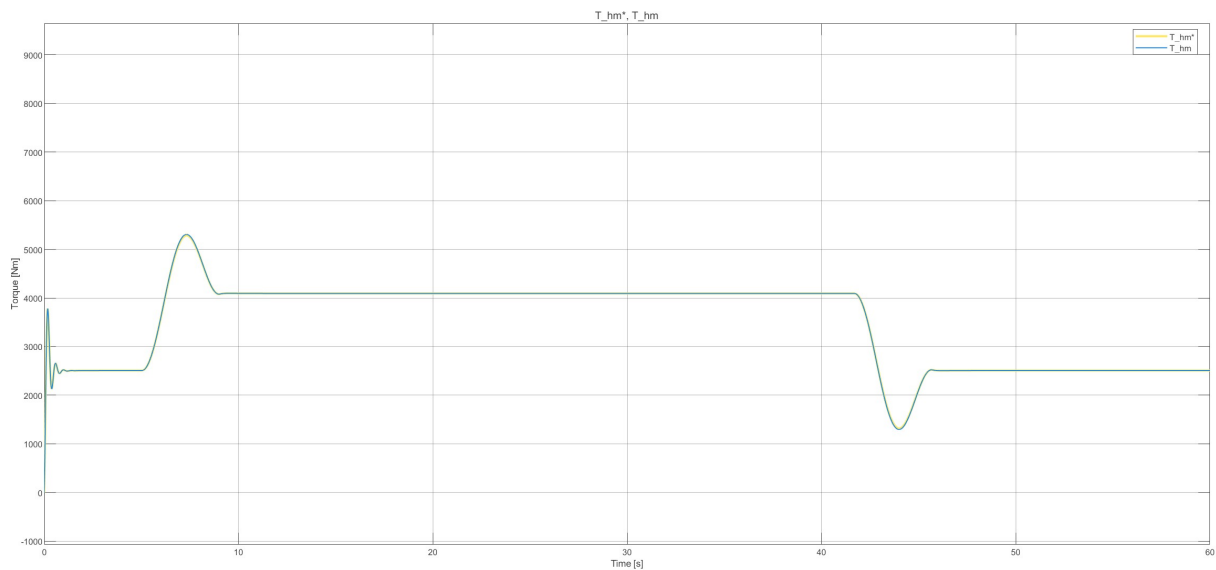


Figura 37: Izaje de carga — comparativa de torque: consigna $T_{hm}^*(t)$ y torque a la salida del modulador $T_{hm}(t)$ (carga máxima, perfil trapezoidal).

El transitorio inicial del torque se debe al paso de $T_{hm}^* = 0$ al valor necesario para sostener el peso del spreader en suspensión. A los $t = 5$ s se incorpora la carga, cuya masa toma un valor aleatorio dentro del rango $m_l \in [M_s + M_{c,min}, M_s + M_{c,max}]$, produciendo un escalón en el torque de consigna y en la salida del modulador. El movimiento de izaje comienza a los $t = 10$ s.

3.1.3. Controlador de oscilación de la carga

Modelo dinámico del péndulo de longitud variable. Se parte de las ecuaciones de movimiento de la carga en los ejes x e y (35)–(38). Asumiendo que la carga se encuentra suspendida en el aire ($F_{cx} = 0$, $F_{cy} = 0$, conforme a (37)–(40)), las ecuaciones se simplifican a:

$$m_l(TLK) \ddot{x}_l(t) = -2 F_{hw}(t, l(t) > l_h(t)) \sin(\theta_l(t)) \quad (101)$$

$$m_l(TLK) \ddot{y}_l(t) = 2 F_{hw}(t, l(t) > l_h(t)) \cos(\theta_l(t)) - m_l(TLK) g \quad (102)$$

Las posiciones de la carga en función de la posición del carro y el ángulo de balanceo son:

$$x_l(t) = x_t(t) + l(t) \sin(\theta_l(t)) \quad (103)$$

$$y_l(t) = Y_{t0} - l(t) \cos(\theta_l(t)) \quad (104)$$

Derivando (103) y (104) respecto al tiempo:

$$\dot{x}_l(t) = \dot{x}_t(t) + \dot{l}(t) \sin(\theta_l(t)) + l(t) \dot{\theta}_l(t) \cos(\theta_l(t)) \quad (105)$$

$$\dot{y}_l(t) = -\dot{l}(t) \cos(\theta_l(t)) + l(t) \dot{\theta}_l(t) \sin(\theta_l(t)) \quad (106)$$

Derivando una vez más y agrupando términos semejantes:

$$\begin{aligned} \ddot{x}_l(t) = & \ddot{x}_t(t) + \ddot{l}(t) \sin(\theta_l(t)) + 2 \dot{l}(t) \dot{\theta}_l(t) \cos(\theta_l(t)) \\ & + l(t) \ddot{\theta}_l(t) \cos(\theta_l(t)) - l(t) \dot{\theta}_l^2(t) \sin(\theta_l(t)) \end{aligned} \quad (107)$$

$$\begin{aligned} \ddot{y}_l(t) = & -\ddot{l}(t) \cos(\theta_l(t)) + 2 \dot{l}(t) \dot{\theta}_l(t) \sin(\theta_l(t)) \\ & + l(t) \ddot{\theta}_l(t) \sin(\theta_l(t)) + l(t) \dot{\theta}_l^2(t) \cos(\theta_l(t)) \end{aligned} \quad (108)$$

Eliminación de la tensión del cable — ecuación del péndulo. Sustituyendo (107) en (101) y (108) en (102), se multiplica la ecuación del eje x por $\cos(\theta_l(t))$ y la del eje y por $\sin(\theta_l(t))$, y se suman miembro a miembro. Los términos de F_{hw} se cancelan mutuamente en el lado derecho, dejando únicamente $-m_l(TLK) g \sin(\theta_l(t))$. En el lado izquierdo, los términos con $\dot{l}(t) \sin(\theta_l) \cos(\theta_l)$ y $l(t) \dot{\theta}_l^2(t) \sin(\theta_l) \cos(\theta_l)$ se cancelan entre sí, mientras que los de $\dot{l}(t) \dot{\theta}_l(t)$ y $l(t) \ddot{\theta}_l(t)$ se unifican mediante la identidad $\cos^2(\theta_l) + \sin^2(\theta_l) = 1$:

$$m_l(TLK) \left[\ddot{x}_t(t) \cos(\theta_l(t)) + 2 \dot{l}(t) \dot{\theta}_l(t) + l(t) \ddot{\theta}_l(t) \right] = -m_l(TLK) g \sin(\theta_l(t)) \quad (109)$$

La masa $m_l(TLK)$ se cancela en ambos miembros — la dinámica oscilatoria es independiente de la carga suspendida. Reordenando se obtiene la **ecuación diferencial del péndulo de longitud variable**:

$$l(t) \ddot{\theta}_l(t) + 2 \dot{l}(t) \dot{\theta}_l(t) + g \sin(\theta_l(t)) = -\ddot{x}_t(t) \cos(\theta_l(t)) \quad (110)$$

Modelo LPV del balanceo. Dado que en la operación de la grúa STS no es válida la aproximación $\sin(\theta_l) \approx \theta_l$, se reescribe el término gravitatorio multiplicando y dividiendo por $\theta_l(t)$: $g \sin(\theta_l(t)) = g [\sin(\theta_l(t))/\theta_l(t)] \theta_l(t)$. Dividiendo (110) por $l(t)$, la ecuación adopta la forma estándar de un sistema de segundo orden con parámetros variables en el tiempo (**modelo qLPV**):

$$\ddot{\theta}_l(t) + B_{eq}(t) \dot{\theta}_l(t) + K_{eq}(t) \theta_l(t) = G_{eq}(t) \ddot{x}_t(t) \quad (111)$$

donde la entrada es la aceleración del carro $\ddot{x}_t(t)$ y los coeficientes dependientes del estado instantáneo son:

$$B_{eq}(t) = \frac{2\dot{l}(t)}{l(t)} \quad (112)$$

$$K_{eq}(t) = \frac{g}{l(t)} \frac{\sin(\theta_l(t))}{\theta_l(t)} \quad \left[\lim_{\theta_l \rightarrow 0} \frac{\sin(\theta_l)}{\theta_l} = 1 \right] \quad (113)$$

$$G_{eq}(t) = -\frac{\cos(\theta_l(t))}{l(t)} \quad (114)$$

$B_{eq}(t)$ refleja el efecto de Coriolis: izar ($\dot{l} < 0$) amortigua la oscilación mientras que descender ($\dot{l} > 0$) la amplifica. $K_{eq}(t)$ es la rigidez equivalente del péndulo, dependiente de la longitud y del ángulo instantáneo; para evitar la división por cero cuando $\theta_l(t) \rightarrow 0$ se impone el límite computacional $\sin(\theta_l)/\theta_l \rightarrow 1$.

Ley de control — corrección de velocidad proporcional al ángulo. El objetivo del controlador es mantener $\theta_l(t) = 0$. En lugar de inyectar una corrección de aceleración, el controlador modifica directamente la consigna de velocidad del carro sumando un término proporcional al ángulo de balanceo:

$$v_t^*(t) = v_{ref}(t) + v_{bal}(t), \quad v_{bal}(t) = \varepsilon(t) K_{do}(t) \theta_l(t) \quad (115)$$

donde $\varepsilon(t) \in [0, 1]$ es un factor de escala progresivo definido más adelante. La corrección de velocidad genera una aceleración efectiva sobre el péndulo. Tratando $K_{do}(t)$ como localmente constante, la derivada de v_{bal} respecto al tiempo es:

$$\dot{v}_{bal}(t) \approx K_{do}(t) \dot{\theta}_l(t) \quad (116)$$

Sustituyendo esta aceleración efectiva en (111) y agrupando, la **ecuación diferencial a lazo cerrado** resulta:

$$\ddot{\theta}_l(t) + [B_{eq}(t) - G_{eq}(t) K_{do}(t)] \dot{\theta}_l(t) + K_{eq}(t) \theta_l(t) = G_{eq}(t) \dot{v}_{ref}(t) \quad (117)$$

El controlador actúa exclusivamente sobre el coeficiente de amortiguamiento: la rigidez $K_{eq}(t)$ —y por tanto la frecuencia natural— no se modifica.

Sintonización — amortiguamiento crítico. Se impone **amortiguamiento relativo crítico** ($\zeta_o = 1$), con polinomio característico deseado $s^2 + 2\omega_o s + \omega_o^2$. La frecuencia de diseño se programa igual a la frecuencia natural libre del péndulo:

$$\omega_o(t) = \sqrt{\frac{g}{l(t)}} \quad (118)$$

Igualando el coeficiente de $\dot{\theta}_l$ en (117) con $2\omega_o$ y sustituyendo (112) y (114):

$$\frac{2\dot{l}(t)}{l(t)} + \frac{\cos(\theta_l(t))}{l(t)} K_{do}(t) = 2\omega_o(t) \quad (119)$$

despejando $K_{do}(t)$:

$$K_{do}(t) = \frac{2\omega_o(t)l(t) - 2\dot{l}(t)}{\cos(\theta_l(t))} \quad (120)$$

Los polos de lazo cerrado quedan en $s = -\omega_o(t)$ (doble), garantizando amortiguamiento crítico a la frecuencia natural del péndulo para cualquier longitud de cable. La frecuencia natural no se altera respecto al péndulo libre.

Activación progresiva — gain scheduling por zona de operación. Para evitar excitaciones innecesarias del carro, la corrección v_{bal} se activa de forma progresiva mediante el factor $\varepsilon(t)$ en función del ángulo instantáneo $|\theta_l(t)|$ y de la zona de operación. Se definen dos umbrales, θ_{lim} y θ_{full} , que varían según el estado del sistema:

- **Izaje/descenso activo** ($|\dot{l}(t)| > \delta_a$): $\theta_{lim} = 15$, $\theta_{full} = 20$. Se permiten mayores oscilaciones durante el movimiento de cable.
- **Carro en movimiento** ($|\dot{\theta}_l(t)| > \delta_v$): $\theta_{lim} = 3,5$, $\theta_{full} = 5$.
- **Sistema en reposo**: $\theta_{lim} = 0$, $\theta_{full} = 1$. Corrección activa incluso para ángulos muy pequeños.

El factor de escala adopta la forma:

$$\varepsilon(t) = \begin{cases} 0 & |\theta_l| \leq \theta_{lim} \\ 0,5 + 0,5 \frac{|\theta_l| - \theta_{lim}}{\theta_{full} - \theta_{lim}} & \theta_{lim} < |\theta_l| < \theta_{full} \\ 1 & |\theta_l| \geq \theta_{full} \end{cases} \quad (121)$$

La transición suave entre 0,5 y 1 en la zona intermedia evita discontinuidades en la consigna de velocidad. Para $|\theta_l| \leq \theta_{lim}$ la corrección es nula, desconectando el controlador cuando el balanceo es despreciable.

Implementación en Simulink como *MATLAB Function*. La implementación completa del controlador de balanceo como MATLAB Function en Simulink es la siguiente:

```
1 function v_balanceo = controlador_balanceo(theta_1, omega_1, l, dl_dt, g)
2   DEG2RAD = pi / 180;
3   THRESH_V = 0.05;
4   THRESH_ACC = 0.1;
5
6   if abs(dl_dt) > THRESH_ACC
7       theta_lim = 15 * DEG2RAD;
8       theta_full = 20 * DEG2RAD;
9   elseif abs(omega_1) > THRESH_V
10       theta_lim = 3.5 * DEG2RAD;
11       theta_full = 5 * DEG2RAD;
12   else
13       theta_lim = 0;
14       theta_full = 1 * DEG2RAD;
15   end
16
17   theta_margen = theta_full - theta_lim;
18   abs_theta = abs(theta_1);
19   if abs_theta >= theta_full
20       escala = 1.0;
21   elseif abs_theta > theta_lim
22       escala = 0.5 + 0.5 * (abs_theta - theta_lim) / theta_margen;
23   else
24       escala = 0.0;
25   end
26
27   l_seg = max(1, 0.1);
28   th_sat = max(-80*DEG2RAD, min(80*DEG2RAD, theta_1));
29   w_o = sqrt(g / l_seg);
30   K_do = (2*w_o*l_seg - 2*dl_dt) / cos(th_sat);
31
32   v_balanceo = escala * K_do * theta_1;
33 end
```

Listing 8: MATLAB Function: controlador de balanceo (Simulink, $T_{s2} = 1$ ms).

Desempeño del controlador de balanceo. Se evalúa el controlador durante una traslación con **carga máxima** ($m_t(TLK = \text{On}) = M_s + M_{c,\text{máx}}$), longitud de cable constante en un valor intermedio $l_h = 30$ m, con perfil trapezoidal de velocidad entre $x_t = -15$ m y $x_t = +45$ m. Se presentan tres gráficas:

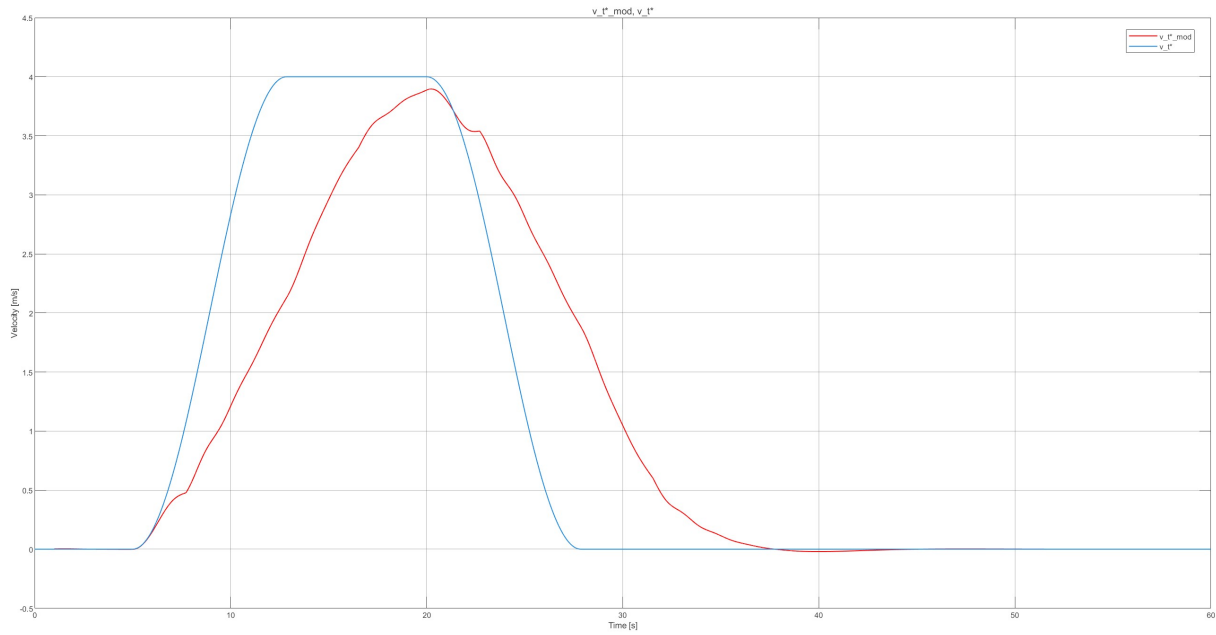


Figura 38: Controlador de balanceo — velocidad: consigna original $v_t^*(t)$ y consigna modificada por el controlador de balanceo $v_{t,mod}^*(t)$.

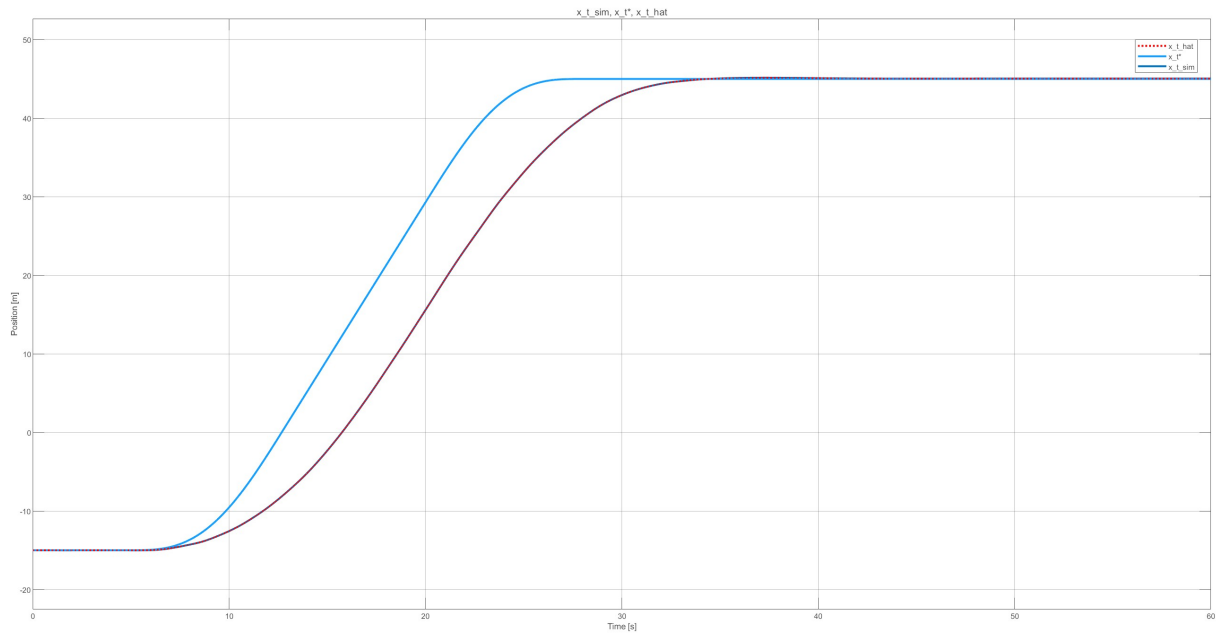


Figura 39: Controlador de balanceo — posición: consigna original $x_t^*(t)$, simulada $x_t(t)$ y estimada $\hat{x}_t(t)$.

Nota: El sistema en su estado actual no garantiza el cumplimiento de la posición final, dado que la modificación del perfil de velocidad por parte del controlador de balanceo altera el recorrido efectivo del carro. Esta limitación será corregida en el Nivel 1 del sistema, a través del planificador de trayectorias, que ajustará la consigna en línea para asegurar que el carro alcance la posición destino.

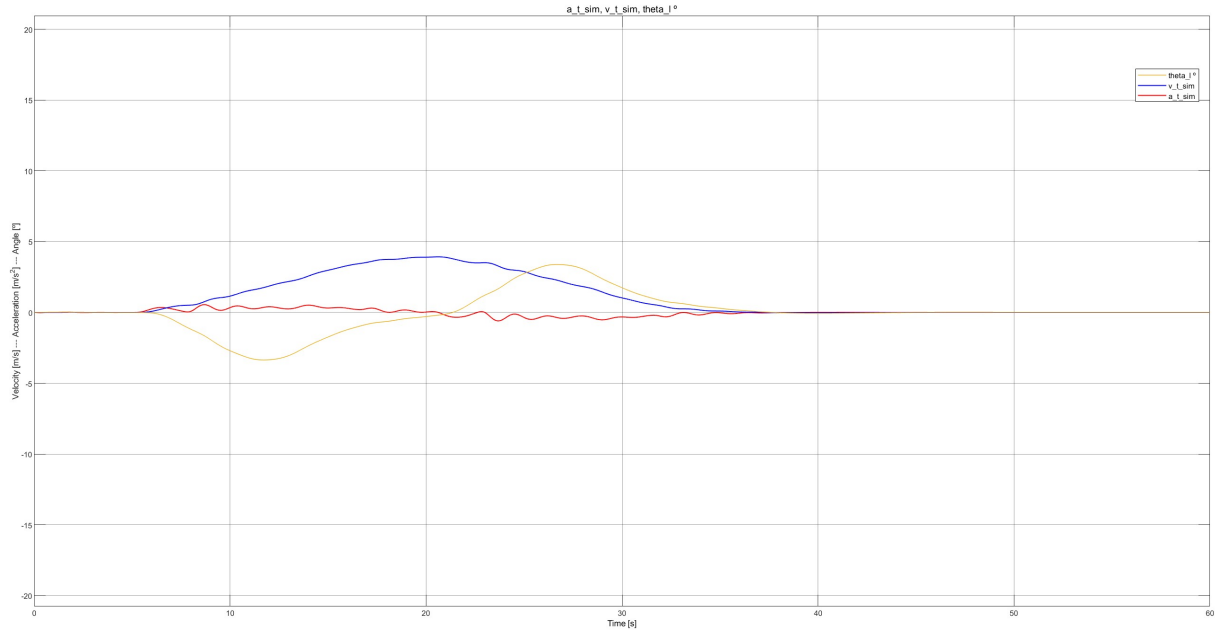


Figura 40: Controlador de balanceo — evolución conjunta de $\theta_l(t)$, $v_t(t)$ y $a_t(t)$ para analizar el comportamiento del ángulo de balanceo en cada región de aceleración y velocidad del carro.

El resultado es satisfactorio: en cada región de operación (aceleración, cruce y desaceleración) el ángulo de balanceo $\theta_l(t)$ se mantiene por debajo del umbral definido, y el amortiguamiento del ángulo es crítico, consistente con el diseño $\zeta_o = 1$ impuesto en la sintonización.

3.1.4. Nivel 2 completo

El Nivel 2 completo resulta de la integración de los tres controladores desarrollados en las secciones anteriores en un único diagrama de bloques de Simulink. La señal de consigna de torque del carro $T_{tm}^*(kT_{s2})$ surge de la superposición del controlador de traslación y del controlador de oscilación: el primero genera la componente de seguimiento de trayectoria a partir del error de posición $x_t^*(t) - \hat{x}_t(t)$, mientras que el segundo inyecta la corrección $a_{PD}(t)$ escalada a torque para amortiguar el balanceo. La consigna de torque del izaje $T_{hm}^*(kT_{s2})$ proviene exclusivamente del controlador de longitud de cable. Ambas consignas son generadas con período de muestreo $T_{s2} = 1$ ms y enviadas a los respectivos moduladores de torque del sistema físico.

Los estados no medidos de cada subsistema ($\omega_{tm}(t)$, $x_t(t)$, $v_t(t)$ para el carro; $\omega_{hm}(t)$ para el izaje) son estimados por sus observadores de Luenberger correspondientes a partir de la única señal sensada de cada encoder. El ángulo de balanceo $\theta_l(t)$ y su derivada $\dot{\theta}_l(t)$ se obtienen directamente del sensor de ángulo del carro, sin necesidad de estimador adicional.

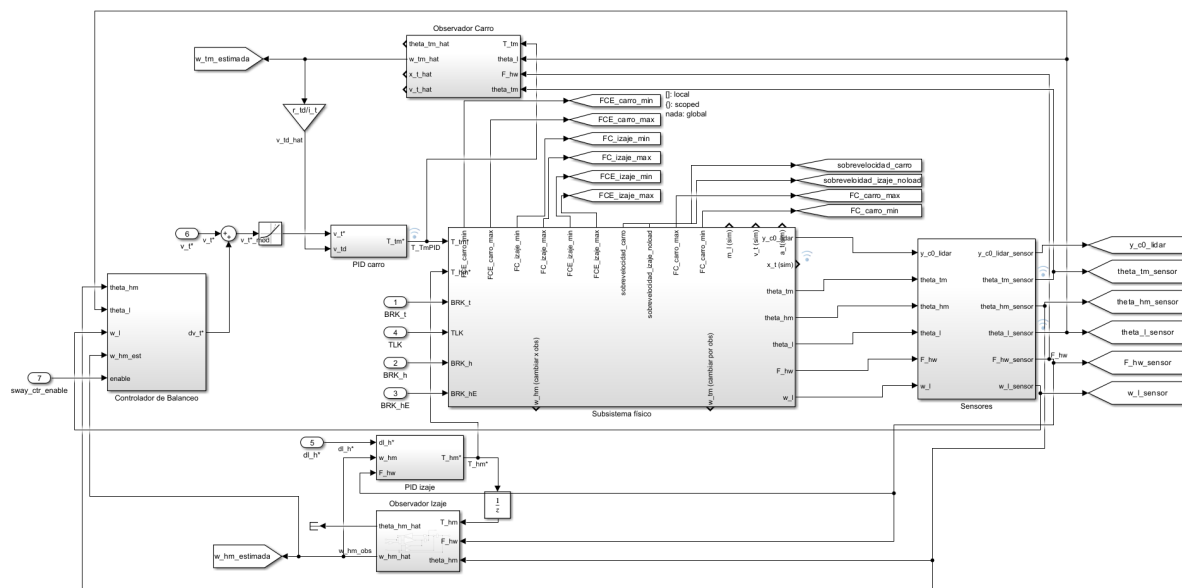


Figura 41: Diagrama de bloques del Nivel 2 completo: controladores de traslación, izaje y oscilación integrados en Simulink ($T_{s2} = 1$ ms). Para mejorar la visibilidad del diagrama se han omitido las entradas a los generadores de trayectorias y otras consignas externas provenientes del Nivel 1.

3.2. Conversión de los diagramas de bloques del Nivel 2 a *MATLAB Function*

3.2.1. Controlador de traslación del carro

El controlador PID de traslación, originalmente implementado como diagrama de bloques con integradores de Tustin en Simulink, se convierte a **MATLAB Function** manteniendo la misma discretización mediante variables persistentes:

```

1 function T_tm_star = PID_carro(v_t_star, v_td_hat)
2
3     persistent int1 int2 k_tsia k_tsa b_ta T_s e_prev int1_prev;
4
5     if isempty(int1)
6         int1 = 0;
7         int2 = 0;
8         k_tsia = 817.1486;
9         k_tsa = 4.09*10^3;
10        b_ta = 4.3394*10^3;
11        T_s = 0.001;
12        e_prev = 0;
13        int1_prev = 0;
14    end
15
16    e_t = v_t_star - v_td_hat;
17
18    int1 = int1 + T_s * (e_t + e_prev)/2;
19    e_prev = e_t;
20
21    int2 = int2 + T_s * (int1 + int1_prev)/2;
22    int1_prev = int1;
23
24    T_tm_star = b_ta * e_t + k_tsa * int1 + k_tsia * int2;
25

```

26 `end`Listing 9: MATLAB Function: controlador PID de traslación del carro ($T_{s2} = 1$ ms).

3.2.2. Observador de estados del carro

El observador de Luenberger del carro estima $\hat{\theta}_{tm}$, $\hat{\omega}_{tm}$, $\hat{x}_t$ y $\hat{v}_t$ a partir del encoder del motor y las entradas de fuerza. Los integradores se discretizan por el método trapezoidal. La ganancia de innovación actúa sobre el error de posición angular $e_\theta = \theta_{tm} - \hat{\theta}_{tm}$:

```

1 function [out_theta_tm_hat, out_omega_tm_hat, out_x_t_hat, out_v_t_hat] = ...
2   fcn(theta_tm, F_hw, theta_l, T_tm)
3
4   persistent r_td i_t K_tw b_tw b_t M_t T_s Lt1 Lt2 Lt3 Lt4 ...
5       J_tEg b_tEg dv_t_hat_prev v_t_hat_prev w_tm_hat_prev ...
6       dw_tm_hat_prev w_tm_hat theta_tm_hat dw_tm_hat x_t_hat ...
7       v_t_hat
8
9   if isempty(r_td)
10       r_td = 0.5;      i_t = 30;
11       K_tw = 480000;   b_tw = 3000;
12       b_t = 90;       M_t = 30000;
13       T_s = 0.001;
14       Lt1 = 17.0420;   Lt2 = 73.1961;
15       Lt3 = 0.0782;   Lt4 = -1.0015;
16       J_tEg = 8.3333; b_tEg = 6.06;
17       dv_t_hat_prev = 0; v_t_hat_prev = 0;
18       dw_tm_hat_prev = 0; w_tm_hat_prev = 0;
19       w_tm_hat = 0;     dw_tm_hat = 0;
20       x_t_hat = theta_tm * r_td / i_t;
21       v_t_hat = 0;
22       theta_tm_hat = theta_tm;
23   end
24
25   e_theta = theta_tm - theta_tm_hat;
26
27   F_tw = (theta_tm*(r_td/i_t) - x_t_hat)*K_tw ...
28         + (w_tm_hat*(r_td/i_t) - v_t_hat)*b_tw;
29
30   dv_t_hat = (1/M_t)*(2*F_hw*sin(theta_l) - b_t*v_t_hat + F_tw) ...
31             + Lt4 * e_theta;
32   v_t_hat = v_t_hat + T_s*(dv_t_hat + dv_t_hat_prev)/2 + Lt3*e_theta;
33   dv_t_hat_prev = dv_t_hat;
34   x_t_hat = x_t_hat + T_s*(v_t_hat + v_t_hat_prev)/2;
35   v_t_hat_prev = v_t_hat;
36
37   dw_tm_hat = (1/J_tEg)*(T_tm - F_tw*(r_td/i_t)) ...
38             - w_tm_hat*b_tEg + Lt2*e_theta;
39   w_tm_hat = w_tm_hat + T_s*(dw_tm_hat_prev + dw_tm_hat)/2 + Lt1*e_theta;
40   dw_tm_hat_prev = dw_tm_hat;
41   theta_tm_hat = theta_tm_hat + T_s*(w_tm_hat + w_tm_hat_prev)/2;
42   w_tm_hat_prev = w_tm_hat;
43
44   out_theta_tm_hat = theta_tm_hat;
45   out_omega_tm_hat = w_tm_hat;
46   out_x_t_hat = x_t_hat;
47   out_v_t_hat = v_t_hat;
48
49 end

```

Listing 10: MATLAB Function: observador de Luenberger del carro ($T_{s2} = 1$ ms).

3.2.3. Controlador de izaje de carga

El controlador PID de izaje incorpora además la compensación de la fuerza del cable F_{hw} como *feedforward*, y reconstruye la velocidad de cable $\dot{l}_h$ a partir de la velocidad angular estimada del motor:

```

1 function T_hm_star = fcn(F_hw, dl_h_star, w_hm_hat)
2
3     persistent int1 int2 k_hsia k_hsa b_ha T_s r_hd i_h e_prev int1_prev
4
5     if isempty(int1)
6         int1 = 0;
7         int2 = 0;
8         k_hsia = -6.4992*10^5;
9         k_hsa = -4.3987*10^5;
10        b_ha = -6.7646*10^4;
11        T_s = 0.001;
12        r_hd = 0.75;
13        i_h = 22;
14        e_prev = 0;
15        int1_prev = 0;
16    end
17
18    dl_h = w_hm_hat * (-r_hd)/(i_h*2);
19
20    e_h = dl_h_star - dl_h;
21
22    int1 = int1 + T_s * (e_h + e_prev)/2;
23    e_prev = e_h;
24
25    int2 = int2 + T_s * (int1 + int1_prev)/2;
26    int1_prev = int1;
27
28    T_hm_star = F_hw*r_hd/(i_h*2) + b_ha * e_h + k_hsa * int1 + k_hsia * int2;
29
30 end

```

Listing 11: MATLAB Function: controlador PID de izaje de carga ($T_{s2} = 1$ ms).

3.2.4. Observador de estados del izaje

El observador de Luenberger del izaje estima $\hat{\omega}_{hm}$ y $\hat{\theta}_{hm}$ a partir del encoder del motor y las entradas de torque y fuerza de cable. La innovación actúa sobre el error de posición angular $e_\theta = \theta_{hm} - \hat{\theta}_{hm}$:

```

1 function [out_wm_hm_hat, out_theta_hm_hat] = fcn(T_hm, F_hw, theta_hm)
2
3     persistent Lh1 Lh2 T_s b_hEq J_hEq r_hd i_h ...
4         w_hm_hat_prev dw_hm_hat_prev w_hm_hat theta_hm_hat
5
6     if isempty(Lh1)
7         Lh1 = 9.5197; Lh2 = 20.3903;
8         J_hEq = 37.8512; b_hEq = 18.0165;
9         r_hd = 0.75; i_h = 22;
10        T_s = 0.001;
11        dw_hm_hat_prev = 0;
12        w_hm_hat_prev = 0;
13        w_hm_hat = 0;
14        theta_hm_hat = theta_hm;
15    end
16
17    e_theta = theta_hm - theta_hm_hat;
18
19    dw_hm_hat = (1/J_hEq)*(T_hm - w_hm_hat*b_hEq - F_hw*(r_hd/i_h)) ...
20        + Lh2 * e_theta;
21    w_hm_hat = w_hm_hat + T_s*(dw_hm_hat_prev + dw_hm_hat)/2 + Lh1*e_theta;

```



```
22 dw_hm_hat_prev = dw_hm_hat;  
23  
24 theta_hm_hat = theta_hm_hat + T_s*(w_hm_hat + w_hm_hat_prev)/2;  
25 w_hm_hat_prev = w_hm_hat;  
26  
27 out_wm_hm_hat = w_hm_hat;  
28 out_theta_hm_hat = theta_hm_hat;  
29  
30 end
```

Listing 12: MATLAB Function: observador de Luenberger del izaje ($T_{s2} = 1$ ms).

3.2.5. Controlador paralelo de balanceo

La versión final del controlador de balanceo recibe directamente las señales del encoder del motor de izaje para calcular internamente la longitud de cable l y su derivada $\dot{l}$, e incorpora una entrada `enable` para desconectarlo desde el Nivel 1:

```
1 function v_balanceo = controlador_balanceo(theta_hm, theta_l, w_l, w_hm_hat, enable)  
2  
3 persistent g r_hd i_h DEG2RAD;  
4  
5 if isempty(g)  
6     DEG2RAD = pi/180;  
7     g = 9.8066;  
8     r_hd = 0.75;  
9     i_h = 22;  
10 end  
11  
12 if ~enable  
13     v_balanceo = 0;  
14     return;  
15 end  
16  
17 l = theta_hm * (-r_hd)/(i_h*2);  
18 dl_dt = w_hm_hat * (-r_hd)/(i_h*2);  
19  
20 l_seg = max(l, 0.1);  
21 th_sat = max(-80*DEG2RAD, min(80*DEG2RAD, theta_l));  
22 w_o = sqrt(g / l_seg);  
23 K_do = (2*w_o*l_seg - 2*dl_dt) / cos(th_sat);  
24  
25 v_balanceo = 1 * K_do * theta_l;  
26 end
```

Listing 13: MATLAB Function: controlador paralelo de balanceo ($T_{s2} = 1$ ms).

3.3. Nivel 1: Control Supervisor

El Nivel 1 es el responsable de ejecutar la **lógica de funcionamiento normal** de la grúa STS. Opera con período de muestreo $T_{s1} = 20$ ms e implementa el movimiento coordinado del carro y el izaje, la gestión de frenos y *twistlocks*, la detección de fines de carrera, el almacenamiento de lecturas del LIDAR, la generación de trayectorias para el Nivel 2 y el pulso de vida (*heartbeat*) que el Nivel 0 monitorea mediante su autómata *Watchdog*.

La Figura 42 muestra el esquema de entradas y salidas del Nivel 1.

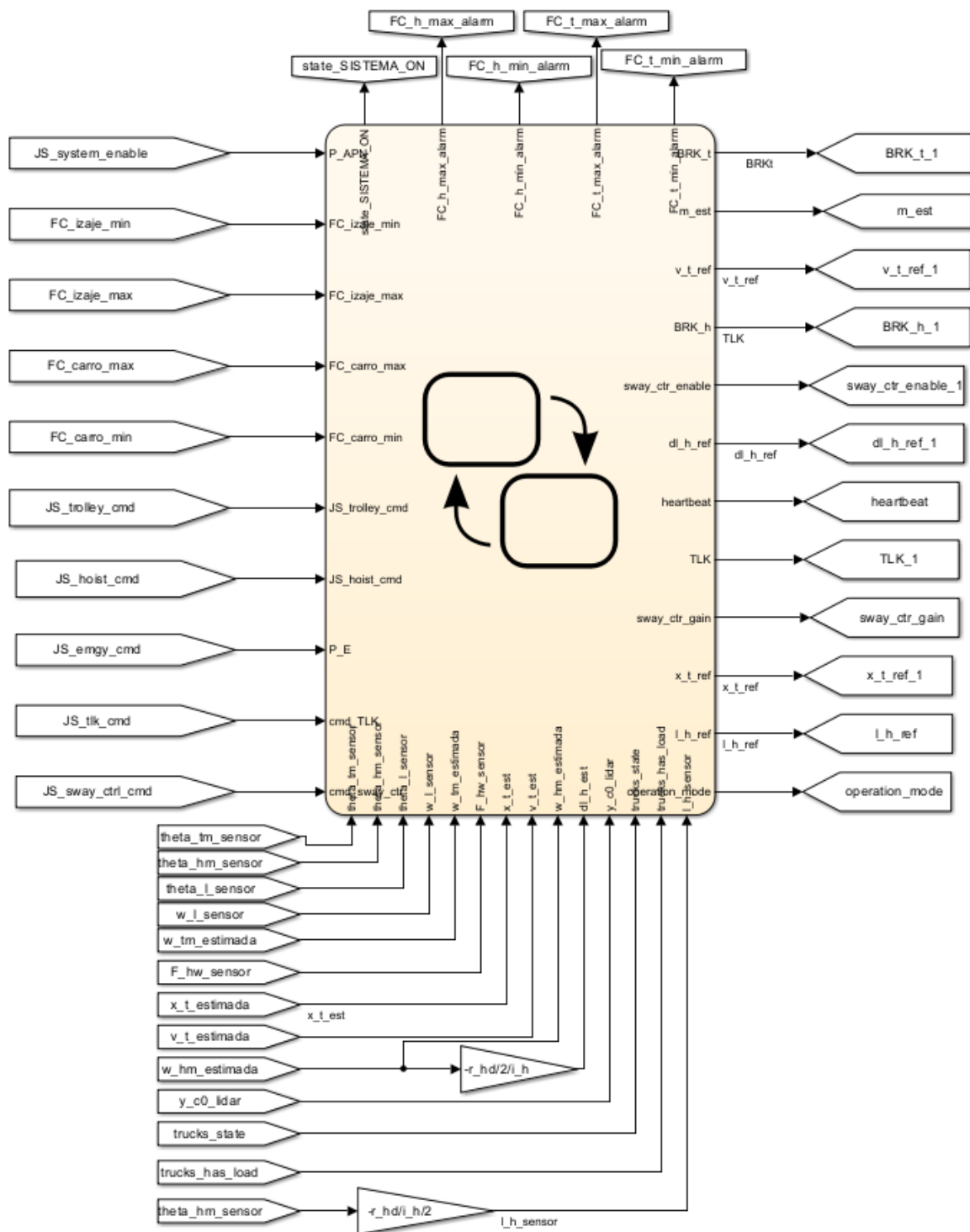


Figura 42: Esquema de entradas y salidas del Nivel 1.

3.3.1. Estructura general — Heartbeat y estado OFF

El autómata se implementa en Stateflow con dos subestados ejecutados en **paralelo** (AND): Heartbeat_Generator y Supervisory_Control (Figura 43).

Heartbeat_Generator alterna entre Heartbeat_High (heartbeat = true) y Heartbeat_Low

(heartbeat = false) cada 200 ms, produciendo una onda cuadrada de 2,5Hz que el Nivel 0 verifica continuamente.

Supervisory_Control arranca en **OFF**, que inicializa todas las salidas a valores seguros: frenos abiertos, TLK y balanceo desactivados, referencias de velocidad nulas, alarmas en **false** y vectores de memoria del LIDAR y estimación de masa en cero. Cuando el operador activa el sistema (**P_APN**), el autómata sale de **OFF** —registrando **state_SISTEMA_ON = true**— y transiciona al estado activo (**ON**). Si **P_APN** se desactiva, el sistema retorna a **OFF** (**[~P_APN]**). Dentro de **ON**, todos los subestados descritos en las secciones siguientes (**BRK_Logic**, **Limit_Switch**, **Mode**, entre otros) se ejecutan de forma **paralela (AND)**, de modo que la lógica de frenos, la detección de fines de carrera y el control de modo operan simultáneamente en cada paso de muestreo.

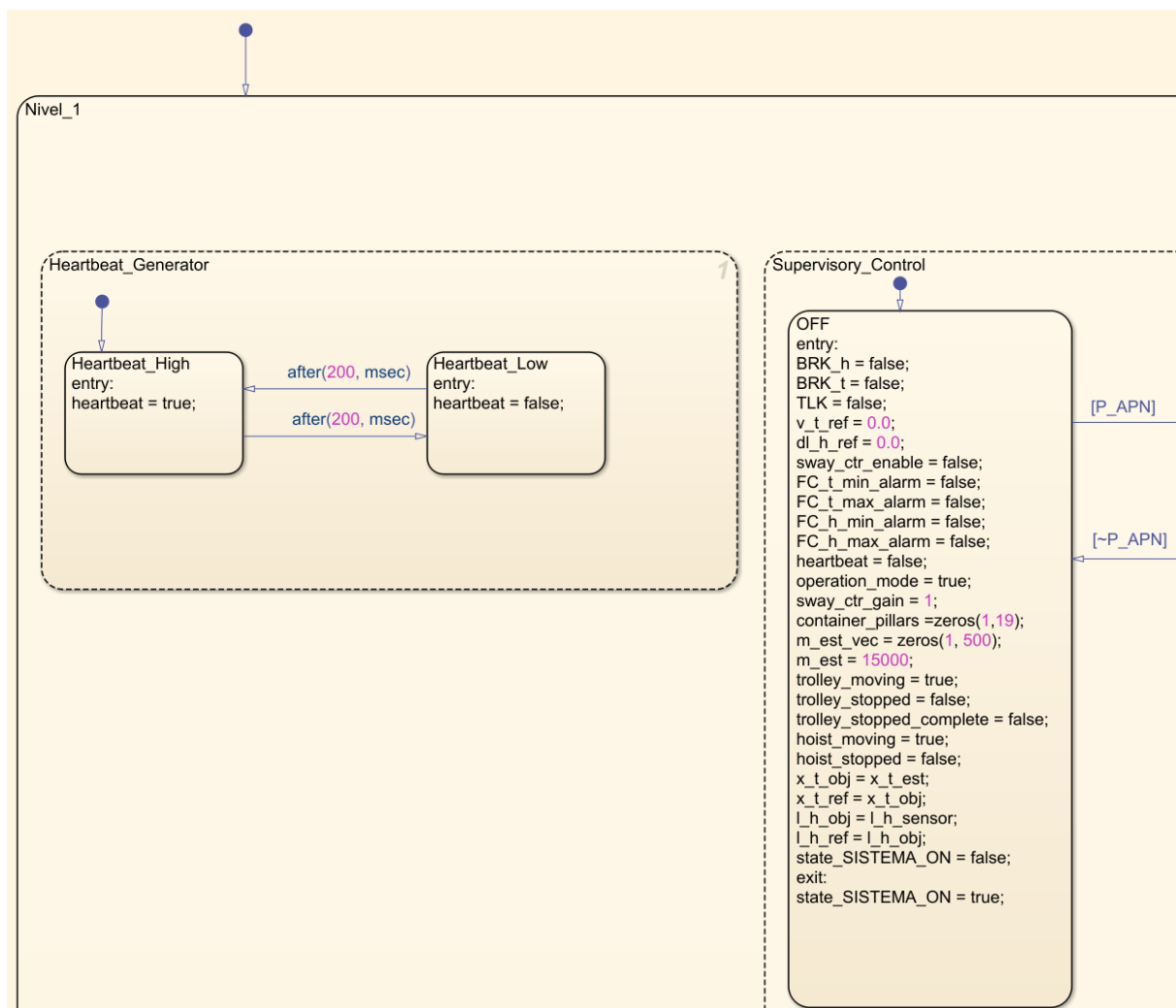


Figura 43: Estructura paralela del Nivel 1: **Heartbeat_Generator** y estado **OFF** de **Supervisory_Control**.

3.3.2. Lógica de frenos

Dentro del estado activo de **Supervisory_Control** opera el subestado paralelo **BRK_Logic**, con dos ramas independientes: **BRK_t_Logic** (freno de carro) y **BRK_h_Logic** (freno de

izaje).

Ambas ramas parten del estado **cerrado** ($BRK_x = \text{false}$) y transicionan al estado **abierto** ($BRK_x = \text{true}$) cuando no hay fines de carrera activos y la posición se encuentra alejada 1,5 m de los límites físicos. Para el carro:

$$\neg(FC_carro_max \vee FC_carro_min) \wedge x_{t,est} < x_{t,max} - 1,5 \wedge x_{t,est} > x_{t,min} + 1,5$$

y análogamente para el izaje con $l_{h,sensor}$, $l_{h,min}$ y $l_{h,max}$.

El freno se cierra cuando se activa un fin de carrera y la velocidad del motor supera el umbral $\omega_{min,BRK} = 0,01 \text{ rad/s}$, evitando cierres bruscos cuando el motor ya está prácticamente detenido.

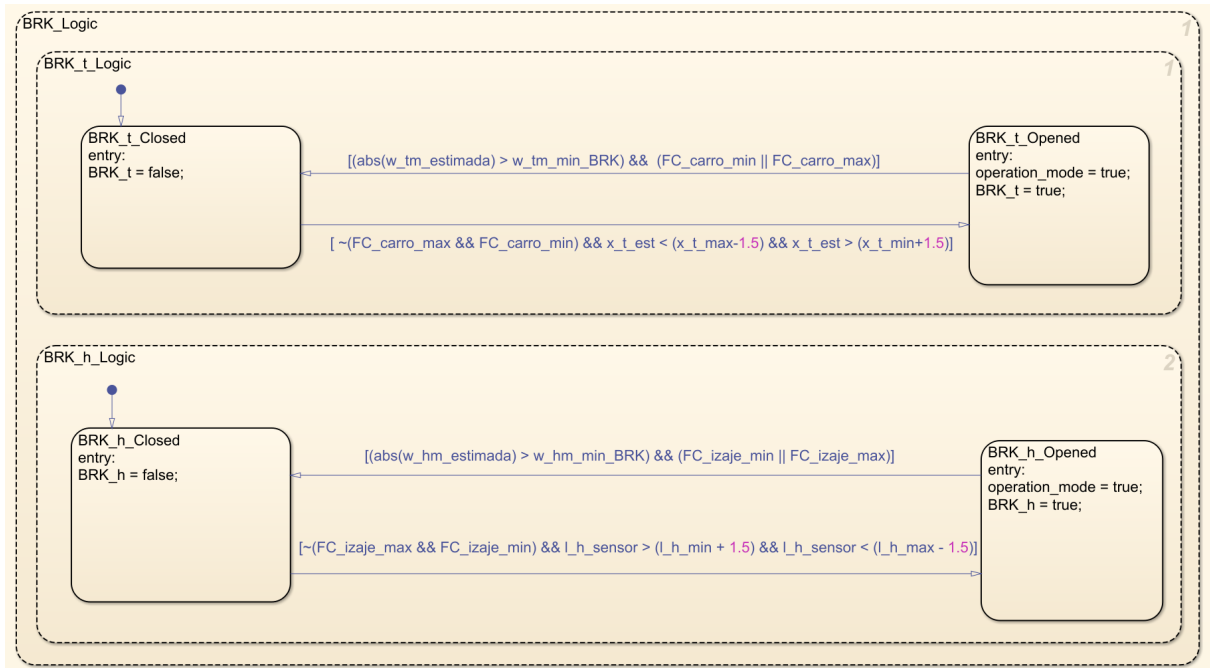


Figura 44: Lógica de frenos: subestados paralelos BRK_t_Logic y BRK_h_Logic.

3.3.3. Comandos de frenos — BRK_Commander

El subestado **BRK_Commander** se encarga de activar o desactivar los frenos de operación del carro y del izaje en función de la demanda de movimiento de cada subsistema. Opera de forma paralela mediante dos lógicas independientes: **BRK_t_Activation_Logic** para el freno del carro y **BRK_h_Activation_Logic** para el freno del izaje.

En cada lógica, el freno se libera ($BRK_close = \text{false}$) cuando se detecta error de posición o velocidad suficiente para justificar movimiento, y se cierra ($BRK_close = \text{true}$) una vez que el subsistema se encuentra dentro de la zona de tolerancia tanto en posición como en velocidad. Para el carro, la transición a demanda de freno ocurre cuando $|x_{t,est} - x_{t,ref}| < 0,01$ y $|v_{t,est}| < 0,01$, mientras que la liberación se activa si $|x_{t,est} - x_{t,ref}| > 0,03$. Para el izaje, la lógica es análoga en términos de longitud de cable y velocidad de enrollamiento.

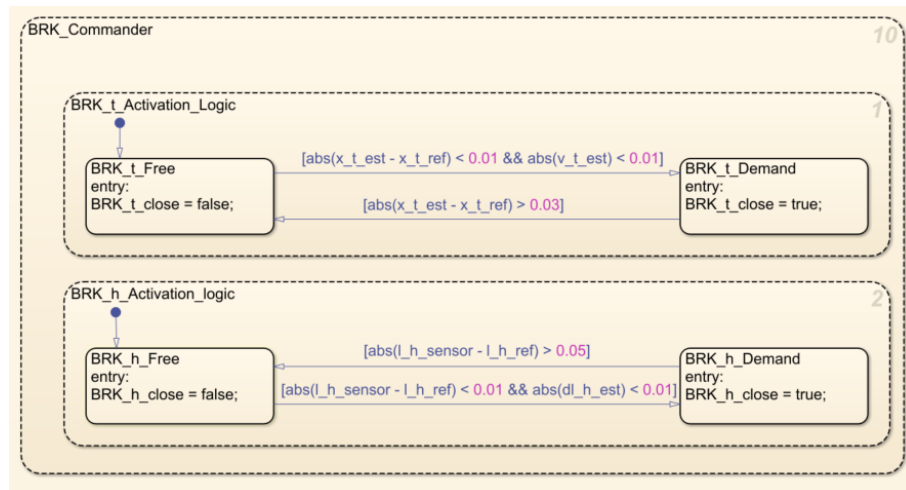


Figura 45: Diagrama del subestado **BRK_Commander**: lógicas de activación de frenos para el carro (**BRK_t**) y el izaje (**BRK_h**).

3.3.4. Fines de carrera

El subestado paralelo **Limit_Switch** monitorea de forma continua (*during*) los cuatro fines de carrera del sistema mediante cuatro ramas paralelas independientes: **Trolley_Min_Logic**, **Trolley_Max_Logic**, **Hoist_Min_Logic** y **Hoist_Max_Logic** (Figura 46).

Cada rama sigue la misma lógica biestable: en estado *Low* la alarma está en `false`; al activarse el sensor correspondiente la alarma pasa a `true` (estado *High*), y regresa a `false` en cuanto el sensor se desactiva. Las cuatro alarmas generadas son `FC_t_min_alarm`, `FC_t_max_alarm`, `FC_h_min_alarm` y `FC_h_max_alarm`.

Estas señales son utilizadas por el ciclo operativo del Nivel 1 para interrumpir o inhibir los movimientos activos: ante la activación de un fin de carrera, la lógica supervisora detiene el movimiento en la dirección correspondiente, evitando que el carro o la carga alcancen los topes mecánicos.

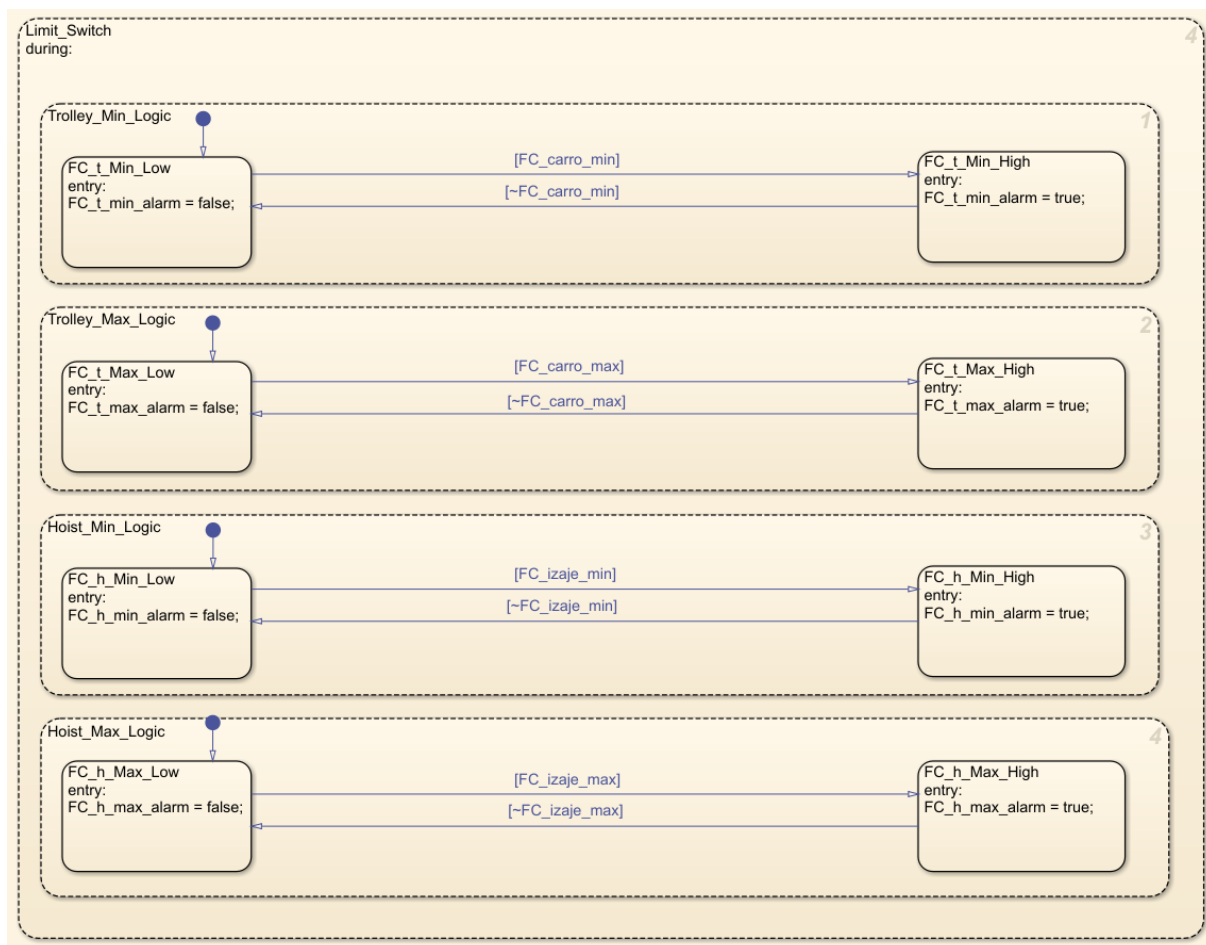


Figura 46: Subestado **Limit_Switch**: cuatro ramas paralelas de detección de fines de carrera.

3.3.5. Modos de operación

El subestado paralelo **Mode** (Figura 47) gestiona los dos modos de operación del sistema. Si **operation_mode** es verdadero se activa el estado **Automatic**, reservado para la ejecución del ciclo operativo automático. Si **operation_mode** es falso el sistema pasa al estado **Manual**, que permite al operador comandar directamente la grúa mediante el joystick.

Al entrar en **Manual** se congelan las referencias actuales ($x_t_ref = x_t_est$, $l_h_ref = l_h_sensor$) para evitar saltos bruscos en las consignas del Nivel 2 (*bumpless transfer*). Durante la operación manual, las referencias de velocidad se calculan aplicando la función **rate_limiter** a los comandos del joystick escalados ($JS_trolley_cmd \times 0,1$, $JS_hoist_cmd \times 0,1$).

Las alarmas de fines de carrera saturan la consigna de velocidad correspondiente: ante **FC_t_min_alarm** se impone $v_{t,ref} = \max(0, v_{t,ref,a})$; ante **FC_t_max_alarm**, $v_{t,ref} = \min(0, v_{t,ref,a})$; y de forma análoga para el izaje, agregando además la condición de sobrecarga que satura el izaje hacia arriba ($\dot{l}_{h,ref} = \max(0, \dot{l}_{h,ref,a})$ ante **overload**), impidiendo bajar la carga en ese estado. Las referencias de posición se integran por el método trape-

zoidal:

$$x_{t,ref}^{(k)} = x_{t,ref}^{(k-1)} + T_{s1} \frac{v_{t,ref}^{(k)} + v_{t,ref}^{(k-1)}}{2}$$

y de forma equivalente para $l_{h,ref}$.

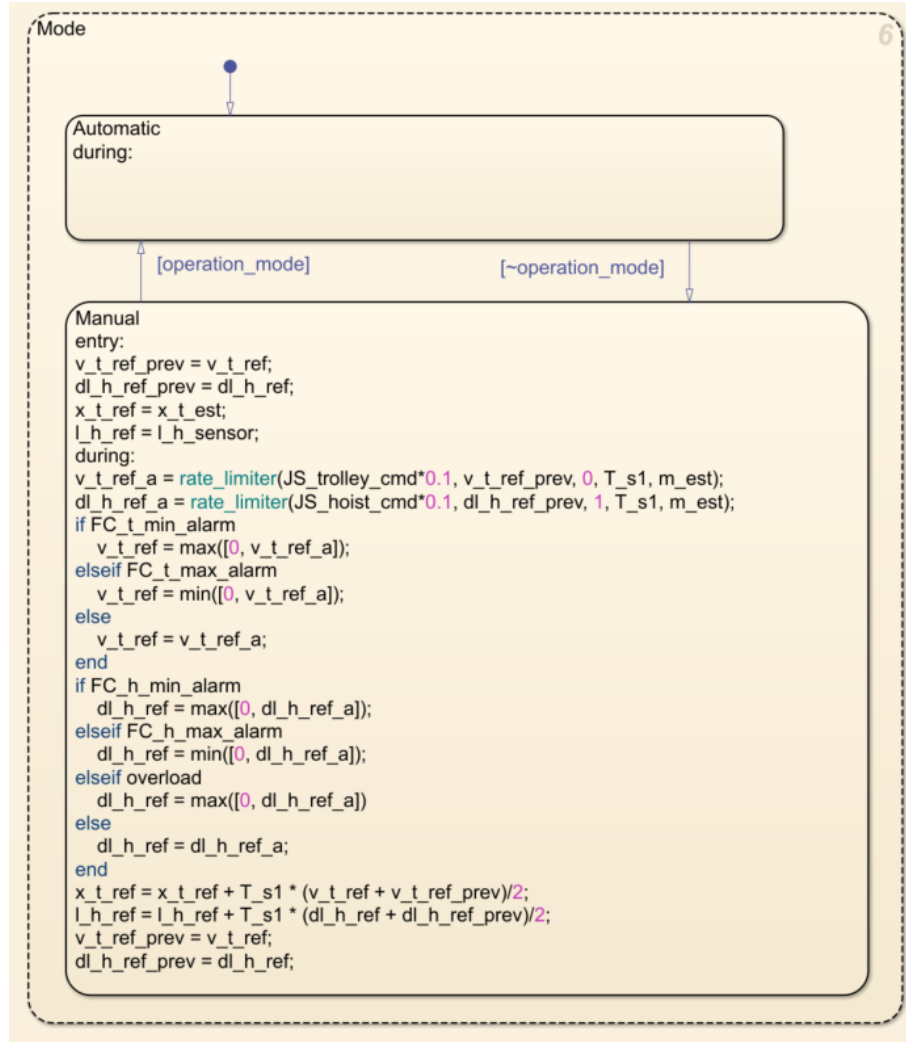


Figura 47: Subestado Mode: lógica de conmutación entre operación manual y automática.

Función rate_limiter.

La función limita la tasa de cambio de la consigna de velocidad para respetar las aceleraciones máximas admisibles por los accionamientos. Para el carro la aceleración máxima es $a_{t,max} = 0,72 \text{ m/s}^2$; para el izaje, $a_{h,max} = 0,675 \text{ m/s}^2$. El incremento máximo por paso es $\Delta v_{max} = a_{max} T_{s1}$, y la salida se obtiene saturando el incremento pedido:

$$v_{out}^{(k)} = v_{prev} + \text{sat}_{[-\Delta v_{max}, \Delta v_{max}]}(v_{in} - v_{prev})$$

Para el izaje se aplica adicionalmente un límite de velocidad por potencia constante antes del limitador de pendiente. Con potencia nominal $P = 956150 \text{ W}$, la velocidad

máxima de izaje en función de la masa total es:

$$v_{h,max}(m) = \frac{P}{mg}$$

de modo que al aumentar la masa la consigna de entrada ya llega acotada al limitador, y la desaceleración resultante respeta $a_{h,max}$ en lugar de producir un escalón.

3.3.6. Lógica de *twistlocks*

El subestado paralelo TLK_Logic (Figura 48) gestiona la apertura y cierre de los *twistlocks*. Consta de tres estados: Init, TLK_Closed (TLK = true) y TLK_Opened (TLK = false). Al ingresar al subestado, Init lee el estado actual del sensor TLK para determinar el estado inicial: si TLK es verdadero transiciona a TLK_Closed (prioridad 1), en caso contrario a TLK_Opened (prioridad 2).

La condición de conmutación entre ambos estados incorpora una restricción de seguridad: el cambio solo se ejecuta cuando la fuerza medida en el cable es nula, es decir, cuando el contenedor está apoyado y no hay carga suspendida. Esto impide abrir los *twistlocks* mientras el contenedor está izado, evitando una caída. La señal de comando cmd_TLK proviene del operador o del ciclo automático: si cmd_TLK es verdadero y la fuerza es nula, el sistema cierra los *twistlocks*; si cmd_TLK es falso y la fuerza es nula, los abre.

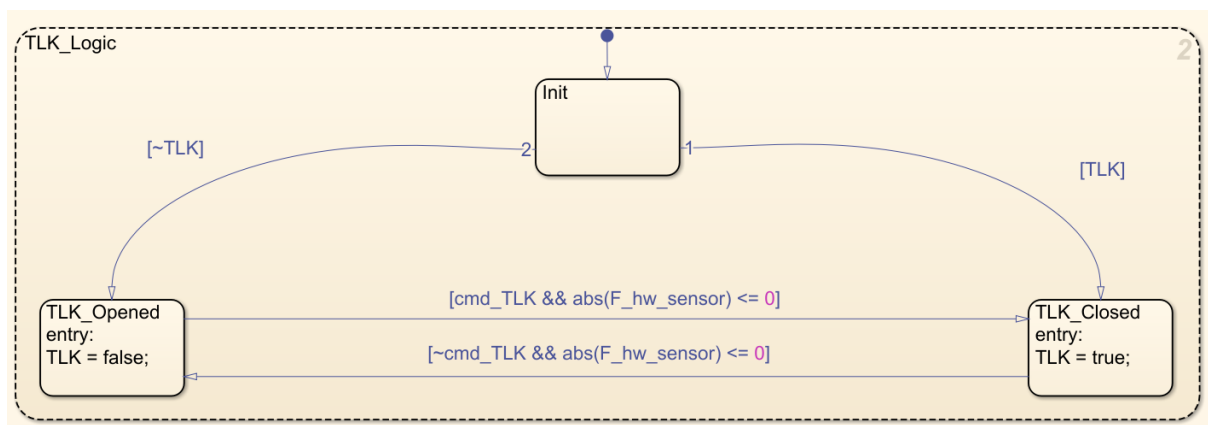


Figura 48: Subestado TLK_Logic: lógica de apertura y cierre de *twistlocks* con condición de seguridad sobre la fuerza en cable.

3.3.7. Control de balanceo de carga

El subestado paralelo Sway_Control_Mode (Figura 49) habilita o inhibe el controlador paralelo de balanceo del Nivel 2. Consta de dos estados: Sway_Control_ON y Sway_Control_OFF, que activan o desactivan la señal sway_ctr_enable, respectivamente. El estado inicial es ON.

La habilitación requiere que el operador lo haya solicitado mediante cmd_sway_ctr y que la longitud del cable sea suficiente ($l_{h,sensor} > 12\text{m}$), condición que garantiza que el péndulo tenga dinámica apreciable. El controlador se inhibe si el operador lo desactiva o si el cable es demasiado corto ($l_{h,sensor} < 10\text{m}$), con una histéresis de 2m entre ambos umbrales para evitar conmutaciones repetidas cerca del límite.

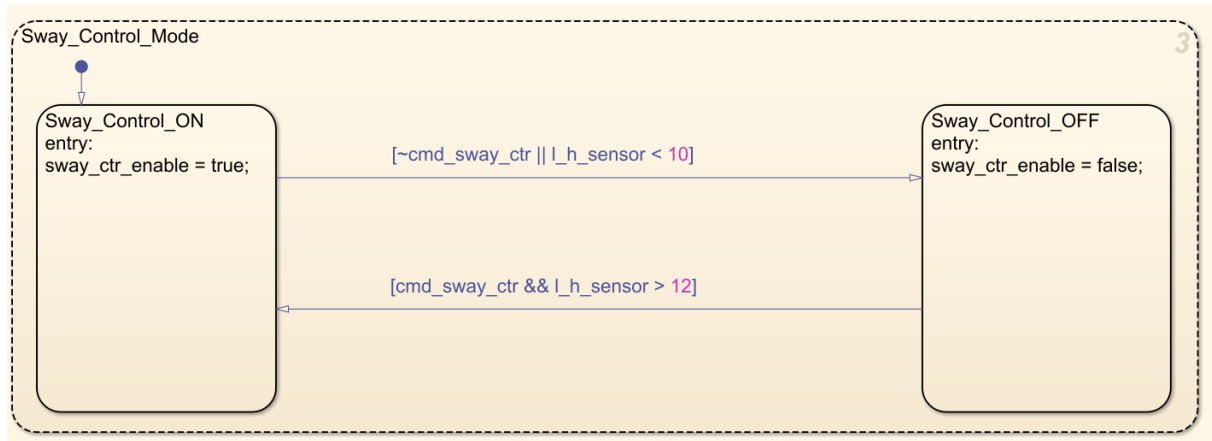


Figura 49: Subestado **Sway_Control_Mode**: habilitación del controlador de balanceo con condición de longitud mínima de cable.

3.3.8. Mapa LIDAR

El subestado paralelo **Lidar_Map** (Figura 50) construye y mantiene un mapa de alturas del entorno a partir de las lecturas del sensor LIDAR montado solidariamente al carro.

Al ingresar al estado (**entry**) se inicializa el mapa: se calcula el índice de grilla correspondiente a la posición $x = 0$ (límite tierra/barco) y todos los elementos desde ese índice hasta el final del vector se inicializan a $y_{h,min}$, reflejo de la altura mínima de la zona terrestre. El contador interno cc se pone en cero.

Durante la operación (**during**), en cada paso de muestreo se actualiza la celda de la grilla correspondiente a la posición actual del LIDAR ($x_{lidar} = x_{t,est} + x_{lidar,offset}$) con la lectura $y_{c0,lidar}$ proveniente del Nivel 2. El mapa se construye así de forma incremental a medida que el carro recorre su trayectoria.

A partir del mapa acumulado se calculan dos salidas de interés para el ciclo operativo:

- **Longitud de cable segura** $l_{h,safe}$: se obtiene como la diferencia entre la altura del pórtico y la máxima altura registrada en todo el mapa, con un margen adicional de $1,5 H_c$:

$$l_{h,safe} = 45 - \text{máx}(\text{lidar_map}) - 1,5 H_c$$

Esta longitud es la máxima a la que puede descender el spreader sin riesgo de colisión, y es utilizada por el planificador de trayectorias.

- **Conteo de pilas de contenedores**: para cada una de las 19 posiciones fijas de pilas se busca el máximo de altura en una ventana espacial centrada en esa posición. El ancho de la ventana se calcula como la distancia recorrida por el carro a velocidad máxima en un paso de muestreo del Nivel 2 ($\text{window_idx} = \lfloor v_{t,max} T_{s2} / \Delta x_{lidar} \rfloor$), compensando el hecho de que las lecturas del LIDAR no siempre coinciden exactamente con el centro de cada pila. La altura neta de contenedores se convierte a número entero de unidades mediante:

$$\text{container_pillars}(i) = \text{máx}\left(0, \left\lceil \frac{\text{pillar_height} - y_{h,min}}{H_c} - \epsilon \right\rceil\right)$$

donde $\epsilon = 10^{-6}$ evita redondeos espurios por errores numéricos.

```

Lidar_Map
entry:
    land_end_element = floor((0 - x_c0_left)/dx_grid_lidar) + 1;
    land_end_element = max(1, min(lidar_grid_size, land_end_element));
    lidar_map(land_end_element:end) = y_h_min;
    cc = 0;
during:
    x_lidar = x_t_est + x_lidar_offset;
    grid_element = max(1, min(lidar_grid_size, floor((x_lidar-x_c0_left)/dx_grid_lidar) + 1));
    lidar_map(grid_element) = y_c0_lidar;
    window_idx = round((v_t_max * T_s2) / dx_grid_lidar);
    n_pilares = 19;
    espaciado = (x_containers_right_lim - x_containers_left_lim) / (n_pilares - 1);
    l_h_safe = 45 - max(lidar_map) - 1.5 * H_c;
    for i = 1:n_pilares
        x_lidar_pillar = x_containers_left_lim + (i-1)*espaciado;
        grid_element_pillar = max(1, min(lidar_grid_size, floor((x_lidar_pillar - x_c0_left)/dx_grid_lidar) + 1));
        window_start = max(1, grid_element_pillar - window_idx);
        window_end = min(lidar_grid_size, grid_element_pillar + window_idx);
        pillar_height = max(lidar_map(window_start:window_end));
        containers_height = pillar_height - y_h_min;
        container_pillars(i) = max(0, ceil(containers_height / H_c - 1e-6));
    end
end

```

Figura 50: Subestado Lidar_Map: construcción incremental del mapa de alturas y estimación de pilas de contenedores a partir del LIDAR.

3.3.9. Detector de sobrecarga — Overload

El subestado paralelo **Overload** monitorea continuamente la masa estimada del contenedor y activa la alarma de sobrecarga cuando ésta supera el límite máximo permitido $M_{c,max}$. En estado **No_Overload** el sistema opera con normalidad (**overload** = **false**, **operation_mode** = **true**); si la masa estimada supera el umbral ($\hat{m} > M_{c,max}$) se transiciona a **Overload** (**overload** = **true**, **operation_mode** = **false**), deshabilitando el modo automático y forzando la transición al modo manual de operación. La recuperación ocurre únicamente cuando la masa estimada desciende por debajo de 16 000 kg, incorporando histéresis para evitar conmutaciones frecuentes en torno al umbral.

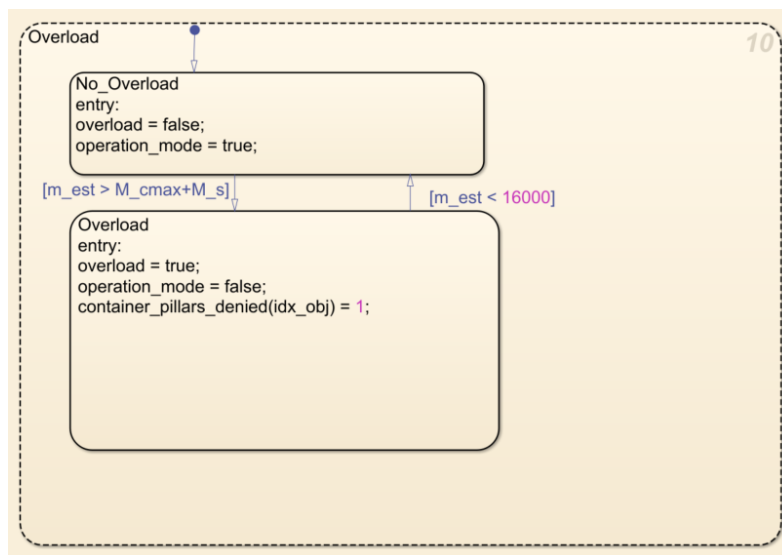


Figura 51: Subestado **Overload**: detección de sobrecarga con activación de alarma y forzado a modo manual.

3.3.10. Banderas de movimiento

Los subestados paralelos **Trolley_Movement_Flags** y **Hoist_Movement_Flags** generan señales de estado que el ciclo operativo del Nivel 1 utiliza para habilitar las transiciones entre los distintos segmentos de movimiento: por ejemplo, iniciar el izaje recién cuando el carro ha llegado a destino y el balanceo se ha disipado.

Carro — Trolley_Movement_Flags.

Tiene tres estados. El estado inicial es **Trolley_Moving** (`trolley_moving = true`). Cuando el error de posición y la velocidad estimada son suficientemente pequeños ($|x_{t,est} - x_{t,obj}| \leq 0,03$ m y $|v_{t,est}| \leq 0,05$ m/s) se transiciona a **Trolley_Stopped**. Desde allí, si el balanceo se estabiliza ($|\omega_{l,sensor}| \leq 0,005$ rad/s y $|\theta_{l,sensor}| \leq 0,005$ rad) el sistema avanza a **Trolley_Stopped_Complete** (`trolley_stopped_complete = true`), señalizando que el carro está detenido y el péndulo amortiguado. Si el balanceo vuelve a crecer ($|\omega_l| > 0,01$ o $|\theta_l| > 0,01$) se retorna a **Trolley_Stopped**. En cualquier estado detenido, si el error de posición o la velocidad superan los umbrales de retorno (0,1 m o 0,15 m según el estado) el autómata vuelve a **Trolley_Moving**.

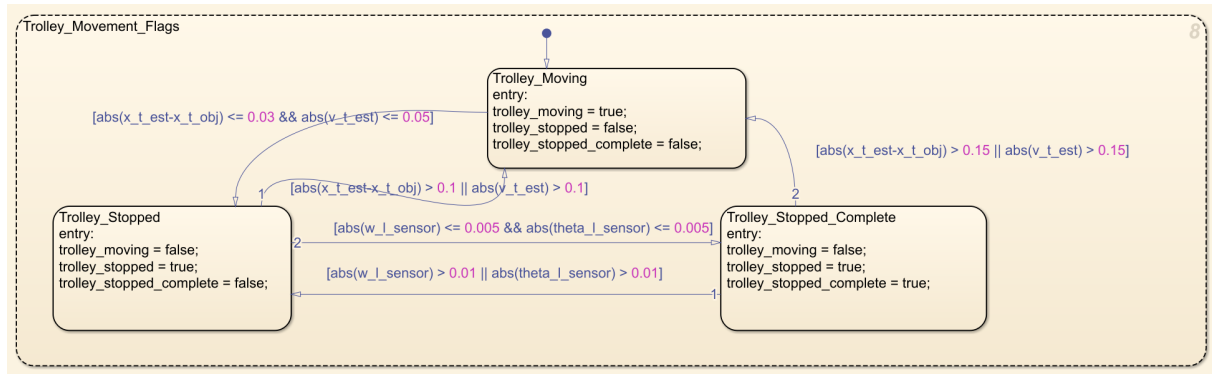


Figura 52: Subestado Trolley_Movement_Flags: detección de carro detenido con condición de balanceo amortiguado.

Izaje — Hoist_Movement_Flags.

Tiene dos estados: Hoist_Moving (inicial) y Hoist_Stopped. La transición a detenido ocurre cuando el error de longitud de cable es menor a 0,05 m y la velocidad estimada de izaje $|\dot{l}_{h,est}| \leq 0,05$ m/s. El regreso a Hoist_Moving se produce si el error supera 0,1 m o la velocidad 0,1 m/s, asegurando histéresis en la detección del estado detenido.

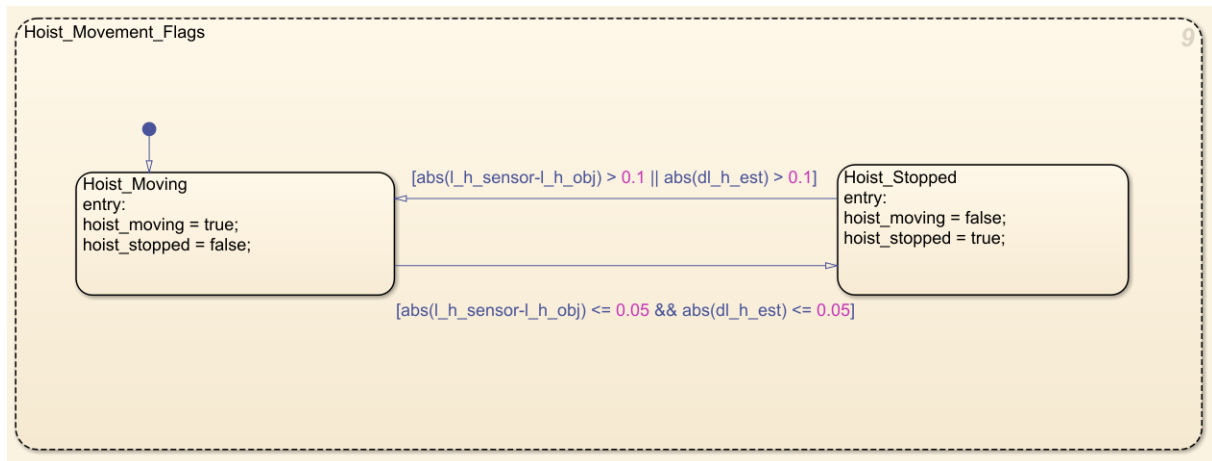


Figura 53: Subestado Hoist_Movement_Flags: detección de izaje detenido.

3.3.11. Agrupador de alarmas — Any_Alarms

El subestado paralelo Any_Alarms agrupa todas las alarmas activas del sistema en una única señal booleana (alarms), simplificando las condiciones de transición en el resto del autómata. El estado Alarms (alarms = true) se activa cuando cualquiera de las alarmas de fines de carrera de operación o la señal de sobrecarga está presente:

$$FC_t_min_alarm \vee FC_t_max_alarm \vee FC_h_min_alarm \vee FC_h_max_alarm \vee overload$$

La vuelta a No_Alarms (alarms = false) ocurre únicamente cuando todas esas condiciones se han desactivado simultáneamente. De este modo, en lugar de evaluar cada

alarma de forma individual en cada transición, el resto del autómata supervisora sólo necesita consultar la señal `alarms`.

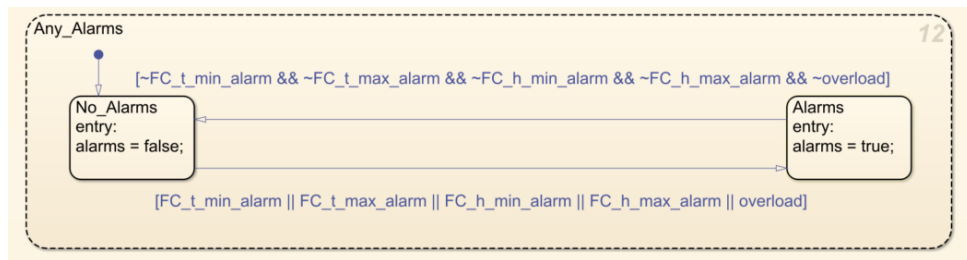


Figura 54: Subestado `Any_Alarms`: agrupación de alarmas de fines de carrera y sobrecarga en una señal booleana unificada.

3.3.12. Planificador de trayectorias

El subestado `Trajectory_Planner` coordina la secuencia completa de movimientos del ciclo operativo. La generación de las trayectorias de referencia en sí se describe en la sección siguiente; aquí se presenta la estructura del planificador y la lógica de escaneo LIDAR, retorno a *homing* y selección de escenario.

Escaneo LIDAR y retorno a *homing*.

Al ingresar al planificador se ejecutan tres fases secuenciales de escaneo:

1. **Line_Scan_Rise**: el spreader sube hasta la altura segura calculada por el mapa LIDAR ($l_{h,obj} = l_{h,safe}$). La trayectoria de izaje se genera con la función `generador_trayectoria_izaje` inicializada en la entrada al estado y actualizada en cada paso. La transición al siguiente estado ocurre cuando el izaje se detiene o cuando el progreso supera el 80 %.
2. **Line_Scan_Move**: el carro recorre en dirección $+x$ hasta la posición $x_{t,obj} = 49,5\text{ m}$, mientras el LIDAR construye el mapa de alturas del barco. Las trayectorias de carro e izaje se actualizan en paralelo en cada paso. La transición ocurre cuando el carro llega a destino (`trolley_stopped`).
3. **Line_Scan_Return**: el carro regresa a la posición de *homing* ($x_{t,obj} = -29,5\text{ m}$). Al detenerse tanto el carro como el izaje (`trolley_stopped && hoist_stopped`), el planificador transiciona a `Scenario_Selector`.

Selección de escenario.

Una vez completado el retorno, `Scenario_Selector` inicializa la ganancia del controlador de balanceo (`sway_ctr_gain = 1`) y determina el ciclo a ejecutar según la variable `escenario`:

- **Escenario 0**: el sistema permanece en `Waiting` ($v_{t,ref} = 0$, $dl_{h,ref} = 0$), sin ejecutar ningún ciclo.
- **Escenario 1**: ciclo de descarga simple — el carro toma un contenedor de tierra y lo deposita en el barco.

- **Escenario 2:** ciclo de carga simple — el carro recoge un contenedor del barco y lo deposita en tierra.
- **Escenario 3:** ciclo doble — combina una operación de carga y una de descarga en un mismo recorrido.

Cuando `escenario` $\neq 0$, el planificador ingresa al estado `Pickup_And_Place_Loop`, que implementa el ciclo operativo correspondiente. Si durante la ejecución se activa cualquier alarma de fin de carrera, el planificador aborta el ciclo y retorna a `Scenario_Selector`. Su lógica se describe en detalle en la sección siguiente.

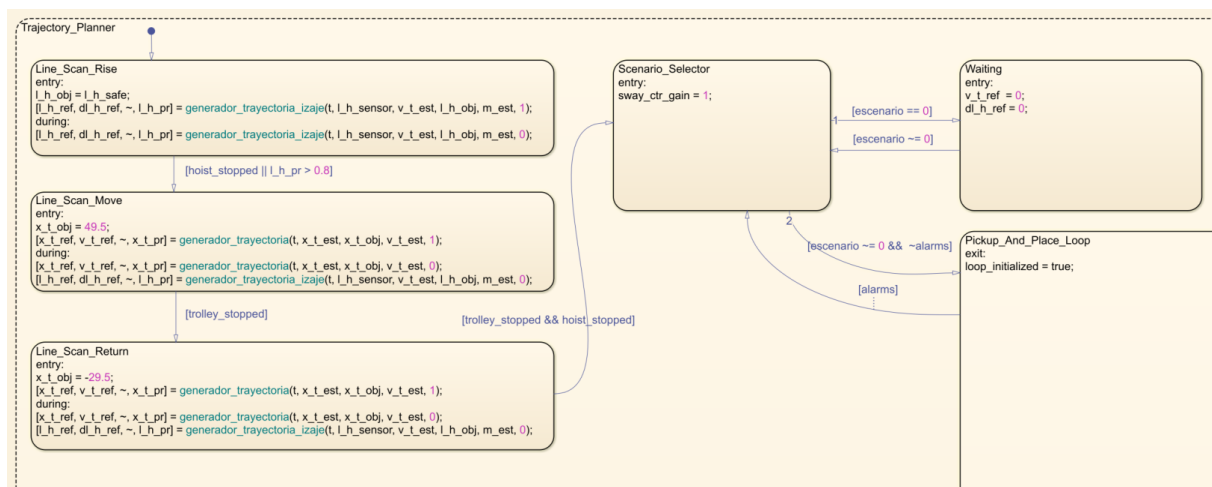


Figura 55: Estructura general del `Trajectory_Planner`: selección de escenario, escaneo LIDAR y retorno a *homing*.

3.3.13. Ciclo operativo — `Pickup_And_Place_Loop`

El estado `Pickup_And_Place_Loop` implementa el ciclo de carga y descarga de contenedores. Está compuesto por una secuencia de estados que encadenan los movimientos de carro e izaje para llevar el spreader desde la posición de origen hasta el destino, realizar la operación (tomar o depositar el contenedor) y regresar. La Figura 57 muestra la estructura completa del ciclo.

Al salir del estado `Pickup_And_Place_Loop` se registra `loop_initialized = true`. De este modo, si el ciclo es interrumpido (por ejemplo por una alarma de fin de carrera) y luego se vuelve a ingresar, el estado `Init` interno detecta esta variable y, en lugar de reiniciar desde el primer estado, delega la decisión a la **unión histórica** (*history junction*), que retoma la ejecución desde el último estado activo antes de la interrupción. Si `loop_initialized` es falso (primera entrada), `Init` transiciona a `Scenario_Selector` para inicializar el ciclo desde cero (Figura 56).

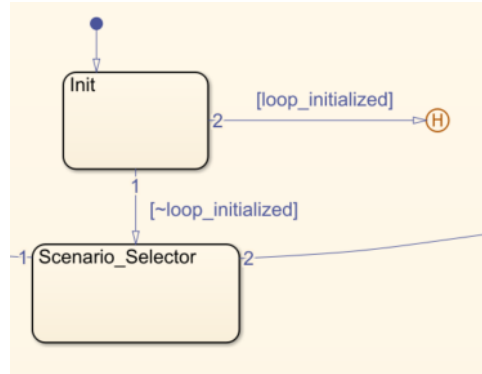


Figura 56: Lógica de Init dentro de Pickup_And_Place_Loop: reanudación por historia o inicialización según loop_initialized.

Las transiciones entre estados se basan en las **banderas de movimiento** descritas anteriormente: cuando un estado genera un movimiento de izaje, la transición al estado siguiente requiere `hoist_stopped`; cuando genera un movimiento de carro, requiere `trolley_stopped` o `trolley_stopped_complete` según corresponda. Esto garantiza que cada segmento del ciclo espera la condición de estaticidad del eje que estuvo en movimiento antes de continuar.

Escenario 1 — Descarga (tierra → barco).

El ciclo recoge un contenedor del camión en tierra y lo deposita en el barco:

1. **Truck_Pickup_Raise**: sube el spreader hasta altura de tránsito. Antes de ejecutar el movimiento se selecciona el camión destino: se recorre el vector de estado de camiones y se elige el primero que esté presente y con carga, calculando su posición como $x_{t,obj} = -25 + (i - 1) \times 5$ m.
2. **Truck_Pickup_Move**: desplaza el carro hasta la posición del camión seleccionado.
3. **Truck_Pickup_Descend**: baja el spreader hasta el contenedor.
4. **Truck_Pickup_Manual**: el sistema pasa a modo manual (`operation_mode = false`) y realiza la **estimación de masa** mientras el operador cierra los TLK. Durante este estado se acumulan muestras de la fuerza medida en el cable: cada muestra válida ($F_{hw,sensor} > 8000$ g) agrega al vector una estimación $\hat{m}_k = 2 F_{hw,sensor} / g$. Al salir del estado se llama a `estimar_masa`, que calcula la media de las muestras válidas con rechazo de *outliers* por criterio 2σ , obteniendo $\hat{m}$ para el limitador de velocidad por potencia del `rate_limiter`.
5. **Ship_Placement_Raise**: sube el contenedor hasta altura de tránsito.
6. **Ship_Placement_Move**: antes de trasladar, se selecciona la posición de destino en el barco como la pila con menor cantidad de contenedores apilados:

$$i^* = \arg \min_{i \in \{1, \dots, 19\}} \text{container_pillars}(i), \quad x_{t,obj} = x_{left} + (i^* - 1) \Delta x$$

Luego el carro se desplaza hasta esa posición.

7. **Ship_Placement_Descend**: la altura objetivo se calcula a partir del mapa LIDAR mediante la función `xpos2ypos`, que convierte la posición horizontal $x_{t,obj}$ al índice de grilla correspondiente y devuelve $l_{h,obj} = 45 - \text{lidar_map}(\text{idx})$, restando luego un margen de seguridad $l_{h,obj} = l_{h,obj} - \Delta_{safety}$. El spreader desciende hasta esa cota.
8. **Ship_Placement_Manual**: el operador abre los TLK para depositar el contenedor. El ciclo retorna a **Truck_Pickup_Raise** para la siguiente iteración.

Escenario 2 — Carga (barco $\rightarrow$ tierra).

El ciclo es el inverso del Escenario 1: el spreader se dirige primero al barco, donde se selecciona la pila con *mayor* cantidad de contenedores como posición de recogida (**Ship_Pickup**), y luego traslada el contenedor a tierra, donde se elige el camión vacío disponible como destino de depósito (**Truck_Placement**). La selección de posiciones y la estimación de masa siguen la misma lógica descripta anteriormente, adaptada al sentido inverso.

Escenario 3 — Ciclo doble (descarga + carga).

Combina ambos ciclos en un único recorrido: primero ejecuta el ciclo del Escenario 1 completo y luego, sin regresar a *homing*, ejecuta el ciclo del Escenario 2, completando un ciclo de ida y vuelta.

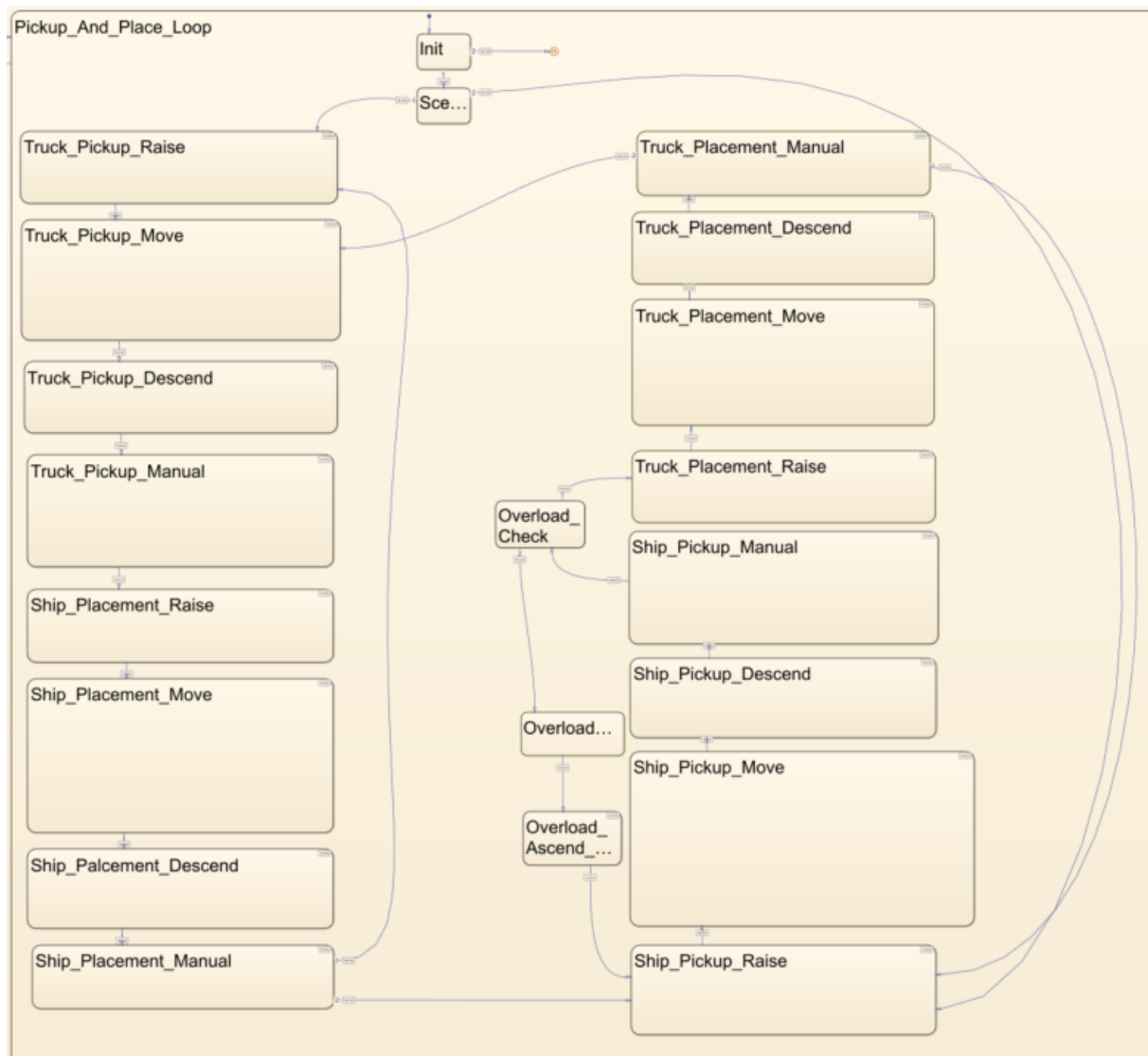


Figura 57: Estados del ciclo Pickup_And_Place_Loop.

3.3.14. Generación de trayectorias

Las referencias de posición y velocidad para el Nivel 2 se generan mediante perfiles de **perfil en S** (aceleración sinusoidal), que limitan el *jerk* —variación temporal de la aceleración— para reducir las excitaciones mecánicas sobre la estructura y el cable. Ambas funciones (`generador_trayectoria` para el carro y `generador_trayectoria_izaje` para el izaje) utilizan variables persistentes para congelar el perfil planificado: cuando se llaman con `NEW_TRAJ = 1` calculan y almacenan los parámetros de la trayectoria; en llamadas sucesivas con `NEW_TRAJ = 0` evalúan ese perfil congelado en función del tiempo real transcurrido, sin recalcular. Todas las funciones devuelven además una salida de **progreso** $p \in [0, 1]$, calculada como la fracción del tiempo total de la trayectoria ya transcurrido, que el planificador utiliza como condición de transición anticipada entre estados.

Trayectoria del carro.

El perfil de aceleración es sinusoidal:

$$a(t) = a_{max} \sin\left(\frac{\pi t}{t_a}\right)$$

con $a_{max} = 0,8 \text{ m/s}^2$ y $v_{max} = 4,0 \text{ m/s}$. Integrando se obtiene la velocidad y la posición:

$$v(t) = \frac{a_{max} t_a}{\pi} \left(1 - \cos \frac{\pi t}{t_a}\right), \quad x(t) = x_0 + \frac{a_{max} t_a}{\pi} \left(t - \frac{t_a}{\pi} \sin \frac{\pi t}{t_a}\right)$$

La duración de la fase de aceleración se obtiene imponiendo $v(t_a) = v_{max}$:

$$t_a = \frac{\pi v_{max}}{2 a_{max}}$$

El perfil completo tiene tres fases: aceleración ($0 \leq t < t_a$), crucero a v_{max} ($t_a \leq t < t_a + t_c$) y desaceleración simétrica ($t_a + t_c \leq t \leq t_{total}$). Si la distancia total D es insuficiente para alcanzar v_{max} , la velocidad pico se reduce a $v_{pico} = \sqrt{2 a_{max} D / \pi}$ y la fase de crucero desaparece.

Trayectoria del izaje.

El generador de izaje extiende el perfil anterior para contemplar dos aspectos adicionales:

- **Velocidad inicial no nula:** cuando se planifica una nueva trayectoria el mecanismo ya puede estar en movimiento con velocidad v_0 . Las fases de aceleración y deceleración son entonces asimétricas: la aceleración lleva el sistema de v_0 a v_p en tiempo $t_a = 2(v_p - v_0)/a_{max}$, mientras que la deceleración va de v_p a 0 en $t_d = 2v_p/a_{max}$.
- **Límite de velocidad por potencia constante:** la potencia disponible del accionamiento de izaje se modela como constante $P = m_{nom} g v_{h,load}$. La velocidad máxima admisible en función de la masa izada estimada $\hat{m}$ es:

$$v_{max}(\hat{m}) = \frac{P}{\hat{m} g} = \frac{m_{nom} v_{h,load}}{\hat{m}}$$

saturada entre $v_{h,load} = 1,5 \text{ m/s}$ (carga nominal) y $v_{h,noload} = 3,0 \text{ m/s}$ (sin carga). Esto garantiza que al aumentar la masa izada la trayectoria se adapta automáticamente para no superar la potencia disponible.

El perfil resultante mantiene la forma sinusoidal en aceleración y deceleración, con la integración ajustada para partir de $v_0 \neq 0$. Se implementan además casos especiales para distancias muy cortas o velocidades iniciales elevadas, donde el sistema pasa directamente a desacelerar sin fase de crucero.

Nota sobre movimiento sincronizado. En el ciclo operativo, el carro y el izaje pueden estar en movimiento simultáneo. Para coordinar ambos ejes sin imponer una sincronización rígida en tiempo, el planificador utiliza la variable de **progreso** $p \in [0, 1]$ como condición de habilitación de las transiciones entre estados: en lugar de esperar a que un eje llegue exactamente a su destino (condición de estaticidad estricta), la transición puede habilitarse en cuanto el progreso de la trayectoria activa supera un umbral predefinido (por ejemplo $p > 0,8$). Esto suaviza las condiciones de transición y permite iniciar el siguiente segmento de movimiento antes de que el eje termine completamente su recorrido, evitando tiempos muertos entre fases y logrando un movimiento coordinado más fluido entre carro e izaje.

3.4. Nivel 0: Seguridad y Protección

El Nivel 0 implementa el sistema de **seguridad y protección** de la grúa STS mediante un autómata de estados discretos que opera con período de muestreo $T_{s0} = 20\text{ ms}$. A diferencia del Nivel 1, que coordina la secuencia operativa normal del sistema, el Nivel 0 es un autómata *separado e independiente*, que permanece en *standby* permanente sin intervenir en la operación habitual de la grúa. Su función es monitorear de forma continua las variables críticas de la planta y, ante la detección de fallas o estados indeseados, tomar el control del sistema y ejecutar las acciones de protección necesarias para llevarlo a un estado seguro de operación, con total independencia del estado en que se encuentre el Nivel 1 en ese instante.

La Figura 58 muestra el esquema del Nivel 0 visto desde el exterior, indicando sus señales de entrada y salida hacia el resto del sistema.

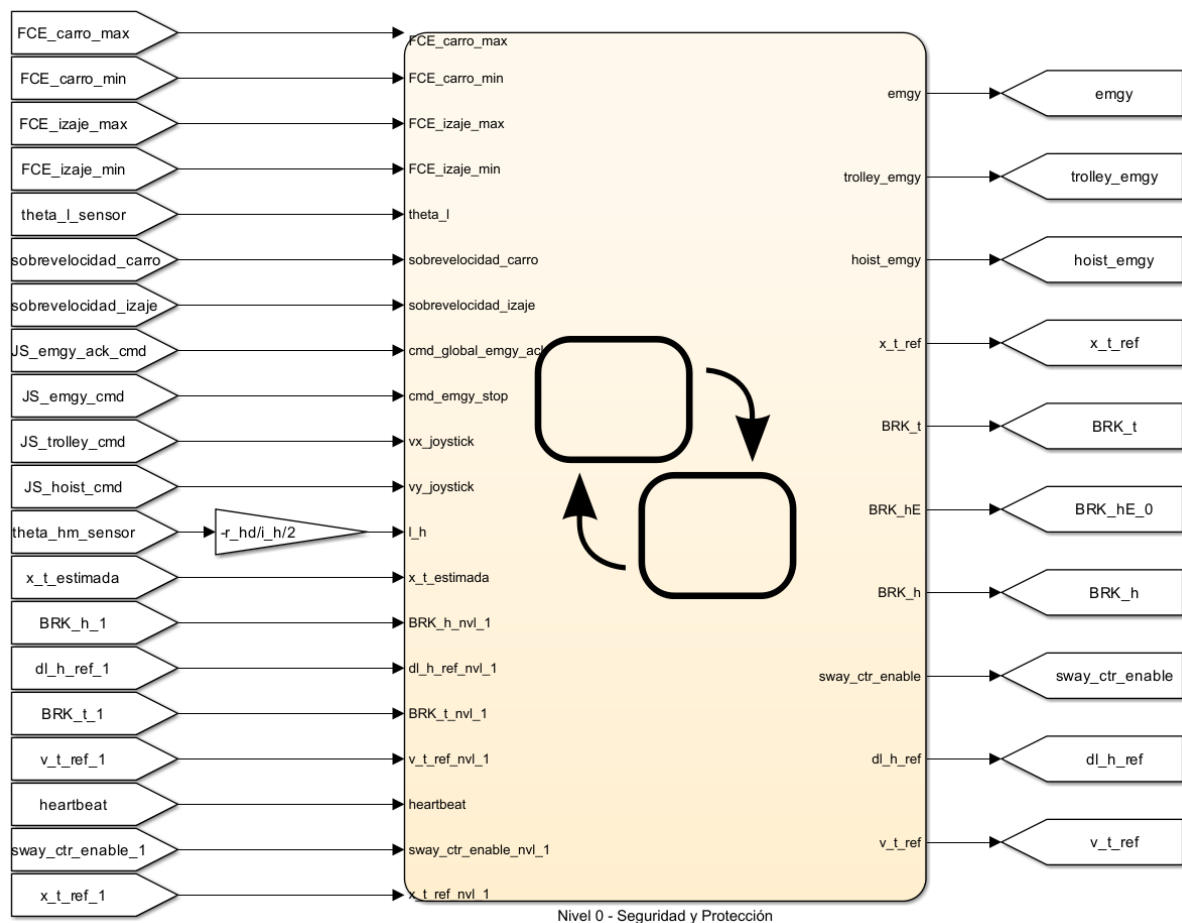


Figura 58: Nivel 0 visto desde el exterior: entradas y salidas del autómata de seguridad y protección (Stateflow).

Internamente, el Nivel 0 ejecuta dos estados **en paralelo** (Figura 59). El primero, **Emergencies**, se encarga de la supervisión y gestión de las emergencias del sistema, clasificándolas según su origen (pulsador de emergencia, fines de carrera de emergencia, detectores de sobrevelocidad, entre otras) y su nivel de criticidad, determinando con selectividad

jerárquica la acción de protección correspondiente a cada caso. El segundo, también en paralelo, es el **Watchdog**, encargado de monitorear la señal de vida (**heartbeat**) proveniente del Nivel 1, de modo de detectar la pérdida de comunicación o el bloqueo del autómata supervisor y escalar a un estado seguro cuando el Nivel 1 deja de responder.

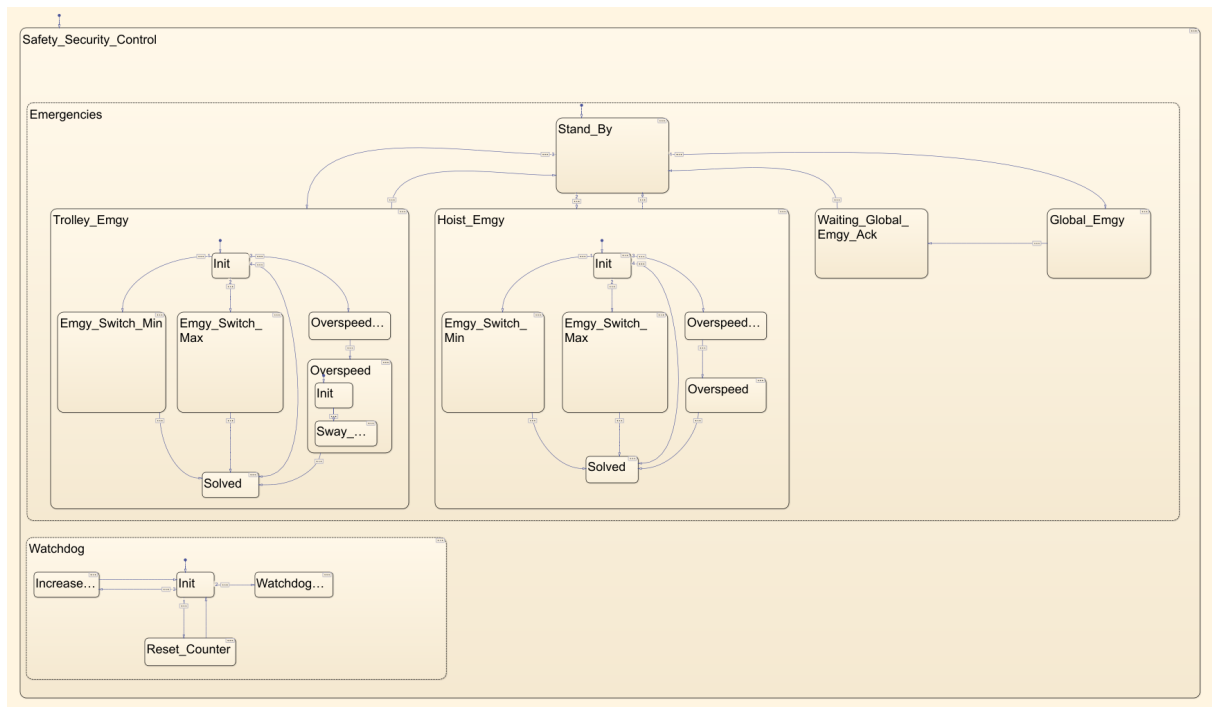


Figura 59: Diagrama de bloques del Nivel 0 completo: estados paralelos **Emergencies** y **Watchdog** (Stateflow, $T_{s0} = 20$ ms).

3.4.1. Emergencies

El estado **Emergencies** resuelve la supervisión y gestión de las emergencias del sistema. Su estado de entrada es **Stand_By**, en el cual el Nivel 0 deja pasar íntegramente las consignas provenientes del Nivel 1 hacia la planta: es decir, en operación normal el Nivel 0 actúa como un canal transparente que permite al Nivel 1 ejercer el control con total libertad. Esta arquitectura implica que es siempre el Nivel 0 quien determina si las salidas del Nivel 1 llegan o no a los accionamientos; ante cualquier emergencia, simplemente deja de retransmitir esas consignas y toma el control directo de las salidas. La acción de *entry* de **Stand_By** inicializa las banderas internas **hoist_solved** y **trolley_solved** en **false**. Durante la permanencia en este estado (*during*), el freno de emergencia de izaje se mantiene energizado de forma fija ($BRK_{hE} = \text{true}$), mientras que los frenos de traslación e izaje normal se retransmiten directamente desde el Nivel 1 ($BRK_t = BRK_{t,nvl1}$, $BRK_h = BRK_{h,nvl1}$). El autómata evalúa de forma continua las condiciones de transición, a la espera de que alguna de ellas se satisfaga para escalar al estado de emergencia correspondiente.

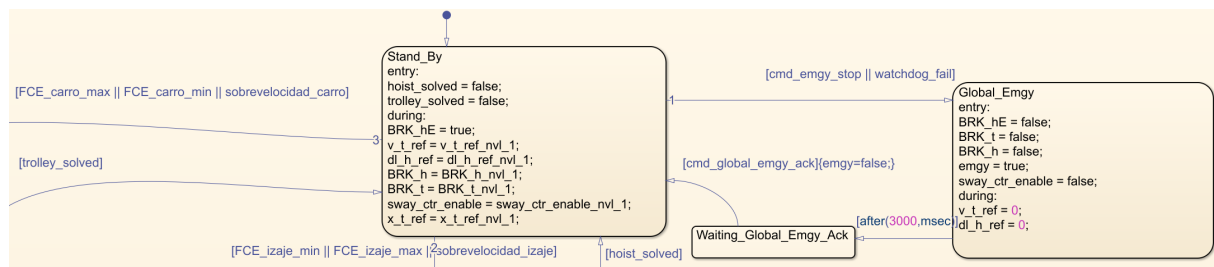


Figura 60: Estado Emergencies: estado de entrada Stand_By (Stateflow).

Desde **Stand_By** se definen tres transiciones, cada una activada por la disyunción de las condiciones que comparten un mismo origen físico, y cada una dirigida a un subestado de emergencia específico:

- $\text{FCE_trolley_max} \vee \text{FCE_trolley_min} \vee \text{sobrevelocidad_carro} \Rightarrow \text{Trolley_Emgy}$
- $\text{FCE_hoist_min} \vee \text{FCE_hoist_max} \vee \text{sobrevelocidad_izaje} \Rightarrow \text{Hoist_Emgy}$
- $\text{cmd_emgy_stop} \vee \text{watchdog_fail} \Rightarrow \text{Global_Emgy}$

Las señales $\text{FCE_trolley_min/max}$ y FCE_hoist_min/max corresponden a los **finés de carrera de emergencia**, ubicados 0,50 m más allá de los fines de carrera de operación atendidos por el Nivel 1 (Sección 2.4); su activación indica que el carro o la carga izada superaron el recorrido normal sin haber sido detenidos a tiempo por el control supervisor, por lo que el Nivel 0 toma el control de forma inmediata. Del mismo modo, $\text{sobrevelocidad_carro}$ y $\text{sobrevelocidad_izaje}$ son las salidas de los detectores de sobrevelocidad (45) de cada mecanismo.

La señal cmd_emgy_stop es una entrada al Nivel 0 proveniente de la HMI del operador, que le permite llevar al sistema a una emergencia global ante cualquier problema o falla que observe, con independencia de que alguna otra condición de emergencia se encuentre activa. La señal watchdog_fail , en cambio, no se origina en un sensor de planta sino en el estado paralelo **Watchdog**: se activa cuando este detecta un funcionamiento anómalo de la señal **heartbeat** del Nivel 1, y constituye la vía por la cual el **Watchdog** escala una falla del autómata supervisor hacia una emergencia global gestionada por **Emergencies**.

La Figura 61 muestra el estado **Emergencies** completo, incluyendo el estado **Stand_By** y las tres transiciones hacia los subestados de emergencia.

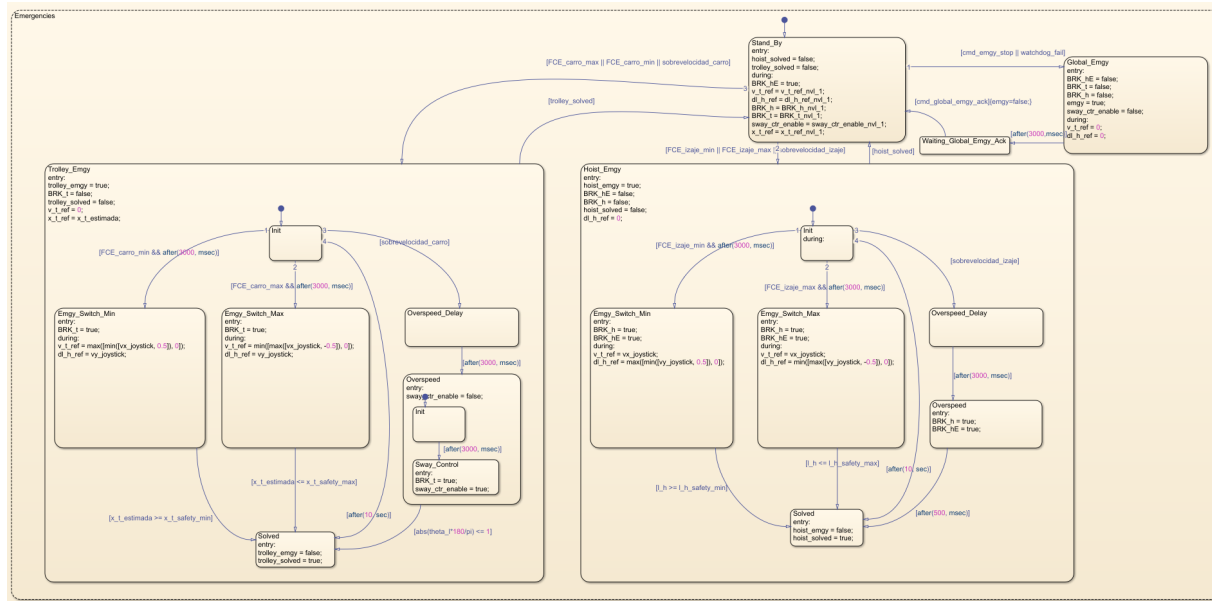


Figura 61: Estado Emergencias completo: Stand_By y transiciones hacia Trolley_Emg, Hoist_Emg y Global_Emg (Stateflow).

3.4.2. Subestado Trolley_Emg

Al ingresar a Trolley_Emg se ejecutan las siguientes acciones de entrada: se activa la variable de emergencia `trolley_emgy`, que informa al Nivel 1 la condición de falla; el Nivel 0 toma el control directo de la señal BRK_t y la desenergiza ($BRK_t = false$), aplicando el freno de traslación de (6) para detener el carro de forma inmediata; se inicializa `trolley_solved = false`; y se fija la consigna de posición al valor estimado actual ($x_t_ref = x_t_estimada$) y la consigna de velocidad a cero ($v_t_ref = 0$), congelando la referencia en el punto donde ocurrió la emergencia.

El estado de entrada es `Init`, encargado de seleccionar el origen de la emergencia entre los tres que activan Trolley_Emg (Sección 3.4.1):

- $FCE_carro_min \wedge after(3000, msec) \Rightarrow Emgy_Switch_Min$
- $FCE_carro_max \wedge after(3000, msec) \Rightarrow Emgy_Switch_Max$
- $sobrevelocidad_carro \Rightarrow Overspeed_Delay$

Para las condiciones de fin de carrera, la espera de 3s está incorporada en la propia condición de transición, permitiendo que el carro termine de detenerse antes de continuar. Para la sobrevelocidad, en cambio, la transición hacia `Overspeed_Delay` es inmediata y es ese estado intermedio quien impone los 3s de espera (`after(3000, msec)`) antes de pasar a `Overspeed`.

Los subestados `Emgy_Switch_Min` y `Emgy_Switch_Max` implementan la misma lógica de forma espejada. En su acción de entrada se energiza el freno ($BRK_t = true$), liberándolo para permitir el reposicionamiento manual del carro. Durante la permanencia en estos subestados (*during*), la referencia de velocidad del carro v_t_ref toma valores acotados y de signo único: solo positivos en `Emgy_Switch_Min`, alejando al carro del extremo mínimo, y solo negativos en `Emgy_Switch_Max`, alejándolo del extremo máximo:

Emgy_Switch_Min: $v_{t_ref} = \max(\min(vx_joystick, 0.5), 0)$

Emgy_Switch_Max: $v_{t_ref} = \min(\max(vx_joystick, -0.5), 0)$

La consigna de velocidad de izaje dl_h_ref se libera al joystick ($vy_joystick$), dado que la emergencia es exclusiva del mecanismo de traslación. La transición hacia **Solved** evalúa que la posición estimada $x_t_estimada$ haya superado una posición de seguridad interior al recorrido ($x_t_safety_min$ o $x_t_safety_max$ según corresponda).

El camino de sobrevelocidad llega a **Overspeed** tras los 3s en **Overspeed_Delay**. La acción de entrada de **Overspeed** desactiva el controlador de balanceo ($sway_ctr_enable = false$) para evitar que interfiera durante la transición. Internamente, **Overspeed** contiene su propio **Init** que, tras otros 3s adicionales, pasa a **Sway_Control**: allí se energiza el freno ($BRK_t = true$) y se rehabilita el controlador de balanceo ($sway_ctr_enable = true$) para disipar la oscilación residual inducida por la frenada brusca. Desde **Sway_Control**, la transición hacia **Solved** se ejecuta cuando el ángulo de balanceo verifica $|\theta_t| < 1^\circ$, o bien de forma incondicional tras 10s (**after(10,sec)**) como salvaguarda temporal.

En el subestado **Solved**, común a los tres caminos anteriores, se resetea **trolley_emgy** y se activa **trolley_solved**, habilitando la transición externa de regreso a **Stand_By**.

La Figura 62 muestra el subestado **Trolley_Emgy** completo con todos sus estados internos y transiciones.

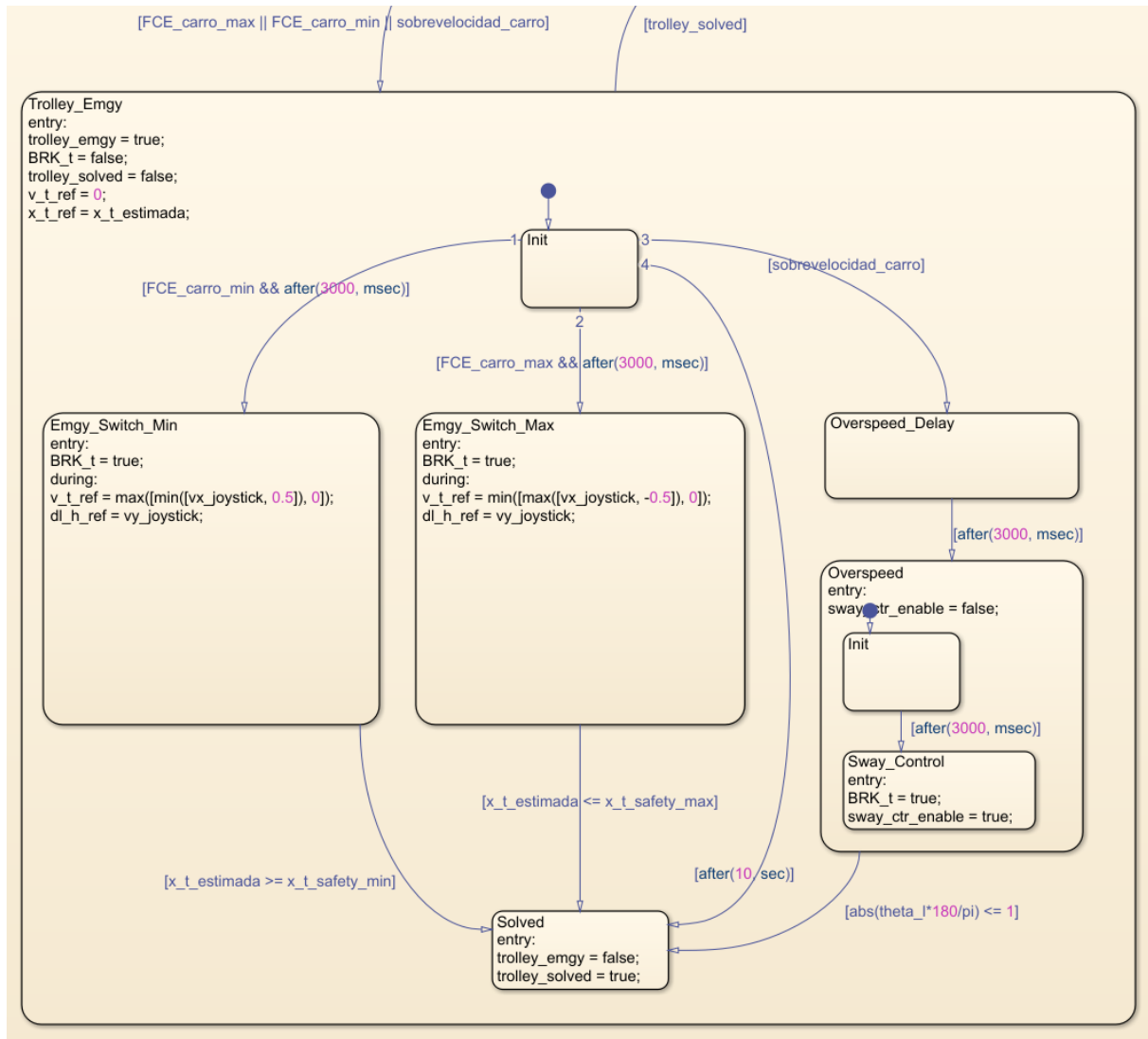


Figura 62: Subestado Trolley_Emg completo: Init, Emgy_Switch_Min, Emgy_Switch_Max, Overspeed_Delay, Overspeed y Solved (Stateflow).

3.4.3. Subestado Hoist_Emg

El subestado Hoist_Emg es análogo a Trolley_Emg pero aplicado al mecanismo de izaje. Al ingresar se ejecutan las acciones de entrada: se activa `hoist_emgy` para informar al Nivel 1 la condición de falla; el Nivel 0 desenergiza ambos frenos del izaje ($BRK_h = \text{false}$, $BRK_{hE} = \text{false}$) para detener el tambor de forma inmediata mediante la acción conjunta del freno de operación (25) y del freno de emergencia (27); se inicializa `hoist_solved = false`; y se fija la consigna de velocidad de cable a cero (`dl_h_ref = 0`).

El estado de entrada es `Init`, que selecciona el origen de la emergencia:

- $\text{FCE_izaje_min} \wedge \text{after}(3000, \text{msec}) \Rightarrow \text{Emgy_Switch_Min}$
- $\text{FCE_izaje_max} \wedge \text{after}(3000, \text{msec}) \Rightarrow \text{Emgy_Switch_Max}$

- `sobrevelocidad_izaje` $\Rightarrow$ `Overspeed_Delay`

Al igual que en `Trolley_Emgy`, la sobrevelocidad transiciona inmediatamente a `Overspeed_Delay`, que impone los 3s de espera antes de pasar a `Overspeed`.

Los subestados `Emgy_Switch_Min` y `Emgy_Switch_Max` replican la lógica de `Trolley_Emgy` de forma espejada. En su acción de entrada se energizan ambos frenos del izaje ($BRK_h = \text{true}$, $BRK_{hE} = \text{true}$), liberando el tambor para el reposicionamiento manual. Durante la permanencia (*during*), la referencia de velocidad del carro `v_t_ref` se libera directamente al joystick (`vx_joystick`) sin restricción de signo, dado que la emergencia es exclusiva del izaje, mientras que `dl_h_ref` se acota con signo único:

`Emgy_Switch_Min: dl_h_ref = max(min(vy_joystick, 0.5), 0)`

`Emgy_Switch_Max: dl_h_ref = min(max(vy_joystick, -0.5), 0)`

La transición hacia `Solved` evalúa que la longitud de cable medida `l_h` haya superado el umbral de seguridad correspondiente (`l_h_safety_min` o `l_h_safety_max`).

En `Overspeed`, la acción de entrada energiza ambos frenos del izaje ($BRK_h = \text{true}$, $BRK_{hE} = \text{true}$). A diferencia de `Trolley_Emgy`, no existe un subestado de control de balanceo: la transición hacia `Solved` se dispara tras 500ms (`after(500,msec)`), con un máximo de 10s como salvaguarda temporal.

En `Solved` se resetea `hoist_emgy` y se activa `hoist_solved`, habilitando la transición externa de regreso a `Stand_By`.

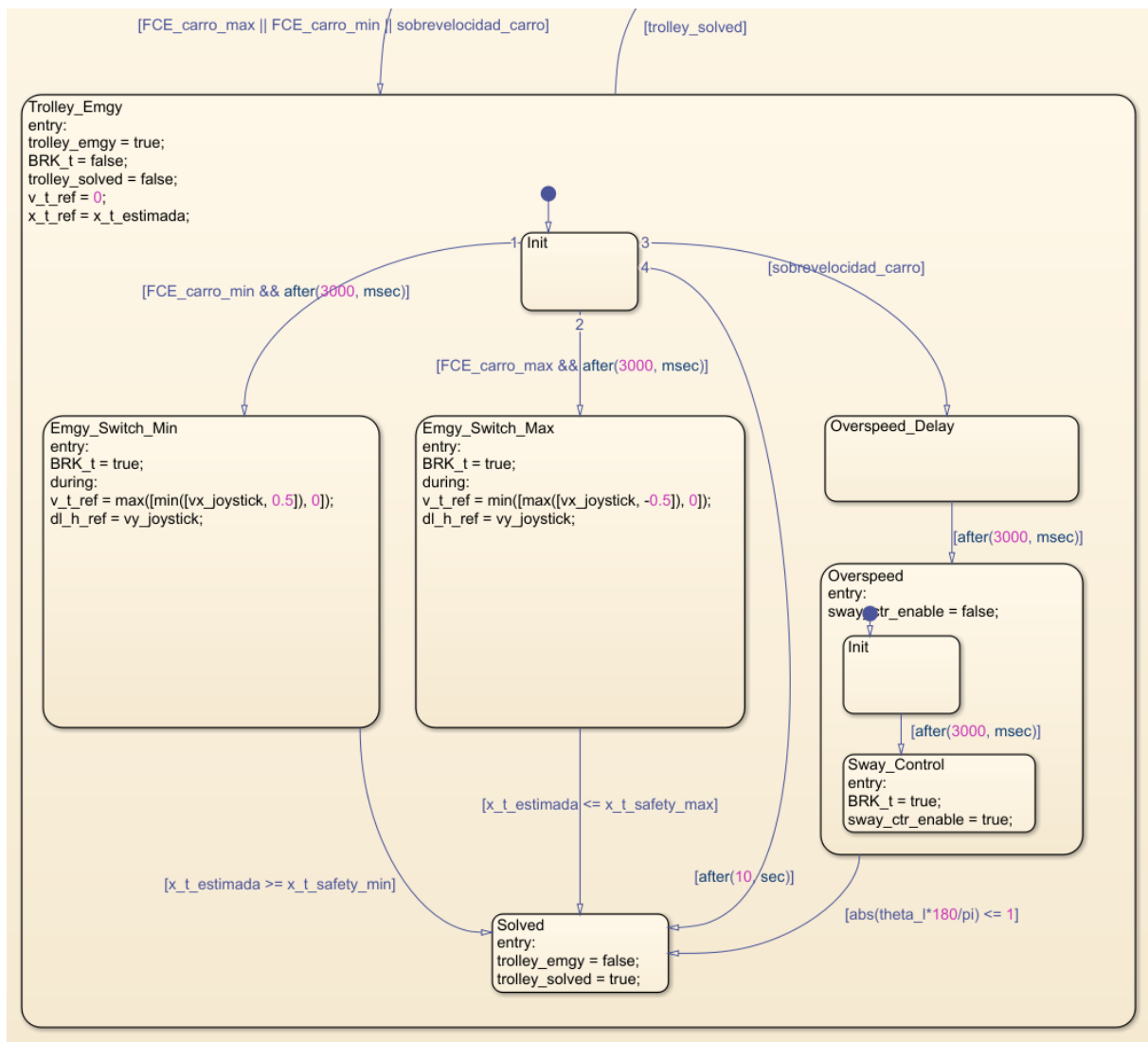


Figura 63: Subestado Hoist_Emg completo: Init, Emgy_Switch_Min, Emgy_Switch_Max, Overspeed_Delay, Overspeed y Solved (Stateflow).

3.4.4. Subestado Global_Emg

A diferencia de Trolley_Emg y Hoist_Emg, el subestado Global_Emg no posee subestados internos de selección de origen ni de reposicionamiento: se alcanza ante el pulsador de emergencia (`cmd_emgy_stop`) o una falla de `watchdog` (`watchdog_fail`), ambas de origen humano o de diagnóstico interno del propio Nivel 0, por lo que su recuperación requiere reconocimiento explícito del operador en lugar de una condición de posición segura evaluada automáticamente.

En su acción de entrada se desenergizan simultáneamente los tres frenos del sistema ($BRK_{hE} = \text{false}$, $BRK_t = \text{false}$, $BRK_h = \text{false}$), deteniendo de forma inmediata tanto el carro como el izaje; se activa la variable `emgy`; y se desactiva el controlador de balanceo (`sway_ctr_enable = false`). Durante la permanencia (*during*) se fuerzan a cero las referencias de velocidad de ambos mecanismos ($v_{t_ref} = 0$, $dl_{h_ref} = 0$), como resguardo adicional a nivel de consigna independiente del estado de los frenos.

Transcurridos 3 s (`after(3000,msec)`) se transiciona a `Waiting_Global_Emg_Ack`, subestado de espera en el que el sistema permanece detenido hasta que el operador reconoce la emergencia desde la HMI. Solo ante la señal `cmd_global_emgy_ack` se ejecuta la transición de regreso a `Stand_By`, con la acción asociada `{emgy = false;}` que desactiva la bandera de emergencia global antes de retornar al modo normal de operación.

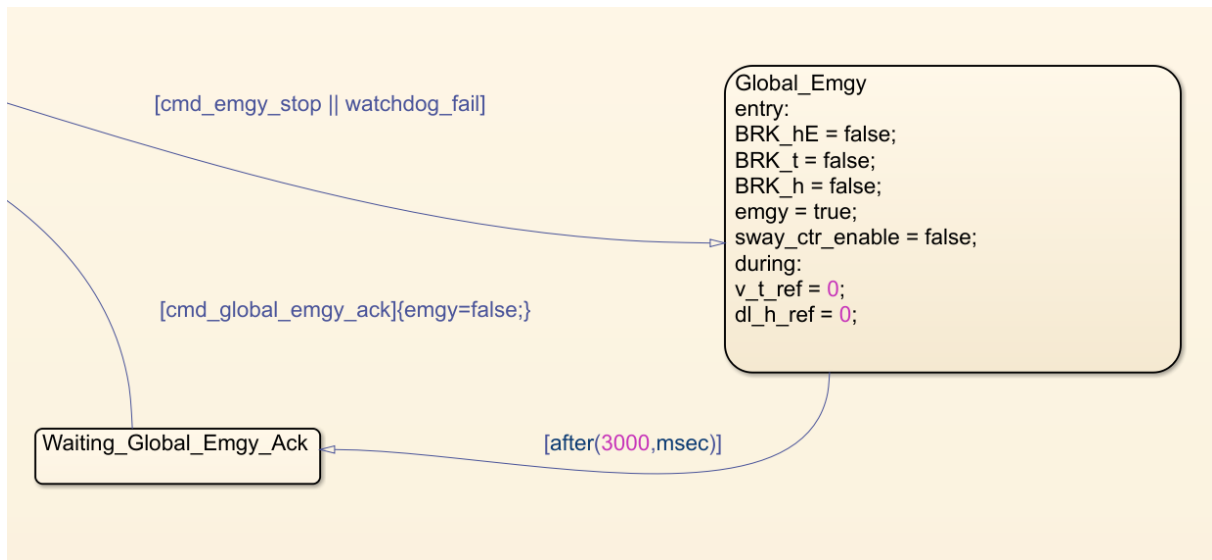


Figura 64: Subestado `Global_Emg` completo: acciones de entrada, espera de 3 s y reconocimiento del operador en `Waiting_Global_Emg_Ack` (Stateflow).

3.4.5. Watchdog

El estado `Watchdog` se ejecuta en paralelo a `Emergencies` (Sección 3.4.1) y es el encargado de supervisar la señal de vida (`heartbeat`) proveniente del Nivel 1, generando la variable `watchdog_fail` que escala a una emergencia global ante la pérdida de comunicación o el bloqueo del autómata supervisor. Su acción de entrada inicializa el contador de fallos y la referencia de comparación del `heartbeat`: `counter = 0` y `heartbeat_prev = 0`.

El estado de entrada es `Init`, en el cual se evalúan, con prioridad estricta ($1 > 2 > 3$), tres condiciones de transición:

- **Prioridad 1** — `heartbeat_prev ≠ heartbeat` ⇒ `Reset_Counter`: si el valor del `heartbeat` cambió respecto de la última verificación, evidenciando que el Nivel 1 continúa con vida, se resetea el contador de fallos (`counter = 0`) y se actualiza la referencia (`heartbeat_prev = heartbeat`), retornando de inmediato a `Init`.
- **Prioridad 2** — `counter ≥ 5` ⇒ `Watchdog_Emg`: si el contador alcanza 5 ciclos sin que el `heartbeat` haya cambiado, se activa `watchdog_fail = true`.
- **Prioridad 3** — `after(200,msec)` ⇒ `Increase_counter`: si transcurren 200 ms en `Init` sin que se satisfagan las dos condiciones anteriores, se incrementa el contador (`counter = counter+1`) y se retorna de inmediato a `Init`.

En conjunto, el `Watchdog` verifica cada 200 ms si el `heartbeat` del Nivel 1 cambió de valor desde la última lectura; si transcurren 5 verificaciones consecutivas ($5 \times 200 \text{ ms} = 1 \text{ s}$)

sin cambio, se considera que el Nivel 1 dejó de responder y se declara la falla. La variable `watchdog_fail` resultante es, junto con `cmd_emgy_stop`, una de las dos condiciones de entrada a `Global_Emg` (Sección 3.4.1).

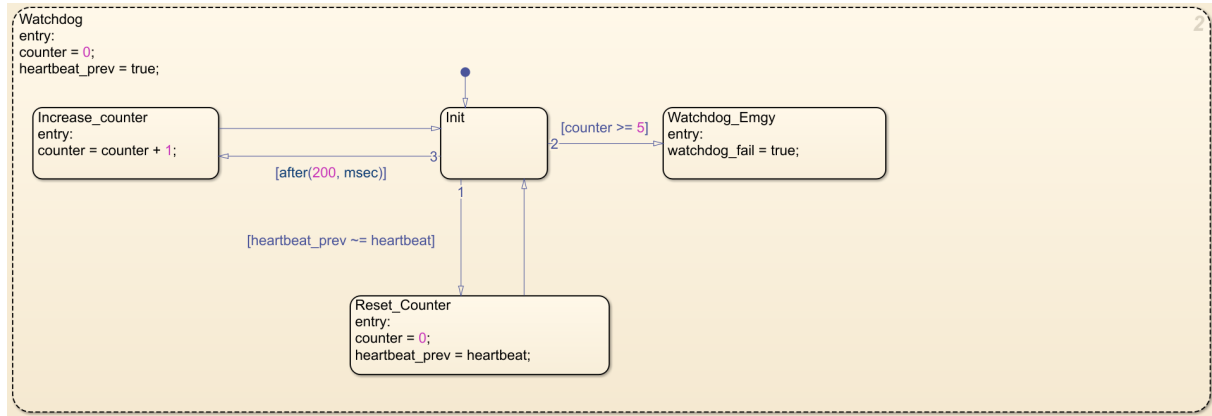


Figura 65: Estado Watchdog completo: Init, Reset_Counter, Increase_counter y Watchdog_Emg (Stateflow).

4. Implementación de los Niveles 0 y 1 en CODESYS

La implementación en hardware real de los Niveles 0 y 1 se realiza sobre un PLC virtual ejecutado en **CODESYS SP20 V3.5**, bajo el estándar IEC 61131-3 [5]. Ambos autómatas se codifican manualmente en *Sequential Function Chart* (SFC), complementado con bloques de *Structured Text* (ST) para las acciones de entrada/salida, temporizadores y condiciones de transición más complejas. La tarea cíclica que los ejecuta se configura con un período de $T_{s1} = T_{s0} = 20$ ms, acorde a la frecuencia de muestreo de ambos niveles de control. Esta elección respecto de la generación automática vía PLC Coder permite un control explícito sobre la estructura del programa y facilita la validación directa del comportamiento secuencial frente a la especificación del diseño.

La verificación del código implementado se lleva a cabo mediante una arquitectura *Hardware-in-the-Loop* (HIL) en la que CODESYS actúa como el controlador real y Simulink conserva el rol de planta y Nivel 2. La comunicación entre ambos entornos se establece mediante el protocolo **OPC UA** [6]: Simulink publica el estado de la planta (posiciones, velocidades, flags de sensores) como nodos OPC UA, y CODESYS los suscribe para calcular las salidas de los Niveles 0 y 1; simétricamente, CODESYS publica sus salidas discretas (consignas de frenos, twistlocks, habilitaciones) y Simulink las lee para cerrar el lazo de co-simulación. Esta configuración permite ejecutar los autómatas SFC en el entorno de PLC real mientras el modelo dinámico continuo de la grúa corre en Simulink, verificando así el comportamiento integrado del sistema con fidelidad equivalente a la prueba sobre hardware físico.

4.1. Estructura general del proyecto

El proyecto en CODESYS se organiza siguiendo la jerarquía de unidades de organización de programa (POU) definida por IEC 61131-3. La Figura 66 muestra el árbol de proyecto tal como queda configurado en el entorno de desarrollo.

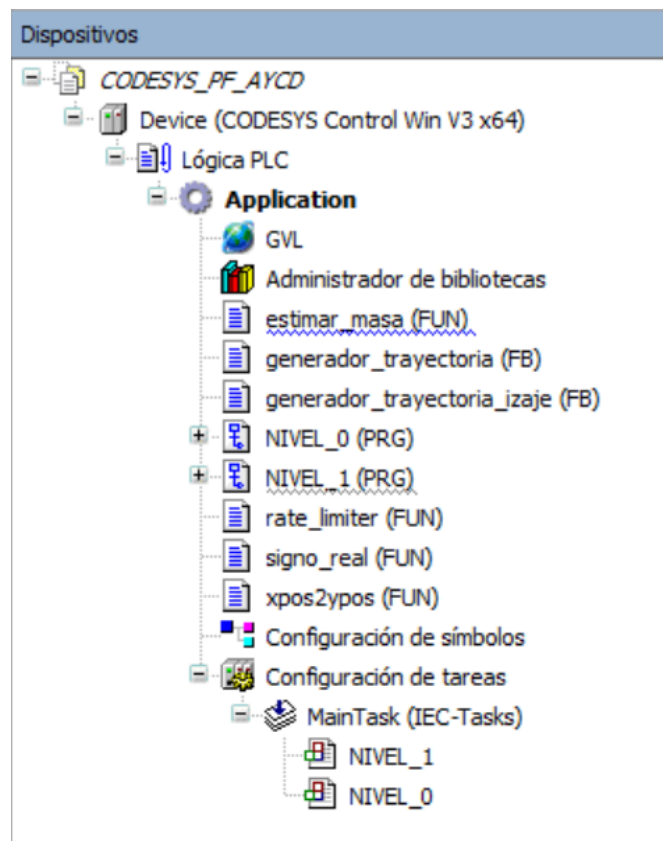


Figura 66: Árbol de proyecto CODESYS_PF_AYCD. Bajo *Application* se agrupan las variables globales (GVL), las funciones auxiliares (*estimar_masa*, *rate_limiter*, *signo_real*, *xpos2ypos*), los bloques de función de generación de trayectorias (*generador_trayectoria*, *generador_trayectoria_izaje*), y las POU tipo *Programa* *NIVEL_0* y *NIVEL_1* (cada una con acciones internas). La *Configuración de símbolos* habilita la interfaz OPC UA. En *Configuración de tareas*, *MainTask* ejecuta cíclicamente ambos programas.

En el nivel superior se encuentran las tareas cíclicas que ejecutan los programas principales. Cada uno de los autómatas está encapsulado en una POU de tipo *Programa*: *NIVEL_0* y *NIVEL_1*, ambas codificadas en SFC. Dentro de cada POU se definen **acciones** (bloques de código ST asociados a los pasos del SFC) que implementan la lógica específica de cada estado del autómata — entradas, salidas, condiciones de transición y llamadas a funciones auxiliares. Estas acciones se describen en detalle en las subsecciones siguientes.

Las variables globales compartidas entre ambos niveles se declaran en el módulo *GVL* (*Global Variable List*), organizado en grupos temáticos que reflejan la jerarquía funcional del sistema:

```

1 VAR_GLOBAL
2
3 (* -- 1. Constantes físicas y matematicas ----- *)
4 PI   : REAL := 3.14159265;
5 g    : REAL := 9.80665;
6
7 (* -- 2. Parametros mecanicos de los accionamientos ----- *)
8 r_td : REAL := 0.5;
9 i_t  : REAL := 30;
10 r_hd : REAL := 0.75;

```

```

11  i_h  : REAL := 22;
12
13  (* -- 3. Limites de posicion y fines de carrera ----- *)
14  Y_t0      : REAL := 45;
15  x_t_min   : REAL := -30;      x_t_max   : REAL := 50;
16  x_t_minE  : REAL := x_t_min - 0.5;
17  x_t_maxE  : REAL := x_t_max + 0.5;
18  y_h_min   : REAL := -20;      y_h_max   : REAL := 40;
19  y_h_minE  : REAL := y_h_min - 0.5;
20  y_h_maxE  : REAL := y_h_max + 0.5;
21  l_h_min   : REAL := Y_t0 - y_h_max;
22  l_h_max   : REAL := Y_t0 - y_h_min;
23  l_h_minE  : REAL := l_h_min - 0.5;
24  l_h_maxE  : REAL := l_h_max + 0.5;
25  x_t_safety_min : REAL := x_t_minE + 3;
26  x_t_safety_max : REAL := x_t_maxE - 3;
27  l_h_safety_min : REAL := l_h_minE + 3;
28  l_h_safety_max : REAL := l_h_maxE - 3;
29
30  (* -- 4. Condiciones iniciales ----- *)
31  x_t0      : REAL := -15;
32  theta_tm0 : REAL := (x_t0) * i_t / r_td;
33  x_l0      : REAL := x_t0;      y_l0 : REAL := 7;
34  l_h0      : REAL := Y_t0 - y_l0;
35  theta_hm0 : REAL := -l_h0 * i_h * 2 / r_hd;
36
37  (* -- 5. Geometria de contenedores ----- *)
38  H_c      : REAL := 2.59;
39  truck_pickup_height : REAL := 3.59;
40  pickup_safety_margin : REAL := 1.5 * H_c;
41  placement_safety_margin : REAL := 1.3 * H_c;
42
43  (* -- 6. Parametros del mapa Lidar (constantes) ----- *)
44  x_containers_left_lim : REAL := 1.5;
45  x_containers_right_lim : REAL := 48.5;
46  n_pilares      : INT := 19;
47  espaciado : REAL := (x_containers_right_lim - x_containers_left_lim) / (19 - 1);
48  lidar_grid_size : INT := 9001;
49  LIDAR_GRID_SIZE_C : INT := 9001;
50  dx_grid_lidar : REAL := 0.01;
51  x_c0_lidar : REAL := -35.0;
52  x_lidar_offset : REAL := 3.0;
53  v_t_max : REAL := 4.0;
54  T_s2 : REAL := 0.001;
55  l_h_safe : REAL := 0;
56
57  (* -- 7. Estado runtime del Lidar ----- *)
58  lidar_map : ARRAY[1..9001] OF REAL := [9001(0.0)];
59  window_idx : DINT := 0;
60  land_end_element : DINT := 1;
61  x_lidar : REAL := 0;      x_lidar_prev : REAL;
62  grid_element : DINT := 1;      grid_element_prev : DINT := 1;
63  y_c0_lidar : REAL := 0;
64  max_pillar_local : REAL := 0.0;
65  j_offset : DINT := 0;      cc : INT := 0;
66  max_lidar_map : REAL := 0.0;
67  pillar_height : REAL := 0.0;      containers_height : REAL := 0.0;
68  x_lidar_pillar : REAL := 0.0;
69  grid_element_pillar : DINT := 1;
70  window_start : DINT := 1;      window_end : DINT := 1;
71
72  (* -- 8. Estimacion de masa ----- *)
73  m_est_vec : ARRAY[1..500] OF REAL := [500(0.0)];
74  m_est_vec_pos : DINT := 1;
75  m_est : REAL := 15000.0;
76  min_containers : REAL := 999;
77  ship_idx : DINT := 0;
78
79  (* -- 9. Estado general del sistema ----- *)
80  t : REAL := 0;

```

```

81     escenario           : REAL := 1;
82     simulink_started    : BOOL := FALSE;
83     state_SISTEMA_ON    : BOOL := FALSE;
84     operation_mode      : BOOL := FALSE;
85     heartbeat           : BOOL := FALSE;
86     sway_ctr_enable     : BOOL := TRUE;
87     sway_ctr_gain       : REAL := 1;
88
89     (* -- 10. Cinematica estimada y sensada ----- *)
90     x_t_est             : REAL := x_t0;   v_t_est             : REAL := 0;
91     x_td_sensor         : REAL := 0.0;
92     l_h_sensor          : REAL := l_h0;   dl_h_est            : REAL := 0;
93     l_h_pr              : REAL := 0.0;   x_t_pr               : REAL := 0.0;
94     w_l_sensor          : REAL := 0;      theta_l_sensor       : REAL := 0;
95     w_tm_estimada       : REAL := 0;      w_hm_estimada       : REAL := 0;
96     F_hw_sensor         : REAL := 0;
97     trolley_moving      : BOOL := TRUE;
98     trolley_stopped     : BOOL := FALSE;
99     trolley_stopped_complete : BOOL := FALSE;
100    hoist_moving        : BOOL := TRUE;
101    hoist_stopped       : BOOL := FALSE;
102
103    (* -- 11. Referencias de trayectoria ----- *)
104    x_t_obj              : REAL := x_t0;   x_t_ref              : REAL := x_t0;
105    v_t_ref              : REAL := 0;
106    x_t_ref_nvl_1        : REAL := x_t0;   v_t_ref_nvl_1       : REAL := 0;
107    l_h_obj              : REAL := l_h0;   l_h_ref              : REAL := l_h0;
108    dl_h_ref             : REAL := 0;
109    l_h_ref_nvl_1        : REAL := l_h0;
110    dl_h_ref_nvl_1       : REAL := -theta_hm0 * r_hd / i_h / 2;
111    BRK_t_nvl_1          : BOOL := FALSE;
112    BRK_h_nvl_1          : BOOL := FALSE;
113    sway_ctr_enable_nvl_1 : BOOL := FALSE;
114
115    (* -- 12. Mando manual (joystick y rate limiter) ----- *)
116    JS_trolley_cmd       : REAL := 0;      JS_hoist_cmd        : REAL := 0;
117    v_t_ref_prev         : REAL := 0;      dl_h_ref_prev       : REAL := 0;
118    v_t_ref_a            : REAL := 0;      dl_h_ref_a          : REAL := 0;
119    T_s1                 : REAL := 0.02;
120    w_tm_min_BRK         : REAL := 0.01;   w_hm_min_BRK        : REAL := 0.01;
121
122    (* -- 13. Actuadores (frenos, TLK, sway) ----- *)
123    BRK_t                : BOOL := FALSE;   BRK_h                : BOOL := FALSE;
124    BRK_hE               : BOOL := FALSE;   TLK                  : BOOL := FALSE;
125    cmd_BRK_t            : BOOL := TRUE;
126    cmd_BRK_h            : BOOL := FALSE;
127    cmd_TLK              : BOOL := TRUE;
128    cmd_sway_ctr         : BOOL := FALSE;
129
130    (* -- 14. Nivel 0 - Seguridad y emergencia ----- *)
131    emgy                 : BOOL := FALSE;
132    FCE_carro_max        : BOOL := FALSE;
133    FCE_carro_min        : BOOL := FALSE;
134    sobrevelocidad_carro : BOOL := FALSE;
135    FCE_izaje_max        : BOOL := FALSE;
136    FCE_izaje_min        : BOOL := FALSE;
137    sobrevelocidad_izaje : BOOL := FALSE;
138    cmd_emgy_stop        : BOOL := FALSE;
139    cmd_global_emgy_ack  : BOOL := FALSE;
140    trolley_emgy         : BOOL := FALSE;
141    hoist_emgy           : BOOL := FALSE;
142
143    (* -- 15. Nivel 1 - Supervisor ----- *)
144    P_APN                : BOOL := FALSE;
145    FC_carro_max         : BOOL := FALSE;   FC_carro_min        : BOOL := FALSE;
146    FC_izaje_max         : BOOL := FALSE;   FC_izaje_min        : BOOL := FALSE;
147    FC_t_min_alarm       : BOOL := FALSE;   FC_t_max_alarm      : BOOL := FALSE;
148    FC_h_min_alarm       : BOOL := FALSE;   FC_h_max_alarm      : BOOL := FALSE;
149
150    (* -- 16. Camiones y barco ----- *)

```



```

151 trucks_state : ARRAY[1..5] OF BOOL;
152 trucks_has_load : ARRAY[1..5] OF BOOL;
153 TS_1 : BOOL := FALSE; TS_2 : BOOL := FALSE; TS_3 : BOOL := FALSE;
154 TS_4 : BOOL := FALSE; TS_5 : BOOL := FALSE;
155 THL_1 : BOOL := FALSE; THL_2 : BOOL := FALSE; THL_3 : BOOL := FALSE;
156 THL_4 : BOOL := FALSE; THL_5 : BOOL := FALSE;
157 container_pillars : ARRAY[1..19] OF INT := [19(0)];
158
159 (* -- 17. Iteradores ----- *)
160 i_lidar : DINT := 0; j_lidar : DINT := 0;
161 i_truck : INT := 0; i_m_est : DINT := 0; i_ship : DINT := 0;
162
163 (* -- 18. Bloques de funcion ----- *)
164 traj_trolley : generador_trayectoria;
165 traj_izaje : generador_trayectoria_izaje;
166
167 END_VAR

```

Listing 14: Variables globales (GVL) del proyecto CODESYS, organizadas por grupo funcional.

4.2. Nivel 0: Seguridad y Protección

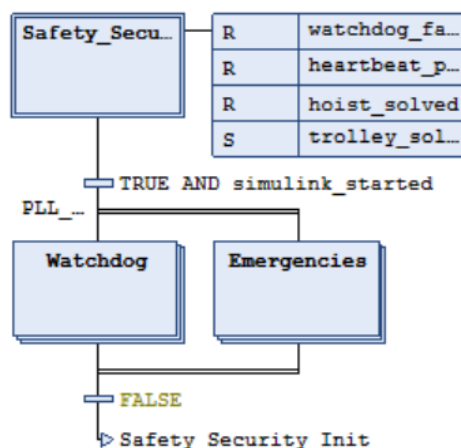


Figura 67: SFC del programa NIVEL_0 en CODESYS. El step inicial **Safety_Secu...** inicializa mediante acciones Reset/Set las variables de diagnóstico (**watchdog_failed**, **heartbeat_present**, **hoist_solved**, **trolley_solved**). La transición **TRUE AND simulink_started** habilita una rama AND que lanza en paralelo los pasos **Watchdog** y **Emergencies**, manteniendo ambas lógicas activas de forma concurrente durante toda la operación. Ante una condición **FALSE**, se ejecuta la acción **Safety_Security_Init** que reinicializa el nivel.

El SFC de NIVEL_0 arranca en el step **Safety_Security**, donde se inicializan por Reset (R) las flags de falla (**watchdog_failed**, **heartbeat_present**, **hoist_solved**) y por Set (S) la variable **trolley_solved**, estableciendo el estado seguro de partida. La transición de salida evalúa **TRUE AND simulink_started**, donde **simulink_started** es una variable booleana publicada por Simulink vía OPC UA que se activa al comenzar la simulación, señalizando a CODESYS que la planta está operativa. De este modo, el autómata de seguridad permanece bloqueado en el step inicial hasta recibir esa confirmación, evitando

que opere sobre una planta aún no inicializada. Al dispararse, el SFC ingresa en una **rama AND** que activa simultáneamente los **macropasos Watchdog** y **Emergencies**, los cuales encapsulan cada uno su propia secuencia interna y corren en paralelo de forma permanente durante toda la operación. Esta estructura refleja directamente el diseño del Statechart de Stateflow descrito en la sección 3.4.1.

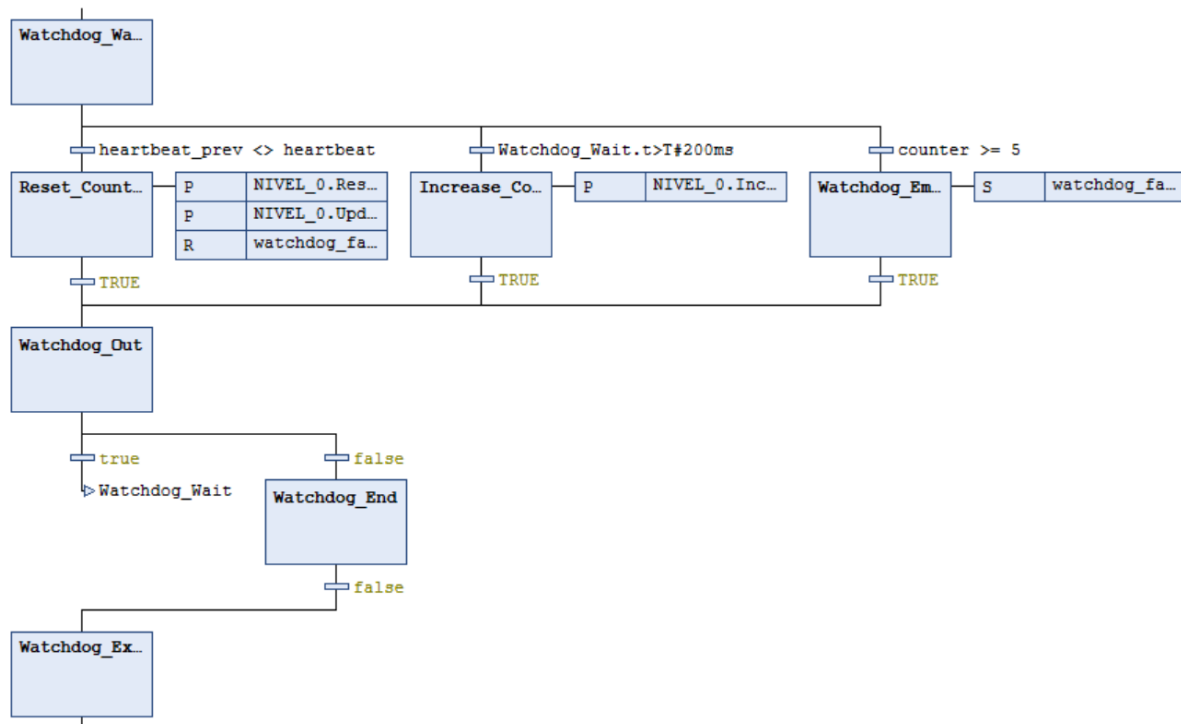


Figura 68: Macropaso Watchdog del programa NIVEL_0. Su funcionamiento es análogo al descrito para el estado homónimo en el Statechart de Stateflow (sección 3.4.1). Las acciones ResetCounter, UpdateHeartbeat e IncreaseCounter están implementadas en ST como acciones asociadas a los pasos de esta POU. Al final del macropaso se incorpora una rama OR con transición false hacia el step Watchdog_Exit, recurso requerido por IEC 61131-3 para que el compilador acepte el diagrama SFC como conforme a la norma.

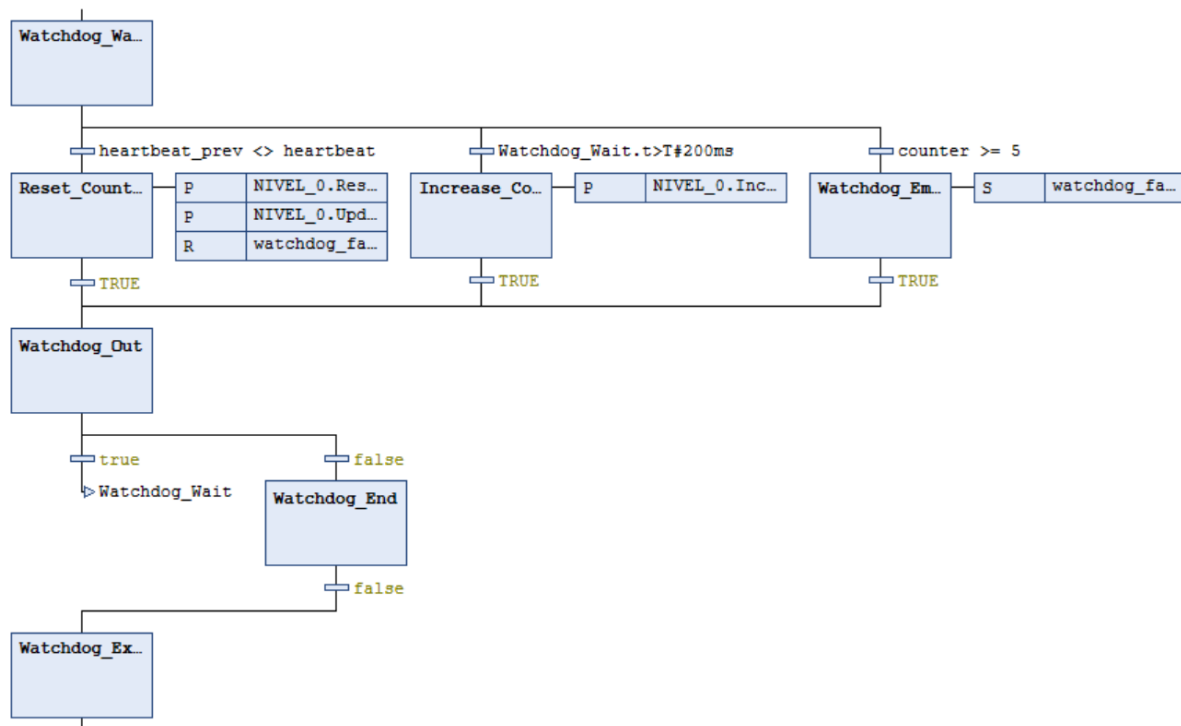


Figura 69: Macropaso Emergencias del programa NIVEL_0, análogo al diseño de Statflow de la sección 3.4.1. La acción LVL1PASS, con calificador N en el step Stand_By, permite el paso de las consignas provenientes del Nivel 1 (BRK_t, BRK_h, v_t_ref, x_t_ref, dl_h_ref, l_h_ref, sway_ctr_enable) mientras no haya falla activa. Las acciones ResetTrolleyRef y ResetHoistRef, activadas ante emergencia, sustituyen dichas consignas por velocidades nulas y referencias de posición iguales al valor actual del sensor o estimación, de modo que no induzcan movimiento sobre el eje afectado. Las tres acciones están implementadas en ST dentro de esta POU.

4.3. Nivel 1: Control Supervisor

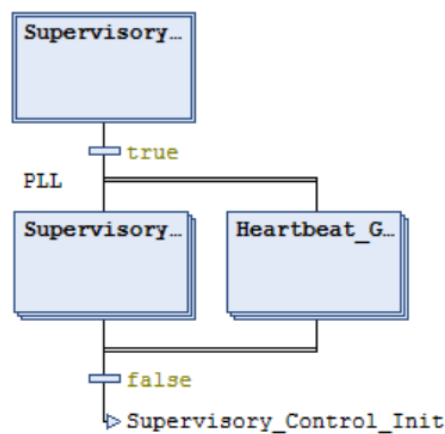


Figura 70: SFC del programa NIVEL_1 en CODESYS. El step inicial `Supervisory_Control` habilita, mediante transición `true`, una rama AND que activa en paralelo los macropasos `Heartbeat_Generator` y `Supervisory_Control`, los cuales permanecen activos de forma concurrente durante toda la operación. La rama OR con transición `false` y acción `Supervisory_Control_Init` provee el camino de salida requerido por IEC 61131-3 para conformidad del compilador.

La estructura es análoga a la del Nivel 0: un step inicial seguido de una **rama AND** que lanza dos macropasos en paralelo. `Heartbeat_Generator` produce el pulso de vida periódico que el autómata *Watchdog* del Nivel 0 monitorea, mientras que `Supervisory_Control` implementa la lógica de coordinación operativa descrita en la subsección 4.2.

4.3.1. Heartbeat Generator

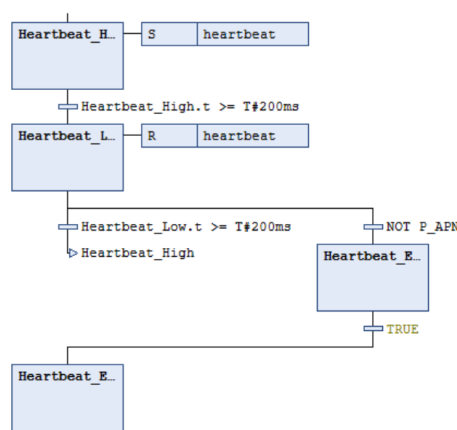


Figura 71: Macropaso `Heartbeat_Generator` del programa NIVEL_1, análogo al diseño de Stateflow de la sección 3.3.1. Al final se provee el camino de escape mencionado anteriormente, requerido por IEC 61131-3 para conformidad del compilador.

4.3.2. Supervisory Control

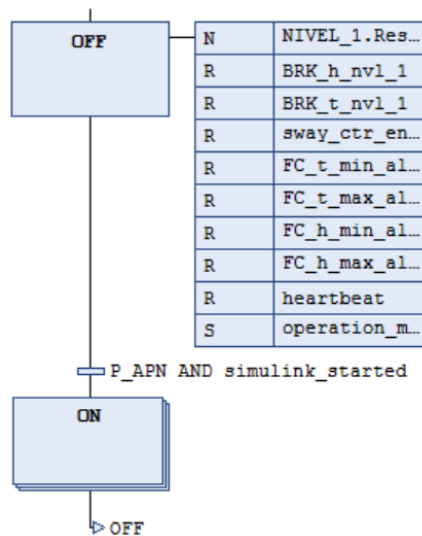


Figura 72: Macropaso Supervisory_Control del programa NIVEL_1. El step OFF inicializa las variables de control y alarma mediante acciones Reset/Set, y permanece activo hasta que se cumple $P_APN \text{ AND } \text{simulink_started}$. El macropaso ON encapsula la lógica de operación normal y retorna incondicionalmente a OFF al finalizar.

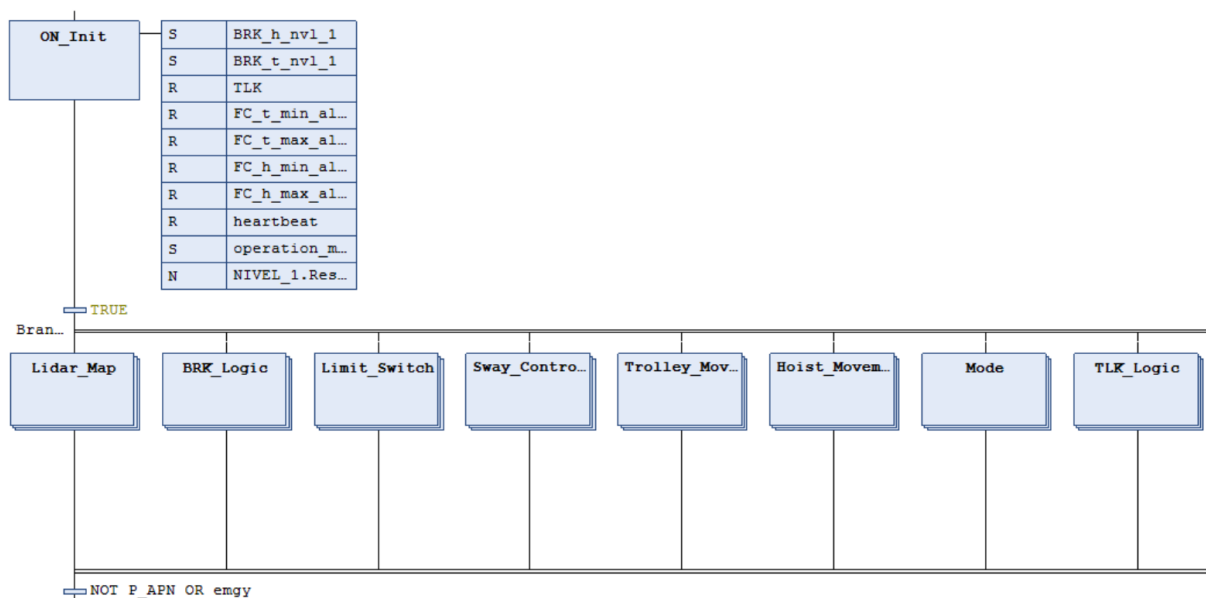


Figura 73: Interior del macropaso ON — rama AND izquierda. Tras el step de inicialización ON_Init, la rama AND lanza en paralelo los macropasos Lidar_Map, BRK_Logic, Limit_Switch, Sway_Control y Trolley_Movement, entre otros.

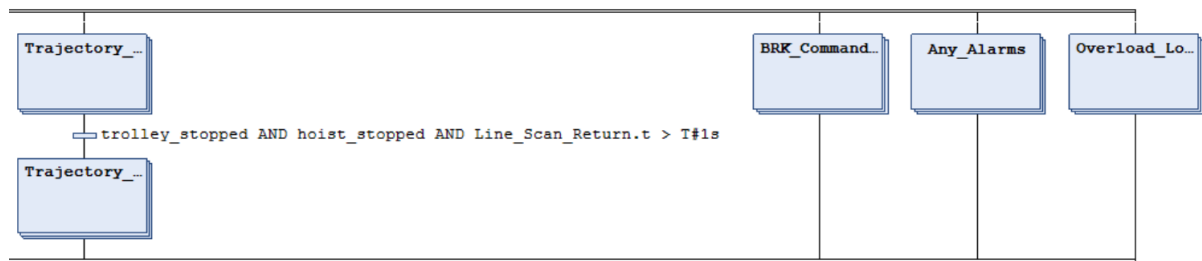


Figura 74: Interior del macropaso ON — rama AND derecha. Contiene los macropasos Hoist_Movement, Mode, TLK_Logic y Trajectory, ejecutándose todos de forma concurrente. La rama AND finaliza con la transición NOT P_APN OR emgy, que retorna el control al step OFF.

Lidar Map.

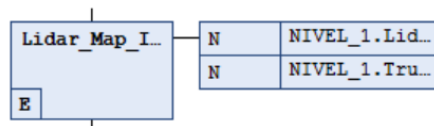


Figura 75: Macropaso Lidar_Map del programa NIVEL_1. Ejecuta las acciones LidarMapEntry (calificador N en el step de entrada E) y LidarMapDuring (calificador N en el step principal), análogas a las acciones *entry* y *during* del estado homónimo en Stateflow (sección 3.3.8), ambas implementadas en ST dentro de esta POU.

BRK Logic.

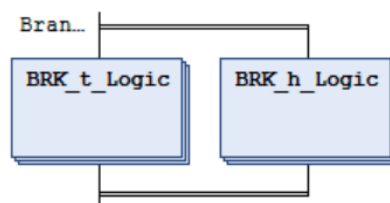


Figura 76: Macropaso BRK_Logic: rama AND que ejecuta en paralelo los macropasos BRK_t_Logic y BRK_h_Logic, uno por cada accionamiento.

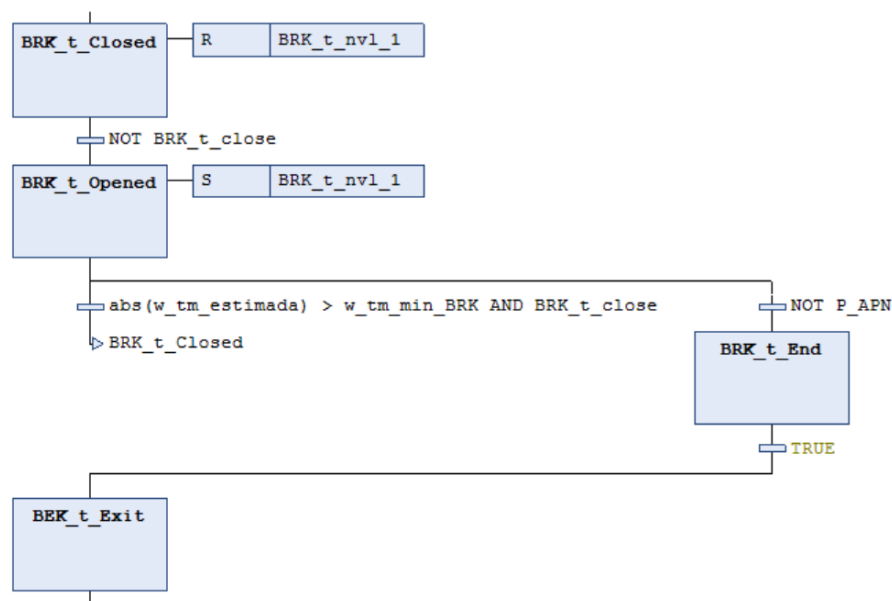


Figura 77: Interior del macropaso BRK_t_Logic. El step BRK_t_Closed mantiene el freno aplicado (R BRK_t_nv1_1) hasta que la posición estimada del carro se encuentra dentro de la zona segura y no hay fin de carrera activo, condición bajo la cual se habilita la apertura (BRK_t_Opened, S BRK_t_nv1_1). El freno se vuelve a aplicar si la velocidad supera el umbral $w_{tm_min_BRK}$ con un fin de carrera activo, retornando a BRK_t_Closed. La rama OR con NOT P_APN provee el camino de salida del macropaso. El caso del izaje (BRK_h_Logic) es enteramente análogo.

BRK Commander.

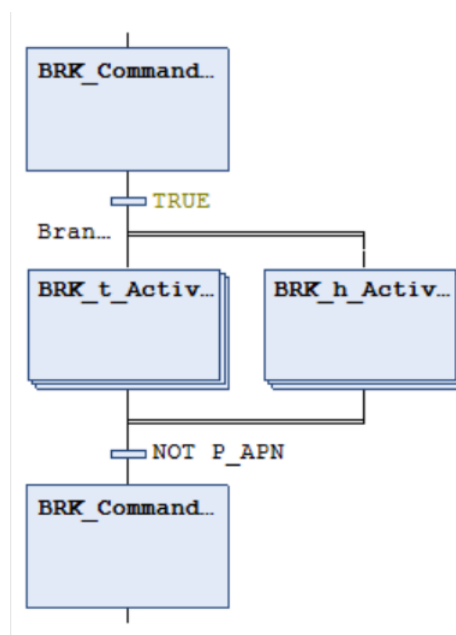


Figura 78: Vista superior del macropaso BRK_Commander: rama AND que lanza en paralelo BRK_t_Activation_Logic y BRK_h_Activation_Logic.

Al ingresar al macropaso se activa una rama AND que ejecuta en paralelo las dos lógicas de activación de frenos. La Figura 79 muestra el interior de BRK_t_Activation_Logic; el comportamiento de BRK_h_Activation_Logic es enteramente análogo. El comportamiento funcional de ambas lógicas es el descrito en la sección 3.3.3.

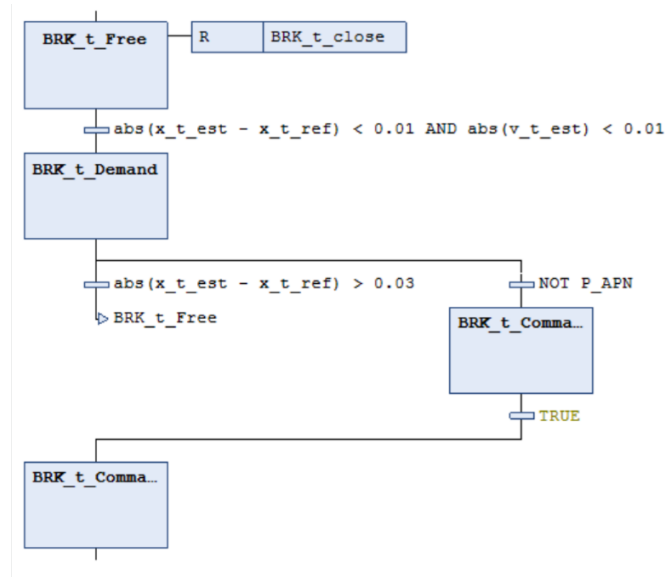


Figura 79: Interior de BRK_t_Activation_Logic: lógica de activación del freno del carro en CODESYS SFC.

Limit Switch.

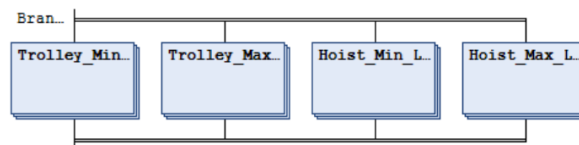


Figura 80: Macropaso Limit_Switch: rama AND con los macropasos Trolley_Min_LS, Trolley_Max_LS, Hoist_Min_LS y Hoist_Max_LS, uno por cada fin de carrera supervisado.

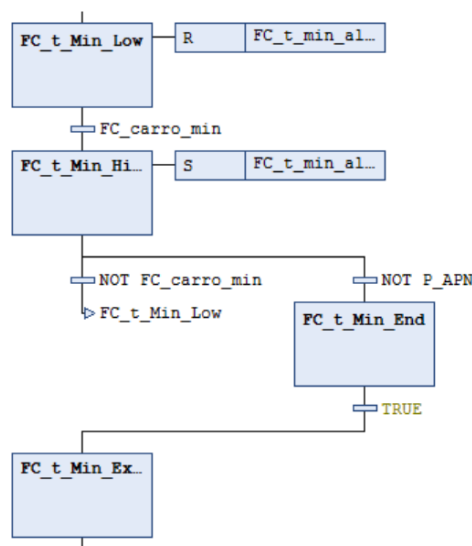


Figura 81: Interior del macropaso Trolley_Min_LS, representativo de los cuatro. Monitorea FC_carro_min para activar o limpiar la alarma FC_t_min_alarm (análogo a la lógica descrita en la sección 2.4). Los demás macropasos (Trolley_Max_LS, Hoist_Min_LS, Hoist_Max_LS) son enteramente análogos sobre sus respectivas señales.

Sway Control.

El comportamiento de este macropaso es el descrito en la sección 3.3.7.

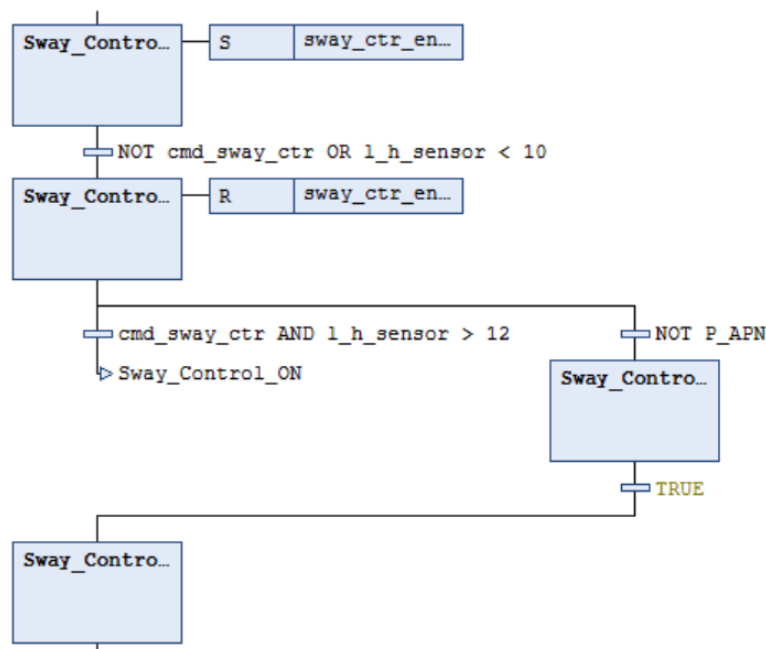


Figura 82: Macropaso Sway_Control_Mode en CODESYS.

Trolley Movement Flags.

El comportamiento de este macropaso es el descrito en la sección 3.3.10.

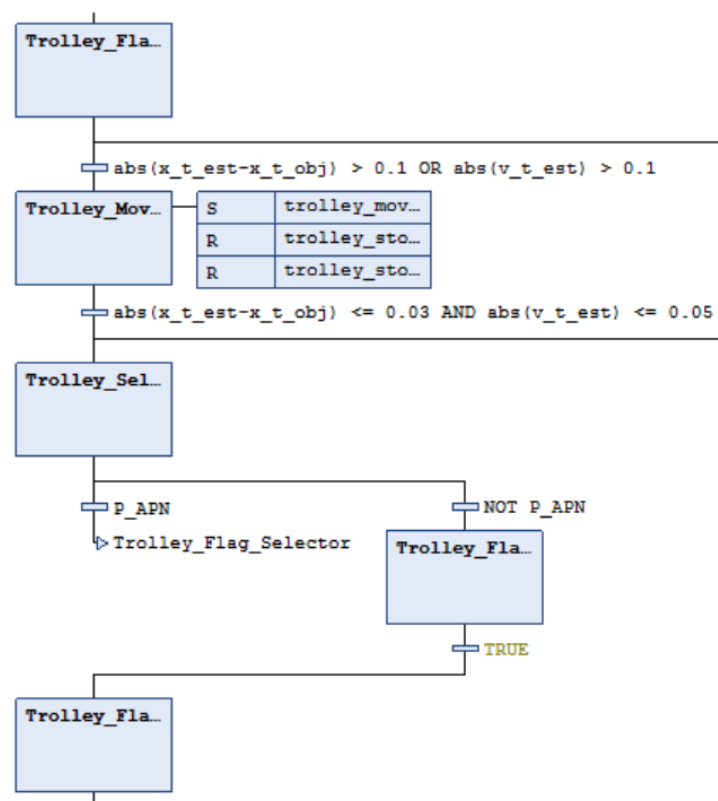


Figura 83: Macropaso Trolley_Movement_Flags en CODESYS: primer elemento de la rama OR (comportamiento descrito en la sección 3.3.10).

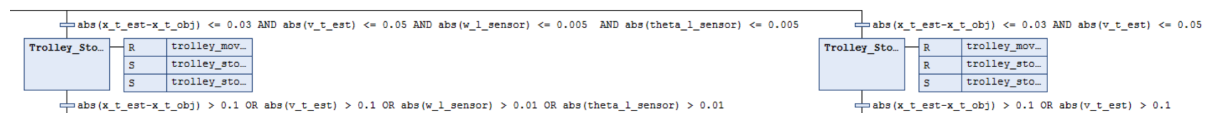


Figura 84: Macropaso Trolley_Movement_Flags en CODESYS: segundo y tercer elemento de la rama OR.

Hoist Movement Flags.

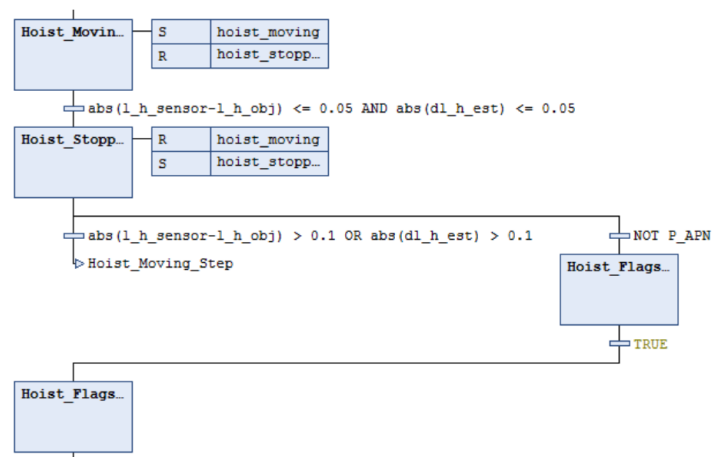


Figura 85: Macropaso Hoist_Movement_Flags en CODESYS (comportamiento descrito en la sección 3.3.10).

Mode.

El comportamiento de este macropaso es el descrito en la sección 3.3.5.

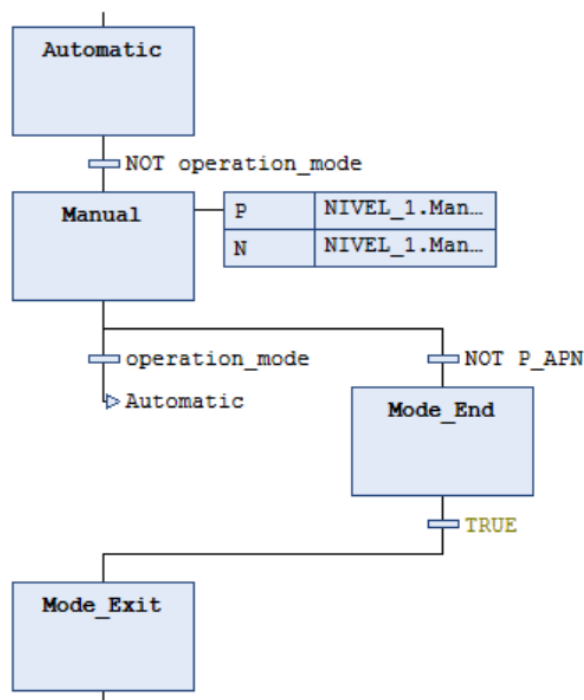


Figura 86: Macropaso Mode en CODESYS.

Overload Logic.

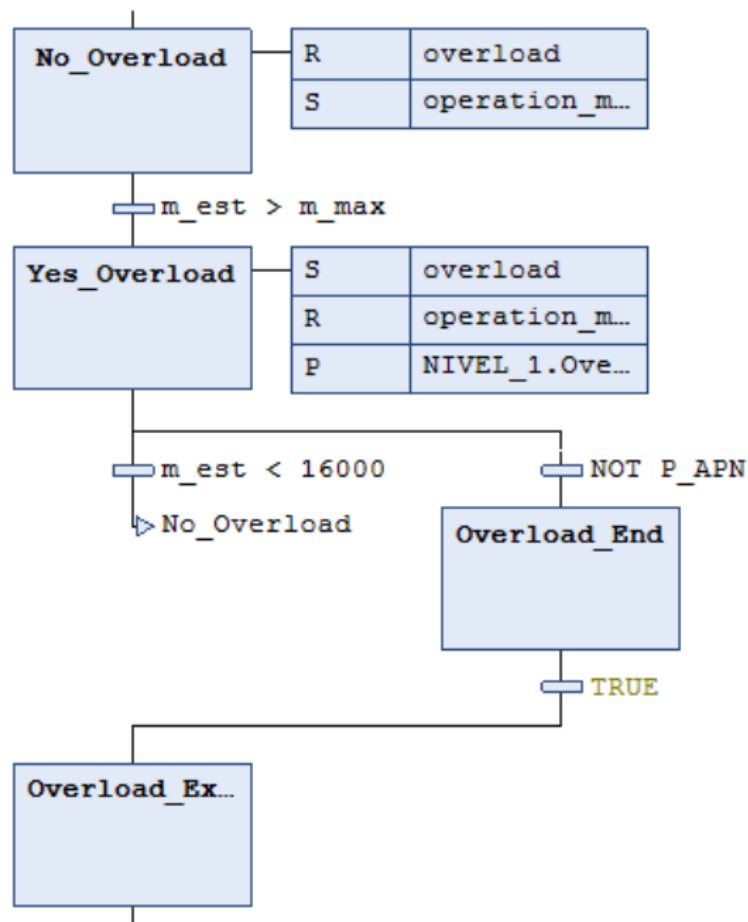


Figura 87: Macropaso Overload_Logics en CODESYS SFC.

El comportamiento de este macropaso es el correspondiente al descrito en la sección 3.3.9.

Any Alarms.

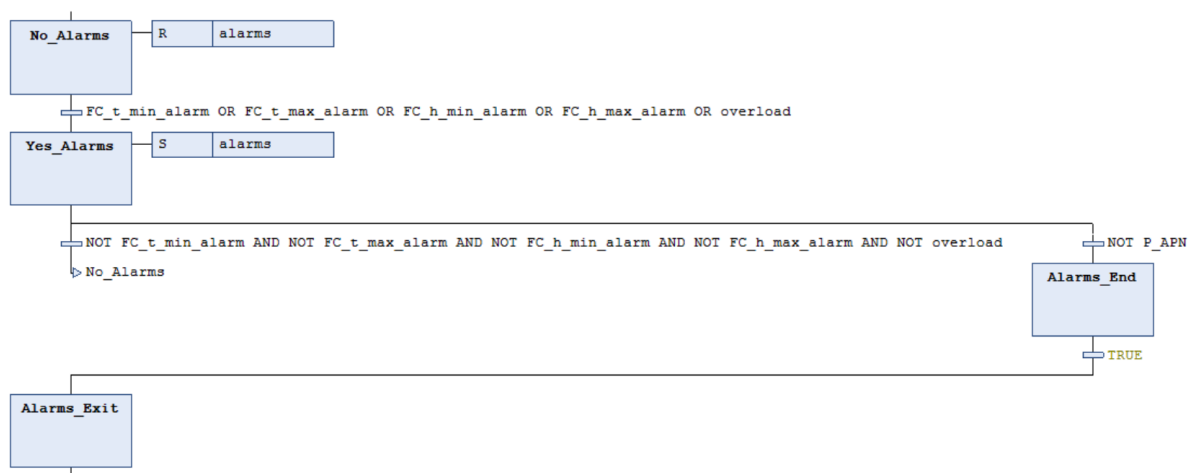


Figura 88: Macropaso Any_Alarms en CODESYS SFC.

El comportamiento de este macropaso es el correspondiente al descrito en la sección 3.3.11.

TLK Logic.

El comportamiento de este macropaso corresponde al descrito en la sección 3.3.6. Cabe destacar que, a diferencia de la implementación en Stateflow, en CODESYS fue necesario agregar lógica adicional para la correcta inversión del valor de TLK, de forma que sea conforme a lo que acepta el estándar IEC 61131-3.

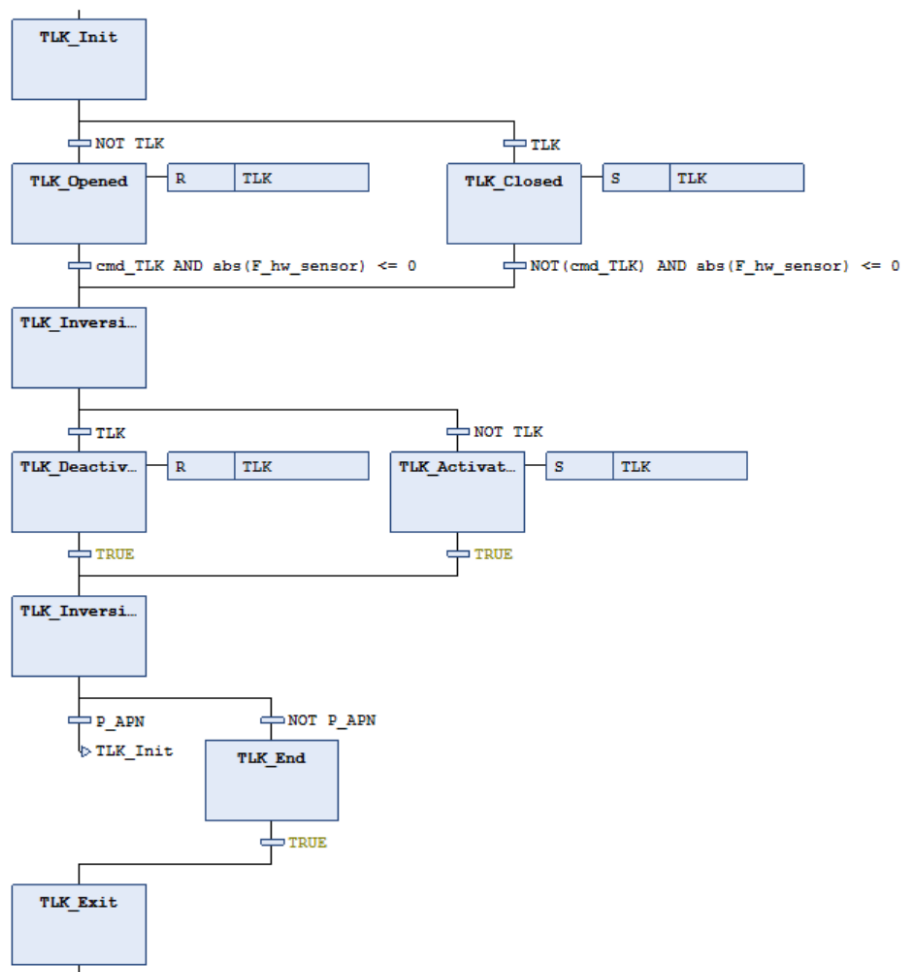


Figura 89: Macropaso TLK_Logic en CODESYS.

Planificador de trayectorias.

La estructura general del planificador corresponde a la descrita en la sección 3.3.12. La figura 90 muestra los pasos de inicialización de la maniobra de *scan*: elevación de la carga, desplazamiento del carro hasta el extremo izquierdo y desplazamiento hasta el extremo derecho.

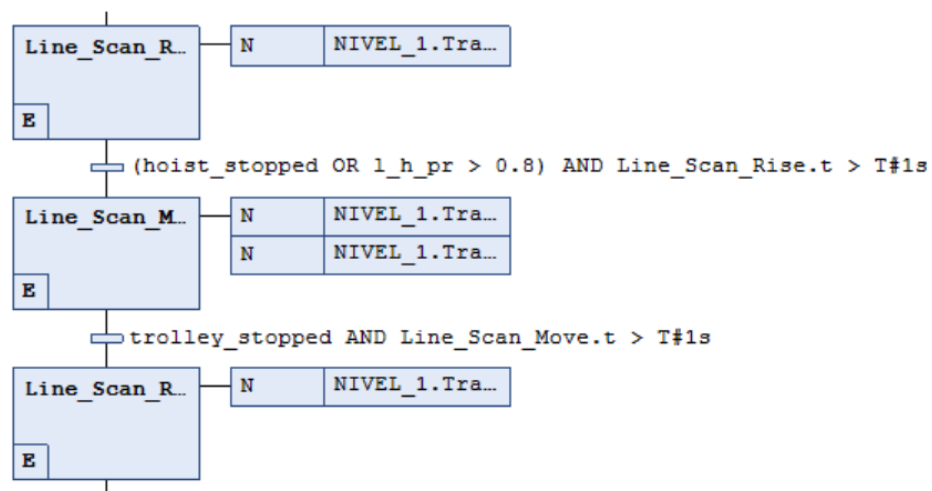


Figura 90: Pasos de inicialización del *scan* en el planificador de trayectorias de CODESYS: elevación, barrido derecha y barrido izquierda.

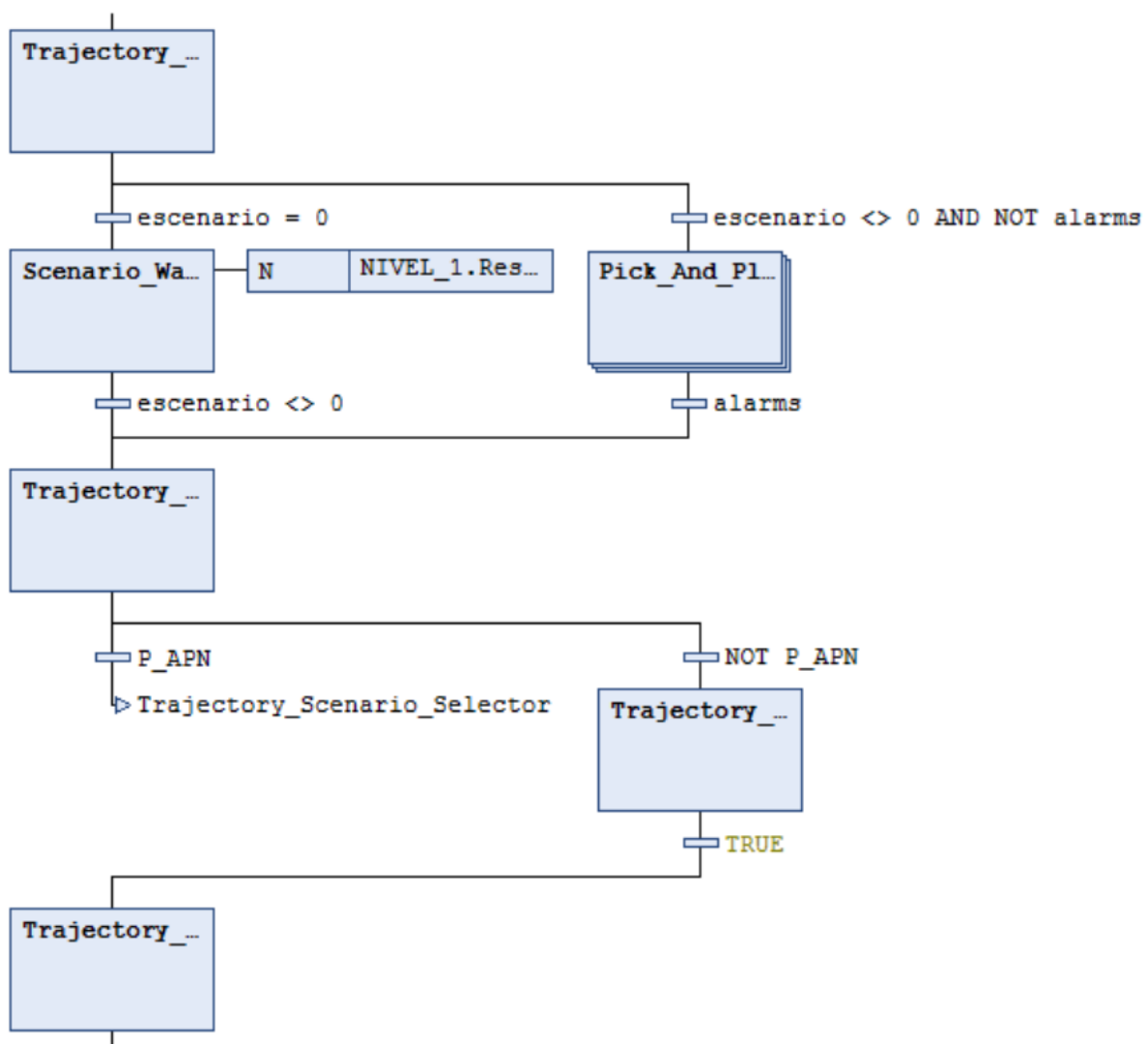


Figura 91: Lógica de selección de escenario y vía de escape del planificador de trayectorias en CODESYS (comportamiento descrito en la sección 3.3.12).

El macropaso **Pick_And_Place_Loop** implementa la lógica de pasos para realizar cargas y descargas de forma cíclica, permitiendo operar tanto en ciclo simple (solo carga o solo descarga) como en ciclo doble (carga seguida de descarga). La referencia completa del comportamiento se encuentra en la sección 3.3.12. Para evitar condiciones de carrera entre las banderas de movimiento y las transiciones del planificador, se incorporan retardos de 5s en las transiciones entre pasos consecutivos.

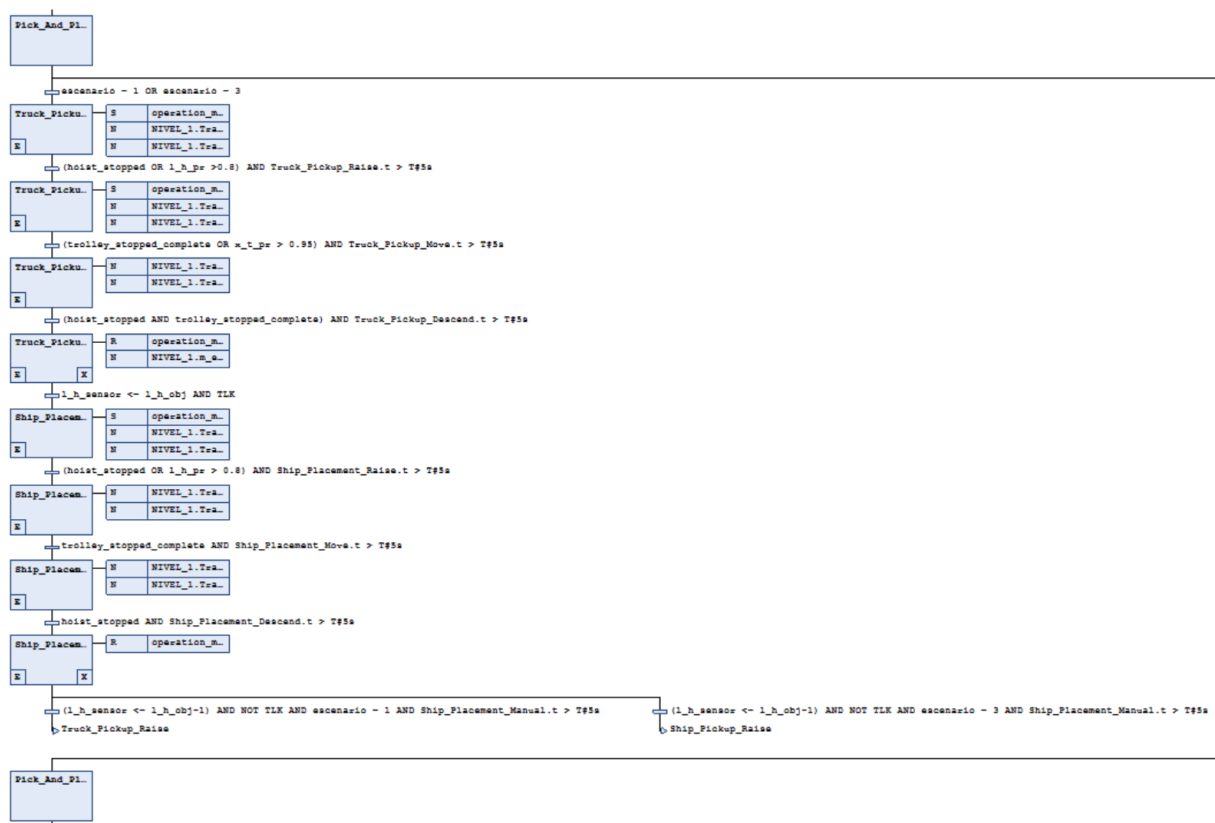


Figura 92: Recorrido de pasos para la operación de **carga** (camión → barco) dentro del macropaso Pick_And_Place_Loop.

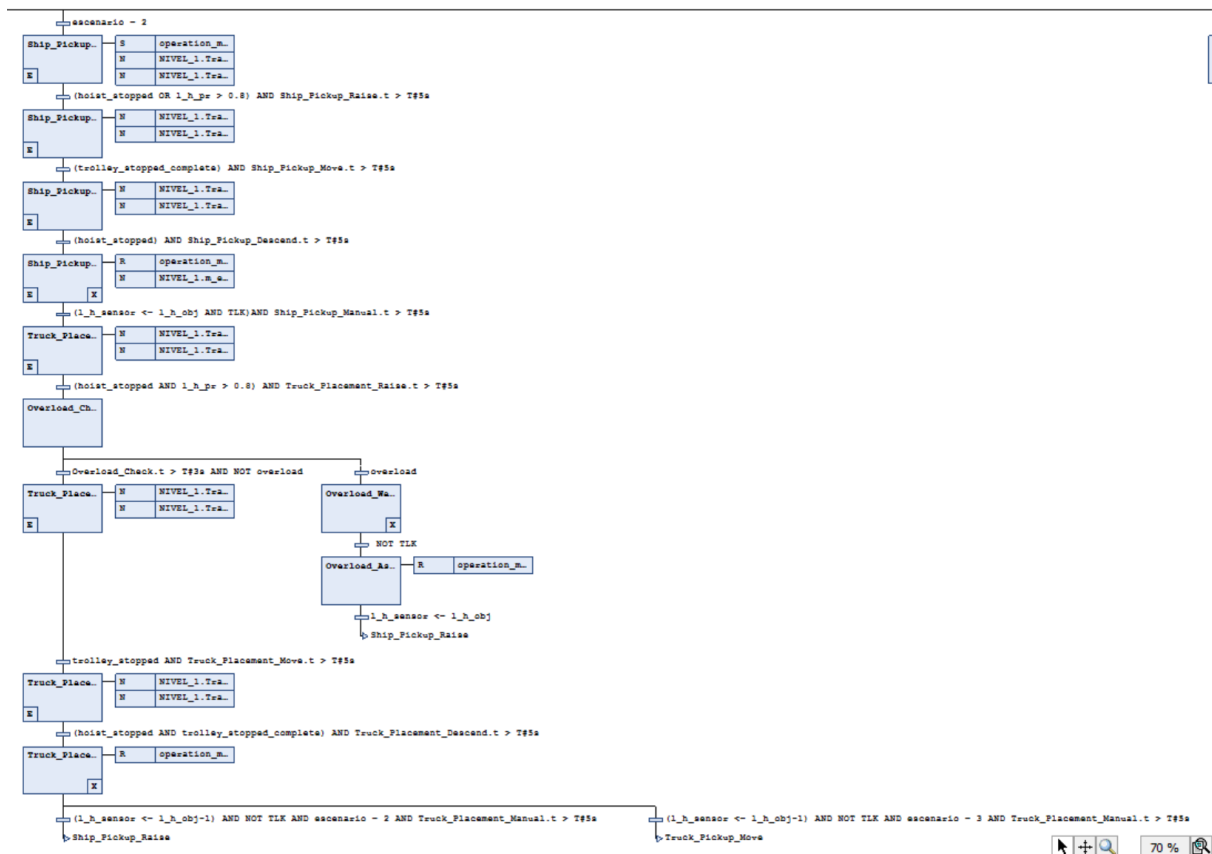


Figura 93: Recorrido de pasos para la operación de **descarga** (barco → camión) dentro del macropaso `Pick_And_Place_Loop`.

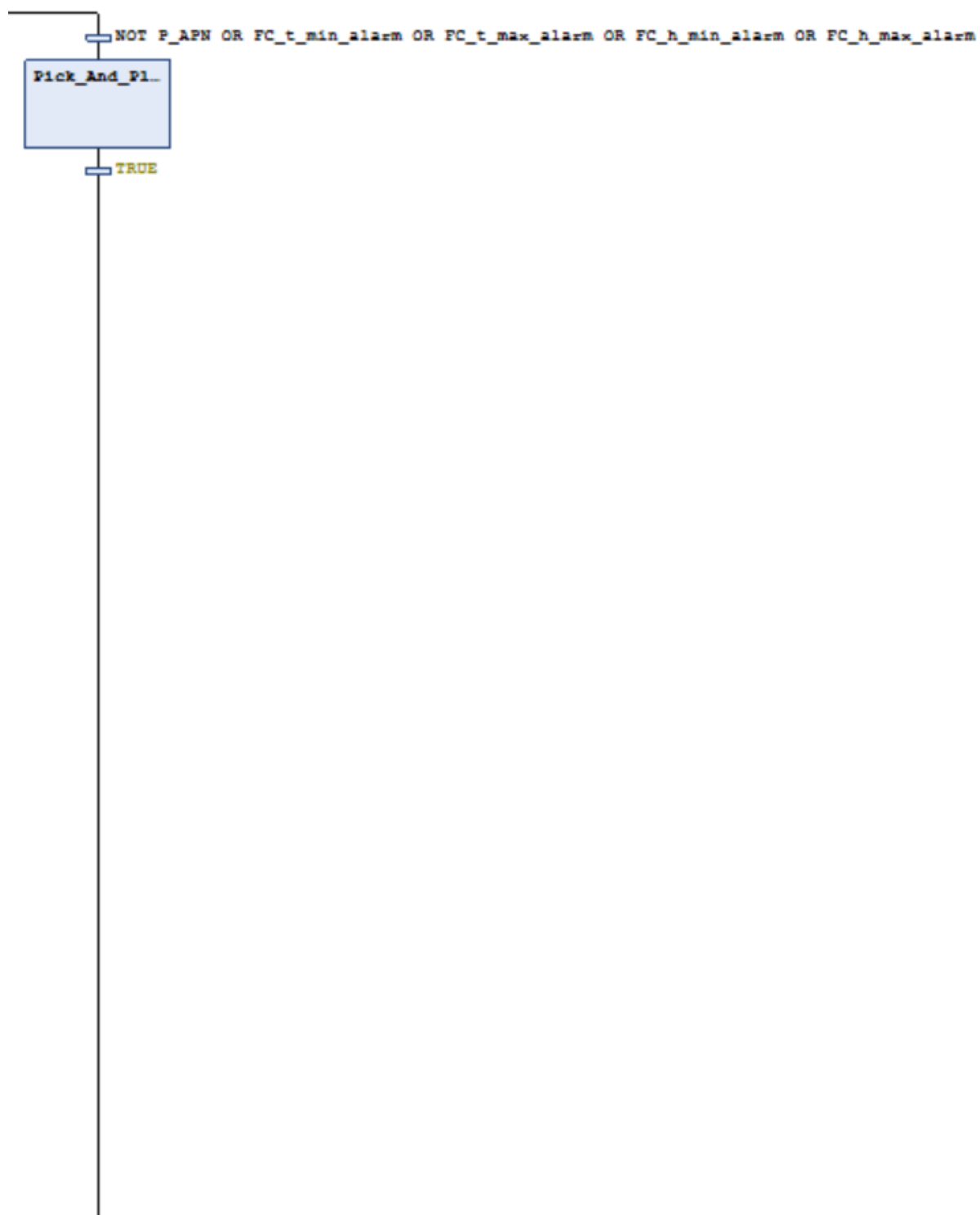


Figura 94: Camino de escape del macropaso `Pick_And_Place_Loop`, requerido para satisfacer la norma IEC 61131-3.

5. Generación automática de código ST mediante PLC Coder

La herramienta **PLC Coder** de Simulink permite exportar automáticamente los diagramas de bloques y Statecharts de Stateflow a código *Structured Text* (ST) compatible con el estándar IEC 61131-3, para su importación directa en CODESYS. Sin embargo, el proceso requiere un conjunto de pasos de acondicionamiento previo para evitar incompatibilidades entre el entorno de simulación y el generador de código.

1. **Subsistemas atómicos con paso de muestreo definido.** Todo bloque que vaya a convertirse debe encapsularse como *Atomic Subsystem* con su período de muestreo T_s explícitamente especificado. Las entradas y salidas deben declararse como *Input Port* y *Output Port* con tipo de dato y tamaño definidos, de modo que PLC Coder pueda inferir las declaraciones de variables del *Function Block* (FB) generado.
2. **Configuración del solver.** El solver debe configurarse en modo *Fixed-step*, activando las opciones *Treat each discrete rate as a separate task* y *Automatically handle rate transitions for data transfer*, para que el generador resuelva correctamente las transferencias entre tareas de distinta frecuencia.
3. **Configuración de PLC Coder.** En la aplicación PLC Coder se selecciona como IDE destino **3S-Smart Software Solutions CODESYS 3.5** y se define la carpeta de salida donde se depositarán los archivos generados.
4. **Tipado completo de símbolos.** Todos los símbolos presentes en los diagramas (señales, parámetros, estados) deben tener tipo y tamaño de dato explícitamente definidos. La presencia de tipos inferidos o ambiguos produce errores en la etapa de generación de código.
5. **Eliminación de funciones MATLAB anidadas en Stateflow.** Las funciones MATLAB embebidas dentro de los Statecharts de Stateflow no son compatibles con PLC Coder y deben eliminarse previamente. Las funcionalidades correspondientes deberán añadirse manualmente en CODESYS una vez importado el código generado.
6. **Exportación.** Se ejecuta la generación de código desde PLC Coder, que produce un archivo `.xml` con la estructura completa del proyecto y los archivos de código ST.
7. **Importación en CODESYS.** El archivo `.xml` generado se importa en CODESYS. Este archivo contiene la información de los archivos a generar y la estructura del proyecto: variables globales, tipos de datos, *Function Blocks* y *Programs*.
8. **Instanciación del FB y conexión de variables.** Se crea una *Program* (PRG) en ST que instancia el FB generado. Las variables globales definidas en `GVL_VARS` se conectan con las entradas y salidas del FB dentro de este PRG, que es el encargado de orquestar la ejecución en cada ciclo de tarea.

Nota: La aplicación generada por PLC Coder (`Application_ST`) coexiste en el mismo dispositivo con la aplicación implementada manualmente en SFC (`Application_SFC`).

Previo a cualquier ejecución debe seleccionarse explícitamente cuál de las dos aplicaciones correrá en el PLC, ya que ambas no pueden ejecutarse simultáneamente.

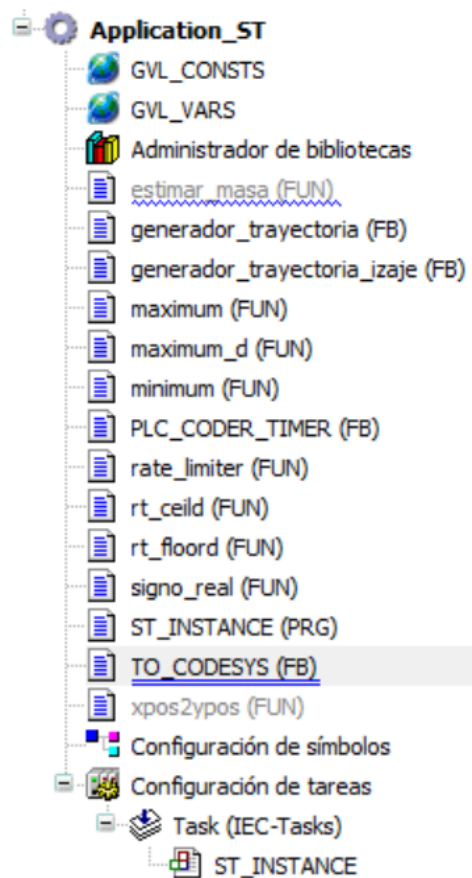


Figura 95: Estructura del proyecto CODESYS generado automáticamente mediante PLC Coder: *Function Blocks*, *Programs*, variables globales y bibliotecas auxiliares.

6. Interfaz HMI

La interfaz HMI del sistema se compone de dos subsistemas independientes: el **control de mando** basado en un joystick PS4 con comunicación UDP hacia Simulink, y la **visualización en tiempo real** de la grúa implementada como una animación 2D en Python.

6.1. Control de mando — Joystick PS4

La interfaz de operación se implementó sobre un control inalámbrico **PS4 DualShock 4**, gestionado desde un equipo de escritorio mediante dos scripts Python desarrollados a medida. El control expone dos sticks analógicos y un conjunto de botones digitales que se mapean directamente a las consignas y señales de mando del sistema.

Script de diagnóstico (`diagnostico_control.py`).

Antes de configurar el mapeado definitivo de ejes y botones, se desarrolló un script auxiliar de diagnóstico que enumera en tiempo real *todos* los ejes analógicos y botones detectados por la biblioteca *pygame*. La visualización muestra barras proporcionales para cada eje (hasta 12) y celdas iluminadas para cada botón (hasta 16), indicando además el índice del último botón presionado. Esta herramienta permitió determinar de forma empírica qué índice de pygame corresponde a cada control físico del mando, independientemente del driver o sistema operativo utilizado.



Figura 96: Script de diagnóstico `diagnostico_control.py`: visualización en tiempo real de ejes analógicos y botones digitales del control PS4 para identificación de índices (Python/pygame/matplotlib).

El resultado del diagnóstico determinó el mapeado definitivo de ejes y botones que se muestra en la Tabla 1.

Tabla 1: Mapeado del control PS4 DualShock 4: índices pygame y función asignada.

Índice	Control físico	Tipo	Señal	Función
Eje 0	Stick izquierdo (X)	Analógico	trolley_cmd	Consigna de velocidad del carro $[-4,0; +4,0]$ m/s
Eje 3	Stick derecho (Y)	Analógico	hoist_cmd	Consigna de velocidad de cable $[-3,0; +3,0]$ m/s
Btn 0	Cruz (X)	Digital (toggle)	system_on	Habilitación del sistema ON/OFF
Btn 2	Cuadrado ($\square$)	Digital (toggle)	twistlock_closed	Apertura/cierre del twistlock
Btn 9	L1	Digital (toggle)	sway_ctrl	Activación del control de balanceo
Btn 6	Options	Digital (toggle)	emergency	Comando de emergencia manual
Btn 10	R1	Digital (toggle)	emgy_ack	Reconocimiento de emergencia global

Script principal (controller_reader.py) — comunicación con Simulink y visualización.

El script principal realiza tres funciones en paralelo mediante un hilo dedicado y un bucle de animación:

1. **Lectura del control:** mediante *pygame* se leen los ejes analógicos del stick izquierdo (eje 0 $\rightarrow$ traslación del carro) y del stick derecho (eje 3 $\rightarrow$ velocidad de cable de izaje), con una zona muerta de $\pm 0,12$ para eliminar derivas mecánicas. Cinco botones se procesan por flanco de subida como biestables: *cross* (sistema ON/OFF), *square* (twistlock), *L1* (control de balanceo), *Options* (emergencia) y *R1* (reconocimiento de emergencia global).
2. **Comunicación con Simulink via UDP:** el script actúa como emisor UDP hacia la dirección de *loopback* 127.0.0.1, puerto 5005, enviando un paquete binario de **13 bytes** en formato *big-endian* cada 50 ms. La estructura del paquete sigue el formato !ffBBBBB:

Bytes	Tipo	Campo	Descripción
0-3	float32	trolley_cmd	Consigna de velocidad del carro [m/s]
4-7	float32	hoist_cmd	Consigna de velocidad de cable [m/s]
8	uint8	system_on	Sistema habilitado (0/1)
9	uint8	twistlock_closed	Twistlock cerrado (0/1)
10	uint8	sway_ctrl	Control de balanceo activo (0/1)
11	uint8	emergency	Emergencia activa (0/1)
12	uint8	emgy_ack	Reconocimiento de emergencia (0/1)

Ante la desconexión del mando, el script envía automáticamente un paquete de *estado seguro* con todos los campos en cero, garantizando que Simulink no retenga la última consigna. Adicionalmente se envía un paquete extendido de 14 bytes al puerto 5006 (añadiendo un byte de presencia del joystick) para la visualización en tiempo real de la grúa 2D.

- Visualización del estado:** una ventana *matplotlib* animada a 50ms muestra en tiempo real la posición de ambos sticks, el valor numérico de las consignas de velocidad, el estado activo/inactivo de cada botón, barras de velocidad normalizadas para carro e izaje, un panel de estado del sistema y una franja de emergencia destacada en rojo cuando la emergencia está activa.

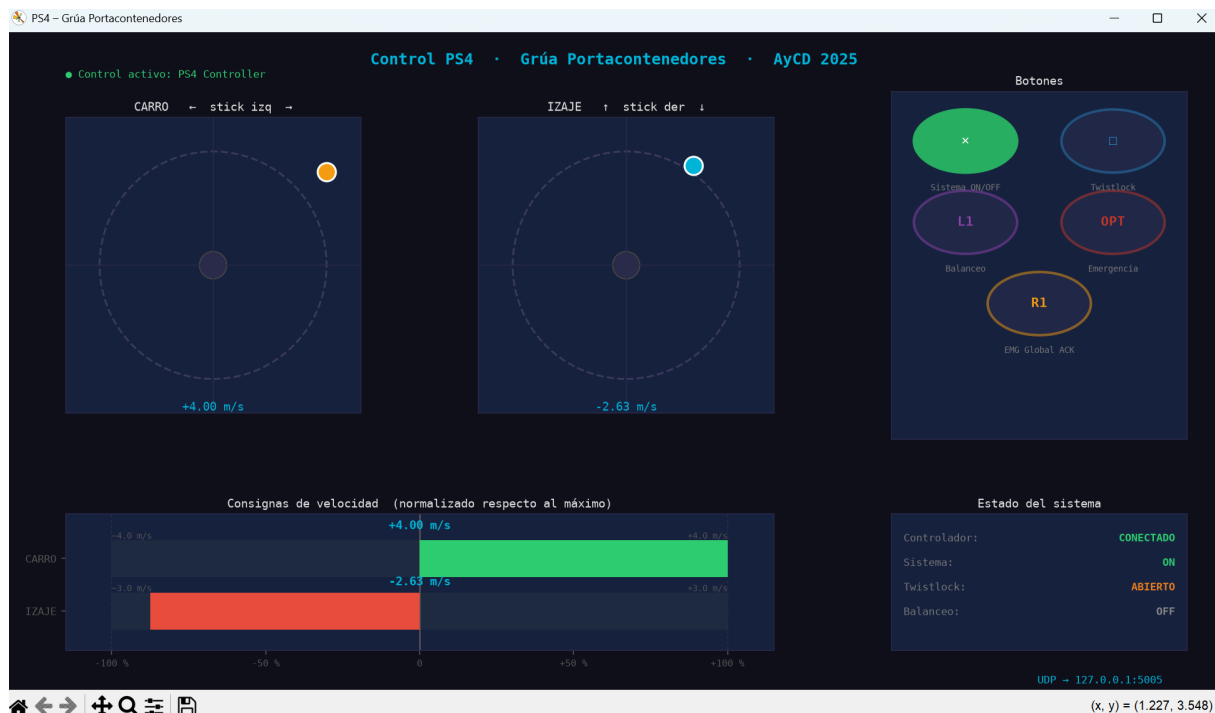


Figura 97: Visualización en tiempo real del script `controller_reader.py`: sticks analógicos, botones, barras de velocidad y panel de estado del control PS4 durante la operación de la grúa (Python/pygame/matplotlib).

Recepción en Simulink — bloque UDP Receive y desempaquetado.

En el lado de Simulink, la recepción del datagrama se realiza mediante el bloque **UDP Receive** del *Instrument Control Toolbox*, configurado para escuchar en el puerto local 5005, aceptar datagramas de cualquier origen (Remote IP: '0.0.0.0'), con un *receive buffer* de **14 bytes** y tipo de dato de salida `uint8`. El período de muestreo del bloque se fija en 1s, de modo que actúa como fuente asíncrona: Simulink toma la última muestra disponible en cada paso de tiempo de la simulación.

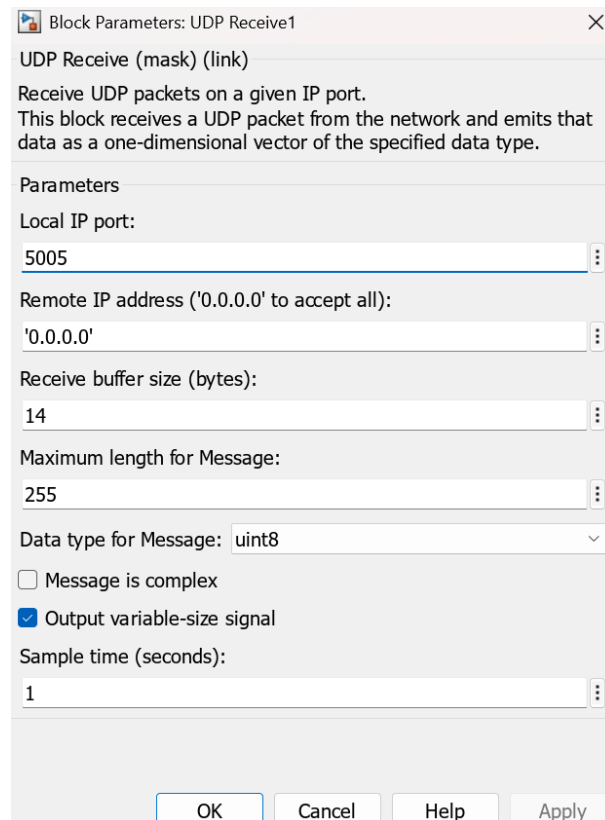


Figura 98: Configuración del bloque UDP Receive en Simulink: puerto 5005, buffer de 14 bytes, salida `uint8` (Instrument Control Toolbox).

La salida del bloque es un vector `uint8(1×13)` con los bytes crudos del datagrama. Su desempaquetado se delega a la *MATLAB Function* `parse_udp_packet`, que reconstruye los valores de punto flotante y los flags booleanos respetando el orden de bytes *big-endian* empleado por Python (`struct.pack('!ffBBBBB', ...)`):

```
1 function [trolley_cmd, hoist_cmd, system_on, twistlock, ...  
2         sway_ctrl, emergency, emgy_ack] = parse_udp_packet(data)  
3  
4 trolley_cmd = 0; hoist_cmd = 0;  
5 system_on   = 0; twistlock = 0;  
6 sway_ctrl   = 0; emergency = 0; emgy_ack = 0;  
7  
8 if numel(data) < 13  
9     return  
10 end  
11  
12 raw = uint8(data(:)');
```

```

13
14 trolley_cmd = double(typecast(raw(4:-1:1), 'single'));
15 hoist_cmd   = double(typecast(raw(8:-1:5), 'single'));
16
17 system_on = double(raw(9));
18 twistlock = double(raw(10));
19 sway_ctrl = double(raw(11));
20 emergency = double(raw(12));
21 emgy_ack  = double(raw(13));

```

Listing 15: *MATLAB Function*: desempaqueado del datagrama UDP proveniente del control PS4.

Los primeros 8 bytes contienen dos valores `float32` en big-endian; dado que MATLAB corre sobre arquitectura little-endian (x86), es necesario invertir el orden de los bytes de cada campo antes de aplicar `typecast` (`raw(4:-1:1)` y `raw(8:-1:5)`) para obtener el valor correcto. Los bytes del 9 al 13 son `uint8` individuales que se convierten directamente a `double` para su uso en los bloques de Simulink. En caso de recibir un paquete con menos de 13 bytes (por ejemplo, durante el arranque antes de que el script Python esté activo), la función retorna el estado seguro con todas las salidas en cero.

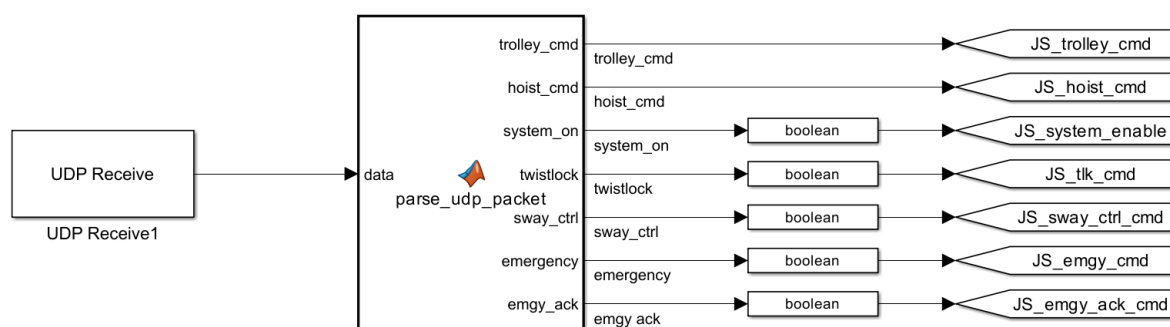


Figura 99: Bloque de lectura UDP completo en Simulink: `UDP Receive` y *MATLAB Function* `parse_udp_packet` con sus siete salidas hacia el sistema de control.

6.2. Visualización en tiempo real — RT_VIEW

La visualización en tiempo real se implementó como una animación 2D de la grúa STS en Python (`visualizacion_grua.py`), animada mediante `matplotlib.animation` a ≈ 30 fps. La ventana replica el estado completo del sistema en tiempo real, mostrando simultáneamente la geometría de la grúa, el estado del entorno (barco, contenedores apilados, camiones) y un panel de variables del sistema.

La simulación en Simulink se configura con **tiempo de simulación infinito** y un **pacing ratio de 1** (tiempo de simulación = tiempo real), de modo que el modelo avanza en tiempo real y el script Python recibe datos coherentes con el reloj del operador.

Fuentes de datos — canales UDP.

El script mantiene tres hilos receptores independientes, cada uno escuchando en un puerto distinto:

- **Puerto 5010 — Simulink:** paquete de *doubles* little-endian cuyo tamaño varía según la versión del bloque UDP Send activo en Simulink (formatos de 8 a 336 bytes). El paquete más completo incluye: posición simulada del carro $x_{t,sim}$, longitud de cable simulada $l_{h,sim}$, ángulo de balanceo simulado $\theta_{l,sim}$, señales discretas de frenos y twistlock (BRK_t , BRK_h , BRK_{hE} , TLK , $SWAY_CTR$), lectura del LIDAR $y_{c0,lidar}$, los 19 contadores de pilas de contenedores en el barco, presencia y carga de los 5 camiones, escenario activo, fuerza medida en cable F_{hw} , modo de operación y masa estimada $\hat{m}$.
- **Puerto 5006 — Joystick:** mirror del paquete enviado por `controller_reader.py` (formato extendido de 15 bytes, véase Sección 6.1), con las consignas de velocidad y el estado de los botones del control PS4.
- **Puerto 5011 — Contenedores** (canal alternativo): vector de 9 *doubles* con los contadores de pilas, utilizado cuando el bloque Simulink envía ese subconjunto en un datagrama separado.

Transmisor UDP en Simulink.

El envío de datos desde Simulink hacia RT_VIEW se realiza mediante un bloque *UDP Send* configurado con dirección de destino 127.0.0.1 y puerto remoto **5010**, operando sobre loopback local. El tamaño del buffer de envío es de 8192 bytes, suficiente para el paquete más extenso (336 bytes de *doubles* little-endian). La Figura 100 muestra la configuración del bloque, y la Figura 101 el subsistema transmisor completo dentro del modelo Simulink.

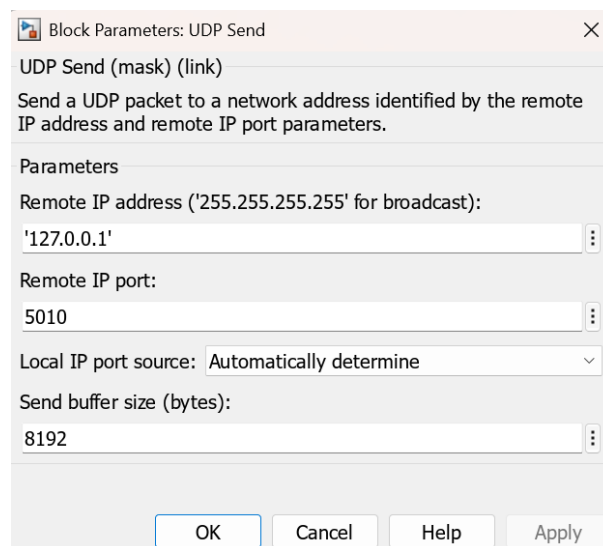


Figura 100: Configuración del bloque *UDP Send* en Simulink para el canal de datos de RT_VIEW (puerto 5010, loopback).

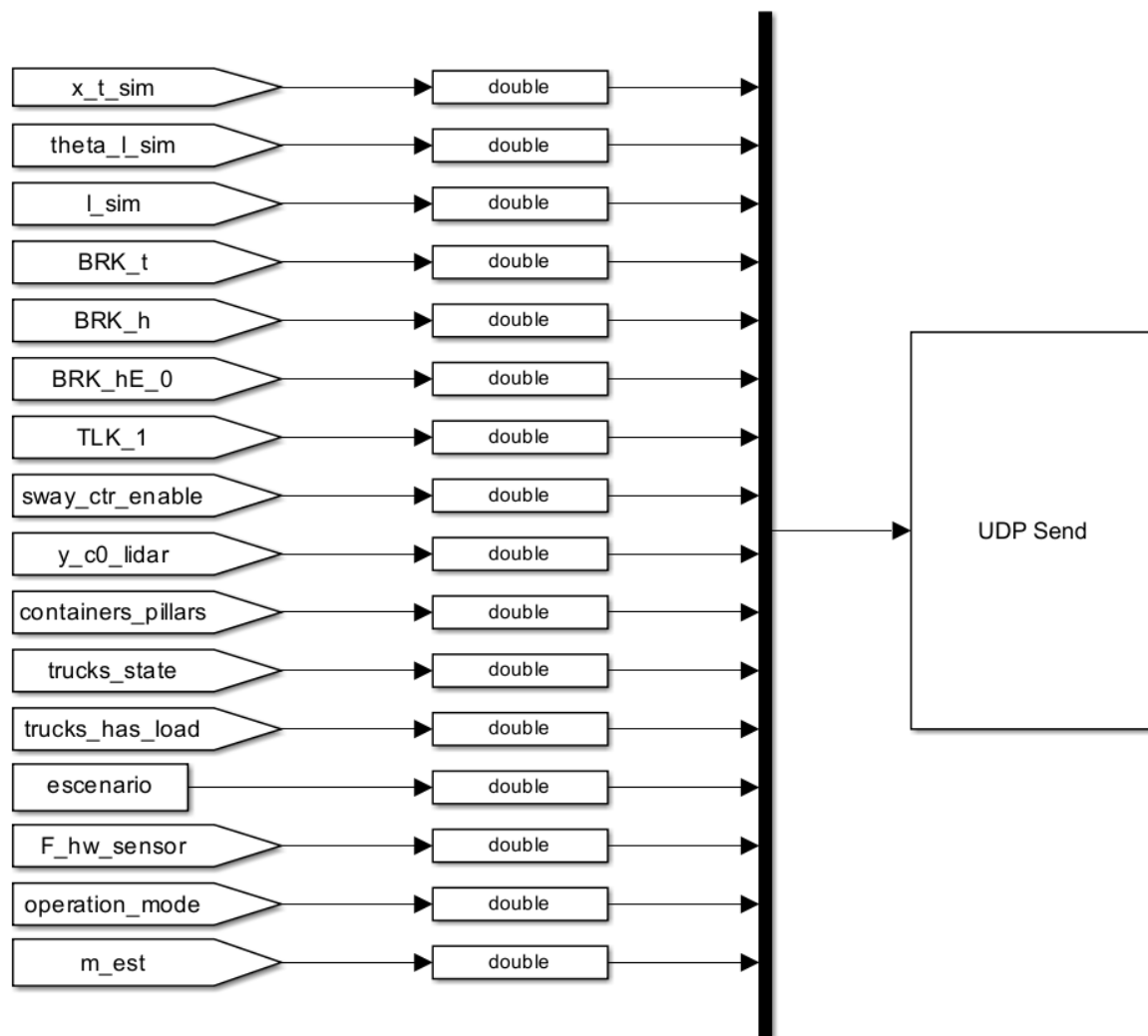


Figura 101: Subsistema transmisor completo en Simulink: multiplexado de señales y bloque *UDP Send* hacia RT_VIEW.

Elementos de la visualización.

La ventana se divide en tres paneles:

- **Panel izquierdo** (fondo oscuro): estado del control PS4 —indicador de conexión del joystick, barras de velocidad normalizadas de carro e izaje, y estado de las señales discretas (sistema ON/OFF, modo, twistlock, balanceo, emergencia).
- **Vista central:** animación 2D a escala real de la grúa, incluyendo la estructura del pórtico, el boom, el carro con sus ruedas y polea, el cable de izaje, el spreader y el contenedor suspendido (rotado con el ángulo de balanceo θ_l). El sensor LIDAR se representa como un cuerpo solidario al carro con su haz de medición y el perfil acumulado de lecturas a lo largo del recorrido. El entorno incluye el muelle, el agua, el barco con sus bodegas, las pilas de contenedores que crecen o se reducen dinámicamente en las 19 posiciones, y hasta 5 camiones de transporte terrestre que aparecen y desaparecen con o sin contenedor según el estado reportado por Simulink.

- **Panel derecho:** variables de estado numéricas actualizadas en tiempo real, separadas en continuas ($x_{t,sim}$, $l_{h,sim}$, $\theta_{l,sim}$, $\hat{m}$, F_{hw}) y discretas con pastillas de color semafórico (BRK_t , BRK_h , BRK_{hE} , TLK , $SWAY_CTR$).

En la parte superior de la ventana se muestran además tres indicadores de estado global: conexión con Simulink, modo de operación (manual/automático) y escenario activo (espera, ciclo carga, ciclo descarga, ciclo doble).

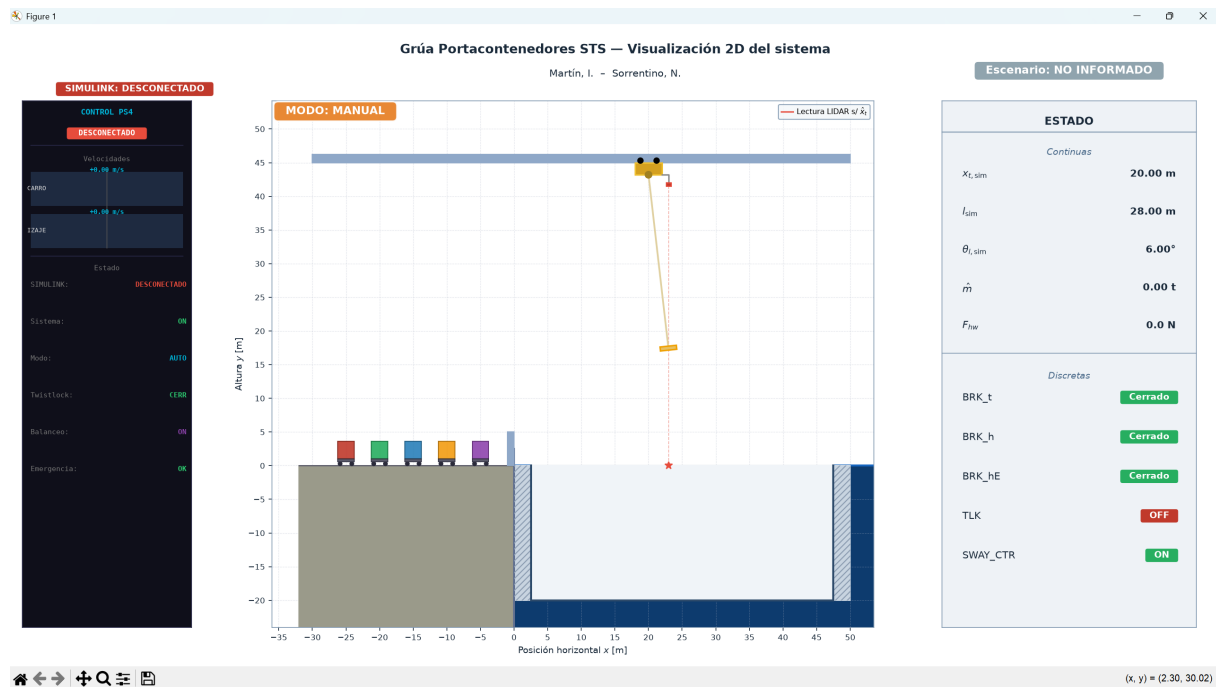


Figura 102: Visualización 2D en tiempo real de la grúa STS (`visualizacion_grua.py`): vista central con geometría animada, panel izquierdo de control PS4 y panel derecho de variables de estado (Python/matplotlib).

Nota sobre la fidelidad temporal de la visualización. Aunque la animación se actualiza nominalmente a ≈ 30 fps y Simulink corre con pacing 1:1, la visualización introduce varias fuentes de latencia que pueden hacer que lo observado en pantalla no refleje el estado instantáneo del sistema: (i) el intervalo de envío del bloque *UDP Send* de Simulink introduce un retardo de hasta un paso de muestreo entre el estado interno del modelo y el datagrama efectivamente emitido; (ii) el hilo receptor Python opera con un *timeout* de 500 ms, por lo que ante una ráfaga de paquetes solo se procesa el último disponible al momento de la lectura, pudiendo saltarse estados intermedios; (iii) el renderizado del canvas de *matplotlib* no está garantizado en tiempo real: en equipos con carga elevada o drivers gráficos lentos, el tiempo de dibujado puede acumularse y generar un desfase visible entre la animación y el modelo. Por todo esto, la visualización *RT_VIEW* debe considerarse como una herramienta de monitoreo cualitativo y diagnóstico rápido, no como una fuente de referencia temporal precisa. Para verificar el estado exacto del sistema, la evolución de señales y el cumplimiento de consignas, siempre debe recurrirse a las gráficas y *scopes* internos de Simulink.

6.3. Panel de monitoreo en Simulink — Dashboard

Como complemento a las herramientas de visualización externas, se desarrolló un panel de monitoreo directamente en el entorno Simulink utilizando la librería *Simulation 3D/Dashboard* de MATLAB. Esta librería permite incorporar elementos gráficos interactivos —indicadores luminosos, medidores analógicos, displays numéricos, entre otros— directamente en el diagrama de bloques, conectados en tiempo de compilación a las señales del modelo. El panel resultante refleja el estado del sistema durante la simulación sin necesidad de herramientas externas, con un diseño gráfico organizado según la lógica de operación de la grúa. La Figura 103 muestra el aspecto del panel durante una sesión de simulación.

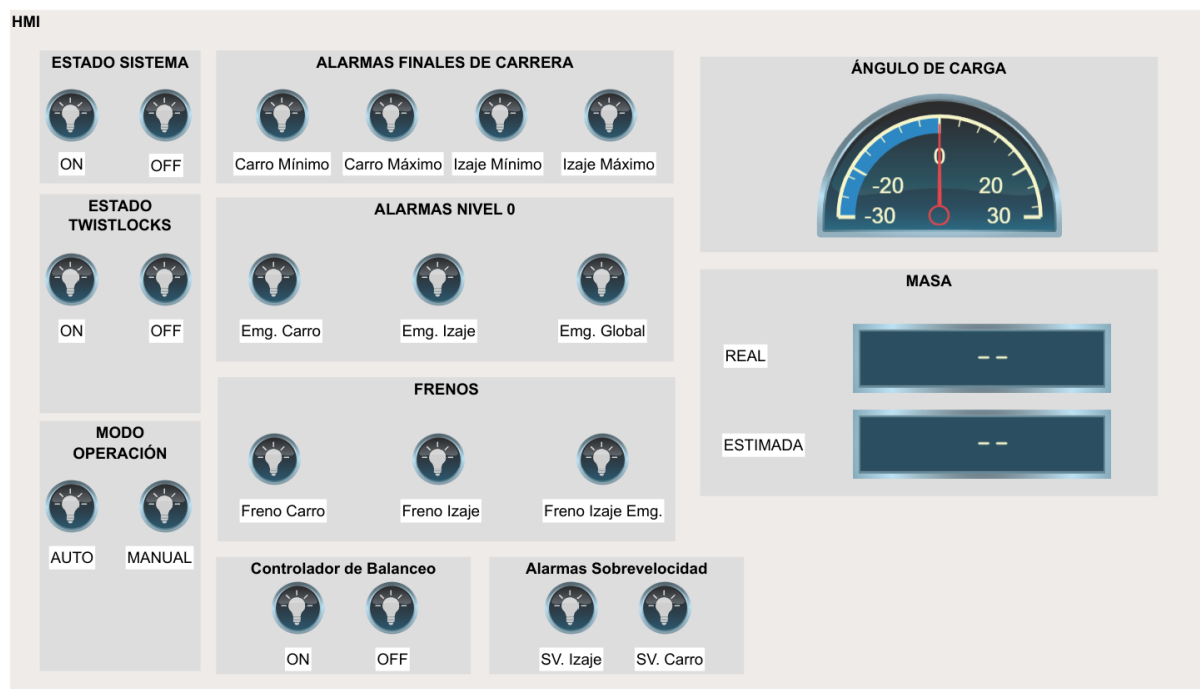


Figura 103: Panel de monitoreo del sistema desarrollado con la librería *Dashboard* de Simulink.

7. Resultados

En esta sección se presentan los resultados obtenidos mediante co-simulación *Software-in-the-Loop* entre Simulink y CODESYS. Se evalúan el rendimiento operativo del sistema en ciclo simple, el comportamiento del control de oscilación a lo largo de múltiples ciclos dobles, la estimación de masa del contenedor, la respuesta ante fines de carrera y la actuación ante condiciones de emergencia. En los videos incluidos en la sección 7.3 se presenta material adicional y pruebas complementarias que proporcionan una visión más clara e integradora del comportamiento del sistema en operación.

7.1. Rendimiento: ciclo simple

Para evaluar el rendimiento del sistema se somete al controlador a un recorrido en las condiciones de mayor desfavor posibles: **máxima distancia de traslación del carro**, **masa de carga máxima** y **máxima profundidad de depósito**. Con el objetivo de calcular la duración del ciclo de forma directa, se saltan los estados manuales del sistema de control, ejecutando únicamente la secuencia automática.

La figura 104 muestra la evolución de la longitud de cable $l_h(t)$ durante el ciclo completo.

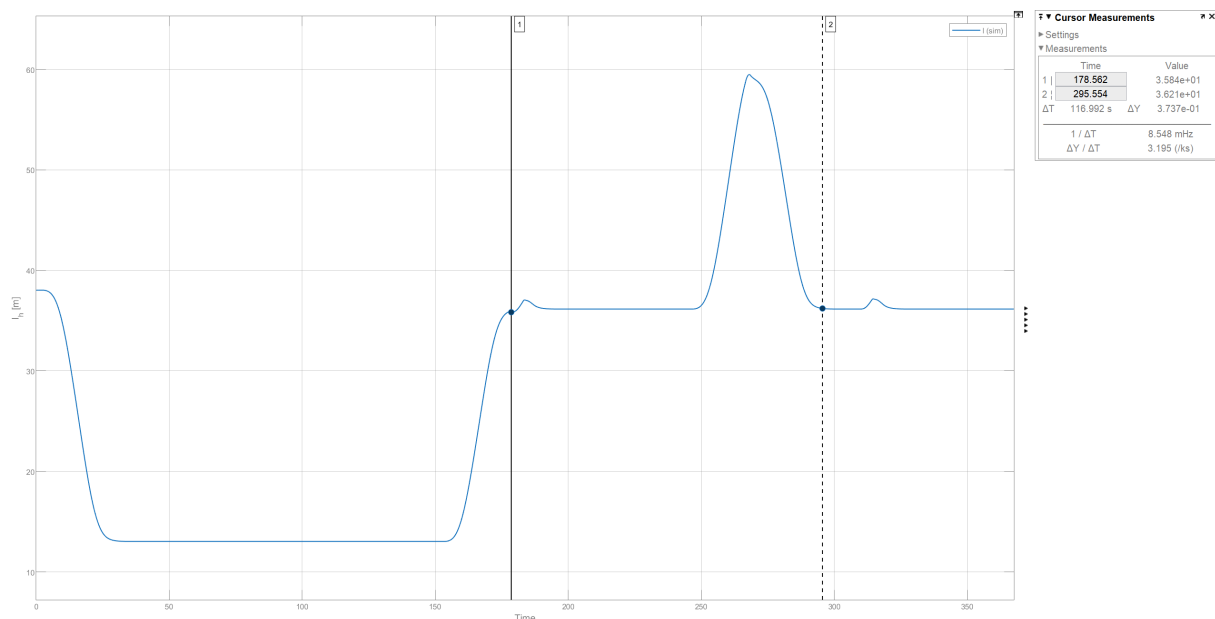


Figura 104: Ciclo simple — evolución de la longitud de cable de izaje $l_h(t)$.

En la imagen se observa un escenario que comienza con el cable suficientemente estirado para tomar una carga del camión y finaliza en ese mismo punto, por lo que mediante esta variable puede obtenerse directamente el tiempo de ciclo. Tomando los instantes $t_0 = 178,562\text{ s}$ y $t_1 = 295,554\text{ s}$ se obtiene:

$$\Delta t = t_1 - t_0 = 295,554 - 178,562 = 116,992\text{ s} \approx 1\text{ min } 57\text{ s}$$

La productividad resultante es:

$$\dot{N} = \frac{3600 \text{ s/h}}{\Delta t} = \frac{3600}{116,992} \approx \mathbf{30,77} \text{ contenedores/hora}$$

Este valor supera el umbral mínimo de **30 contenedores por hora** establecido como requisito de diseño (sección 2.6), confirmando que el sistema cumple con la especificación de productividad en ciclo simple bajo condiciones de máximo desfavor.

Los pequeños picos visibles en torno a los instantes $t \approx 180 \text{ s}$ y $t \approx 315 \text{ s}$ corresponden a pequeñas ineficiencias del sistema de control debidas a la cercanía entre el margen aplicado en la toma de la carga y la altura segura de traslación del carro.

La eficiencia del ciclo reside en la **coordinación de movimientos X–Y** entre el carro y el izaje: se han definido umbrales de compleción de cada movimiento para habilitar el inicio del siguiente, fijados entre el 80 % y el 95 % de la trayectoria según la necesidad de precisión y el riesgo asociado a cada fase de la maniobra.

7.2. Pruebas adicionales

7.2.1. Evolución del ángulo de balanceo en múltiples ciclos dobles

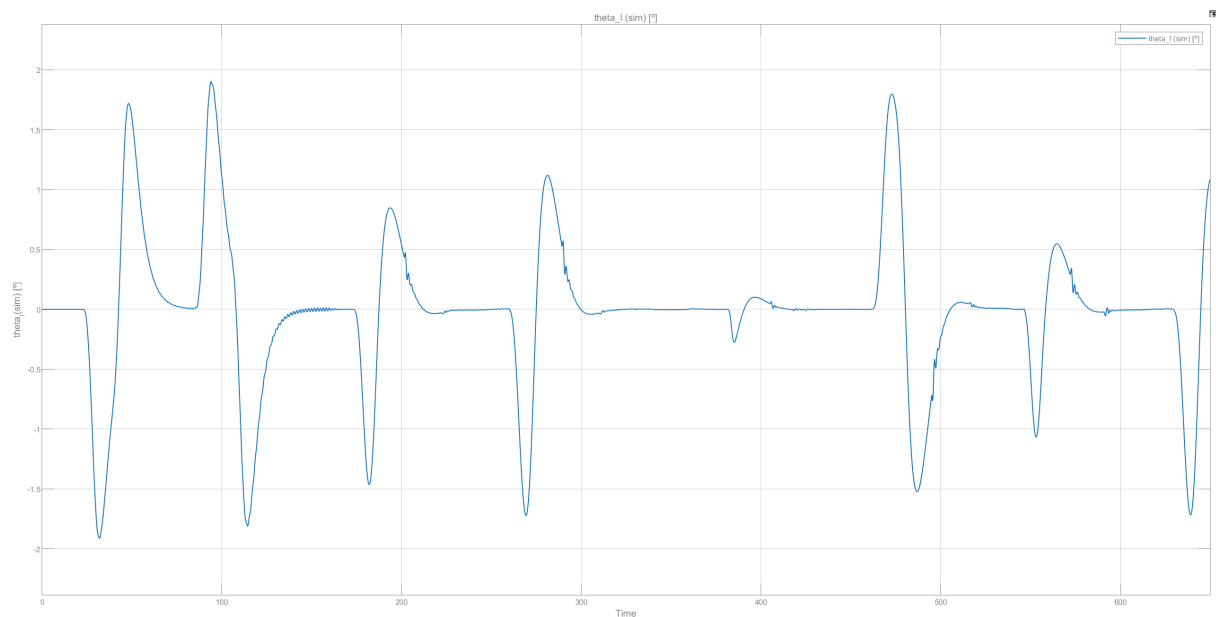


Figura 105: Evolución del ángulo de balanceo $\theta_l(t)$ a lo largo de múltiples ciclos dobles (carga + descarga) con el controlador de oscilación activo.

Con el controlador de oscilación en paralelo activo, el ángulo de balanceo $\theta_l(t)$ **nunca supera los 2** a lo largo de toda la operación, sin afectar notablemente al rendimiento del ciclo — como lo confirma el resultado de la sección 7.1. El diseño garantiza un **amortiguamiento crítico** ($\zeta = 1$) del ángulo en todas las fases de traslación, y valores **inferiores a 1** durante las operaciones de *pick and place*, donde la precisión de posicionamiento es más exigente. Las pequeñas oscilaciones residuales observadas en la señal son introducidas por la actuación de los **frenos de operación** al detener el movimiento del carro.

7.2.2. Fines de carrera

Para verificar el funcionamiento de los fines de carrera de operación se genera una trayectoria en la que el carro se desplaza desde -30 hasta 50 m, activando ambos límites laterales de operación. En la Figura 106 se muestra la evolución de la posición del carro durante esta prueba.

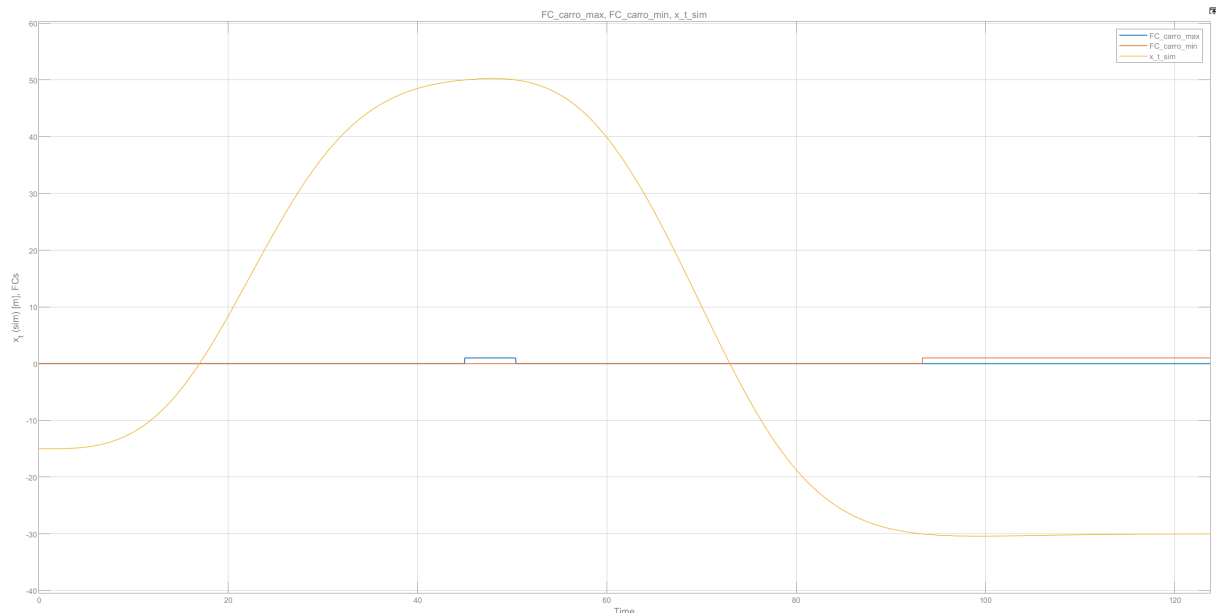


Figura 106: Activación de fines de carrera de operación del carro: trayectoria desde -30 hasta 50 m.

En ambas figuras se observa la activación y desactivación de los finales de carrera conforme el sistema alcanza o abandona la posición de activación correspondiente.

De forma análoga, para el eje de izaje se genera una trayectoria desde 5 hasta 65 m de longitud de cable, activando los fines de carrera de operación superior e inferior. La Figura 107 muestra la evolución de la longitud de cable durante esta prueba.

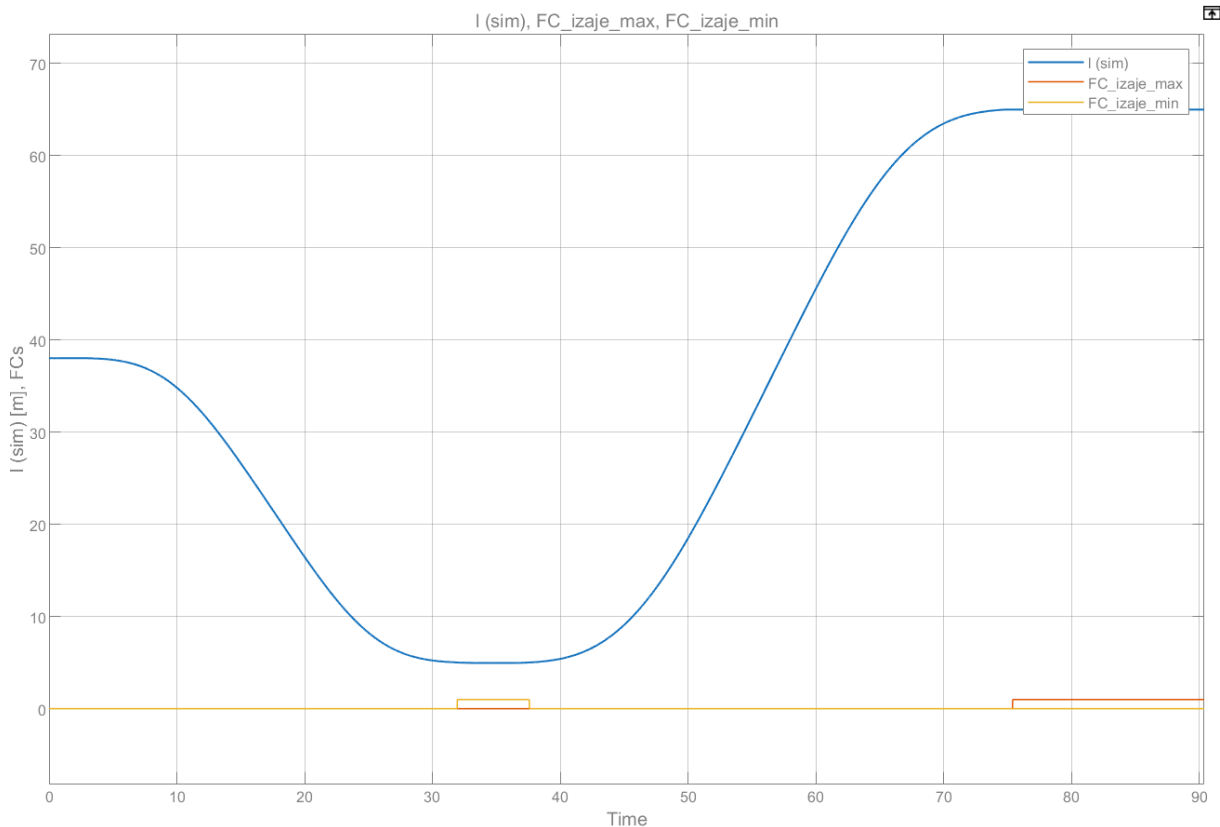


Figura 107: Activación de fines de carrera de operación del izaje: trayectoria desde 5 hasta 65 m de longitud de cable.

7.2.3. Parada de emergencia

Se evalúa el escenario en que el operador introduce, mediante el controlador de la HMI, una señal de parada de emergencia. En la figura se presentan las señales discretas relevantes: la entrada de operador P_E , las señales de frenos BRK_t , BRK_h y BRK_{hE} , y las referencias de velocidad $v_{t,ref}$ y $\dot{h}_{h,ref}$, que se ponen a cero de forma inmediata ante la activación de la señal de emergencia. En la Figura 108 se observa cómo ambas referencias caen a cero de forma inmediata en el instante en que se activa la parada de emergencia.

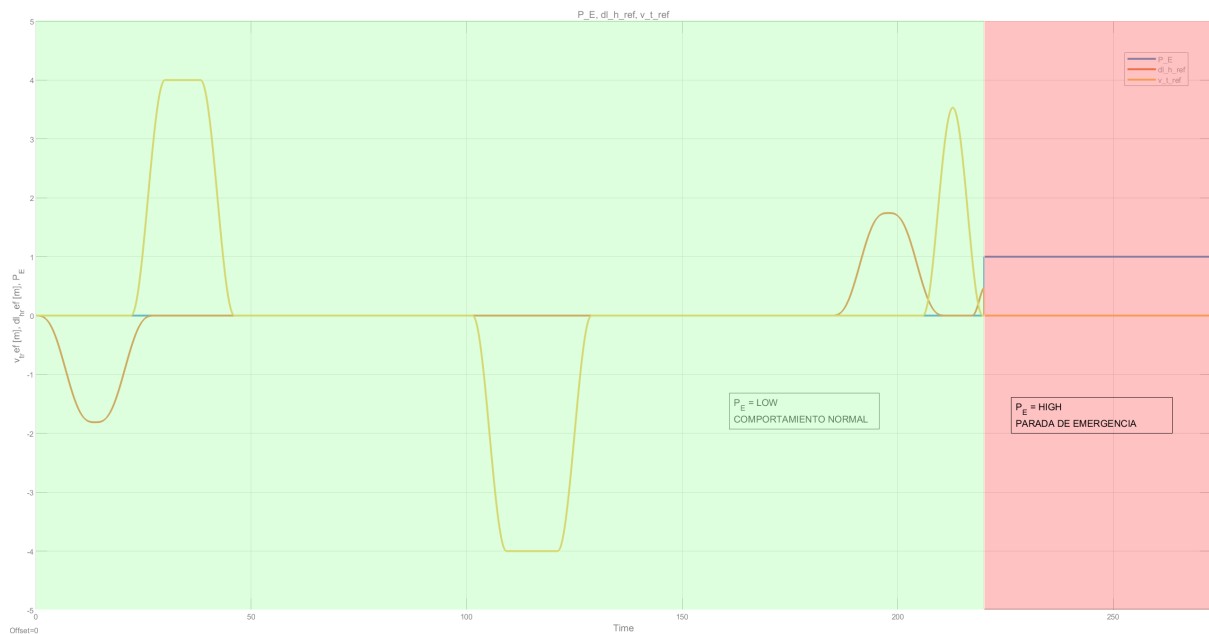


Figura 108: Referencias de velocidad del carro ($v_{t,ref}$) y del izaje ($\dot{l}_{h,ref}$) ante una parada de emergencia: caída inmediata a cero al activarse P_E .

En la Figura 109 se muestra el comportamiento de las señales de frenos. Se observa la actuación normal de los frenos de operación durante el funcionamiento previo a la emergencia y, a partir de la activación de P_E , la energización inmediata de todos los frenos (BRK_t , BRK_h y BRK_{hE}) de forma simultánea.

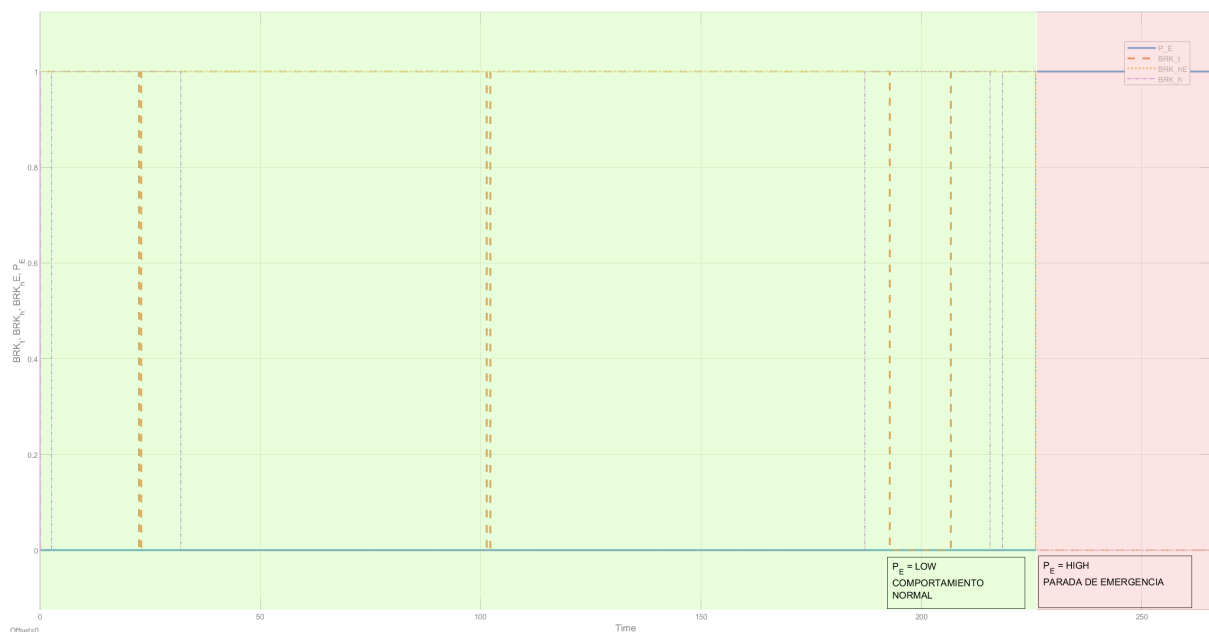


Figura 109: Señales de frenos ante una parada de emergencia: actuación normal de operación seguida de la energización simultánea de todos los frenos al activarse P_E .

7.2.4. Estimación de masa

La estimación de masa del contenedor se evalúa a lo largo de múltiples ciclos de carga y descarga, con el objetivo de analizar el comportamiento del estimador ante distintos casos de operación. En la Figura 110 se presenta la evolución de la masa estimada durante la secuencia completa de prueba.

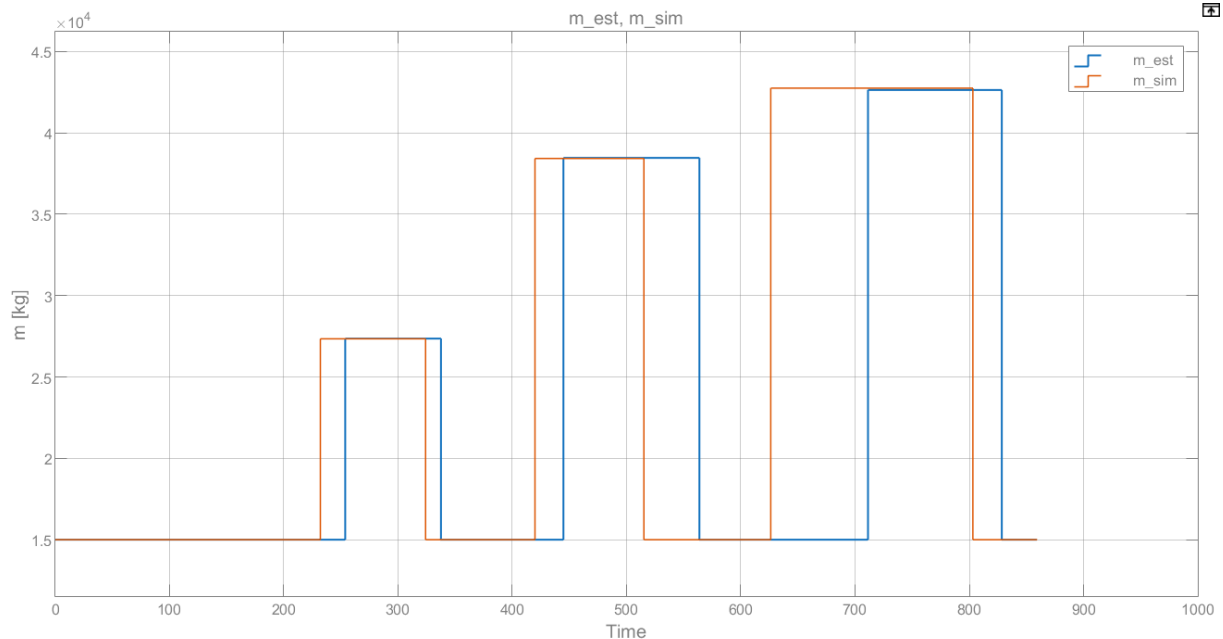


Figura 110: Evolución de la masa estimada $\hat{m}(t)$ a lo largo de múltiples ciclos de carga y descarga para la evaluación del estimador en distintos escenarios de operación.

La diferencia temporal observable entre la evolución del valor simulado y el valor estimado se debe a que el primero se actualiza en el instante en que se produce el cambio en la señal de TLK, mientras que la estimación se activa al abandonar el modo manual de operación. En todos los casos evaluados, la estimación converge con una precisión **ampliamente superior al 1%** requerido por las especificaciones del sistema.

7.3. Enlaces a videos

Los videos de las pruebas realizadas se alojan en una carpeta compartida de Google Drive, accesible mediante el siguiente enlace:

[Enlace a videos de pruebas](#)

8. Conclusiones

El presente trabajo permitió diseñar, modelar, simular e implementar un sistema híbrido de control y protección para una grúa portacontenedores de muelle tipo pórtico, cumpliendo con la totalidad de los objetivos y especificaciones planteados. Se desarrolló un modelo matemático completo de la planta, incluyendo los subsistemas de traslación e izaje con sus respectivos accionamientos, cables elástico-amortiguados y la dinámica pendular de la carga suspendida. Sobre este modelo se diseñaron e implementaron en tiempo discreto los controladores del Nivel 2, incorporando realimentación de estados con observador de Luenberger, compensación de no linealidades y control proporcional de balanceo con ganancia adaptada a la longitud de cable. El Nivel 1 implementó la lógica supervisora mediante un autómata jerárquico que gestiona los modos de operación, la coordinación de trayectorias automáticas, la estimación de masa, el relevamiento del perfil de obstáculos y la habilitación de los accionamientos. El Nivel 0, por su parte, opera de forma independiente monitoreando continuamente las condiciones de emergencia y tomando el control de los accionamientos ante cualquier situación que comprometa la seguridad del sistema.

En cuanto al desempeño, el ángulo de oscilación de la carga se mantuvo dentro de los límites establecidos en todas las pruebas, el tiempo de ciclo simple resultó inferior al requerido para garantizar una productividad mínima de 30 contenedores por hora, y la estimación de masa presentó un error relativo promedio inferior al 1 %, permitiendo adaptar la velocidad de izaje a la característica de potencia constante del accionamiento. La operación semiautomática funcionó correctamente en ambos modos, con transferencias suaves entre ellos (*bumpless transfer*), trayectorias coordinadas que evitaron colisiones con el perfil de obstáculos, y una gestión correcta de los twistlocks y la detección de cable flojo. El Nivel 0 respondió adecuadamente ante todas las condiciones de emergencia evaluadas: activación de finales de carrera, sobrevelocidad y parada de emergencia. Adicionalmente, se integró una interfaz de operación compuesta por un mando de joystick físico y una visualización 2D del entorno portuario en tiempo real, conformando un entorno de simulación semi-inmersivo que permitió validar la operación del sistema de forma representativa.

Desde el punto de vista metodológico, el flujo *Model-in-the-Loop* (MiL) $\rightarrow$ *Software-in-the-Loop* (SiL) demostró ser una estrategia eficaz para el desarrollo de sistemas de control industriales complejos. La etapa MiL en Stateflow/Simulink permitió validar y ajustar la lógica del autómata en un entorno ágil, mientras que la etapa SiL expuso aspectos no evidentes en simulación pura: diferencias semánticas entre Stateflow y el estándar IEC-61131-3, restricciones de nomenclatura entre charts y la necesidad de adaptaciones manuales sobre el código generado por Simulink PLC Coder. El resultado es un algoritmo verificado y listo para ser cargado en un controlador PLC real, lo que representa un valor concreto más allá del entorno académico.

Uno de los desafíos más significativos fue la sincronización en co-simulación: la coexistencia de tres períodos de muestreo distintos y la latencia del canal OPC UA requirieron un diseño cuidadoso de la arquitectura de comunicación para garantizar la coherencia de los datos en cada ciclo. La separación clara entre los tres niveles jerárquicos facilitó tanto el desarrollo como la depuración, y la arquitectura resultó naturalmente compatible con la estructura de POU's del estándar IEC-61131-3.

Finalmente, el trabajo permitió integrar conocimientos de las cátedras de Automática

y Máquinas Eléctricas y de Autómatas y Control Discreto, evidenciando cómo ambos dominios del problema de control —el continuo y el discreto— se articulan de forma complementaria en una aplicación industrial real. El uso simultáneo de MATLAB, Simulink y CODESYS, junto con los protocolos de comunicación que los vinculan, fortaleció el conocimiento sobre programación de PLCs, lenguajes del estándar IEC-61131-3 y en particular sobre el lenguaje SFC.

9. Nota sobre el uso de agentes de Inteligencia Artificial

Durante el desarrollo del presente proyecto se hizo uso de agentes de Inteligencia Artificial (IA) como herramienta de apoyo en diversas etapas del trabajo. En particular, se emplearon para el desarrollo de las interfaces visuales en Python: el visualizador en tiempo real de la planta en 2D y el panel de monitoreo de las entradas del control de mando.

Adicionalmente, los agentes de IA fueron utilizados para:

- **Mejora de coherencia, redacción y formato del informe:** revisión de la prosa técnica, consistencia terminológica y estructura del documento.
- **Investigación y sugerencia de pruebas:** propuesta de escenarios de validación, alternativas de diseño y búsqueda de errores en desarrollos matemáticos, de programación y conceptuales.

En cuanto a la metodología de interacción, se adoptó un enfoque de tipo *black-box*: al agente se le asignaron tareas pequeñas y específicas con entradas y salidas claramente definidas, lo que permitió una mejor modularización del desarrollo. Este enfoque resultó en un proceso incremental y escalable, en el que cada entregable parcial fue verificado antes de integrarse al conjunto, sin perder la visión global del sistema que se estaba desarrollando.

Finalmente, se destaca que el uso de la Inteligencia Artificial en este trabajo se realizó de forma consciente y responsable, entendiéndola como una herramienta complementaria a las capacidades propias de los integrantes del equipo. En ningún momento se buscó delegar en el agente el razonamiento, la comprensión ni la toma de decisiones de diseño: los desarrollos matemáticos, las decisiones de arquitectura y la interpretación de resultados fueron siempre responsabilidad de los autores. La IA fue utilizada para potenciar, corregir y mejorar el trabajo humano, no para reemplazarlo.

Referencias

- [1] Julián, G. L., *Proyecto Global Integrador 2025: Control de Accionamientos y Automatización de Grúa Portacontenedores de Muelle tipo Pórtico – Guía de Trabajo y Especificaciones, Rev. 0*. UNCuyo – Ingeniería Mecatrónica, Autómatas y Control Discreto, Mendoza, 2025.
- [2] UNCTAD, *Review of Maritime Transport 2023*. United Nations Conference on Trade and Development, Ginebra, 2023. Disponible en: <https://unctad.org/rmt2023>
- [3] Drewry Maritime Research, *Container Terminal Productivity: Key Performance Indicators 2022*. Drewry Shipping Consultants Ltd., Londres, 2022.
- [4] ZPMC (Zhenhua Port Machinery Company), *Company Profile and Global Installations*, 2023. Disponible en: <https://www.zpmc.com>
- [5] IEC, *IEC 61131-3: Programmable Controllers – Part 3: Programming Languages*, 3ra ed. International Electrotechnical Commission, Ginebra, 2013.
- [6] OPC Foundation, *OPC Unified Architecture Specification, Part 1: Overview and Concepts*. OPC Foundation, Scottsdale, AZ, 2017. Disponible en: <https://opcfoundation.org>